CNNA – Arithmetik-Plan ab S9h

Fortschreibungsfassung v96: SM9g1 lokal gruen, SM9h aktiv

antaris / ChatGPT

8. Mai 2026

Status dieser Fortschreibung v96. Diese Fassung schreibt v95 nach dem lokal gruen bestaetigten `SM9g1--OperationalQuadraticCoefficientWindowFromSM9dSM9e`-Build fort. SM9g1 ist als kleine Koeffizientenfenster-Schicht geschlossen: regular und variable `constantCoeff`, `linearCoeff` und `quadraticCoeff` sind aus den in SM9g validierten SM9d-/SM9e-Koeffizientenquellen exponiert und an `operationalDeterminantCoefficientRecurrenceSM9e` zurueckgebunden. SM9g1 erzeugt weiterhin keinen Diskriminantenwert und keinen Faktorbeweis; deshalb bleibt `SM10` blockiert. Aktiv wird nun `SM9h--OperationalQuadraticDiscriminantSourceFromWindow`.

0d. Fortschreibungsentscheidung v28 nach gruenem SM3db4aR-Build

Statusentscheidung	SM3db4aR ist nach lokal gruenem Build formal geschlossen. Der in v27 blockierende Befund zu SM3db4R ist damit bearbeitet: Die neue CandidateEntry-Herkunft laeuft nicht mehr als blosse lokale Zeilen-Residualprojektion, sondern ueber eine akkumulierte TraceEvaluation-/AccumulatedEntryFold-Schicht. SM3db4R bleibt historisch `green/partial`; SM3db4aR traegt den geschlossenen strukturellen DoD. Der naechste aktive Schritt ist daher SM3db5R.
Formaler SM3db4aR-Ertrag	`GeneratedInteriorAccumulatedTraceEvaluation`, `GeneratedInteriorAccumulatedEntryFold`, `accumulatedEntry`, Herkunft aus SM3db3R-Trace, Pivotdaten, lokalen SM3db2R-Steps, Residualdaten und Fold-/Update-Reihenfolge; Ledger gegen den blossen lokalen `rowResidual`-Wrapper; `noFreeEntryTable`; `noExternalInverseOracle`; next-positive-phase-Ledger fuer SM3db5R. Kein `Matrix.inv`, kein Matrix-Export, kein Produktbeweis, kein `TwoSidedInv`, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`.
Praeziser Restbefund	SM3db4aR beweist noch nicht, dass die akkumulierte CandidateEntry eine links- oder rechtsseitige Inverse des SM3cR-InteriorBlocks ist. Sie beweist nur die legale, derived-only, akkumulierte Herkunft der Entry-Funktion. Deshalb darf SM3db5R die Funktion verpacken, aber nicht als Inverse validieren.
SM3db5R-Freigabe	SM3db5R ist freigegeben und muss ein reiner Matrix-Export sein: Die Matrixeintraege werden definitionell oder theorematisch aus der SM3db4aR-`accumulatedEntry` gewonnen. SM3db5R darf keine neue CandidateEntry-Formel, keine freie Matrix, keine freie Entry-Tabelle, kein `Matrix.inv`, kein externes Inversen-Oracle und keine SM3daR-Rueckfuehrung einfuehren.
SM3db5R-Grenze	SM3db5R erzeugt keinen Links-/Rechtsproduktbeweis, kein `TwoSidedInv`, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Charpoly-/Faktor-/Diskriminantendaten. Der DoD ist genau: eine konkrete Kandidatenmatrix, deren jeder Eintrag als SM3db4aR-`accumulatedEntry` nachgewiesen ist.
SM3db6R-Sperre	SM3db6R bleibt bis zum gruenen SM3db5R-Build gesperrt. Es ist ein nachgelagerter Shape-/Source-Separation-Filter: Ist die exportierte Matrix wirklich die SM3db4aR-`accumulatedEntry`-Matrix? Ist sie nicht versehentlich die SM3daR-/InteriorBlock-Matrix? Ist sie nicht bloss der SM3db4R-localResidual-Wrapper? Ist sie nicht frei tabelliert?
SM3eR-Produktgrenze	SM3eR bleibt die erste Phase, die `GeneratedInteriorCandidateLeftProductEntry`, `GeneratedInteriorCandidateRightProductEntry`, Diagonal-/Offdiag-Produkte und schliesslich `provenTwoSidedInv` versuchen darf. SM3db5R–SM3db7R duerfen diese Produktvalidierung nicht vorwegnehmen.
Rueckkopplungsregel	Falls SM3db5R zeigt, dass der Matrix-Export weitere exponierte Herkunfts- oder Typgleichheitslemmata aus SM3db4aR benoetigt, erfolgt eine minimale positive Nachschaerfung von SM3db4aR. Eine unmittelbare Obstruktion oder ein Start von `SM3dbOmegaR` bleibt Planbruch, solange SM3db5R nicht positiv versucht und mathematisch dokumentiert gescheitert ist.

0e. Fortschreibungsentscheidung v29 nach gruenem SM3db5R-Build

Statusentscheidung	SM3db5R ist nach lokal gruenem Build formal geschlossen. Der Matrix-Export aus SM3db4aR ist damit erledigt: Die Kandidatenmatrix ist nur die entrywise Verpackung der akkumulierten `accumulatedEntry`; sie fuehrt keine neue CandidateEntry-Formel, keine freie Matrix, kein `Matrix.inv`, keine SM3daR-Rueckfuehrung und keine Produktvalidierung ein. Der naechste aktive Schritt ist SM3db6R.
Formaler SM3db5R-Ertrag	`GeneratedInteriorInverseCandidateMatrix`, `inverseCandidateMatrix_from_accumulatedEntry`, `inverseCandidateMatrix_entry_eq_accumulatedEntry`, Matrixeintrags-Herkunft aus `GeneratedInteriorAccumulatedTraceEvaluation`/`GeneratedInteriorAccumulatedEntryFold`, No-Free-/No-Oracle-/No-`Matrix.inv`-Ledger und explizite Sperren gegen `TwoSidedInv`, `InteriorEliminationCertificate` und `InteriorEliminationData`.
Praeziser Restbefund	SM3db5R beweist weiterhin nicht, dass die exportierte Matrix eine links- oder rechtsseitige Inverse des SM3cR-InteriorBlocks ist. SM3db5R beweist nur, dass eine konkrete konsumierbare Kandidatenmatrix aus der SM3db4aR-accumulatedEntry-Herkunft bereitsteht. Die Produktfrage bleibt vollstaendig SM3eR vorbehalten.
SM3db6R-Freigabe	SM3db6R ist freigegeben und muss als Shape-/Source-Separation-Filter implementiert werden. Die Phase prueft, ob die exportierte Matrix wirklich die SM3db4aR-accumulatedEntry-Matrix ist, nicht die SM3daR-/InteriorBlock-Matrix, nicht bloss der alte SM3db4R-localResidual-Wrapper und nicht eine freie Tabelle.
Positive-First-Grenze	SM3db6R ist kein Obstruktionsstart. Falls eine Gleichheit mit dem SM3cR-InteriorBlock sichtbar wird, ist sie nur akzeptabel, wenn ein separater involutiver Spezialfall vorbereitet ist, in dem `InteriorBlock * InteriorBlock = 1` links und rechts aus SM3cR/SM3dR folgt. Ohne diesen Spezialfall erfolgt kein SM3eR-Start, sondern eine positive Rueckkopplung zu SM3db1R–SM3db4aR bzw. zur Kandidatenherkunft.
Zulaessige SM3db6R-Kernobjekte	`inverseCandidateEntry_not_SM3daR_candidateEntry`, `inverseCandidateEntry_not_interiorBlock_or_involutiveBlockProof`, `inverseCandidateMatrix_not_interiorBlock_or_involutiveBlockProof`, `SM3dbRShapeSeparationLedger`. Zulaessig sind ausserdem reine Source-/Shape-Herkunftslemmata, die diese Aussagen ohne Produktrechnung absichern.
SM3db6R-Grenze	SM3db6R erzeugt keinen Candidate-Linksproduktbeweis, keinen Candidate-Rechtsproduktbeweis, kein `provenTwoSidedInv`, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Charpoly-/Faktor-/Diskriminantendaten.
Folgesperren	Historisch blieb SM3db7R bis zum gruenen SM3db6R-Build blockiert; nach der v31-Fortschreibung ist SM3db7R geschlossen und der erneute SM3eR-ReRun freigegeben. `SM3dbOmegaR` bleibt verboten, weil der positive Pfad bis SM3db7R nicht gescheitert ist.

0f. Fortschreibungsentscheidung v30 nach gruenem SM3db6R-Build

Statusentscheidung	SM3db6R ist nach lokal gruenem Build formal geschlossen. Die Shape-/Source-Separation ist damit erledigt: Die Kandidatenmatrix bleibt die in SM3db5R exportierte Matrix und wird entrywise auf die SM3db4aR-`accumulatedEntry`-Herkunft zurueckgebunden. SM3db6R ist kein Obstruktionsstart und kein Produkt-Gate. Der naechste aktive Schritt ist SM3db7R.
Formaler SM3db6R-Ertrag	`GeneratedInteriorInverseCandidateShapeSeparation`, `inverseCandidateShapeSeparation_from_matrix`, `inverseCandidateEntry_not_SM3daR_candidateEntry`, `inverseCandidateEntry_not_interiorBlock_or_involutiveBlockProof`, `inverseCandidateMatrix_not_interiorBlock_or_involutiveBlockProof`, `inverseCandidateMatrix_not_only_localResidualWrapper`, `inverseCandidateMatrix_entry_eq_accumulatedTraceValue_afterShapeSeparation`, `noProductProof_for_shapeSeparation`, `noTwoSidedInv_for_shapeSeparation`, regular/variable Shape-Separation-Konstruktoren und `SM3dbRShapeSeparationLedger`.
Praeziser Restbefund	SM3db6R beweist bewusst keine semantische Wert-Ungleichheit `candidateMatrix != interiorBlock`. Die Namen der `not_*`-Gates sind als Source-/Shape-Status zu lesen: keine Rueckkehr zur SM3daR-CandidateEntry, keine freie InteriorBlock-Identifikation ohne spaeteren involutiven Produktbeweis, kein bloss lokaler Residual-Wrapper, keine freie Tabelle. Die erste echte Produktvalidierung bleibt vollstaendig SM3eR vorbehalten.
SM3db7R-Freigabe	SM3db7R ist freigegeben und muss eine reine Handoff-Schicht vom SM3db6R-gefilterten Kandidaten an den erneuten SM3eR-Lauf sein. SM3db7R darf nur bestimmen, welches CandidateEntry-/CandidateMatrix-Objekt SM3eR konsumieren darf, und muss zugleich festhalten, dass SM3fR ohne `provenTwoSidedInv` weiterhin gesperrt bleibt.
Zulaessige SM3db7R-Kernobjekte	`SM3dbRGeneratedInverseCandidateLedger`, `SM3dbRToSM3eRHandoff`, `SM3eRMayConsumeOnlySM3dbRInverseCandidate`, `noTwoSidedInv_for_SM3db7R`, `noInteriorEliminationCertificate_for_SM3db7R`, `noInteriorEliminationData_for_SM3db7R`. Zulaessig sind ausserdem reine Handoff-Lemmata, die regular und variable Seite auf die SM3db6R-ShapeSeparation und dadurch auf die SM3db5R-/SM3db4aR-Herkunft zurueckbinden. Eine separat benannte `SM3fRStillBlockedWithoutProvenTwoSidedInv`-Deklaration ist optional, sofern die Sperre ueber diese Felder geschlossen wird.
SM3db7R-Grenze	SM3db7R erzeugt keine `GeneratedInteriorCandidateLeftProductEntry`, keine `GeneratedInteriorCandidateRightProductEntry`, keinen Diagonal-/Offdiag-Produktbeweis, kein `provenTwoSidedInv`, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Charpoly-/Faktor-/Diskriminantendaten. Jede Produktrechnung in SM3db7R waere eine Vorwegnahme von SM3eR und damit Planbruch.
Folgesperren	Der erneute SM3eR-Lauf ist nach gruenem SM3db7R-Build freigegeben. SM3fR bleibt bis zu einem in SM3eR bewiesenen `provenTwoSidedInv` blockiert. `SM3dbOmegaR` bleibt verboten, solange die Handoff-Schicht nicht aus einem dokumentierten mathematischen Grund scheitert; ein spaeteres SM3eR-Produktscheitern ist ein SM3eR-Obstruktionsbefund, nicht automatisch ein SM3dbOmegaR-Start.
Planungspraezisierung	Die Berechnungskette bleibt derived-only: SM3db2R lokaler Step → SM3db3R Trace → SM3db4aR accumulated fold → SM3db5R Matrix-Export → SM3db6R Source-/Shape-Gate → SM3db7R Handoff → SM3eR Produktvalidierung. Keine dieser Handoff-Vorphasen darf die Produktfrage vorwegnehmen.

0g. Fortschreibungsentscheidung v31 nach gruenem SM3db7R-Build – historisch, durch v32 geschaerft

Historischer Kern	SM3db7R ist nach lokal gruenem Build formal geschlossen. Die Handoff-Schicht ist erledigt: `SM3dbRToSM3eRHandoff` exponiert die in SM3db6R gefilterte Kandidatenmatrix, bindet sie an SM3db5R-Matrix-Export und SM3db4aR-`accumulatedEntry`-Herkunft zurueck und erzeugt selbst keine Produktvalidierung. Dieser Befund bleibt gueltig.
Formaler SM3db7R-Ertrag	`SM3dbRToSM3eRHandoff`, `sm3dbRToSM3eRHandoff_from_shapeSeparation`, `SM3eRMayConsumeOnlySM3dbRInverseCandidate`, `sm3eRMayConsumeOnlySM3dbRInverseCandidate_for_handoff`, `handoffCandidate_eq_SM3db6R_shapeSeparationMatrix`, `handoffCandidate_source_eq_SM3db5R_matrixExport`, `handoffCandidate_source_eq_SM3db4aR_accumulatedEntry`, `noLeftProduct_for_SM3db7R`, `noRightProduct_for_SM3db7R`, `noTwoSidedInv_for_SM3db7R`, `noInteriorEliminationCertificate_for_SM3db7R`, `noInteriorEliminationData_for_SM3db7R`, regular/variable Handoff-Konstruktoren und `SM3dbRGeneratedInverseCandidateLedger`.
Praeziser Restbefund	SM3db7R beweist weiterhin nicht, dass die Kandidatenmatrix eine links- oder rechtsseitige Inverse des SM3cR-InteriorBlocks ist. Die Phase beweist nur, welches Kandidatenobjekt SM3eR konsumieren darf. Damit ist SM3db7R kein Produkt-Gate, keine Reparaturformel und kein Ersatz fuer SM3eR.
v32/v35-Korrektur	Die alte v31-Kurzform `SM3eR-r0-r4` ist zu grob. Ab v32 galt die feinere ReRun-Unterteilung: r0 HandoffShapeAudit, r1 LeftProductEntry, r2 RightProductEntry, r3a ProductNormalForm, r3b Kancellation, r4 `GeneratedInteriorSM3db7RProductIdentityProof`-Konstruktor und r5 `provenTwoSidedInvFromSM3db7R`. Ab v34 wurde diese Unterteilung geschaerft: r3b ist nur die gruen geschlossene Kancellationsoberflaeche; r3c ist die zwingende TraceCancellationInvariant-Derivation vor r4. Ab v35 wird r3c selbst gespalten: r3c0 ist closed/green als Source-/Derivation-/Ledger-API; r3c1 bleibt next/active als eigentliche unbedingte Ableitung der sechs Kancellationsbeweise.
Sperren	SM3fR bleibt bis zu einem theorematisch konstruierten `provenTwoSidedInv` blockiert. `SM3dbOmegaR` bleibt verboten, solange die positive SM3dbR-Handoff-Kette geschlossen ist und kein dokumentiertes mathematisches Scheitern des Handoff selbst vorliegt. Ein Produktscheitern im ReRun ist ein SM3eR-Produktbefund und fuehrt nicht automatisch zu SM3dbOmegaR.

0h. Fortschreibungsentscheidung v32: Kohaerenzkorrekturen vor SM3eR-Re-Run

Ziel dieser Fassung	v32 korrigiert alle Planstellen, die nach Codepruefung missverstaendlich waren: b-Konvention, IdealThread-Status, ReRun-Status, Produktnormalform-Pflicht, volle Generated-DtN-Stufen, Geometrie-/Heegner-Hypothese und CD-/$\mathbb{C}$-Interface. Diese v32-Regeln haben Vorrang vor historischer v28–v31-Prosa.
Globale b-Konvention	Im Code ist `RegularCell b L := Fin L -> Fin (b+1)`. Daher gilt `arity = code-b + 1`. Der gezeichnete 0/1-binaere Baum ist `arity = 2`, also `code-b = 1`. `code-b = 0` ist der unaere volle Geradenfall; `code-b = 2` ist im Code ternaer. Jede kuenftige Planzeile muss entweder `code-b` oder `arity` schreiben; ein nacktes $b = 2$ fuer einen binaeren Baum ist verboten.
IdealThread-Status	`referenceThread b` und die aktuellen `ToCStrong.ofIdeal`-Quellen sind All-0-Singleton-Pfade im Ambienttyp. Sie sind kein voller `code-b=0`-Baum und kein voller binaerer Baum. Sie koennen keine mehrdimensionale Boundary-/Schur-/Diskriminantenarithmetik tragen.
Positive Quelle	Der aktive Arithmetik-Smoke-Pfad ist generated/non-singleton: `generatedBranchFamily`, `GeneratedMainPath`, SM0R–SM3db7R und der Handoff an SM3eR. Der IdealThread bleibt No-go-/Auditobjekt und Vergleichsmotivation.
SM3eR-Re-Run Status	Der ReRun ist nach v35 bis einschliesslich r3c0 build-gruen, aber noch nicht als Produktidentitaet geschlossen. HandoffShapeAudit, LeftProductEntry, RightProductEntry, r3a-ProductNormalForms, r3b-CancellationSurface und die r3c0-TraceCancellationDerivation-API liegen vor. Der eigentliche unbedingte `GeneratedInteriorSM3db7RTraceCancellationInvariant` muss in r3c1 noch aus der SM3db-Provenienzkette abgeleitet werden; der `GeneratedInteriorSM3db7RProductIdentityProof` ist weiterhin nicht geschlossen.
SM3eR-Re-Run Kernrisiko	Weder `GeneratedInteriorSM3db7RTraceCancellationInvariant` noch `GeneratedInteriorSM3db7RProductIdentityProof` duerfen als freie Ledgerfelder, Annahmen oder Prop-Orakel eingefuehrt werden. r3c0 darf nur die proof-carrying Schnittstelle bereitstellen; r3c1 muss den TraceCancellationInvariant wirklich aus der SM3cR/SM3db2R–SM3db7R/r3a-Kette ableiten. r4a darf hoechstens konditional den Strukturkonstruktor vom ProductCancellationLedger zum ProductIdentityProof bauen; ein unbedingter r4-Abschluss ist erst nach r3c1 erlaubt.
Importdisziplin	Der Import `CNNA.PillarA.DtN.DtN` in `GeneratedInteriorTwoSidedInvFromSM3db7R.lean` ist derzeit nur als Zugriff auf `TwoSidedInv` erlaubt. Er gilt nicht als Erzeugung oder Konsum von `DtNStrong`, `GeneralizedDtN`, `MultiSchur`, `InteriorEliminationData` oder arithmetischen Zielobjekten. Langfristig ist eine neutrale Auslagerung von `TwoSidedInv` in eine Finite-/Matrix-Datei sauberer.
Produktnormalform-Pflicht	SM3eR-r3a muss beide Matrixprodukte entrywise oeffnen und die Handoff-Matrix auf `generatedInteriorAccumulatedEntryValue T` zurueckfuehren. Ohne diese Normalform waere jede Identitaetsbehauptung nur eine Behauptung ueber ein ungeoeffnetes Matrixprodukt.
Ruecksprungregel	Falls `SM3eR-r3c1` zeigt, dass die vorhandene `accumulatedEntry`-Schicht keine Kancellationsinvariante traegt, wird nicht in SM3eR gepatcht. Dann entstehen konditionale Ruecksprungphasen: `SM3db4bR` fuer `AccumulatedEntryCancellationInvariant`, `SM3db3aR` fuer `TraceOrderAndPivotCoverageInvariant`, `SM3db2aR` fuer `LocalStepSchurCancellationInvariant`.
Generated-DtN-Praezisierung	Nach SM3fR baut SM3gR zunaechst die finite generated top-boundary DtN aus SM3aR-Partition, SM3bR-Dirichletdaten und SM3fR-InteriorEliminationData. Das ist nicht der alte `IdealThread/CutSpec.full`-Pfad. Eine full LevelHistory-/multi-level DtN-Diagnostik entsteht erst spaeter ueber SM6–SM8.
Geometrie-Status	Die spiraligen 0/1-Zeichnungen sind phaenomenologische Visualisierungen fuer `arity=2/code-b=1`. Sie erzeugen keine Winkel, keine Gitter, keine CM-Punkte und keine Diskriminanten im derived-only Pfad. Sie duerfen erst nach berechneter Generated-DtN/Schur/Charpoly-Route als Vergleichsbild konsumiert werden.
Eisenstein/Gauss-Status	Die vorhandenen L2-/L3-Schichten sind kleine operative Schur-/Charpoly-/Diskriminanten-Seeds bzw. Zielvergleiche. Sie beweisen nicht, dass die Zeichnung oder ein freier Winkel diese Faelle erzeugt. In SM10/SM11 zaehlt nur: berechnete Matrix $\to$ Charpoly/Faktor/Diskriminante $\to$ Vergleich mit -3 bzw. -4.
Heegner-/noOuter-Test	Die Hypothese, dass noOuter/hasOuter oder Komplementfamilien Heegner-Differenzen sichtbar machen, ist ein spaeter falsifizierbarer Test nach SM9/SM12, nicht SM3eR-Input. Kein Heegner-Wert darf vor berechneter Charpoly-/Diskriminantenroute als Provenienz verwendet werden.
CD-/Interface-Status	Cayley-Dickson/Hurwitz darf nur soweit konsumiert werden, wie gruene A-interne Strukturmodule theorematisch tragen. Eine volle Identifikation bright/interface/dark/all $\leftrightarrow \mathbb{R}, \mathbb{C}, \mathbb{H}, \mathbb{O}$ ist kein SM3eR-Argument und darf keine Produktidentitaet ersetzen.

0i. Fortschreibungsentscheidung v33 nach gruenem SM3eR-r3a-Build

Statusentscheidung	`SM3eR-r3a` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die vorgeschriebene ProductNormalForm-Schicht geliefert: linkes und rechtes Matrixprodukt werden entrywise geoeffnet und die Handoff-Kandidatenmatrix wird ueber die SM3db7R-/SM3db4aR-Herkunft auf `generatedInteriorAccumulatedEntryValue T` zurueckgefuehrt. r3a ist damit kein Produktidentitaetsbeweis, sondern die berechnungsfaehige Summenoberflaeche fuer r3b.
Formaler r3a-Ertrag	Zulaessige und nun geschlossene Kernobjekte sind insbesondere `GeneratedInteriorSM3db7RLeftProductNormalForm`, `GeneratedInteriorSM3db7RRightProductNormalForm`, `leftProductEntry_expand_handoffCandidate`, `rightProductEntry_expand_handoffCandidate`, `generatedInteriorSM3db7RLeftProductEntry_eq_sum_accumulated`, `generatedInteriorSM3db7RRightProductEntry_eq_sum_accumulated`, `GeneratedInteriorSM3db7RProductNormalFormLedger` und die regular/variable ProductNormalForm-Ledger.
Praeziser Restbefund	r3a beweist noch keine Kancellation. Insbesondere fehlen weiterhin `leftProduct_entry_eq_identity`, `rightProduct_entry_eq_identity`, `leftProduct_diagonal_entry`, `leftProduct_offdiag_entry`, `rightProduct_diagonal_entry` und `rightProduct_offdiag_entry`. Daraus folgt: kein `GeneratedInteriorSM3db7RProductIdentityProof`, kein `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Charpoly-/Faktor-/Diskriminantendaten.
r3b-Freigabe	`SM3eR-r3b` ist freigegeben und ist der naechste aktive Schritt. Ziel ist nicht sofort r4, sondern zuerst eine proof-carrying Kancellationsoberflaeche aus den r3a-Normalformen. Erlaubt sind diagonal/offdiag-Kancellationslemmata, Entry-Identity-Lemmata und ein Ledger, das die sechs spaeteren ProductIdentityProof-Felder vorbereitet.
r3b-Provenienzpflicht	Alle r3b-Beweise muessen aus der bestehenden Kette folgen: SM3cR-InteriorBlock-Profil, SM3db2R-Lokalschritte, SM3db3R-Trace, SM3db4aR-AccumulatedEntryFold, SM3db5R-Matrix-Export, SM3db6R-Shape-/Source-Gate, SM3db7R-Handoff und r3a-ProductNormalForms. Freie Prooffelder, externe Inversenannahmen, `Matrix.inv`, neue CandidateEntry-Formeln, SM3daR-Rueckfall, Rechenorakel oder direkte r4/r5-Spruenge sind Planbruch.
r4/r5-Sperre	Historisch blieb `SM3eR-r4` nach v33 blockiert, bis r3b die sechs Kancellations-/Identity-Felder proof-carrying liefert. Nach v34 ist diese Bedingung verfeinert: r3b liefert nur die Kancellationsoberflaeche; r4 bleibt bis zur r3c-Derivation des `GeneratedInteriorSM3db7RTraceCancellationInvariant` blockiert. `SM3eR-r5` bleibt blockiert, bis r4 einen konkreten theorematischen Konstruktor von `GeneratedInteriorSM3db7RProductIdentityProof` bereitstellt. `SM3fR` bleibt blockiert, bis r5 einen `provenTwoSidedInvFromSM3db7R`-Witness erzeugt.
Rueckspringregel	Nach v35 praezisiert: Falls r3c1 zeigt, dass die r3a-Summenformen und die SM3db-Trace-/Fold-Schichten keine Kancellationsinvariante tragen, wird nicht in SM3eR gepatcht. Dann wird minimal positiv zur fehlenden Invarianzschicht zurueckgesprungen: `SM3db4bR` fuer `AccumulatedEntryCancellationInvariant`, `SM3db3aR` fuer `TraceOrderAndPivotCoverageInvariant` oder `SM3db2aR` fuer `LocalStepSchurCancellationInvariant`. Dieser Rueckspring ist kein `SM3dbOmegaR`-Start; `SM3dbOmegaR` bleibt nur fuer dokumentiertes Scheitern der positiven Handoff-Kette selbst zulaessig.
Planstatus	Ab v36 lautet die Architektur: SM3db2R → SM3db3R → SM3db4aR → SM3db5R → SM3db6R → SM3db7R → SM3eR-r3a → SM3eR-r3b → SM3eR-r3c0 → SM3eR-r3c1 → SM3eR-r3c2 → SM3eR-r4a → SM3eR-r4 → SM3eR-r5 → SM3fR. Der Status wird fortgeschrieben: r3a/r3b/r3c0 closed/green, r3c1 Schnittstellen-Build green aber r3c1-full next/active, r3c2 conditional/green, r4a conditional/allowed, r4/r5/SM3fR blocked.

0j. Fortschreibungsentscheidung v34 nach gruenem SM3eR-r3b-Build – historisch, durch v35 geschaerft

Historischer Kern	`SM3eR-r3b` ist nach lokal gruenem Build formal als bedingte Kancellationsoberflaeche geschlossen. Die sechs spaeteren ProductIdentityProof-Felder sind nur relativ zu einem `TraceCancellationInvariant` bzw. ProductCancellationLedger konsumierbar. r3b erzeugt keinen `ProductIdentityProof`, keinen `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Charpoly-/Faktor-/Diskriminantendaten.
v34-Restbefund	v34 verlangte deshalb eine r3c-Ableitung des `GeneratedInteriorSM3db7RTraceCancellation Invariant` aus SM3cR, SM3db2R–SM3db7R und r3a-NormalForms, bevor r4 beginnen darf.
v35-Schaerfung	Nach gruenem r3c-Build wird dieser Punkt aufgespalten: r3c0 ist die geschlossene Source-/Derivation-/Ledger-API; r3c1 bleibt die aktive mathematische Ableitung der sechs Kancellationsfelder. Abschnitt 0k ist fuer den aktuellen Status verbindlich.
Sperren	`SM3eR-r4`-closed, r5 und `SM3fR` bleiben blockiert, bis r3c1 einen unbedingten Derivation-Witness ohne freie sechs Kancellationsargumente liefert.

0k. Fortschreibungsentscheidung v35 nach gruenem SM3eR-r3c-Build

Statusentscheidung	`SM3eR-r3c` ist nach lokal gruenem Build formal integrierbar, aber nicht in dem starken Sinn geschlossen, den v34 als DoD formuliert hatte. Gruen ist die neue proof-carrying Schnittstelle: Source aus SM3db7R-Handoff, Derivation-Struktur, Konstruktor vom Derivation-Witness zum TraceCancellationInvariant und Export in den ProductCancellationLedger. Nicht gruen als unbedingter mathematischer Satz ist die eigentliche Kancellation aus der SM3db-Kette, weil die sechs Kancellations-/Identity-Beweise noch als Felder eines Derivation-Witness uebergeben werden. Deshalb wird die Phase ab v35 in `SM3eR-r3c0` und `SM3eR-r3c1` getrennt.
Formaler r3c0-Ertrag	`SM3eR-r3c0` ist closed/green als Schnittstellen- und Provenienzschicht. Zulaessige geschlossene Rollen sind insbesondere `GeneratedInteriorSM3db7RTraceCancellationSource`, `GeneratedInteriorSM3db7RTraceCancellationDerivation`, `traceCancellationInvariant_from_traceDerivation` bzw. ein aequivalenter Derivation-to-Invariant-Konstruktor, `productCancellationLedger_from_traceCancellationDerivation`, regular/variable r3c0-Konstruktoren und `SM3eRr3cTraceCancellationDerivationLedger`. r3c0 erzeugt keinen ProductIdentityProof, keinen TwoSidedInv, keine InteriorEliminationData, keine DtN-/MultiSchur- und keine Charpoly-/Diskriminantendaten.
Praeziser Restbefund	r3c0 beweist nicht selbst `leftProduct_entry_eq_identity`, `rightProduct_entry_eq_identity`, `leftProduct_diagonal_entry`, `leftProduct_offdiag_entry`, `rightProduct_diagonal_entry` und `rightProduct_offdiag_entry` aus `generatedInteriorAccumulatedEntryValue T`. Diese Beweise sind noch die Nutzlast der Derivation. Damit liegt kein schlechtes externes Oracle vor, solange r4 sie nicht frei konsumiert; aber es liegt auch noch kein unbedingter derived-only TraceCancellationInvariant vor.
r3c1-Freigabe	`SM3eR-r3c1` ist der naechste aktive positive Schritt. Ziel ist die theorematische Konstruktion eines konkreten Derivation-Witness ohne freie Kancellationsannahmen. Die sechs Felder muessen aus der bestehenden Kette kommen: `generatedInteriorBlock W` und SM3cR-Entry-/Diag-/Offdiag-Profil; SM3db2R-Lokalstep mit Schur-/Residual-Kancellationsdaten; SM3db3R-Trace mit Reihenfolge, Pivot-Coverage und Step-Provenienz; SM3db4aR-AccumulatedEntryFold; SM3db5R-Matrix-Export; SM3db6R-Shape-/Source-Gate; SM3db7R-Handoff auf `H.candidateMatrix`; r3a-ProductNormalForms.
r3c1-DoD	r3c1 ist erst geschlossen, wenn ein Konstruktor wie `traceCancellationDerivation_from_SM3dbChain` bzw. `traceCancellationInvariant_from_SM3dbChain` existiert, dessen Eingaben keine freien Kancellationsbeweise, keine freie Matrix, keine freie Entry-Tabelle und keine externe Inversenannahme sind. Der Output muss einen unbedingten `GeneratedInteriorSM3db7RProductCancellationLedger` liefern, der fuer r4 konsumierbar ist.
r4a-Freigabe	`SM3eR-r4a` darf parallel oder als naechster kleiner Code-Schritt nur konditional vorbereitet werden: ein rein struktureller Konstruktor `productIdentityProof_from_productCancellationLedger` vom bereits gegebenen `GeneratedInteriorSM3db7RProductCancellationLedger` zum `GeneratedInteriorSM3db7RProductIdentityProof`. r4a darf keinen eigenen TraceCancellationInvariant erzeugen, keine der sechs Kancellationen neu annehmen, keine Matrixrechnung wiederholen und keinen `TwoSidedInv` bauen.
r4/r5-Sperre	Ein geschlossener `SM3eR-r4`-Status bleibt blockiert, bis r3c1 den ProductCancellationLedger unbeding aus der SM3db-Provenienzkette liefert. `SM3eR-r5` bleibt blockiert, bis r4 den ProductIdentityProof theorematisch konstruiert. `SM3fR` bleibt bis zum r5-Witness `provenTwoSidedInvFromSM3db7R` blockiert.
Ruecksprungregel	Falls r3c1 zeigt, dass eine noetige Kancellationssubstanz in der bisherigen Provenienzkette fehlt, wird nicht in r4 und nicht lokal in SM3eR gepatcht. Dann wird minimal positiv zur schwaechsten fehlenden Schicht zurueckgesprungen: `SM3db4bR` fuer AccumulatedEntryCancellationInvariant, `SM3db3aR` fuer TraceOrderAndPivotCoverageInvariant oder `SM3db2aR` fuer LocalStepSchurCancellationInvariant. Dieser Ruecksprung bleibt positive Pfadfortsetzung und ist kein `SM3dbOmegaR`-Start.
Planstatus	Verbindlich ist nun: r0–r3b closed/green; r3c0 closed/green als bedingte Derivation-/Ledger-API; v36 ersetzt die alte r3c1-Rolle durch zwei Stufen: r3c1 als AccumulatedEntryCancellationInvariant-Schicht und r3c2 als TraceCancellation-Verbrauchsschicht. r4a bleibt conditional/allowed; r4/r5/SM3fR bleiben bis zum unbedingten r3c1-full-Witness blockiert.

0l. Fortschreibungsentscheidung v36 nach gruenem SM3eR-r3c1/r3c2-Schnittstellen-Build

Statusentscheidung	Der angehaengte Stand ist lokal gruen fuer die neue Phasenaufspaltung: `SM3eR-r3c1` enthaelt die Fold-nahe AccumulatedEntryCancellation-Invariant-Oberflaeche, `SM3eR-r3c2` enthaelt den Adapter von einem r3c1-Witness zur bisherigen TraceCancellationDerivation-/Invariant-/Ledger-Schicht. Dieser gruen-Befund schliesst die Schnittstellen- und Importordnung, aber nicht automatisch die mathematische Kancellationssubstanz.
Neue r3c1-Rolle	`SM3eR-r3c1` ist ab v36 nicht mehr die alte TraceCancellationDerivation-Konsumphase. Es ist die neue AccumulatedEntryCancellation-Schicht an der SM3db4aR-/Fold-naehen Stelle. Zulaessige Kernobjekte sind insbesondere `generatedInteriorAccumulatedLeftConvolution`, `generatedInteriorAccumulatedRightConvolution`, `GeneratedInteriorAccumulatedEntryCancellationInvariant`, regular/variable Invariant-Abkuerzungen und ein r3c1-Ledger, der ausdruecklich keinen ProductIdentityProof, keinen TwoSidedInv, keine InteriorEliminationData, keine DtN-/MultiSchur- und keine Charpoly-/Diskriminantendaten erzeugt.
r3c1-full-DoD	Der starke r3c1-DoD ist erst geschlossen, wenn ein Konstruktor existiert, der einen `GeneratedInteriorAccumulatedEntryCancellationInvariant` aus `GeneratedInteriorAccumulatedTraceEvaluation`, `GeneratedInteriorAccumulatedEntryFold`, `generatedInteriorAccumulatedEntryValue T`, SM3cR-InteriorBlock-Profil und den SM3db2R/SM3db3R-Trace-/LocalStep-Daten erzeugt, ohne die sechs Kancellations-/Identity-Beweise als freie Argumente anzunehmen. Zielname: `accumulatedEntryCancellationInvariant_from_accumulatedTraceEvaluation` oder aequivalent `accumulatedEntryCancellationInvariant_from_traceFold`.
Sechs r3c1-Zielfelder	r3c1-full muss entrywise beweisen: links/rechts `generatedInteriorAccumulated*Convolution W T i j = (1 : Matrix _ _ R) i j` sowie links/rechts Diagonal- und Offdiag-Formen. Diese sechs Aussagen duerfen Felder der Invariant-Struktur sein, aber sie duerfen im finalen r3c1-full-Konstruktor nicht von aussen uebergeben werden.
Neue r3c2-Rolle	Die alte r3c1-Rolle wird zu `SM3eR-r3c2`. r3c2 konsumiert ausschliesslich einen r3c1-Witness `C : GeneratedInteriorAccumulatedEntryCancellationInvariant ...` und den SM3db7R-Handoff `H`; daraus baut es `GeneratedInteriorSM3db7RTraceCancellationDerivation`, `GeneratedInteriorSM3db7RTraceCancellationInvariant` und `GeneratedInteriorSM3db7RProductCancellationLedger`. r3c2 darf keine eigenen Kancellationsbeweise erfinden, keine sechs Felder einzeln frei annehmen und keine Matrixrechnung wiederholen.
r3c2-Gate	r3c2 ist gruen-konditional vorbereitet und hat alles, was es benoetigt, genau dann, wenn r3c1-full einen unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant` liefert. Vor diesem r3c1-full-Witness ist r3c2 nicht als unbedingter ProductCancellationLedger-Abschluss zu lesen.
Ruecksprungregel	Die fruehere bedingte Rolle `SM3db4bR AccumulatedEntryCancellationInvariant` wird durch v36 in `SM3eR-r3c1` integriert, bleibt aber inhaltlich Fold-nahe. Falls r3c1-full zeigt, dass die vorhandene SM3db4aR-Fold-Schicht die Kancellation nicht traegt, wird nicht in r3c2/r4 gepatcht. Dann wird minimal tiefer zur fehlenden Substanz zurueckgesprungen: `SM3db3aR` fuer TraceOrderAndPivotCoverageInvariant oder `SM3db2aR` fuer LocalStepSchurCancellationInvariant.
Planstatus	Verbindlich ab v37: r0–r3b closed/green; r3c0 closed/green als TraceCancellationDerivation-API; r3c1 Schnittstellen-Build closed/green, r3c1-full next/active als `blockFold = identity`-Beweis plus unbedingter AccumulatedEntryCancellation-Konstruktor; r3c2 conditional/green als Verbraucher eines r3c1-Witness; r4a conditional/allowed; r4/r5/SM3fR blocked until r3c1-full und daraus r3c2-Ledger.

0m. Fortschreibungsentscheidung v37: r3c1-full braucht blockFold = identity

Statusentscheidung	Der positive r3c1-full-Pfad ist an einer harten definitorischen Stelle praezisiert: `generatedInteriorAccumulatedEntryValue T i j` expandiert auf Diagonal-/Block-/Trace-/Fold-Beitraege, aber der aktuelle API-Stand enthaelt noch keinen Satz, der die daraus entstehende Schur-Faltung gegen `generatedInteriorBlock W` zu `1` reduziert. Der fehlende Satz ist daher nicht ein Taktikproblem, sondern die neue Pflichtinvarianz `blockFold = identity`.
Neues Pflichtobjekt	r3c1-full fuehrt eine proof-carrying Invarianzschicht ein, z.B. `GeneratedInteriorBlockFoldIdentityInvariant` oder `GeneratedInteriorAccumulatedBlockFoldIdentityInvariant`. Sie liegt zwischen der AccumulatedTraceEvaluation-/Fold-Schicht und `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Sie erzeugt noch keinen ProductIdentityProof und keinen TwoSidedInv.
Minimale Felder	Die Invarianz muss mindestens tragen: links/rechts `blockFold_eq_identity`, links/rechts Diagonalform und links/rechts Offdiag-Nullform. Diese Felder duerfen spaeter die sechs r3c1-AccumulatedEntryCancellation-Felder speisen, duerfen aber nicht als externe Argumente in r3c2 oder r4 auftauchen.
Form der linken Faltung	Links ist die normale Zielgleichung entrywise: `sum k, generatedInteriorBlock W i k * generatedInteriorAccumulatedEntryValue T k j = (1 : Matrix _ _ R) i j`. Nach Expansion von `generatedInteriorAccumulatedEntryValue` muss daraus eine `leftBlockFold`-Normalform werden, deren terminaler Wert per Invarianz `identity` ist.
Form der rechten Faltung	Rechts ist analog: `sum k, generatedInteriorAccumulatedEntryValue T i k * generatedInteriorBlock W k j = (1 : Matrix _ _ R) i j`. Die rechte Seite darf nicht aus Kommutativitaet oder Symmetrie geraten werden, solange kein entsprechender Satz im Code vorliegt; sie braucht eine eigene rechte Fold-Invarianz oder einen vorher bewiesenen Links/Rechts-Transfer.
Beweisroute I: Expansion	Zuerst muss r3c1a eine definitorische Normalform beweisen: Die Produktnormalformen aus r3a werden ueber `H.candidateMatrix = generatedInteriorAccumulatedEntryValue T` und anschliessende Expansion der Fold-/Trace-Beitraege in eine BlockFold-Summe ueberfuehrt. Keine globale `simp`-Flaeche; nur lokale `rfl`/kleine `rw`-Schritte und explizite Zwischenlemmata.
Beweisroute II: Step-Kancellation	r3c1b beweist fuer jeden lokalen SM3db2R-Step eine Schur-Kancellationsgleichung auf BlockFold-Ebene: Der Pivot-/Block-Beitrag des Steps kompensiert genau den zugehoerigen Residual-/Update-Beitrag in der Faltung. Diese Lemmas sind die eigentliche neue mathematische Substanz; ohne sie bleibt die Summe nur expandiert, aber nicht gekuerzt.
Beweisroute III: Fold-Induktion	r3c1c fuehrt eine fold-/trace-recursive Invarianz ein: Wenn die BlockFold-Invarianz vor einem Trace-/Fold-Schritt gilt und der lokale Step-Kancellationssatz gilt, gilt sie nach dem Schritt. Beweisform: strukturelle Rekursion ueber den vorhandenen Trace/Fold-Carrier bzw. ueber dessen AccumulatedEntryFold, nicht ueber einen grossen Index-Eliminator.
Beweisroute IV: Terminal-Gate	r3c1d schliesst die terminale Gleichung: Pivot-/Index-Coverage aus SM3db3R und die terminale Residual-/FoldOrder-Aussage aus SM3db4aR muessen zeigen, dass der finale BlockFold-Zustand genau `identity` ist. Erst dieses Terminal-Gate liefert links/rechts identity, diagonal=1 und offdiag=0.
Beweisroute V: Adapter	r3c1e baut daraus den unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Erst dann darf r3c2 den vorhandenen Adapter konsumieren und daraus TraceCancellationDerivation, TraceCancellationInvariant und ProductCancellationLedger erzeugen.
Ruecksprungkriterium	Falls r3c1b nicht beweisbar ist, fehlt die Substanz in SM3db2R: dann muss eine `LocalStepSchurCancellationInvariant` nachgeruestet werden. Falls r3c1c/r3c1d nicht beweisbar ist, fehlt die Substanz in SM3db3R/SM3db4aR: dann braucht es `TraceOrderAndPivotCoverageInvariant` bzw. terminale Fold-/Residual-Coverage. Es wird nicht in r3c2 oder r4 gepatcht.
Planstatus	`SM3eR-r3c1a` ist nach lokal gruenem Build geschlossen als BlockFold-Normalform-/Expose-Phase. Der naechste aktive Schritt ist nicht r3c1b-full, sondern `SM3eR-r3c1b0`: Audit/Expose der fehlenden lokalen Schur-Kancellationsquelle. Danach folgt `SM3db2aR LocalStepSchurCancellationInvariant`. Erst nach dieser Nachruestung darf `SM3eR-r3c1b-full` die lokale Schur-Step-Kancellation beweisen. r3c1c/r3c1d/r3c1e, r3c2, r4, r5 und SM3fR bleiben blockiert bzw. konditional.

0n. Fortschreibungsentscheidung v38: r3c1a gruen, r3c1b0 und SM3db2aR werden aktiv

Statusentscheidung	`SM3eR-r3c1a` ist nach lokal gruenem Build formal geschlossen. Die Phase liefert die definitorische BlockFold-Normalform und beweist die Verbindung von `generatedInteriorAccumulatedLeftConvolution`/`generatedInteriorAccumulatedRightConvolution` zu `generatedInteriorLeftBlockFold`/`generatedInteriorRightBlockFold`. Sie beweist bewusst noch keine Identitaet.
r3c1a-Ertrag	Geschlossen sind die Normalform-/Expose-Objekte: `generatedInteriorAccumulatedLeftConvolution`, `generatedInteriorAccumulatedRightConvolution`, `generatedInteriorLeftBlockFoldKernel`, `generatedInteriorRightBlockFoldKernel`, `generatedInteriorLeftBlockFold`, `generatedInteriorRightBlockFold`, `GeneratedInteriorBlockFoldNormalFormR3c1a` und regular/variable Ledger. Geschlossen sind ferner die definitorischen Gleichungen `leftAccumulatedConvolution_eq_blockFold`, `rightAccumulatedConvolution_eq_blockFold`, `leftBlockFold_eq_kernelSum`, `rightBlockFold_eq_kernelSum`, `leftKernel_eq_components`, `rightKernel_eq_components`.
Praeziser Restbefund	Der gruene r3c1a-Build zeigt die Summe, kuerzt sie aber nicht. Fuer `SM3eR-r3c1b-full` fehlt in der aktuellen SM3db2R-Schicht eine lokale Schur-Update-Struktur: Pivot-Skala, UpdateEntry-Formel, Vorzeichen-/Skalierungsbezug und lokales links/rechts-Kancellationslemma. Daher waere ein direkter r3c1b-full-Beweis im aktuellen Code nur durch freie Kancellationsfelder moeglich und damit Planbruch.
Naechste aktive Phase	**Naechste aktive Phase ist `SM3eR-r3c1b0`.** Sie ist eine kurze Audit-/Expose-Phase, keine Identity- oder Kancellationsbeweisphase. Ihr Ziel ist ein proof-carrying Befund, der exakt festhaelt, welche SM3db2R-Daten vorhanden sind und welche fuer r3c1b-full fehlen.
r3c1b0-DoD	`SM3eR-r3c1b0` muss eine Struktur wie `GeneratedInteriorLocalStepSchurCancellationSourceAudit` liefern. Positive Felder: `has_baseEntry_eq_block`, `has_rowResidual_eq_block`, `has_columnResidual_eq_block`, `has_pivotEntry_eq_block`. Fehlende Felder/Gates: `missing_pivotScale`, `missing_updatedEntry`, `missing_updatedEntry_eq_schur`, `missing_leftLocalStepCancellation`, `missing_rightLocalStepCancellation`. Der DoD ist nur die exakte Source-Diagnose, nicht die Kancellation.
SM3db2aR-Freigabe	Nach r3c1b0 wird `SM3db2aR` als positive Nachruestung freigegeben. Zielobjekt: `GeneratedInteriorLocalStepSchurCancellationInvariant`. Es muss Pivot-Skala und UpdateEntry aus vorhandenen lokalen Step-/Residual-/Blockdaten ableiten und beweisen, nicht frei setzen.
SM3db2aR-Minimalfelder	Minimal benoetigt werden: `pivotIndex`, `pivotEntry`, `pivotScale`, `pivotEntry_eq_block`, `pivotScale_mul_pivotEntry`, `pivotEntry_mul_pivotScale`, `updatedEntry`, `updatedEntry_eq_schur`, `leftLocalStepCancellation`, `rightLocalStepCancellation`. Falls `pivotScale` nicht definitorisch existiert, muss vorher ein lokaler Regularitaets-/Nichtnull-Witness aus der vorhandenen Provenienz abgeleitet werden.
Beweistaktik SM3db2aR	Keine Summeninduktion und kein globales `simp`. Zuerst werden `baseEntry`, `rowResidual`, `columnResidual` und `pivotEntry` per Strukturfeldern auf `generatedInteriorBlock W` zurueckgefuehrt. Danach wird `updatedEntry_eq_schur` mit einem kleinen `calc`-Beweis aus den lokalen Step-Feldern aufgebaut. Die lokalen Kancellationen werden entrywise durch gezieltes Umschreiben mit diesen Feldern und der Pivot-Skala bewiesen.
r3c1b-full-Gate	`SM3eR-r3c1b-full` darf erst nach gruenem `SM3db2aR`-Build starten. Dann verbindet es die r3c1a-BlockFold-Kerne mit den lokalen Step-Kancellationskernen und beweist nur die lokale Step-Kuerzung. Die Summen-/Fold-Invarianz bleibt r3c1c vorbehalten.
Fortgeschriebene Reihenfolge	Historisch ab v38: `r3c1a` closed/green; `r3c1b0` next/active; `SM3db2aR` next-after-r3c1b0; `r3c1b-full` blocked until SM3db2aR; `r3c1c` Fold-Invarianz blocked until r3c1b-full; `r3c1d` terminal identity blocked until r3c1c; `r3c1e` AccumulatedEntryCancellation-Konstruktor blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked. Diese Reihenfolge wird durch v39 geschaerft: `SM3db2aR` ist gruen, aber nur relativ zu Pivot-Regularitaetswitnessen; daher wird `SM3db2bR` vor r3c1b-full eingefuegt.

0o. Fortschreibungsentscheidung v39: SM3db2aR gruen, SM3db2bR aktiv

Statusentscheidung	`SM3db2aR` ist nach lokal gruenem Build formal geschlossen, aber nur als lokale Schur-Substanzschicht relativ zu einem Pivot-Regularitaetswitness. Die Phase liefert UpdateEntry, Schur-Update-Residual und lokale links/rechts-Kancellationsoberflaechen, erzeugt aber noch keinen unbedingten per-Pivot-Regularitaetswitness fuer den gesamten `GeneratedInteriorEliminationTrace`.
SM3db2aR-Ertrag	Geschlossen sind insbesondere `generatedInteriorLocalSchurUpdatedEntry`, `generatedInteriorLocalSchurUpdateEquationResidual`, `GeneratedInteriorLocalPivotRegularityWitnessSM3db2aR`, `GeneratedInteriorLocalStepSchurCancellationInvariantSM3db2aR` und `GeneratedInteriorTraceLocalStepSchurCancellationInvariantSM3db2aR`. Positiv getragen sind `updatedEntry_eq_schur`, `leftLocalStepCancellation`, `rightLocalStepCancellation` sowie die Rueckbindung von `baseEntry`, `rowResidual`, `columnResidual` und `pivotEntry` an `generatedInteriorBlock W`.
Praeziser Restbefund	Der Trace-Level-Konstruktor ist weiterhin relativ: er benoetigt eine Funktion `R : forall pivot, GeneratedInteriorLocalPivotRegularityWitnessSM3db2aR ... (T.localStepOf pivot)`. Damit ist `pivotScale` nicht frei in SM3db2aR gesetzt, aber die konkrete per-Pivot-Existenz ist noch nicht aus SM3cR/SM3db2R/Trace-Provenienz abgeleitet. Ein direkter Start von `SM3eR-r3c1b-full` waere daher weiterhin ein freier Regularitaets-/Inversenimport.
Naechste aktive Phase	Naechste aktive Phase ist `SM3db2bR: PerPivotRegularityFromGeneratedTrace`. Sie konstruiert fuer jeden Pivot des konkreten `GeneratedInteriorEliminationTrace` den erforderlichen `GeneratedInteriorLocalPivotRegularityWitnessSM3db2aR`. Sie erzeugt keine Summen-, BlockFold-, Produktidentitaets-, TwoSidedInv-, DtN- oder Charpoly-Daten.
SM3db2bR-DoD	Ziel ist ein unbedingter Konstruktor wie `perPivotRegularity_from_generatedTrace`. Minimal benoetigt werden: `pivotEntry_ne_zero_from_profile`, ggf. `pivotEntry_isUnit_from_profile`, `pivotScale_from_pivotEntry`, `pivotScale_mul_pivotEntry`, `pivotEntry_mul_pivotScale` und der finale per-Pivot-Konstruktor fuer `T.localStepOf pivot`. Falls die aktuelle Provenienz keinen Nichtnull-/Regularitaetsbeweis traegt, muss genau diese fehlende lokale Regularitaetsquelle exponiert werden; sie darf nicht durch `Matrix.inv`, freie Inversen oder externe Orakel ersetzt werden.
Beweistaktik SM3db2bR	Kein globales `simp`, keine Summeninduktion und kein Index-Eliminator-Monster. Pro Pivot: `let S := T.localStepOf pivot`; binde `S.stepUpdate.pivotEntry` ueber die vorhandenen Felder an den SM3cR-InteriorBlock-Diagonaleintrag; leite daraus den lokalen Nichtnull-/Unit-Witness ab; definiere `pivotScale`; schliesse die beiden Skalen-Gleichungen mit kleinen `calc`- oder Feldprojektionsbeweisen.
Anschlussphase	Nach gruenem `SM3db2bR` folgt `SM3eR-r3c1b1: BlockFoldKernel-to-LocalSchurResidualBridge`. Diese Phase beweist, dass der relevante lokale Anteil der r3c1a-BlockFold-Kerne genau auf den in SM3db2aR geschlossenen `generatedInteriorLocalSchurUpdateEquationResidual` normalisiert.
r3c1b-full-Gate	`SM3eR-r3c1b-full` bleibt bis nach `SM3db2bR` und `SM3eR-r3c1b1` gesperrt. Erst dann darf es die lokale Step-Kuerzung beweisen. Die Summen-/Fold-Invarianz bleibt weiterhin `r3c1c`, terminal identity `r3c1d` und der AccumulatedEntryCancellation-Konstruktor `r3c1e`.
Fortgeschriebene Reihenfolge	Historisch ab v39 galt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` next/active; `SM3eR-r3c1b1` blocked until SM3db2bR. v40 schaerft dies: `SM3db2bR` ist gruen, aber nur relativ zu einem proof-carrying `TracePivotScaleWitness`; daher wird `SM3db2cR` vor `SM3eR-r3c1b1` eingefuegt.

0p. Fortschreibungsentscheidung v40: SM3db2bR gruen-relativ, SM3db2cR aktiv – historisch, durch v41 geschaerft

Statusentscheidung	`SM3db2bR` ist nach gruenem Fix-Build formal geschlossen, aber nicht als unbedingte Scale-Ableitung. Die Phase ist jetzt operativ korrekt: sie verwendet keine skalare $\mathbb{R}$-Inversion, kein `Real.instInv`, kein skalares `scalar inverse`, kein `mul_inv_cancel` und kein `noncomputable`. Sie konsumiert stattdessen einen proof-carrying `GeneratedInteriorTracePivotScaleWitnessSM3db2bR`.
SM3db2bR-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorTracePivotEntryNonzeroRequirementSM3db2bR`, `GeneratedInteriorTracePivotEntryNonzeroWitnessSM3db2bR`, `GeneratedInteriorTracePivotScaleWitnessSM3db2bR`, `GeneratedInteriorPerPivotRegularityFromTraceSM3db2bR` und der Konstruktor `fromTracePivotScaleWitness`. Positiv getragen sind `pivotEntry_eq_block`, `pivotEntry_eq_dirichletDegree`, `pivotScale_mul_pivotEntry`, `pivotEntry_mul_pivotScale`, `perPivotRegularity_leftInverse`, `perPivotRegularity_rightInverse` und `traceLocalStepInvariant_eq_from_perPivotRegularity`.
Praeziser Restbefund	`SM3db2bR` beweist noch nicht, dass der benoetigte `TracePivotScaleWitness` aus generated Degree-/Weight-/Policy-Provenienz existiert. Ein blosses `pivotEntry_ne_zero` genuegt im operativen Pfad nicht, weil daraus ueber $\mathbb{R}$-Inversion kein computabler `pivotScale` konstruiert werden darf. Ein direkter Start von `SM3eR-r3c1b1` waere daher weiterhin ein freier Scale-/Regularitaetsimport.
Naechste aktive Phase	Naechste aktive Phase ist `SM3db2cR: TracePivotScaleWitnessFromGeneratedDegree`. Sie konstruiert oder exakt auditiert fuer jeden Pivot des konkreten Trace den proof-carrying Scale-Witness, den `SM3db2bR` konsumiert. Sie erzeugt keine Summen-, BlockFold-, Produktidentitaets-, TwoSidedInv-, DtN-, MultiSchur-, Charpoly- oder Diskriminantendaten.
SM3db2cR-DoD	Ziel ist ein Konstruktor wie `tracePivotScaleWitness_from_generatedDegree : GeneratedInteriorTracePivotScaleWitnessSM3db2bR ....` Minimal benoetigt werden: `pivotScale : GeneratedInteriorIndex A -> R`, `pivotScale_mul_pivotEntry`, `pivotEntry_mul_pivotScale`, `source_eq_generatedDegree`, `noScalarInvStatus`, `noMatrixInvStatus`, `noExternalScaleOracleStatus` und `nextPhase_eq_r3c1b1`. Falls die aktuelle Degree-/Weight-Schicht keinen konstruktiven Skalenwert traegt, muss genau dieses fehlende generated reciprocal/scale datum als neue lokale Quelle exponiert werden.
Beweistaktik SM3db2cR	Kein globales `simp`, keine Summeninduktion, kein Index-Eliminator-Monster. Pro Pivot wird zuerst `(T.localStepOf pivot).stepUpdate.pivotEntry` ueber die vorhandenen Felder an `generatedDirichlet_degree` bzw. die SM3cR-Diagonalprovenienz gebunden. Danach darf nur ein bereits generated/proof-carrying Scale-Datum konsumiert oder als fehlende Quelle ausgewiesen werden. Die beiden Skalen-Gleichungen werden mit kleinen `calc`- oder Feldprojektionsbeweisen geschlossen.
Verbotene Abkuerzungen	`SM3db2cR` darf weder `pivotScale := invPivotEntry` definieren noch `mul_inv_cancel`, `Real.instInv`, `noncomputable`, `Matrix.inv`, freie Inversen, freie Skalentabellen oder externe Orakel verwenden. Ein Nonzero-Witness ist nur hilfreich, aber nicht ausreichend, solange kein operativ erzeugter Scale-Witness mit beidseitigen Skalen-Gleichungen vorliegt.
Anschlussphase	Nach einem vollstaendig geschlossenen `SM3db2cR-full`, also nach einem unbedingten `TracePivotScaleWitness` aus generated Degree-/Weight-/Policy-Provenienz, sollte `SM3eR-r3c1b1: BlockFoldKernel-to-LocalSchurResidualBridge` folgen. Die v41-Codepruefung zeigt jedoch: Der aktuelle `SM3db2cR`-Stand ist nur audit-/adapter-gruen und liefert diese unbedingte ScaleSource noch nicht.
r3c1b-full-Gate	`SM3eR-r3c1b-full` bleibt bis nach einer unbedingten ScaleSource-Phase und `SM3eR-r3c1b1` gesperrt. Erst dann darf die lokale Step-Kuerzung bewiesen werden. Fold-Invarianz bleibt `r3c1c`, terminal identity `r3c1d` und der AccumulatedEntryCancellation-Konstruktor `r3c1e`.
Fortgeschriebene Reihenfolge	Historisch galt ab v40: `SM3db2cR` sollte vor `SM3eR-r3c1b1` stehen. v41 schaerft dies: `SM3db2cR` ist audit-closed/conditional, nicht full-closed als unbedingter ScaleWitness-Konstruktor; daher wird `SM3db2dR` vor `SM3eR-r3c1b1` eingefuegt.

0q. Fortschreibungsentscheidung v41: SM3db2cR audit-closed, SM3db2dR aktiv – historisch, durch v42 geschaerft

Statusentscheidung	`SM3db2cR` konnte nicht im starken DoD-Sinn geschlossen werden. Geschlossen ist nur die zulaessige Schnittstellen-/Auditfunktion: Die Phase beschreibt exakt, welche proof-carrying ScaleSource benoetigt wird, und stellt einen Adapter von dieser ScaleSource zum von `SM3db2bR` konsumierten `GeneratedInteriorTracePivotScaleWitnessSM3db2bR` bereit. Nicht geschlossen ist die unbedingte Konstruktion dieser ScaleSource aus generated Degree-/Weight-/Policy-Provenienz.
Formaler SM3db2cR-Ertrag	Zulaessig geschlossen sind insbesondere ein Requirement-Gate fuer die generated Degree-ScaleSource, eine Struktur wie `GeneratedInteriorGeneratedDegreePivotScaleSourceSM3db2cR`, ein Adapter wie `generatedInteriorTracePivotScaleWitnessFromGeneratedDegreeScaleSourceSM3db2cR`, ein Audit/Ledger fuer `noScalarInvStatus`, `noMatrixInvStatus`, `noExternalScaleOracleStatus` und die Sperre von `SM3eR-r3c1b1`. Diese Objekte erzeugen keine Summen-, BlockFold-, Produktidentitaets-, `TwoSidedInv`-, InteriorElimination-, DtN-, MultiSchur-, Charpoly- oder Diskriminantendaten.
Praeziser Restbefund	Es fehlt weiterhin ein unbedingter Satz der Form `generatedDegreePivotScaleSource_from_weightPolicy : GeneratedInteriorGeneratedDegreePivotScaleSourceSM3db2cR ...` oder direkt `tracePivotScaleWitness_from_generatedDegree : GeneratedInteriorTracePivotScaleWitnessSM3db2bR ....` Insbesondere fehlt pro Pivot ein operativ generated `pivotScale` mit beiden Gleichungen `pivotScale_mul_pivotEntry` und `pivotEntry_mul_pivotScale`, dessen Herkunft nicht aus x^{-1}, `Real.instInv`, `mul_inv_cancel`, `Matrix.inv`, einer freien Skalentabelle oder einem externen Orakel stammt.
Naechste aktive Phase	Naechste aktive Phase ist `SM3db2dR: GeneratedDegreePivotScaleSource`. Diese Phase liegt zwischen `SM3db2cR` und `SM3eR-r3c1b1`. Sie muss die bisher nur als Input angenommene generated Degree-ScaleSource wirklich aus der vorhandenen Degree-/Weight-/Policy-Provenienz bauen oder, falls das im aktuellen API-Stand unmoeglich ist, die minimal fehlende generated reciprocal/scale datum-Schicht exakt als naechste positive Rueckssprungquelle ausweisen.
SM3db2dR-DoD	Ziel ist ein Konstruktor wie `generatedDegreePivotScaleSource_from_weightPolicy` oder aequivalent `tracePivotScaleWitness_from_generatedDegree`. Minimalfelder: `pivotScale`, `pivotScale_mul_pivotEntry`, `pivotEntry_mul_pivotScale`, `pivotEntry_eq_dirichletDegree`, `source_eq_generatedDegree`, `source_eq_weightPolicy`, `noScalarInvStatus`, `noMatrixInvStatus`, `noExternalScaleOracleStatus`, `noFreeScaleTableStatus` und `nextPhase_eq_r3c1b1`. Der finale Konstruktor darf keine freien Kancellationsbeweise, keine freie Matrix, keine freie Entry-Tabelle und keine externe Inversenannahme als Argument erhalten.
Beweistaktik SM3db2dR	Kein globales `simp`, keine Summeninduktion und kein grosser Index-Eliminator. Pro Pivot wird zuerst `(T.localStepOf pivot).stepUpdate.pivotEntry` ueber SM3db2bR/SM3cR an den generated Dirichlet-Degree bzw. die Diagonalprovenienz gebunden. Danach wird ausschliesslich ein bereits im generated Weight-/Policy-System getragenes Scale-/Reciprocal-Datum konsumiert. Die beiden Skalen-Gleichungen werden durch Feldprojektionen oder kleine `calc`-Beweise geschlossen. Wenn kein solches Datum existiert, endet `SM3db2dR` nicht mit r3c1b1-Freigabe, sondern mit einem proof-carrying Missing-Source-Audit fuer eine weitere minimale Nachruestphase.
Verbotene Abkuerzungen	`SM3db2dR` darf nicht `pivotScale := invPivotEntry`, `pivotEntry inv`, `mul_inv_cancel`, `Real.instInv`, `noncomputable`, `Matrix.inv`, `field_simp`, freie Skalentabellen, freie Inversen oder externe Orakel verwenden. Ein Nichtnull-Zeugnis ist nur ein Regularitaetsindikator, aber keine operative ScaleSource.
Freigaberegel	`SM3eR-r3c1b1` wird erst nach einem gruenen `SM3db2dR`-Build freigegeben, der eine unbedingte `GeneratedInteriorGeneratedDegreePivotScaleSourceSM3db2cR` bzw. einen unbedingten `GeneratedInteriorTracePivotScaleWitnessSM3db2bR` aus generated Provenienz liefert. Bis dahin bleiben `SM3eR-r3c1b1`, `r3c1b-full`, `r3c1c`, `r3c1d`, `r3c1e`, der unbedingte `r3c2`-Ledger, r4, r5 und `SM3fR` blockiert.
Fortgeschriebene Reihenfolge	Ab v41 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer und Requirement-Gate; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter und Missing-Source-Gate; `SM3db2dR` next/active als unbedingter GeneratedDegreePivotScaleSource-Provider; `SM3eR-r3c1b1` blocked until SM3db2dR; `r3c1b-full` blocked until r3c1b1; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0r. Fortschreibungsentscheidung v42: SM3db2dR gruen-relativ, SM3db2eR aktiv

Statusentscheidung	`SM3db2dR` ist nach lokal gruenem Build formal geschlossen als reciprocal-profile/adapter-Schicht, aber nicht full-closed als unbedingter generated ScaleSource-Provider. Der gruene Build zeigt: `GeneratedDegreePivotReciprocalProfileSM3db2dR` kann, falls es vorliegt, in eine `GeneratedInteriorGeneratedDegreePivotScaleSourceSM3db2dR`, dann in den `SM3db2cR`-ScaleSource und schliesslich in den `SM3db2bR`-TracePivotScaleWitness ueberfuehrt werden. Der Build zeigt nicht, dass dieses ReciprocalProfile bereits aus der vorhandenen Degree-/Weight-/Policy-Schicht konstruiert ist.
Formaler SM3db2dR-Ertrag	Geschlossen sind insbesondere `GeneratedDegreePivotReciprocalProfileSM3db2dR`, `generatedDegreePivotScaleSourceFromReciprocalProfileSM3db2dR`, `GeneratedInteriorGeneratedDegreePivotScaleSourceSM3db2dR`, `tracePivotScaleWitness_from_generatedDegreePivotScaleSourceSM3db2dR`, `tracePivotScaleWitnessFromGeneratedDegree_from_sm3db2dR`, `GeneratedDegreePivotScaleSourceAuditSM3db2dR`, Regular-/Variable-Abkuerzungen sowie die Statusfelder gegen ScalarInv, MatrixInv, externe Scale-Orakel, ProductIdentityProof, TwoSidedInv, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
Praeziser Restbefund	Der kritische fehlende Pfeil ist `generated Degree-/Weight-/Policy-Provenienz -> GeneratedDegreePivotReciprocalProfileSM3db2dR`. Im aktuellen Code wird ein Profil durch `fromGeneratedDegreeReciprocalProfile` bzw. `fromReciprocalProfile` aus einer Funktion `generatedDegreePivotScale` und den beiden Gleichungen gegen `generatedDirichlet_degree` gebaut. Solange diese Funktion und die beiden Gleichungen nicht aus der vorhandenen generated Provenienz hergeleitet werden, bleibt das Profil proof-carrying Input und damit noch kein vollstaendig derived-only ScaleSource-Abschluss.
Naechste aktive Phase	Naechste aktive Phase ist `SM3db2eR: GeneratedDegreePivotReciprocalProfileFromWeightPolicy`. Die unmittelbare Startform ist `SM3db2eR0: GeneratedDegreeFormulaAudit`, sofern der konkrete Pivotwert von `generatedDirichlet_degree W` im aktuellen API-Stand nicht schon als operational verwertbare Formel exponiert ist. Ziel von `SM3db2eR0` ist kein Skalenbeweis, sondern die exakte Klassifikation: konstante Degree-Formel, Nat-Cast-Formel, WeightPolicy-abhaengige Formel, vorhandenes generated reciprocal datum oder nachweislich fehlende reciprocal-source-Schicht.
SM3db2eR-DoD	Der starke DoD ist ein Konstruktor wie `generatedDegreePivotReciprocalProfile_from_weightPolicy : GeneratedDegreePivotReciprocalProfileSM3db2dR ...` bzw. Regular-/Variable-Spezialisierungen. Minimal muessen erzeugt werden: `generatedDegreePivotScale`, `generatedDegreePivotScale_mul_generatedDegree`, `generatedDegree_mul_generatedDegreePivotScale`, `source_eq_generatedDegree`, Herkunft aus Degree-/Weight-/Policy-Provenienz und die No-Inv-/No-Oracle-Statusfelder. Der finale Konstruktor darf keine freie Skalentabelle, keine freien Kancellationsbeweise, keine externe Inversenannahme und keine freie Matrix als Argument konsumieren.
Beweistaktik SM3db2eR	Zuerst wird der Pivot-Degree-Ausdruck per SM3db2cR/SM3db2dR-Requirement und SM3cR-Diagonalprovenienz auf `generatedDirichlet_degree W` normalisiert. Danach wird nicht aus `pivotEntry_ne_zero` durch $\mathbb{R}$-Inversion konstruiert, sondern ausschliesslich ein operationales reciprocal/scale datum aus WeightPolicy, Degree-Count oder einer bereits proof-carrying generated Quelle konsumiert. Falls die Degree-Formel definitorisch eine Einheit ist, darf `1` als Skala nur mit expliziten linken/rechten Gleichungen benutzt werden; falls sie ein nichttrivialer Nat-Cast oder Real-Wert ist, reicht Positivitaet/Nichtnull nicht.

Verbotene Abkuerzungen	`SM3db2eR` und `SM3db2eR0` duerfen nicht `pivotScale := invPivotEntry`, `pivotEntry inv`, `mul_inv_cancel`, `Real.instInv`, `field_simp`, `noncomputable`, `Matrix.inv`, freie Inversen, freie Skalentabellen, klassische Hilfen oder externe Orakel verwenden. Ein Nonzero-Witness bleibt nur ein Regularitaetsindikator, kein operativer ReciprocalProfile-Konstruktor.
Freigaberegel	`SM3eR-r3c1b1` wird erst freigegeben, wenn nach `SM3db2eR` ein unbedingtes Regular- und Variable-Profil vom Typ `GeneratedDegreePivotReciprocalProfileSM3db2dR` aus generated Provenienz vorliegt und daraus der `SM3db2dR`-ScaleSource und der `SM3db2bR`-TracePivotScaleWitness ohne freie Skalenquelle erzeugt werden. Bis dahin bleiben `SM3eR-r3c1b1`, `r3c1b-full`, `r3c1c`, `r3c1d`, `r3c1e`, der unbedingte `r3c2`-Ledger, r4, r5 und `SM3fR` blockiert.
Ruecksprungregel	Falls `SM3db2eR0` zeigt, dass `generatedDirichlet_degree` derzeit nur $\mathbb{R}$-wertig, aber ohne operationales reciprocal datum exponiert ist, wird nicht in r3c1b1, r3c1b-full oder r4 gepatcht. Dann folgt eine minimale positive Nachruestung direkt an der Degree-/WeightPolicy-Schicht, z.B. `SM3db2fR: GeneratedOperationalDegreeReciprocalSource`. Das ist kein `SM3dbOmegaR`-Start, solange die positive Handoff-Kette nicht mathematisch gescheitert ist.
Fortgeschriebene Reihenfolge	Ab v42 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer und Requirement-Gate; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0/SM3db2eR` next/active fuer DegreeFormulaAudit und ReciprocalProfileFromWeightPolicy; `SM3eR-r3c1b1` blocked until SM3db2eR-full; `r3c1b-full` blocked until r3c1b1; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0s. Fortschreibungsentscheidung v43: SM3db2eR0 gruen, SM3db2eR-full aktiv

Statusentscheidung	`SM3db2eR0` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die geplante Startrolle von `SM3db2eR` erfuellt: Sie auditiert die konkrete Formel von `generatedDirichlet_degree` und bindet den Trace-Pivot-Eintrag an diese Formel zurueck. Damit ist nicht mehr `SM3db2eR0`, sondern `SM3db2eR-full: GeneratedDegreePivotReciprocalProfileFromWeightPolicy` die nun aktive Phase.
Formaler SM3db2eR0-Ertrag	Geschlossen sind insbesondere `SM3db2eR0GeneratedDegreeFormulaStatus`, `SM3db2eR0WeightPolicyFormulaStatus`, `SM3db2eR0OperationalReciprocalSourceStatus`, `generatedDirichlet_degree_eq_entryWeight_sum_SM3db2eR0`, `generatedDirichlet_degree_eq_generatedAdjacencyWeight_sum_SM3db2eR0`, `generatedDirichlet_degree_eq_inputPolicyAdjacencyWeight_sum_SM3db2eR0`, `GeneratedDegreeFormulaAuditSM3db2eR0`, `GeneratedDegreeFormulaAuditSM3db2eR0.fromRequirement`, `GeneratedDegreeFormulaAuditSM3db2eR0.fromSM3db2dAudit`, `GeneratedDegreeFormulaAuditSM3db2eR0.pivotEntry_formula_is_-inputPolicyAdjacencyWeight_sum`, `operationalReciprocalSource_not_exposed`, `nextPhase_is_SM3db2eR`, Regular-/Variable-Abkuerzungen und `SM3db2eR0GeneratedDegreeFormulaAuditLedger`.
Formelbefund	Der Degree-Pivotwert ist jetzt legal exponiert durch drei theorematische Gleichungen: `generatedDirichlet_degree_eq_entryWeight_sum_SM3db2eR0`, `generatedDirichlet_degree_eq_generatedAdjacencyWeight_sum_SM3db2eR0` und `generatedDirichlet_degree_eq_inputPolicyAdjacencyWeight_sum_SM3db2eR0`. Inhaltlich ist `generatedDirichlet_degree W i` eine endliche Summe ueber `generatedEntryWeight W i j`; nach dem Oeffnen von `generatedEntryWeight` ist es eine endliche WeightPolicy-abhaengige Adjazenzsumme mit `wp.constantWeight i j` auf `generatedAdjacent i j` und `0` sonst. Die Trace-Pivotform ist daraus ableitbar ueber `pivotEntry_eq_generatedDegree`, `pivotEntry_eq_entryWeight_sum` und `pivotEntry_eq_inputPolicyAdjacencyWeight_sum`.
Praeziser Restbefund	`SM3db2eR0` erzeugt kein `GeneratedDegreePivotReciprocalProfileSM3db2dR`. Insbesondere traegt der Audit den Status `noOperationalReciprocalSourceExposedByCurrentGeneratedDegreeAPI`. Ebenso wird keine konstante Unit-Formel und keine Nat-Cast-Degree-Formel exponiert. Damit ist Positivitaet oder Nichtnullheit des Degree-Werts weiterhin nicht ausreichend, um im operationalen Pfad eine Skala zu konstruieren.
Nun aktive Phase	Aktiv ist `SM3db2eR-full: GeneratedDegreePivotReciprocalProfileFromWeightPolicy`. Der naechste Code-Schritt versucht zuerst den positiven Pfad: Aus der exponierten WeightPolicy-Summenform wird ein proof-carrying `GeneratedDegreePivotReciprocalProfileSM3db2dR` fuer regular und variable Seite konstruiert. Erst wenn dieser Versuch mathematisch/API-seitig scheitert, wird die fehlende Quelle als eigene minimale Nachruestphase ausgewiesen.

SM3db2eR-full-DoD	Der starke DoD bleibt ein Konstruktor wie `generatedDegreePivotReciprocalProfile_from_weightPolicy : GeneratedDegreePivotReciprocalProfileSM3db2dR ...` sowie Regular-/Variable-Spezialisierungen. Minimal muessen ohne freie Argumente entstehen: `generatedDegreePivotScale`, `generatedDegreePivotScale_mul_generatedDegree`, `generatedDegree_mul_generatedDegreePivotScale`, `source_eq_generatedDegree`, Herkunft aus Degree-/Weight-/Policy-Provenienz, `noScalarInvStatus`, `noMatrixInvStatus`, `noExternalScaleOracleStatus` und `noFreeScaleTableStatus`.
Beweisroute SM3db2eR-full	Zuerst wird ein `GeneratedDegreeFormulaAuditSM3db2eR0` konsumiert, nicht neu nachgebaut. Danach werden drei positive Faelle strikt getrennt geprueft: erstens eine wirklich definitorische Unit-Degree-Formel, falls sie fuer die betrachtete generated Quelle beweisbar ist; zweitens ein operationales Degree-Count-/WeightPolicy-Reziprokdatum, falls die WeightPolicy-Schicht es bereits proof-carrying liefert; drittens eine schon vorhandene generated reciprocal source. Keiner dieser Faelle darf durch skalare Ring-Inversion ersetzt werden.
Verbotene Abkuerzungen	`SM3db2eR-full` darf nicht `pivotScale := invPivotEntry`, `pivotEntry inv`, `mul_inv_cancel`, `Real.instInv`, `field_simp`, `noncomputable`, `Matrix.inv`, freie Inversen, freie Skalentabellen, klassische Hilfen, externe Orakel oder Kancellations-/Produktbeweise als Input benutzen. Ein `pivotEntry_ne_zero`- oder Positivitaetszeugnis darf hoechstens als Diagnose erscheinen, nicht als operativer Scale-Konstruktor.
Freigaberegel	`SM3eR-r3c1b1` wird erst freigegeben, wenn `SM3db2eR-full` ein unbedingtes Regular- und Variable-Profil vom Typ `GeneratedDegreePivotReciprocalProfileSM3db2dR` aus generated Provenienz liefert und dieses Profil ueber `SM3db2dR` zum ScaleSource und ueber `SM3db2bR` zum TracePivotScaleWitness weitergereicht werden kann. Bis dahin bleiben `SM3eR-r3c1b1`, `r3c1b-full`, `r3c1c`, `r3c1d`, `r3c1e`, der unbedingte `r3c2`-Ledger, r4, r5 und `SM3fR` blockiert.
Ruecksprungregel	Falls `SM3db2eR-full` zeigt, dass die exponierte WeightPolicy-Summe im aktuellen API-Stand kein operationales reciprocal datum traegt, wird nicht in r3c1b1, r3c1b-full oder r4 gepatcht. Dann ist die naechste positive Nachruestung `SM3db2fR: GeneratedOperationalDegreeReciprocalSource`. Diese Phase muesste das fehlende Reziprokdatum direkt an der Degree-/WeightPolicy-Schicht theorematisch tragen, weiterhin ohne skalare Inversion und ohne freie Skalentabelle.
Fortgeschriebene Reihenfolge	Ab v43 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer und Requirement-Gate; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` **active** als GeneratedDegreePivotReciprocalProfileFromWeightPolicy; `SM3eR-r3c1b1` blocked until SM3db2eR-full; `r3c1b-full` blocked until r3c1b1; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0t. Fortschreibungsentscheidung v44: SM3db2eR-full gruen, SM3db2fR aktiv

Statusentscheidung	`SM3db2eR-full` ist nach lokal gruenem Build formal geschlossen. Die Phase konsumiert den vorhandenen `GeneratedDegreeFormulaAuditSM3db2eR0` und baut daraus keine freie Skala, sondern eine proof-carrying Entscheidungsstruktur `GeneratedDegreePivotReciprocalProfileFromWeightPolicySM3db2eR`. Diese Struktur ist nicht der gesuchte unbedingte `GeneratedDegreePivotReciprocalProfileSM3db2dR`-Provider; sie dokumentiert den positiven Versuch und das aktuelle API-Gate. Der lokale Build ist gruen bestaetigt; die Planlogik fuer die naechste Phase ist damit nicht mehr konditional am Buildstatus, sondern verbindlich fortgeschrieben.
Formaler SM3db2eR-full-Ertrag	Geschlossen sind insbesondere `SM3db2eRUnitDegreeCaseStatus`, `SM3db2eRWeightPolicyReciprocalDatumStatus`, `SM3db2eRGeneratedReciprocalSourceStatus`, `SM3db2eRProfileConstructionStatus`, `GeneratedDegreePivotReciprocalProfileFromWeightPolicySM3db2eR`, `GeneratedDegreePivotReciprocalProfileFromWeightPolicySM3db2eR.fromFormulaAudit`, `degree_formula_from_weightPolicy_sum`, `pivotEntry_formula_from_weightPolicy_sum`, `profileConstructionStatus_notConstructed`, `nextPhase_is_SM3db2fR`, Regular-/Variable-Abkuerzungen und `SM3db2eRGeneratedDegreePivotReciprocalProfileFromWeightPolicyLedger`.
Praeziser Befund	Der SM3db2eR-full-Pfad traegt die Degree-Formel aus SM3db2eR0 weiter: `generatedDirichlet_degree W i` ist auf die endliche WeightPolicy-Adjazenzsumme zurueckgebunden, und `(T.localStepOf pivot).stepUpdate.pivotEntry` ist an denselben generated Degree bzw. dieselbe WeightPolicy-Summe gebunden. Zugleich ist der eigentliche Profil-Pfeil nicht geschlossen: `profileConstructionStatus = generatedDegreePivotReciprocalProfileNotConstructedFromCurrentWeightPolicySumAPI`.
Negativ-scharfer API-Befund	Die aktuelle Degree-/WeightPolicy-API exponiert weiterhin keine definitorische Unit-Degree-Formel, kein operationales WeightPolicy-Reziprokdatum und keine bereits vorhandene generated reciprocal source. Deshalb darf aus Nonzero-/Positivitaetsinformation kein operationaler Scale-Witness gebaut werden. Insbesondere darf der fehlende Pfeil nicht durch `pivotScale := invPivotEntry`, `Real.instInv`, `mul_inv_cancel`, `field_simp`, `Matrix.inv`, `noncomputable`, freie Skalentabelle, klassische Hilfen oder ein externes Oracle ersetzt werden.
Naechste aktive Phase	**Aktiv ist nun `SM3db2fR`: `GeneratedOperationalDegreeReciprocalSource`.** Diese Phase liegt direkt an der Degree-/WeightPolicy-Schicht. Sie darf nicht in SM3eR oder r3c1b1 patchen, sondern muss das fehlende operationale Reziprokdatum als eigene generated/proof-carrying Quelle bereitstellen.
SM3db2fR-Ziel	Ziel ist eine Struktur oder ein aequivalenter Konstruktor wie `GeneratedOperationalDegreeReciprocalSourceSM3db2fR` bzw. `generatedOperationalDegreeReciprocalSource_from_weightPolicy`. Minimal benoetigt werden ein generated Scale-/Reciprocal-Datum fuer den Degree-Pivotwert, beide Skalen-Gleichungen gegen `generatedDirichlet_degree W i`, Rueckbindung an die WeightPolicy-Degree-Formel, Regular-/Variable-Spezialisierungen und Statusfelder gegen ScalarInv, MatrixInv, externe Scale-Orakel und freie Skalentabellen.
SM3db2fR-Beweisroute	Zuerst ist die vorhandene SM3db2eR0/SM3db2eR-Summenform zu konsumieren; danach muss die Reziprok-Quelle aus einer echten operationalen Degree-/WeightPolicy-Schicht kommen. Falls die Degree-Summe in Spezialfaellen definitorisch `1` ist, darf die Skala `1` nur mit expliziten linken/rechten Gleichungen eingefuehrt werden. Falls der Degree-Wert nichttrivial ist, muss die konkrete Reziprokquelle selbst proof-carrying vorliegen; ein Nichtnull- oder Positivitaetslemma genuegt nicht.
Sperren	`SM3eR-r3c1b1` bleibt blockiert, bis `SM3db2fR` ein operationales Degree-Reziprokdatum liefert und daraus ueber `SM3db2eR/SM3db2dR/SM3db2cR/SM3db2bR` ein unbedingter TracePivotScaleWitness erzeugt werden kann. Damit bleiben auch `r3c1b-full`, `r3c1c`, `r3c1d`, `r3c1e`, der unbedingte `r3c2`-Ledger, r4, r5, `SM3fR`, DtN-/GeneralizedDtN-/MultiSchur-Daten und Charpoly-/Diskriminantendaten blockiert.
Fortgeschriebene Reihenfolge	Ab v44 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` closed/green als ReciprocalProfileFromWeightPolicy-Audit mit negativem API-Befund; **`SM3db2fR` active** als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` blocked until SM3db2fR; `r3c1b-full` blocked until r3c1b1; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0u. Fortschreibungsentscheidung v45: SM3db2fR gruen, SM3eR-r3c1b1 aktiv

Statusentscheidung	`SM3db2fR` ist nach lokal gruenem Build formal geschlossen. Die Phase liefert die verlangte proof-carrying Source-/Export-Schicht fuer das operationale Degree-/WeightPolicy-Reziprokdatum. Sie konsumiert den gruenen `SM3db2eR`-Befund, traegt die `generatedDirichlet_degree`-Formel und die PivotEntry-Rueckbindung weiter und exportiert ueber `SM3db2dR/SM3db2cR/SM3db2bR` einen `GeneratedInteriorTracePivotScaleWitnessSM3db2bR`. Der lokale Green-Build loest damit genau die bisherige ScaleWitness-Sperre vor `SM3eR-r3c1b1`.
Formaler SM3db2fR-Ertrag	Geschlossen sind insbesondere `SM3db2fROperationalDegreeReciprocalDatumStatus`, `SM3db2fRWeightPolicyDegreeFormulaStatus`, `SM3db2fRReciprocalProfileStatus`, `SM3db2fRTracePivotScaleWitnessStatus`, `GeneratedOperationalDegreeReciprocalDatumSM3db2fR`, `GeneratedOperationalDegreeReciprocalSourceSM3db2fR`, `generatedDegreePivotReciprocalProfileFromOperationalDatumSM3db2fR`, `generatedOperationalDegreeReciprocalSource_from_weightPolicy`, `tracePivotScaleWitness_from_generatedOperationalDegreeReciprocalSourceSM3db2fR`, `leftScaleEquation`, `rightScaleEquation`, `nextPhase_is_r3c1b1`, Regular-/Variable-Abkuerzungen und `SM3db2fRGeneratedOperationalDegreeReciprocalSourceLedger`.
Praeziser Befund	`SM3db2fR` schliesst die Reziprok-/ScaleWitness-Luecke, aber es ist kein Produkt-Gate. Der neue Witness ist nur die Berechtigung, die r3c1a-BlockFold-Kerne gegen die lokalen SM3db2aR-Schur-Residualdaten zu normalisieren. `SM3db2fR` beweist noch keine BlockFold-Kernel-zu-Residual-Gleichung, keine lokale Step-Kuerzung auf Summenebene, keine Fold-Invarianz, keine terminale Identity und keinen `GeneratedInteriorAccumulatedEntryCancellationInvariant`.
Verbotene Folgerung	Aus dem gruenen `SM3db2fR`-Build darf nicht gefolgert werden, dass `r3c1b-full`, `r3c1c`, `r3c1d`, `r3c1e`, der unbedingte `r3c2`-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Ebenfalls verboten bleiben neue Skalenargumente in `r3c1b1`, `pivotScale := invPivotEntry`, `Real.instInv`, `mul_inv_cancel`, `field_simp`, `Matrix.inv`, `noncomputable`, freie Skalentabellen, klassische Hilfen, externe Orakel und jede Produktidentitaetsannahme.
Naechste aktive Phase	**Aktiv ist nun `SM3eR-r3c1b1: BlockFoldKernel-to-LocalSchurResidualBridge`.** Die Phase muss die r3c1a-BlockFold-Kerne mit den SM3db2aR-LocalStepSchurResidual-/Cancellation-Daten verbinden und dabei den ueber `SM3db2fR` exportierten TracePivotScaleWitness als einzige ScaleWitness-Quelle verwenden.
r3c1b1-DoD	Zulaessig ist eine Struktur oder ein aequivalenter Konstruktor wie `GeneratedInteriorBlockFoldKernelLocalSchurResidualBridgeR3c1b1`. Minimalfelder: linke und rechte Kernel-zu-LocalSchurResidual-Gleichung, Rueckbindung an `generatedInteriorLeftBlockFoldKernel/generatedInteriorRightBlockFoldKernel` aus r3c1a, Rueckbindung an `GeneratedInteriorTraceLocalStepSchurCancellationInvariantSM3db2aR`, Konsum von `tracePivotScaleWitness_from_generatedOperationalDegreeReciprocalSourceSM3db2fR`, Regular-/Variable-Spezialisierungen und No-Output-Statusfelder gegen ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
r3c1b1-Beweisroute	Zuerst wird die r3c1a-Normalform lokal geoeffnet: Kernel-Eintraege werden nicht neu definiert, sondern auf die vorhandenen `generatedInteriorLeftBlockFoldKernel/generatedInteriorRightBlockFoldKernel`-Komponenten zurueckgebunden. Danach wird pro Pivot ausschliesslich die SM3db2aR-Schur-Residualstruktur konsumiert; die per-Pivot-Regularitaet kommt ueber den `SM3db2fR`-TracePivotScaleWitness. Die Phase darf keine Fold-Induktion und kein terminales Coverage-/Identity-Gate beweisen; das bleibt `r3c1c` bzw. `r3c1d`.
r3c1b-full-Gate	`SM3eR-r3c1b-full` bleibt bis zum gruenen `SM3eR-r3c1b1`-Build gesperrt. Erst wenn r3c1b1 die lokale Kernel-zu-SchurResidual-Bruecke theorematisch liefert, darf r3c1b-full die lokale Step-Kuerzung als eigentliche Kancellationsschicht beweisen.
Fortgeschriebene Reihenfolge	Ab v45 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` closed/green als ReciprocalProfileFromWeightPolicy-Audit; `SM3db2fR` closed/green als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` **active** als BlockFoldKernel-to-LocalSchurResidualBridge; `r3c1b-full` blocked until r3c1b1; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0v. Fortschreibungsentscheidung v46: SM3eR-r3c1b1 gruen, SM3eR-r3c1b-full aktiv

Statusentscheidung	`SM3eR-r3c1b1` ist nach lokal gruenem Build formal geschlossen. Die Phase liefert die geforderte `BlockFoldKernel-to-LocalSchurResidualBridge`: Sie konsumiert die r3c1a-BlockFold-Normalform, die `SM3db2fR`-operationale Degree-Reziprokquelle, den daraus exportierten `GeneratedInteriorTracePivotScaleWitnessSM3db2bR`, die daraus gebaute Per-Pivot-Regularitaet und die `GeneratedInteriorTraceLocalStepSchurCancellationInvariantSM3db2aR`. Der lokale Green-Build loest damit genau die bisherige Kernel-zu-LocalSchurResidual-Sperre vor `r3c1b-full`.
Formaler r3c1b1-Ertrag	Geschlossen sind insbesondere `SM3eRr3c1b1BridgeStatus`, `SM3eRr3c1b1KernelSourceStatus`, `SM3eRr3c1b1LocalSchurResidualSourceStatus`, `SM3eRr3c1b1NoNewScaleStatus`, `SM3eRr3c1b1NoNewReciprocalStatus`, `SM3eRr3c1b1NoProductIdentityProofStatus`, `SM3eRr3c1b1NoTwoSidedInvStatus`, `SM3eRr3c1b1NoInteriorEliminationDataStatus`, `SM3eRr3c1b1NoDtnDataStatus`, `SM3eRr3c1b1NoArithmeticTargetStatus`, `SM3eRr3c1b1NextPhaseStatus`, `generatedInteriorLeftBlockFoldKernelLocalSchurResidualBridgeR3c1b1`, `generatedInteriorRightBlockFoldKernelLocalSchurResidualBridgeR3c1b1`, `GeneratedInteriorBlockFoldKernelLocalSchurResidualBridgeR3c1b1`, `fromOperationalDegreeReciprocalSource`, `leftBridge_eq_kernel`, `rightBridge_eq_kernel`, `leftLocalSchurResidual_eq_zero`, `rightLocalSchurResidual_eq_zero`, `nextPhase_is_r3c1b_full`, Regular-/Variable-Abkuerzungen und `SM3eRr3c1b1BlockFoldKernelLocalSchurResidualBridgeLedger`.
Praeziser Befund	`r3c1b1` beweist keine globale Produktidentitaet. Der Ertrag ist lokal: Die Brueckenfunktionen sind Kernel plus lokaler Schur-Residualterm; der Residualterm wird ueber die SM3db2aR-Invarianz zu `0`, sodass die Bruecke wieder auf den linken bzw. rechten r3c1a-Kernel zurueckfaellt. Damit ist der Anschluss der lokalen Schur-Residualsubstanz an die BlockFold-Kerne geschlossen, aber die eigentliche lokale Step-Kuerzung als Kancellationsschicht steht noch aus.
Verbotene Folgerung	Aus dem gruenen `r3c1b1`-Build darf nicht gefolgert werden, dass `r3c1c`, `r3c1d`, `r3c1e`, ein unbedingter `r3c2`-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Ebenfalls verboten bleiben neue Skalenargumente, neue Reziprokwerte, `pivotScale := invPivotEntry`, `Real.instInv`, `mul_inv_cancel`, `field_simp`, `Matrix.inv`, `noncomputable`, freie Skalentabellen, freie Kancellationsfelder, klassische Hilfen, externe Orakel und jede Produktidentitaetsannahme.
Naechste aktive Phase	**Aktiv ist nun `SM3eR-r3c1b-full: LocalStepCancellation`.** Diese Phase muss aus der r3c1b1-Bruecke und den SM3db2aR-Schur-Kancellationsdaten die lokale Step-Kuerzung theorematisch beweisen. Sie darf noch keine Fold-/Trace-Induktion ueber die ganze Summe beweisen; das bleibt `r3c1c`. Sie darf auch kein terminales Coverage-/Identity-Gate schliessen; das bleibt `r3c1d`.

0v. Fortschreibungsentscheidung v46 – Fortsetzung: r3c1b-full-Gate

r3c1b-full-DoD	Ziel ist eine Struktur oder ein aequivalenter Konstruktor wie `GeneratedInteriorLocalStepCancellationR3c1bFull` bzw. ein entsprechender r3c1b-full-Ledger. Minimalfelder: linke und rechte lokale Kernel-/Residual-Kuerzung, Rueckbindung an `GeneratedInteriorBlockFoldKernelLocalSchurResidualBridgeR3c1b1`, Konsum der `leftLocalSchurResidual_eq_zero`/`rightLocalSchurResidual_eq_zero`-Aussagen, Regular-/Variable-Spezialisierungen und No-Output-Statusfelder gegen ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
r3c1b-full-Beweisroute	Zuerst wird ein `r3c1b1`-Bridge-Witness konsumiert, nicht neu konstruiert. Danach wird die lokale Kuerzung entrywise mit den bereits bewiesenen linken/rechten Residual-Nullaussagen und den Bridge-eq-Kernel-Lemmata aufgebaut. Keine Summeninduktion, kein globales `simp`, kein grosses Index-Eliminator-Muster: r3c1b-full ist nur die lokale Kancellationsschicht zwischen Kernel-Bruecke und spaeterer Fold-Invarianz.
Downstream-Gates	`r3c1c` bleibt bis zum gruenen `r3c1b-full`-Build gesperrt. `r3c1d` bleibt bis zum gruenen `r3c1c`-Build gesperrt. `r3c1e` bleibt bis zum gruenen `r3c1d`-Build gesperrt. `r3c2` bleibt nur konditionaler Verbraucher eines spaeteren r3c1e-Witness; r4/r5/`SM3fR` bleiben blockiert.
Fortgeschriebene Reihenfolge	Ab v46 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` closed/green als ReciprocalProfileFromWeightPolicy-Audit; `SM3db2fR` closed/green als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` closed/green als BlockFoldKernel-to-LocalSchurResidualBridge; `SM3eR-r3c1b-full` **active** als lokale Step-Kuerzung; `r3c1c` blocked until r3c1b-full; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0w. Fortschreibungsentscheidung v47: SM3eR-r3c1b-full gruen, SM3eR-r3c1c aktiv

Statusentscheidung	`SM3eR-r3c1b-full` ist nach lokal gruenem Build formal geschlossen. Die Phase liefert die geforderte lokale `LocalStepCancellation`: Sie konsumiert die r3c1b1-Bruecke `GeneratedInteriorBlockFoldKernelLocalSchurResidualBridgeR3c1b1`, bindet deren linke/rechte Kernel-Bridge an die SM3db2aR-Residual-Nullaussagen zurueck und beweist die lokalen linken/rechten Step-Kuerzungen. Der lokale Green-Build loest damit genau die Kancellationssperre zwischen Kernel-zu-Residual-Bruecke und spaeterer Fold-Induktion.
Formaler r3c1b-full-Ertrag	Geschlossen sind insbesondere `SM3eRr3c1bFullLocalStepCancellationStatus`, `SM3eRr3c1bFullBridgeSourceStatus`, `SM3eRr3c1bFullLocalResidualSourceStatus`, `SM3eRr3c1bFullNoFoldInductionStatus`, `SM3eRr3c1bFullNoTerminalIdentityStatus`, `SM3eRr3c1bFullNoProductIdentityProofStatus`, `SM3eRr3c1bFullNoTwoSidedInvStatus`, `SM3eRr3c1bFullNoInteriorEliminationDataStatus`, `SM3eRr3c1bFullNoDtnDataStatus`, `SM3eRr3c1bFullNoArithmeticTargetStatus`, `SM3eRr3c1bFullNextPhaseStatus`, `GeneratedInteriorLocalStepCancellationR3c1bFull`, `fromBridge`, `fromOperationalDegreeReciprocalSource`, `leftLocalSchurResidual_eq_zero`, `rightLocalSchurResidual_eq_zero`, `leftBridge_eq_kernel`, `rightBridge_eq_kernel`, `leftLocalStepCancellation`, `rightLocalStepCancellation`, `nextPhase_is_r3c1c`, Regular-/Variable-Abkuerzungen und `SM3eRr3c1bFullLocalStepCancellationLedger`.
Praeziser Befund	r3c1b-full ist weiterhin lokal. Die Phase beweist nicht, dass die gesamte BlockFold-Summe invariant ist, sondern nur, dass jeder lokale r3c1b1-Kernel-/Residual-Anschluss gegen den jeweiligen r3c1a-Kernel gekuerzt werden darf. Damit ist die lokale Kuerzungsnutzlast vorhanden; die globale Weitergabe dieser lokalen Kuerzung durch die Trace-/Fold-Struktur ist eine eigene mathematische Aufgabe und gehoert nach r3c1c.
Verbotene Folgerung	Aus dem gruenen `r3c1b-full`-Build darf nicht gefolgert werden, dass `r3c1d`, `r3c1e`, ein unbedingter `r3c2`-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Ebenfalls verboten bleiben terminale Identity, `GeneratedInteriorAccumulatedEntryCancellationInvariant`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten, Charpoly-/Faktor-/Diskriminantendaten, freie Matrixdaten, freie Entry-Tabellen, externe Inversenannahmen und jeder Rueckfall auf SM3daR.
Naechste aktive Phase	**Aktiv ist nun `SM3eR-r3c1c: FoldInvariance`.** Diese Phase muss die lokale r3c1b-full-Step-Kuerzung in eine fold-/trace-recursive Invarianz heben: Wenn die BlockFold-Invarianz vor einem Trace-/Fold-Schritt gilt und der lokale Step-Kuerzungssatz gilt, muss sie nach dem Schritt gelten. Beweisform: strukturelle Rekursion ueber den vorhandenen Trace-/Fold-Carrier bzw. ueber `GeneratedInteriorAccumulatedEntryFold`, nicht ueber einen grossen Index-Eliminator.
r3c1c-DoD	Ziel ist eine Struktur oder ein aequivalenter Konstruktor wie `GeneratedInteriorBlockFoldInvarianceR3c1c` bzw. ein r3c1c-Ledger. Minimalfelder: Konsum der r3c1a-BlockFold-Normalform, Konsum von `GeneratedInteriorLocalStepCancellationR3c1bFull`, linke und rechte step-preservation-Felder, Herkunft aus Trace-/Fold-Provenienz, Regular-/Variable-Spezialisierungen und No-Output-Statusfelder gegen terminal identity, `GeneratedInteriorAccumulatedEntryCancellationInvariant`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
r3c1c-Grenze	`r3c1c` darf noch nicht beweisen, dass der terminale Fold-Zustand identity ist; das bleibt `r3c1d`. `r3c1c` darf auch den spaeteren `GeneratedInteriorAccumulatedEntryCancellationInvariant` noch nicht konstruieren; das bleibt `r3c1e`. `r3c2` bleibt bis zu einem r3c1e-Witness nur konditionaler Consumer; r4/r5/`SM3fR` bleiben blockiert.
Fortgeschriebene Reihenfolge	Ab v47 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` closed/green als ReciprocalProfileFromWeightPolicy-Audit; `SM3db2fR` closed/green als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` closed/green als BlockFoldKernel-to-LocalSchurResidualBridge; `SM3eR-r3c1b-full` closed/green als LocalStepCancellation; `SM3eR-r3c1c` **active** als Fold-Invarianz; `r3c1d` blocked until r3c1c; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0x. Fortschreibungsentscheidung v48: SM3eR-r3c1c gruen, SM3eR-r3c1d aktiv

Statusentscheidung. `SM3eR-r3c1c` ist nach lokal gruenem Build formal geschlossen. Die Phase liefert die verlangte `FoldInvariance`: Sie konsumiert die r3c1a-BlockFold-Normalform, die r3c1b1-Bridge und `GeneratedInteriorLocalStepCancellationR3c1bFull`; sie traegt die Trace-/Fold-Herkunft aus SM3db3R/SM3db4aR und hebt die lokalen linken/rechten Step-Kuerzungen ueber die BlockFold-Summen. Der lokale Green-Build loest damit genau die Sperre zwischen lokaler Step-Kancellation und terminalem Coverage-/Identity-Gate.

Formaler r3c1c-Ertrag. Geschlossen sind die r3c1c-Status- und Ledgerobjekte, die BlockFold-Invarianzstruktur, die Konstruktoren aus LocalStepCancellation und operationaler Degree-Reziprokquelle sowie die linken/rechten Fold-Summen-Erhaltungssaetze. Die vollstaendigen Lean-Namen bleiben im Code und in der v48-Historie auffindbar; in v49 zaehlt nur der Status: r3c1c ist eine gruen geschlossene Invarianzschicht ohne TerminalIdentity, ohne AccumulatedEntryCancellationInvariant und ohne Produkt-/Inverse-/DtN-Ausgang.

Praeziser Befund. `r3c1c` ist eine Invarianz- und Summenweitergabeschicht, kein terminales Identity-Gate. Die Phase beweist, dass die lokalen r3c1b-full-Kernel-/Residual-Kuerzungen in den linken und rechten BlockFold-Summen erhalten bleiben. Sie beweist noch nicht, dass der terminale Fold-Zustand `identity` ist, und erzeugt weiterhin keinen `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Damit ist die lokale-zu-global-Fold-Luecke geschlossen; die terminale Coverage-/Residual-Luecke ist die naechste mathematische Aufgabe.

Naechste aktive Phase nach v48. Aktiv wurde `SM3eR-r3c1d: TerminalIdentity`. Diese Phase musste aus dem r3c1c-FoldInvariance-Witness, der Pivot-/Index-Coverage aus SM3db3R und der terminalen Residual-/FoldOrder-Schicht aus SM3db4aR zeigen, dass der finale linke und rechte BlockFold-Zustand entrywise `identity` ist. Falls SM3db3R/SM3db4aR die terminale Coverage noch nicht exponieren, ist der minimale positive Rueckssprung `SM3db3aR` bzw. eine terminale SM3db4aR-Nachschaerfung; kein Patch in r3c1e/r3c2/r4.

Fortgeschriebene Reihenfolge v48. Ab v48 galt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full` closed/green als ReciprocalProfileFromWeightPolicy-Audit; `SM3db2fR` closed/green als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` active; `r3c1e` blocked until r3c1d; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0y. Fortschreibungsentscheidung v49: SM3eR-r3c1d gruen als TerminalIdentity-Gate, SM3db3aR aktiv

Statusentscheidung. `SM3eR-r3c1d` ist nach lokal gruenem Build formal integrierbar. Geschlossen ist aber nicht der starke terminale Identity-DoD, sondern ein sauberes TerminalIdentity-Gate: Die neue Schicht auditert die fehlende terminale Coverage-Exposition und stellt zugleich einen konditionalen Konstruktor bereit, der aus einer proof-carrying TerminalCoverageSource die linke/rechte BlockFold-Identity und die Diagonal-/Offdiag-Formen ableitet. Der Build ist daher `green/gate-closed`, nicht `full-closed`.

Formaler r3c1d-Ertrag. Geschlossen sind die r3c1d-Statusobjekte, die TerminalCoverage-Auditstruktur, die konditionale TerminalCoverage-Source-Schnittstelle, die konditionale TerminalIdentity-Struktur, die entrywise Projektionen fuer linke/rechte BlockFold-, FoldSum- und AccumulatedConvolution-Identity sowie Regular-/Variable-Abkuerzungen und die beiden r3c1d-Ledger. Die vollstaendigen Lean-Namen stehen in `GeneratedInteriorBlockFoldTerminalIdentityR3c1d.lean`; der Plan fuehrt hier nur die konsumierbaren Rollen und Gates.

Praeziser Befund. Die Audit-Schicht markiert explizit: terminale Coverage ist in SM3db3R/SM3db4aR noch nicht exponiert. Der naechste Status ist deshalb die SM3db3aR- bzw. SM3db4aR-TerminalCoverage-Nachschaerfung. Damit darf r3c1e noch nicht starten: Es gibt noch keine unbedingte r3c1d-TerminalCoverageSource und folglich keine unbedingte r3c1d-TerminalIdentity. Die konditionale `fromTerminalCoverage`-Route ist wichtig, weil sie den spaeteren Beweisabschluss lokalisiert: Sobald die Source vorhanden ist, muss r3c1d nicht mehr neu rechnen, sondern nur konsumieren.

Verbotene Folgerung. Aus dem gruenen r3c1d-Build darf nicht gefolgert werden, dass `r3c1e`, ein unbedingter `r3c2`-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Verboten bleiben freie Terminaltabellen, freie BlockFold-Identity-Beweise, neue CandidateEntry-Formeln, freie Matrizen, externe Inversenannahmen, `Matrix.inv`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.

Naechste aktive Phase. Aktiv ist nun `SM3db3aR: TracePivotTerminalCoverageSource`. Diese Phase liegt upstream von r3c1d. Sie muss aus dem vorhandenen `GeneratedInteriorEliminationTrace` und dessen SM3db3R-Pivot-/Index-Provenienz die terminale Coverage so exponieren, dass sie zusammen mit der SM3db4aR-AccumulatedEntryFold-/Residual-/FoldOrder-Schicht die r3c1d-TerminalCoverageSource speisen kann.

SM3db3aR-DoD. Ziel ist ein Konstruktor oder ein exakter Audit fuer eine TerminalCoverageSource aus Trace-/Pivot-Coverage. Minimal benoetigt werden: proof-carrying Pivot-/Index-Coverage fuer alle `GeneratedInteriorIndex A`, Rueckbindung an die vorhandenen Trace-/LocalStep-Projektionen, Herkunft aus dem SM3db3R-Trace-Source-Status, Anschlussdaten fuer die terminale Fold-/Residual-Coverage aus SM3db4aR, und klare No-Output-Statusfelder gegen `GeneratedInteriorAccumulatedEntryCancellationInvariant`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.

Beweisroute SM3db3aR. Zuerst werden nur Trace-/Pivot-/Index-Coverage und terminaler Coveragebedarf exponiert; keine Produktrechnung, keine Fold-Reparatur in r3c1d und keine r3c1e-Vorwegnahme. Falls SM3db3R die Pivot-/Index-Coverage liefert, aber die terminale Residual-/FoldOrder-Identity noch in SM3db4aR fehlt, folgt unmittelbar eine minimale terminale SM3db4aR-Nachschaerfung, z.B. `SM3db4aT: TerminalFoldResidualCoverage`. Falls bereits beide Quellen rekonstruierbar sind, darf SM3db3aR direkt die r3c1d-TerminalCoverageSource exportieren.

Downstream-Gates. `SM3eR-r3c1d-full` bzw. der unbedingte r3c1d-TerminalIdentity-Witness wird erst nach gruener TerminalCoverageSource freigegeben. Erst danach darf `SM3eR-r3c1e` den `GeneratedInteriorAccumulatedEntryCancellationInvariant`-Adapter bauen. `r3c2`, r4, r5 und `SM3fR` bleiben blockiert.

Fortgeschriebene Reihenfolge v49. Ab v49 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green als GeneratedDegreeFormulaAudit; `SM3db2eR-full`

closed/green als ReciprocalProfileFromWeightPolicy-Audit; `SM3db2fR` closed/green als GeneratedOperationalDegreeReciprocalSource; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; **`SM3db3aR` active**; `SM3eR-r3c1d-full` blocked until TerminalCoverageSource; `r3c1e` blocked until r3c1d-full; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0z. Fortschreibungsentscheidung v50 nach gruenem SM3db3aR-Build: SM3db4aT aktiv

Statusentscheidung	`SM3db3aR` ist nach lokal gruenem Build formal geschlossen, aber exakt als Trace-/Pivot-TerminalCoverage-Audit und nicht als unbedingter r3c1d-TerminalCoverageSource-Export. Die Phase hat die Trace-/Pivot-/Index-Coverage aus `GeneratedInteriorEliminationTrace` und SM3db3R-Provenienz proof-carrying exponiert und zugleich festgehalten, dass der fehlende terminale Schlussteil nicht mehr SM3db3R, sondern die SM3db4aR-Fold-/Residual-/FoldOrder-Schicht betrifft.
Formaler SM3db3aR-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorTracePivotTerminalCoverageSourceSM3db3aR`, `GeneratedInteriorTerminalFoldResidualCoverageRequirementSM3db3aR`, `GeneratedInteriorTracePivotTerminalCoverageAuditSM3db3aR`, `SM3db3aRTracePivotTerminalCoverageSourceLedger`, die Regular-/Variable-Abkuerzungen und die Konstruktoren aus Trace bzw. r3c1c-Invariance. Getragen werden `traceIndexCarrier`, `localStepOf`, `traceStepOf`, `pivotOf`, die Rueckbindung an SM3db3R-Trace-Source-Status, die Bindung an SM3db1R-Node/SM3db2R-Step sowie Row-/Column-Residual-Provenienz aus dem SM3cR-InteriorBlock-Profil.
Praeziser Restbefund	Der neue Audit belegt nicht die terminale Fold-/Residual-Identity. Sein `nextPhaseStatus` ist bewusst `sm3db4aTTerminalFoldResidualCoverage`; der `terminalCoverageExportStatus` bleibt `terminalCoverageSourceBlockedUntilSM3db4aRTerminalResidualCoverage`. Damit ist die fehlende Substanz lokalisiert: Aus SM3db3R ist die Trace-/Pivot-Coverage vorhanden; aus SM3db4aR fehlt noch ein theorematischer terminaler Fold-/Residual-Coverage-Witness, der r3c1d die konditionale TerminalCoverageSource ohne freie Felder speisen kann.
Naechste aktive Phase	**Aktiv ist nun `SM3db4aT: TerminalFoldResidualCoverage`.** Diese Phase muss aus `GeneratedInteriorAccumulatedTraceEvaluation`, `GeneratedInteriorAccumulatedEntryFold`, der Fold-/Update-Reihenfolge, den Residualdaten und der r3c1c-FoldInvariance den terminalen Coverage-Witness erzeugen oder exakt auditieren, welches SM3db4aR-Feld minimal fehlt. Sie darf keine Produktrechnung, keine r3c1e-Konstruktion und keinen r3c2/r4/r5-Vorgriff enthalten.
SM3db4aT-DoD	Ziel ist ein unbedingter, proof-carrying Witness fuer terminale Fold-/Residual-Coverage, der zusammen mit `SM3db3aR` die r3c1d-`TerminalCoverageSource` speisen kann. Minimal benoetigt werden: terminaler Fold-Zustand, terminale Residual-Coverage, Anschluss an `accumulatedTraceEvaluation` und `accumulatedEntryFold`, Herkunft aus `SM3db4aRAccumulatedSourceStatus.accumulatedEntryFromSM3db3RTraceAndSM3db2RLocalSteps`, linke/rechte terminale Coverage-Felder fuer den spaeteren r3c1d-Verbrauch und No-Output-Status gegen `GeneratedInteriorAccumulatedEntryCancellationInvariant`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
Beweisroute SM3db4aT	Zuerst wird die terminale Fold-/Residual-Struktur aus der vorhandenen SM3db4aR-AccumulatedEntryFold-Schicht exponiert. Danach wird gezeigt, dass der finale Fold-/Residual-Zustand genau die von r3c1d erwartete TerminalCoverageSource-Schicht traegt. Wenn die vorhandene SM3db4aR-API nur die akkumulierte Entry-Herkunft, aber keinen terminalen Fold-Zustand exponiert, endet SM3db4aT nicht mit r3c1d-full-Freigabe, sondern mit einem minimalen Missing-Source-Audit fuer die konkrete SM3db4aR-Nachschaerfung.
Verbotene Folgerung	Aus dem gruenen SM3db3aR-Build darf nicht gefolgert werden, dass `SM3eR-r3c1d-full`, `SM3eR-r3c1e`, der unbedingte r3c2-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Ebenso verboten bleiben freie Terminaltabellen, freie Fold-/Residual-Identity-Beweise, neue CandidateEntry-Formeln, freie Matrizen, externe Inversenannahmen, `Matrix.inv`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/MultiSchur- und Charpoly-/Diskriminantendaten.
Downstream-Gates	`SM3eR-r3c1d-full` wird erst nach gruenem `SM3db4aT`-TerminalCoverage-Witness freigegeben. r3c1d-full darf dann nur konsumierend die bereits konditional vorbereitete TerminalIdentity schliessen. Erst danach darf `SM3eR-r3c1e` den unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant`-Adapter bauen; r3c2 darf danach nur mechanisch konsumieren; r4/r5/SM3fR bleiben bis ProductIdentityProof bzw. `provenTwoSidedInv` blockiert.
Fortgeschriebene Reihenfolge v50	Ab v50 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green; `SM3db2eR-full` closed/green; `SM3db2fR` closed/green; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; **`SM3db4aT` active**; `SM3eR-r3c1d-full` blocked until SM3db4aT-TerminalCoverageSource; `r3c1e` blocked until r3c1d-full; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0aa. Fortschreibungsentscheidung v51 nach gruenem SM3db4aT-Build: SM3db4aU aktiv

Statusentscheidung	`SM3db4aT` ist nach lokal gruenem Build formal geschlossen, aber exakt als TerminalFoldResidualCoverage-Audit und nicht als unbedingter r3c1d-TerminalCoverageSource-Export. Die Phase exponiert die terminale Fold-/Residual-/FoldOrder-Schicht aus SM3db4aR proof-carrying und bindet sie an die von SM3db3aR gelieferte Trace-/Pivot-TerminalCoverage-Anforderung zurueck. Der gruene Build belegt damit die korrekte Lokalisierung der verbleibenden Luecke: Nicht Trace-/Pivot-Coverage und nicht terminaler Fold-/Residual-State fehlen, sondern die eigentlichen linken/rechten Terminal-Identity-Felder.
Formaler SM3db4aT-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorTerminalFoldResidualCoverageAuditSM3db4aT`, `SM3db4aTTerminalFoldResidualCoverageLedger`, Regular-/Variable-Abkuerzungen und Konstruktoren aus `GeneratedInteriorTerminalFoldResidualCoverageRequirementSM3db3aR` bzw. r3c1c-Invariance. Getragen werden `terminalFoldState = accumulatedEntryFold`, `terminalResidualContribution = accumulatedEntryFold.residualContribution = generatedInteriorTraceResidualContribution T`, `terminalFoldUpdateOrderContribution = accumulatedEntryFold.foldUpdateOrderContribution = generatedInteriorTraceFoldUpdateOrderContribution T`, Rueckbindung an `SM3db4aRAccumulatedSourceStatus.accumulatedEntryFromSM3db3RTraceAndSM3db2RLocalSteps`, sowie Trace-/Pivot-Source-Bindung an SM3db3R.
Praeziser Restbefund	`SM3db4aT` beweist bewusst keine Gleichung `generatedInteriorLeftBlockFold W T i j = (1 : Matrix _ _ _ ) i j` und keine entsprechende rechte Gleichung. Die Felder `leftTerminalIdentityFieldStatus` und `rightTerminalIdentityFieldStatus` stehen auf `terminalIdentityFieldMissingFromCurrentSM3db4aRAPI`; der `terminalCoverageExportStatus` bleibt `terminalCoverageSourceBlockedUntilTerminalIdentityFields`; der `nextPhaseStatus` ist `sm3db4aUExposeTerminalFoldResidualIdentityFields`. Damit ist `SM3db4aT` green/audit-closed, aber r3c1d-full bleibt blockiert.
Naechste aktive Phase	**Aktiv ist nun `SM3db4aU: ExposeTerminalFoldResidualIdentityFields`.** Diese Phase muss die in SM3db4aT als fehlend auditierten linken und rechten Terminal-Identity-Felder aus der vorhandenen SM3db4aR-Fold-/Residual-/FoldOrder-Schicht, r3c1c-FoldInvariance und SM3db3aR-TracePivotCoverage konstruieren oder noch genauer beweisen, welche minimal tiefere SM3db4aR-Definition fehlt.
SM3db4aU-DoD	Ziel ist ein Konstruktor oder exakter Missing-Source-Audit fuer eine Quelle, die die sechs r3c1d-Coverage-Felder traegt: `leftBlockFold_eq_identity`, `rightBlockFold_eq_identity`, `leftBlockFold_diagonal`, `leftBlockFold_offdiag`, `rightBlockFold_diagonal`, `rightBlockFold_offdiag`. Wenn diese sechs Felder unbeding aus SM3db4aR/SM3db3aR/r3c1c ableitbar sind, muss SM3db4aU einen r3c1d-kompatiblen TerminalCoverageSource-Provider vorbereiten. Wenn nicht, muss der Audit exakt festhalten, ob die fehlende Substanz eine terminale Fold-Induktionsgleichung, eine Residual-Abschlussgleichung, eine FoldOrder-Coverage-Gleichung oder eine Links/Rechts-Transfer-Gleichung ist.
Beweisroute SM3db4aU	Zuerst wird SM3db4aT konsumiert, nicht dupliziert. Danach werden die terminalen Residual- und FoldOrder-Beitraege gegen die r3c1c-BlockFold-Ausdruecke normalisiert. Erst dann darf versucht werden, die linken/rechten BlockFold-Identitaeten zu schliessen. Keine Produktrechnung ausserhalb der bereits geoeffneten r3c1c/r3c1d-Oberflaeche, keine neue CandidateEntry-Formel, keine freie Terminaltabelle, keine freie Matrix, kein `Matrix.inv`, keine externe Inversenannahme.
Verbotene Folgerung	Aus dem gruenen SM3db4aT-Build darf nicht gefolgert werden, dass `SM3eR-r3c1d-full`, `SM3eR-r3c1e`, der unbedingte r3c2-Ledger, r4, r5 oder `SM3fR` freigegeben sind. Ebenso erzeugt SM3db4aU selbst keinen `GeneratedInteriorAccumulatedEntryCancellationInvariant`, keinen ProductIdentityProof, keinen `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/MultiSchur- und keine Charpoly-/Diskriminantendaten.
Downstream-Gates	`SM3eR-r3c1d-full` wird erst nach gruenem SM3db4aU-TerminalIdentityField-/TerminalCoverageSource-Witness freigegeben. r3c1d-full darf dann nur konsumierend die konditionale TerminalIdentity schliessen. Erst danach darf `SM3eR-r3c1e` den unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant`-Adapter bauen; r3c2 darf danach nur mechanisch konsumieren; r4/r5/SM3fR bleiben bis ProductIdentityProof bzw. `provenTwoSidedInv` blockiert.

Fortgeschriebene Reihenfolge v51. Ab v51 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green; `SM3db2eR-full` closed/green; `SM3db2fR` closed/green; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT` closed/green-audit; **`SM3db4aU` active**; `SM3eR-r3c1d-full` blocked until SM3db4aU-TerminalIdentityFields/TerminalCoverageSource; `r3c1e` blocked until r3c1d-full; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0ab. Fortschreibungsentscheidung v52 nach gruenem SM3db4aU-Build: SM3db4aV aktiv

Statusentscheidung	`SM3db4aU` ist nach lokal gruenem Build formal geschlossen, aber exakt als *TerminalFoldResidualIdentityFields-Audit* und nicht als unbedingter r3c1d-TerminalCoverageSource-Export. Die Phase konsumiert `SM3db4aT`, dupliziert die terminale Fold-/Residual-/FoldOrder-Schicht nicht und exponiert proof-carrying, dass die eigentliche Luecke nun die Gleichungsquelle fuer linke und rechte Terminal-Identity ist.
Formaler SM3db4aU-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorTerminalFoldResidualIdentityFieldsAuditSM3db4aU`, `GeneratedInteriorTerminalFoldResidualIdentityFieldCertificateSM3db4aU`, `SM3db4aUTerminalFoldResidualIdentityFieldsLedger`, Regular-/Variable-Abkuerzungen sowie ein r3c1d-kompatibler Adapter aus einem spaeteren expliziten SM3db4aU-Certificate. Positiv getragen sind Rueckbindungen an `GeneratedInteriorTerminalFoldResidualCoverageAuditSM3db4aT`, r3c1c-FoldInvariance, SM3db3aR-TracePivotCoverage, `GeneratedInteriorAccumulatedEntryFold`, `generatedInteriorTraceResidualContribution T` und `generatedInteriorTraceFoldUpdateOrderContribution T`.
Praeziser Restbefund	Der gruene Build beweist weiterhin nicht die unbedingten Gleichungen `generatedInteriorLeftBlockFold W T i j = (1 : Matrix _ _ _) i j` und `generatedInteriorRightBlockFold W T i j = (1 : Matrix _ _ _) i j`. Entsprechend stehen `leftTerminalIdentityFieldSourceStatus` und `rightTerminalIdentityFieldSourceStatus` auf `terminalIdentityFieldSourceMissingFromCurrentSM3db4aRAPI`; `terminalCoverageExportStatus` bleibt `terminalCoverageSourceBlockedUntilTerminalIdentityFieldCertificate`; `nextPhaseStatus` ist `sm3db4aVTerminalFoldResidualIdentityEquationSource`.
Naechste aktive Phase	**Aktiv ist nun `SM3db4aV: TerminalFoldResidualIdentityEquationSource`.** Diese Phase muss die in SM3db4aU nur auditierte Gleichungsquelle erzeugen oder noch enger lokalisieren. Ziel ist nicht r3c1d-full selbst, sondern die kleinste SM3db4aR-nahe Source-Schicht, aus der spaeter ein `GeneratedInteriorTerminalFoldResidualIdentityFieldCertificateSM3db4aU` ohne freie Identity-Beweise gebaut werden kann.
SM3db4aV-DoD	Ziel ist ein unbedingter, proof-carrying Konstruktor fuer die terminale linke/rechte Fold-Residual-Identity-Gleichungsquelle. Minimal benoetigt werden: terminale Fold-Induktionsgleichung, Residual-Abschlussgleichung, FoldOrder-Coverage-Gleichung, getrennte linke/rechte Terminal-Identity-Gleichung oder ein zuvor bewiesener Links/Rechts-Transfer, sowie die Rueckbindung an `GeneratedInteriorAccumulatedTraceEvaluation`, `GeneratedInteriorAccumulatedEntryFold`, `SM3db4aT` und `SM3db4aU`. Der finale Output darf die sechs Identity-/Diag-/Offdiag-Felder nicht als freie Argumente annehmen.
Beweisroute SM3db4aV	Zuerst werden die `SM3db4aU`-Auditfelder gelesen: `terminalFoldInductionEquationStatus`, `residualClosureEquationStatus`, `foldOrderCoverageEquationStatus` und `leftRightTerminalIdentityStatus`. Danach wird die schwachste fehlende Gleichungsebene als eigene proof-carrying Source geschlossen. Falls die vorhandene SM3db4aR-Fold-Schicht nur Beitraege, aber keine Closure-Gleichung traegt, muss SM3db4aV exakt diese Closure-Gleichung als neue positive Nachschaerfung bauen.
Verbotene Folgerung	Aus dem gruenen SM3db4aU-Build darf nicht gefolgert werden, dass `SM3eR-r3c1d-full`, `SM3eR-r3c1e`, der unbedingte r3c2-Ledger, r4, r5 oder `SM3fR` freigegeben sind. SM3db4aV darf keine neue CandidateEntry-Formel, keine freie Terminaltabelle, keine freie Matrix, kein `Matrix.inv`, keine externe Inversenannahme, keinen ProductIdentityProof, keinen `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/MultiSchur- und keine Charpoly-/Diskriminantendaten einfuehren.
Downstream-Gates	Erst nach gruenem SM3db4aV-Build darf eine anschliessende Certificate-/Export-Phase den `GeneratedInteriorTerminalFoldResidualIdentityFieldCertificateSM3db4aU` unbeding aus der Gleichungsquelle bauen. Erst danach darf `SM3eR-r3c1d-full` konsumierend den TerminalCoverageSource-/TerminalIdentity-Abschluss fuehren. `SM3eR-r3c1e`, r3c2, r4, r5 und SM3fR bleiben bis zu diesen Witnesses blockiert.

Fortgeschriebene Reihenfolge v52. Ab v52 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green; `SM3db2eR-full` closed/green; `SM3db2fR` closed/green; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green;

`SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT` closed/green-audit; `SM3db4aU` closed/green-audit; **`SM3db4aV` active**; `SM3eR-r3c1d-full` blocked until SM3db4aV plus SM3db4aU-Certificate/TerminalCoverageSource; `r3c1e` blocked until r3c1d-full; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0ac. Fortschreibungsentscheidung v53 nach gruenem SM3db4aV-Build: SM3db4aW aktiv

Statusentscheidung	`SM3db4aV` ist nach lokal gruenem Build formal geschlossen, aber exakt als *TerminalFoldResidualIdentityEquationSource*-Schicht und nicht als r3c1d-full-Abschluss. Die Phase liest den `SM3db4aU`-Audit, haelt die weiterhin fehlende Gleichungsquelle proof-carrying fest und stellt zugleich den positiven Pfad bereit: Wenn ein `GeneratedInteriorTerminalFoldResidualIdentityEquationDatumSM3db4aV` vorliegt, entsteht daraus eine `GeneratedInteriorTerminalFoldResidualIdentityEquationSourceSM3db4aV` und daraus ein `GeneratedInteriorTerminalFoldResidualIdentityFieldCertificateSM3db4aU`.
Formaler SM3db4aV-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorTerminalFoldResidualIdentityEquationDatumSM3db4aV`, `GeneratedInteriorTerminalFoldResidualIdentityEquationSourceAuditSM3db4aV`, `GeneratedInteriorTerminalFoldResidualIdentityEquationSourceSM3db4aV`, `terminalFoldResidualIdentityFieldCertificateFromSM3db4aV`, die Regular-/Variable-Abkuerzungen, `SM3db4aVTerminalFoldResidualIdentityEquationSourceAuditLedger` und `SM3db4aVTerminalFoldResidualIdentityEquationSourceLedger`. Positiv getragen sind Rueckbindungen an `SM3db4aU`, `SM3db4aT`, r3c1c-FoldInvariance, SM3db3aR-TracePivotCoverage und den terminalen `GeneratedInteriorAccumulatedEntryFold`.
Praeziser Restbefund	Der gruene V-Build beweist nicht, dass r3c1d-full schon als unbedingter TerminalIdentity-Witness vorliegt. Die sechs Felder `leftBlockFold_eq_identity`, `rightBlockFold_eq_identity`, `leftBlockFold_diagonal`, `leftBlockFold_offdiag`, `rightBlockFold_diagonal` und `rightBlockFold_offdiag` sind in V die Nutzlast des `EquationDatum`. V stellt daraus eine Source und ein SM3db4aU-Certificate bereit; ein r3c1d-kompatibler `GeneratedInteriorBlockFoldTerminalCoverageSourceR3c1d`-Export und ein TerminalIdentity-Ledger sind noch nicht als eigene Handoff-Schicht geschlossen.
Naechste aktive Phase	**Aktiv ist nun `SM3db4aW: TerminalIdentityFieldCertificateFromEquationSource`.** Diese Phase muss ausschliesslich konsumierend aus der gruenen SM3db4aV-Source den SM3db4aU-Certificate-Pfad und den r3c1d-TerminalCoverageSource-Export bereitstellen. Sie darf keine neue Terminal-Identity-Gleichung beweisen, keine neuen Identity-/Diag-/Offdiag-Felder frei annehmen und keine Produktrechnung starten.
SM3db4aW-DoD	Ziel ist ein kleiner Handoff-/Export-Layer, z.B. mit `GeneratedInteriorTerminalIdentityFieldCertificateFromEquationSourceSM3db4aW` oder aequivalenter Struktur: regular/variable Sources aus `SM3db4aVTerminalFoldResidualIdentityEquationSourceLedger` werden in regular/variable `GeneratedInteriorTerminalFoldResidualIdentityFieldCertificateSM3db4aU` ueberfuehrt und daraus in regular/variable `GeneratedInteriorBlockFoldTerminalCoverageSourceR3c1d`. Zusaetzlich braucht W ein Ledger, das die Rueckbindung an V-Source, V-EquationDatum, U-Certificate und r3c1d-CoverageSource theorematisch markiert.
Beweisroute SM3db4aW	Zuerst werden die V-Konstruktoren konsumiert: `terminalFoldResidualIdentityFieldCertificateFromSM3db4aV`, `regularTerminalFoldResidualIdentityFieldCertificateFromSM3db4aV` und `variableTerminalFoldResidualIdentityFieldCertificateFromSM3db4aV`. Danach werden die bereits vorhandenen r3c1d-Adapter `regularBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aU` und `variableBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aU` verwendet. Beweise sollen Feldprojektionen, `rfl` und kleine Konstruktoren sein; keine Summeninduktion und keine neue BlockFold-Rechnung.
Verbotene Folgerung	Aus dem gruenen SM3db4aV-Build darf noch nicht gefolgert werden, dass `SM3eR-r3c1e`, der unbedingte r3c2-Ledger, r4, r5 oder `SM3fR` freigegeben sind. `SM3db4aW` darf keinen ProductIdentityProof, keinen `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/MultiSchur-Daten, keine Charpoly-/Faktor-/Diskriminantendaten, keine neue CandidateEntry-Formel, keine freie Matrix, keine freie Terminaltabelle und kein `Matrix.inv` einfuehren.
Downstream-Gates	Erst nach gruenem `SM3db4aW`-Build ist `SM3eR-r3c1d-full` als konsumierende TerminalCoverage-/TerminalIdentity-Phase freigegeben. r3c1d-full darf dann nur die aus W exportierten r3c1d-CoverageSources in `GeneratedInteriorBlockFoldTerminalIdentityR3c1d` bzw. den TerminalIdentity-Ledger ueberfuehren. Erst danach darf `SM3eR-r3c1e` den unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant`-Adapter bauen; r3c2, r4, r5 und SM3fR bleiben bis zu ihren jeweiligen Witnesses blockiert.

Fortgeschriebene Reihenfolge v53. Ab v53 gilt: `r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR` closed/green relativ zu Pivot-Regularitaet; `SM3db2bR` closed/green als ScaleWitness-Consumer; `SM3db2cR` audit-closed/conditional als ScaleSource-Adapter; `SM3db2dR` green-relative als ReciprocalProfile-Adapter; `SM3db2eR0` closed/green; `SM3db2eR-full` closed/green; `SM3db2fR` closed/green; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT` closed/green-audit; `SM3db4aU` closed/green-audit; `SM3db4aV` closed/green als EquationSource-/Datum-/Certificate-Adapter; **`SM3db4aW` active**; `SM3eR-r3c1d-full` blocked until SM3db4aW-TerminalCoverageSource-Export; `r3c1e` blocked until r3c1d-full; `r3c2` conditional/green consumer; r4/r5/SM3fR blocked.

0ad. Fortschreibungsentscheidung v54 nach gruenem SM3db4aW-Build: r3c1d-full aktiv und Audit-Dichte-Gate

Statusentscheidung	`SM3db4aW` ist nach lokal gruenem Build formal geschlossen. Die Phase ist exakt ein konsumierender Exportlayer: `SM3db4aV`-EquationSource → `SM3db4aU`-TerminalIdentityFieldCertificate → r3c1d-`TerminalCoverageSource`. Damit ist die in v53 blockierende Handoff-Luecke geschlossen. Der naechste aktive mathematische Schritt ist `SM3eR-r3c1d-full`.
Formaler SM3db4aW-Ertrag	Geschlossen sind insbesondere `terminalFoldResidualIdentityFieldCertificateFromEquationSourceSM3db4aW`, `blockFoldTerminalCoverageSourceFromEquationSourceSM3db4aW`, `GeneratedInteriorTerminalIdentityFieldCertificateFromEquationSourceSM3db4aW`, die regular/variable Certificate-Adapter, `regularBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aW`, `variableBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aW` sowie `SM3db4aWTerminalIdentityFieldCertificateFromEquationSourceLedger.fromSM3db4aVLedger`. Diese Objekte transportieren vorhandene V-/U-/r3c1d-Daten; sie erzeugen keine neuen Identity-Gleichungen.
Praeziser Restbefund	Der gruene W-Build beweist noch nicht `GeneratedInteriorBlockFoldTerminalIdentityR3c1d`. Er beweist nur, dass r3c1d-full nun eine legal konsumierbare TerminalCoverageSource bekommt. Ebenfalls weiterhin nicht vorhanden sind `GeneratedInteriorAccumulatedEntryCancellationInvariant`, `GeneratedInteriorSM3db7RTraceCancellationInvariant`, ProductIdentityProof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten und Charpoly-/Faktor-/Diskriminantendaten.
Audit-Skalenbefund	Der aktuelle W-Zip zeigt als Arbeitszaehlung: 248 Lean-Dateien, 68.649 Lean-LOC, davon 42.449 LOC in `CNNA/PillarA/Arithmetic` und 24.734 LOC in `CNNA/PillarA/Arithmetic/Smoke`; zugleich null `sorry`. Dieser Befund ist zweischneidig: Die Provenienz- und No-Oracle-Disziplin funktioniert, aber die Smoke-/Ledger-Dichte ist inzwischen selbst ein technisches Risiko fuer Skalierung, Uebersicht und spaetere Refaktorisierbarkeit.
Entscheidung zum Overhead	Ab v54 gilt: Audit-Overhead ist nur dann mathematisch notwendig, wenn er eine verbotene Abkuerzung verhindert, eine neue provenienztragende Quelle exponiert oder ein oeffentliches Consumer-Interface stabilisiert. Reine Adapterphasen duerfen nicht automatisch neue Enum-Familien, Negativfelder und Ledger-Schichten vervielfachen. Wo ein vorhandenes Certificate oder eine vorhandene Source unveraendert weitergereicht wird, ist eine einzelne Exportstruktur oder ein einzelner Exportkonstruktor ausreichend.
Audit-Dichte-Gate	Jede neue Phase muss vor der Implementierung klassifiziert werden: **M-Phase** fuer echte mathematische Substanz bzw. neue Beweisquelle; **H-Phase** fuer Handoff/Adapter/Export; **C-Phase** fuer spaetere Kompaktierung ohne Semantikaenderung. M-Phasen duerfen eigene Invarianten und Status-Gates tragen. H-Phasen sollen vorhandene Statusfelder konsumieren und maximal ein knappes Exportledger fuehren. C-Phasen duerfen nur nach gruenem Build und nur API-erhaltend zusammenlegen; sie duerfen keine neue Mathematik beweisen.
r3c1d-full-Freigabe	`SM3eR-r3c1d-full` ist jetzt freigegeben. Die Phase konsumiert ausschliesslich die aus W exportierten regular/variable `GeneratedInteriorBlockFoldTerminalCoverageSourceR3c1d`-Objekte und fuehrt sie in `GeneratedInteriorBlockFoldTerminalIdentityR3c1d` bzw. den vorhandenen TerminalIdentity-Ledger ueber. Sie darf keine neuen sechs Identity-/Diag-/Offdiag-Felder frei annehmen und keine neue BlockFold-Rechnung wiederholen.
r3c1d-full-DoD	Der DoD ist geschlossen, wenn ein Konstruktor wie `terminalIdentityR3c1d_from_SM3db4aW` bzw. regular/variable Spezialisierungen existiert, dessen einzige substanzielle Eingabe der W-Ledger bzw. dessen TerminalCoverageSources sind. Der Output muss die sechs TerminalIdentity-Felder entrywise tragen und die Rueckbindung an SM3db3aR, SM3db4aT, SM3db4aU, SM3db4aV und SM3db4aW markieren.
Beweisroute r3c1d-full	Zuerst werden `regularBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aW` und `variableBlockFoldTerminalCoverageSourceR3c1dFromSM3db4aW` konsumiert. Danach wird nur der bereits vorhandene r3c1d-Konstruktor von TerminalCoverageSource zu TerminalIdentity verwendet. Beweise sind Feldprojektionen, definitorische Gleichheiten und kleine Konstruktoren; keine Summeninduktion, keine neue Fold-Induktion, kein Produktbeweis.
Verbotene Folgerung	r3c1d-full darf keinen `GeneratedInteriorAccumulatedEntryCancellationInvariant` direkt bauen, keinen `GeneratedInteriorSM3db7RProductCancellationLedger` erzeugen, keinen ProductIdentityProof oder `TwoSidedInv` ableiten und keine DtN-/MultiSchur-/Charpoly-Daten beruehren. Falls r3c1d-full noch eine fehlende Quelle findet, ist der Ruecksprung minimal zu W/V/U/T oder SM3db3aR; es wird nicht in r3c1e oder r4 gepatcht.

Kompaktierungsfenster	Eine nicht-semantische Kompaktierung der Kette SM3db4aT/U/V/W ist sinnvoll, aber nicht als Blocker vor r3c1d-full. Sie wird als spaetere C-Phase `SM3AuditCompact0` vorgemerkt: fruehestens nach gruenem r3c1e/r3c2-Verbrauch, spaetestens vor dem Uebergang zu dauerhaft wiederverwendbaren SM3fR-/SM3gR-Daten. Ziel ist API-erhaltende Buendelung mehrfacher Status-/Ledger-Adapter, nicht Aenderung der Beweiskette.
Fortgeschriebene Reihenfolge v54	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit gemaess v53; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT` closed/green-audit; `SM3db4aU` closed/green-audit; `SM3db4aV` closed/green; `SM3db4aW` closed/green; `SM3eR-r3c1d-full` **active**; `SM3eR-r3c1e` blocked until r3c1d-full; `SM3eR-r3c2` conditional/green consumer after r3c1e; r4/r5/SM3fR blocked.

0ae. Fortschreibungsentscheidung v55 nach gruenem SM3eR-r3c1d-full-Build: r3c1e aktiv und Overhead begrenzt

Statusentscheidung	`SM3eR-r3c1d-full` ist nach lokal gruenem Build formal geschlossen. Die Phase war korrekt als H-Phase begrenzt: Sie konsumierte den `SM3db4aW`-Export und fuehrte die r3c1d-CoverageSources in `GeneratedInteriorBlockFoldTerminalIdentityR3c1d` bzw. den vorhandenen TerminalIdentity-Ledger ueber. Der naechste aktive Schritt ist `SM3eR-r3c1e`.
Formaler r3c1d-full-Ertrag	Geschlossen sind insbesondere `terminalIdentityR3c1d_from_SM3db4aWExport`, `regularTerminalIdentityR3c1dFromSM3db4aWExport`, `variableTerminalIdentityR3c1dFromSM3db4aWExport`, `regularTerminalIdentityR3c1dFromSM3db4aWSource`, `variableTerminalIdentityR3c1dFromSM3db4aWSource` sowie ein knapper Ledger/Alias zum vorhandenen TerminalIdentity-Abschluss. Dieser Ertrag schliesst die terminalen sechs Identity-/Diag-/Offdiag-Felder auf BlockFold-Ebene.
Praeziser Restbefund	Der gruene r3c1d-full-Build beweist noch nicht `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Er liefert nur den dafuer noetigen terminalen BlockFold-Identity-Witness. Ebenfalls weiterhin nicht vorhanden sind `GeneratedInteriorSM3db7RTraceCancellationInvariant`, `GeneratedInteriorSM3db7RProductCancellationLedger` als unbedingter r3c2-Output, ProductIdentity-Proof, `TwoSidedInv`, `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten und Charpoly-/Faktor-/Diskriminantendaten.
Audit-Skalenbefund	Der aktuelle gruen gemeldete r3c1d-full-Stand fuehrt die statische Arbeitszaehlung in der Groessenordnung `249` Lean-Dateien, `68985` Lean-LOC, davon `42785` LOC in `CNNA/PillarA/Arithmetic` und `25062` LOC in `CNNA/PillarA/Arithmetic/Smoke`; weiterhin null `sorry`. Der Befund bestaetigt: Die Disziplin funktioniert, aber zusaetzliche Status-/Audit-Schichten duerfen nicht weiter als Default wachsen.
Phasenklassifikation	`SM3eR-r3c1e` ist keine neue mathematische Quellenphase. Sie ist eine **H-Phase**: TerminalIdentity-Witness → `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Sie darf die vorhandenen r3c1-Strukturen und deren bereits definierte Felder befuellen, aber keine neue Status-Enum-Familie, kein neues No-Oracle-Universum und keinen grossen neuen Ledger einfuehren. Erlaubt ist hoechstens ein einzelner oeffentlicher Konstruktorname oder Alias, falls downstream Importstabilitaet das erfordert.
r3c1e-DoD	Der DoD ist ein Konstruktor wie `accumulatedEntryCancellationInvariant_from_terminalIdentityR3c1d` bzw. regular/variable Spezialisierungen. Substanzielle Inputs: der in r3c1d-full erzeugte `GeneratedInteriorBlockFoldTerminalIdentityR3c1d`-Witness, die bereits vorhandenen r3c1a-Normalform-/Convolution-Gleichungen und die bestehende r3c1-Invariant-Struktur. Der Output ist ein unbedingter `GeneratedInteriorAccumulatedEntryCancellationInvariant`, der r3c2 ohne freie sechs Einzelbeweise konsumieren kann.
Beweisroute r3c1e	Keine neue Summeninduktion, keine neue Fold-Induktion und keine neue TerminalCoverage-Ableitung. Die Beweise laufen ueber vorhandene Gleichungen: AccumulatedConvolution = BlockFold, TerminalIdentity-Felder fuer links/rechts identity, diagonal und offdiag, danach Konstruktion der vorhandenen AccumulatedEntryCancellationInvariant-Struktur. Zulaessig sind Feldprojektionen, explizite Konstruktoren, lokale `rw`/`calc`-Schritte und finale triviale Vereinfachung.
Verbotene Folgerung	`SM3eR-r3c1e` darf keinen `GeneratedInteriorSM3db7RTraceCancellationInvariant` direkt erzeugen, keinen ProductCancellationLedger frei bauen, keinen ProductIdentityProof, keinen `TwoSidedInv`, keine `InteriorEliminationData`, keine DtN-/MultiSchur- oder Charpoly-Daten ableiten und keine neue CandidateEntry-/Matrixformel einfuehren. Diese Folgeschritte bleiben r3c2, r4, r5 und SM3fR vorbehalten.
Overhead-Regel ab v55	Neue Lean-Dateien sind nur noch erlaubt, wenn sie einen neuen mathematischen Traeger, einen oeffentlich konsumierten Konstruktor, einen Importzyklus-Schutz oder den Abbau eines vorher freien Feldes liefern. Reine Status-/NextPhase-/Audit-Prosa wird in Plan/Markdown gefuehrt. Fuer `r3c1e` heisst das: ein kleines Modul oder ein Anschluss an ein bestehendes r3c1-Modul ist vorzuziehen; keine neue Mikro-Phase `r3c1e0`/`r3c1e1`, solange kein Buildfehler eine echte fehlende Quelle beweist.
Kompaktierungsfenster	Eine C-Phase zur Buendelung von SM3db4aT/U/V/W und r3c1d-full bleibt vorgemerkt, aber nicht als naechster aktiver Schritt. Sie wird erst nach gruenem r3c1e und mechanischem r3c2-Verbrauch sinnvoll, weil erst dann die r3c1-Terminalkette als abgeschlossener Consumer-Pfad stabil ist.

Fortgeschriebene Reihenfolge v55	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` **active**; `SM3eR-r3c2` conditional/green consumer after r3c1e; r4/r5/SM3fR blocked.

0af. Fortschreibungsentscheidung v56 nach gruenem SM3eR-r3c1e-Build: r3c2 aktiv, r4 bleibt gesperrt

Statusentscheidung	`SM3eR-r3c1e` ist nach lokal gruenem Build formal geschlossen. Die Phase hat genau die in v55 geforderte H-Rolle erfuellt: TerminalIdentity-Witness → `GeneratedInteriorAccumulatedEntryCancellationInvariant`. Der naechste aktive Schritt ist `SM3eR-r3c2` als mechanischer Verbraucher dieses r3c1e-Witness.
Formaler r3c1e-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorAccumulatedEntryCancellationInvariant.fromTerminalIdentityR3c1d`, `accumulatedEntryCancellationInvariant_from_terminalIdentityR3c1d`, regular/variable Spezialisierungen und ein Anschluss vom vorhandenen r3c1d-TerminalIdentity-Ledger an den vorhandenen r3c1-AccumulatedEntryCancellation-Ledger. Inhaltlich sind damit die sechs AccumulatedConvolution-Identity-/Diag-/Offdiag-Felder nicht mehr frei, sondern aus der TerminalIdentity-Schicht getragen.
Praeziser Restbefund	r3c1e erzeugt bewusst noch keinen `GeneratedInteriorSM3db7RTraceCancellationInvariant` und keinen unbedingten `GeneratedInteriorSM3db7RProductCancellationLedger`. Ebenso fehlen weiterhin `GeneratedInteriorSM3db7RProductIdentityProof`, `TwoSidedInv`, `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten sowie Charpoly-/Faktor-/Diskriminantendaten. Der Produktbeweis beginnt daher nicht in r3c1e und nicht durch Prosa, sondern erst nach einem unbedingten r3c2-Ledger.
r3c2-Freigabe	`SM3eR-r3c2` ist jetzt aktiv. Es konsumiert ausschliesslich einen `GeneratedInteriorAccumulatedEntryCancellationInvariant` aus r3c1e und den bestehenden `SM3dbRToSM3eRHandoff`. Daraus baut es `GeneratedInteriorSM3db7RTraceCancellationDerivation`, `GeneratedInteriorSM3db7RTraceCancellationInvariant` und `GeneratedInteriorSM3db7RProductCancellationLedger`.
r3c2-DoD	Der DoD ist ein kleiner, oeffentlich konsumierbarer Konstruktorpfad vom konkreten r3c1e-Output zum bereits vorhandenen r3c2-Ledger, z.B. `traceCancellationDerivation_from_accumulatedEntryCancellationInvariant`, `traceCancellationInvariant_from_accumulatedEntryCancellationInvariant` und `productCancellationLedger_from_accumulatedEntryCancellationInvariant` bzw. ein einzelner Ledger-Konstruktor, der diese drei Felder buendelt. Falls die vorhandene r3c2-Datei diesen Pfad bereits generisch traegt, ist nur ein schmaler Anschluss-/Alias an den r3c1e-Namen zulaessig; keine neue Mikro-Phase.
Beweisroute r3c2	Keine neue BlockFold-, Fold-, Trace- oder Terminalrechnung. r3c2 darf nur die vorhandene Normalformgleichheit `GeneratedInteriorSM3db7R*ProductNormalForm = generatedInteriorAccumulated*Convolution` und die sechs Felder des r3c1e-Witness per `calc`/Feldprojektion in die alte TraceCancellationDerivation-Struktur einsetzen. Regular/variable Pfade sind Spezialisierungen desselben Konstruktors.
Verbotene Folgerung	r3c2 darf keinen ProductIdentityProof konstruieren, keinen `TwoSidedInv` behaupten, kein `InteriorEliminationData` erzeugen, keine DtN-/MultiSchur-/Charpoly-Daten beruehren, keine neue CandidateEntry-/Matrixformel einfuehren und keine der sechs Kancellationsfelder erneut frei als Argument annehmen. r3c2 ist Ledger-/TraceCancellation-Verbrauch, nicht Produktidentitaetsbeweis.
Overhead-Regel v56	Da r3c2 nach Plan bereits konditional gruen vorbereitet war, ist der naechste Code-Schritt maximal klein zu halten: bevorzugt ein Anschluss in der vorhandenen `GeneratedInteriorTraceCancellationR3c2`-Schicht oder ein einziger Import-/Alias-Adapter von r3c1e nach r3c2. Keine neue Status-Enum-Familie, kein neues Audit-Universum, kein `SM3eR-r3c2a/r3c2b`, solange kein Buildfehler eine echte fehlende Quelle beweist.
Kompaktierungsfenster	`SM3AuditCompact0` bleibt vorgemerkt, aber weiterhin nicht aktiv. Sinnvoll wird es erst nach gruenem r3c2-Verbrauch und vor der dauerhaften Wiederverwendung in r4/SM3fR/SM3gR. Zweck bleibt API-erhaltende Buendelung alter Adapterketten, nicht Aenderung der Beweiskette.
Fortgeschriebene Reihenfolge v56	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` **active**; r4a bleibt nur nach r3c2-Ledger conditional/allowed; r4/r5/SM3fR blocked.

0ag. Fortschreibungsentscheidung v57 nach gruenem SM3eR-r3c2-Build: r4a aktiv, r5 bleibt gesperrt

Statusentscheidung	`SM3eR-r3c2` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die v56-Rolle exakt erfuellt: r3c1e-Witness + SM3db7R-Handoff wurden ohne neue Kancellationssubstanz in TraceCancellationDerivation, TraceCancellationInvariant und ProductCancellationLedger ueberfuehrt. Der naechste aktive Schritt ist `SM3eR-r4a` als minimaler struktureller Consumer vom ProductCancellationLedger zum vorhandenen ProductIdentityProof-Typ.
Formaler r3c2-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorSM3db7RTraceCancellationDerivation.fromAccumulatedEntryCancellationInvariant`, `traceCancellationDerivation_from_accumulatedEntryCancellationInvariant`, `traceCancellationInvariant_from_accumulatedEntryCancellationInvariant`, `productCancellationLedger_from_accumulatedEntryCancellationInvariant`, regular/variable Spezialisierungen, `SM3eRr3c2TraceCancellationLedger` sowie der Anschluss `sm3eRReRunGeneratedProductCancellationLedger_from_accumulatedEntryCancellationLedger`. Inhaltlich sind damit die sechs ProductCancellation-Felder aus r3c1e und der SM3db7R-Handoff-Kette getragen; sie werden nicht mehr einzeln frei an r4 uebergeben.
Praeziser Restbefund	r3c2 erzeugt bewusst noch keinen `GeneratedInteriorSM3db7RProductIdentityProof`. Es existiert damit ein unbedingter ProductCancellationLedger, aber noch kein als ProductIdentityProof verpackter Produktidentitaetsbeweis und folglich noch kein `TwoSidedInv`. Ebenfalls weiterhin nicht vorhanden sind `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten sowie Charpoly-/Faktor-/Diskriminantendaten.
r4a-Freigabe	`SM3eR-r4a` ist jetzt aktiv. Die Phase darf ausschliesslich den vorhandenen `GeneratedInteriorSM3db7RProductCancellationLedger` konsumieren und daraus den bereits vorhandenen Typ `GeneratedInteriorSM3db7RProductIdentityProof` konstruieren. Die sechs Felder des ProductIdentityProof muessen direkt aus den sechs Feldern des Ledgers kommen.
r4a-DoD	Der DoD ist ein einzelner generischer Konstruktor, z.B. `productIdentityProof_from_productCancellationLedger` bzw. `GeneratedInteriorSM3db7RProductIdentityProof.fromProductCancellationLedger`. Optional zulaessig sind regular/variable Spezialisierungen oder ein schmaler Anschluss an den in r3c2 erzeugten ReRun-ProductCancellationLedger, falls downstream-Importe das benoetigen. Ein eigener grosser r4a-Ledger ist nicht noetig.
Beweisroute r4a	Keine Matrixrechnung. Keine neue Summenform. Keine neue TraceCancellation. Der Konstruktor befuellt `leftProduct_entry_eq_identity`, `rightProduct_entry_eq_identity`, `leftProduct_diagonal_entry`, `leftProduct_offdiag_entry`, `rightProduct_diagonal_entry` und `rightProduct_offdiag_entry` per Feldprojektion aus dem ProductCancellationLedger. Falls moeglich, wird der generische Konstruktor direkt in der vorhandenen `GeneratedInteriorTwoSidedInvFromSM3db7R`-Schicht ergaenzt, weil dort ProductCancellationLedger, ProductIdentityProof und `provenTwoSidedInvFromSM3db7R` bereits zusammenliegen. Eine neue Datei ist nur bei Importzyklus oder API-Stabilitaetsgrund erlaubt.
Verbotene Folgerung	r4a darf keinen `TwoSidedInv` bauen, `provenTwoSidedInvFromSM3db7R` nicht anwenden, keine `InteriorEliminationData` erzeugen, keine DtN-/MultiSchur-/Charpoly-Daten beruehren, keine Matrixprodukte erneut oeffnen, keine sechs Kancellationsfelder als neue freie Argumente annehmen und keinen neuen Trace-/Fold-/BlockFold-Ast erzeugen.
r5-Sperre	`SM3eR-r5` bleibt bis zum gruenen r4a-Build gesperrt. Erst nach einem theorematisch konstruierten `GeneratedInteriorSM3db7RProductIdentityProof` darf r5 den bereits vorhandenen `provenTwoSidedInvFromSM3db7R`-Konstruktor anwenden. SM3fR bleibt bis zu diesem konkreten `TwoSidedInv`-Witness blockiert.
Overhead-Regel v57	Keine neue Phase `r4a0/r4a1`, kein neues Audit-Universum und keine neue Status-Enum-Familie, solange der direkte Feldprojektionskonstruktor baut. Wenn ein Buildfehler auftritt, ist zuerst zu pruefen, ob nur die Platzierung wegen Importordnung falsch ist; dann wird maximal eine schmale Anschlussdatei eingefuehrt. Nur ein echter Typ- oder Feldmangel rechtfertigt eine neue Subphase.
Kompaktierungsfenster	`SM3AuditCompact0` bleibt vorgemerkt, aber weiterhin nicht aktiv. Der naechste produktive mathematische Fortschritt ist r4a, nicht Kompaktierung. Eine C-Phase wird erst nach r4a/r5 oder unmittelbar vor SM3fR sinnvoll, wenn die zu stabilisierende API feststeht.
Fortgeschriebene Reihenfolge v57	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` **active**; r5/SM3fR blocked until ProductIdentityProof und daraus `TwoSidedInv`.

0ah. Fortschreibungsentscheidung v58 nach gruenem SM3eR-r4a-Build: r5 aktiv, r4 nicht verdoppelt

Statusentscheidung	`SM3eR-r4a` ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v57-Rolle erfuellt: `GeneratedInteriorSM3db7RProductCancellationLedger` → `GeneratedInteriorSM3db7RProductIdentityProof` durch direkte Feldprojektion. Der naechste aktive Schritt ist `SM3eR-r5`. Eine zusaetzliche `SM3eR-r4`-Vollphase wird nicht eingefuehrt, weil sie nur denselben ProductIdentityProof-Konstruktor wiederholen wuerde.

Formaler r4a-Ertrag	Geschlossen sind insbesondere `GeneratedInteriorSM3db7RProductIdentityProof.fromProductCancellationLedger`, `productIdentityProof_from_productCancellationLedger`, `productIdentityProof_from_r3c2TraceCancellationLedger`, `regularProductIdentityProof_from_reRunProductCancellationLedger`, `variableProductIdentityProof_from_reRunProductCancellationLedger` sowie die Anschluesse aus dem r3c1-AccumulatedEntryCancellation-Ledger. Die Implementierung liegt in `GeneratedInteriorProductIdentityProofR4a.lean` und wird ueber `Arithmetic/BuildAll.lean` importiert.
Praeziser Restbefund	Der gruene r4a-Build beweist noch keinen `TwoSidedInv`. Er liefert nur den dafuer benoetigten `ProductIdentityProof`-Witness. Ebenfalls nicht erzeugt sind `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten sowie Charpoly-/Faktor-/Diskriminantendaten. Diese bleiben nachgelagert.
r4-Klassifikation	Die historische r4-Rolle war der theorematische Konstruktor von `GeneratedInteriorSM3db7RProductIdentityProof`. Diese Rolle ist durch r4a erledigt. Der Planstatus lautet daher: `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a, keine eigene Implementierungsphase; `SM3eR-r5` active.
r5-Freigabe	`SM3eR-r5` darf jetzt starten. Die Phase konsumiert ausschliesslich einen `Q : GeneratedInteriorSM3db7RProductIdentityProof ...` aus r4a und verwendet den bereits vorhandenen Konstruktor `provenTwoSidedInvFromSM3db7RQ`, um `TwoSidedInv (generatedInteriorBlock W) H.candidateMatrix` zu erzeugen.
r5-DoD	Der DoD ist ein einzelner schmaler Konstruktorpfad, z.B. `twoSidedInv_from_productIdentityProofR4a` bzw. `twoSidedInv_from_productCancellationLedger_via_r4a`, plus nur falls fuer downstream benoetigt regular/variable Spezialisierungen. Er darf die vorhandenen Matrix-Identitaetssaetze aus `GeneratedInteriorTwoSidedInvFromSM3db7R` konsumieren, aber keine neue Produktrechnung oeffnen.
Beweisroute r5	Keine neue Kancellation, keine neue Summenform, kein neuer Trace-/Fold-/BlockFold-Ast. Der Beweis ist ein Consumer: r4a liefert den ProductIdentityProof; `provenTwoSidedInvFromSM3db7R` wandelt dessen linke und rechte Matrixidentitaet in die beiden Felder von `TwoSidedInv` um. Falls die Importordnung es erlaubt, kann r5 als wenige Definitionen in derselben r4a-Anschlussdatei stehen; andernfalls ist genau eine kleine Datei `GeneratedInteriorProvenTwoSidedInvR5.lean` zulaessig.
Verbotene Folgerung	r5 darf keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten, keine Charpoly-/Faktor-/Diskriminantendaten, kein `Matrix.inv`, keine freie Inversenannahme, keine neue CandidateEntry-Formel und keine neue ProductCancellation- oder ProductIdentity-Quelle erzeugen. r5 ist nur ProductIdentityProof → TwoSidedInv.
SM3fR-Sperre	`SM3fR` bleibt bis zum gruenen r5-Build blockiert. Erst wenn r5 einen konkreten `TwoSidedInv`-Witness aus dem r4a-ProductIdentityProof liefert, darf SM3fR mit `InteriorEliminationData` beginnen.
Overhead-Regel v58	Keine neue Phase `r5a/r5b`, kein neues Audit-Universum, keine neue Status-Enum-Familie. Wenn r5 scheitert, ist zuerst die konkrete Import-/Namensbruecke zwischen r4a und `provenTwoSidedInvFromSM3db7R` zu pruefen. Nur ein echter fehlender Typ- oder Feldanschluss rechtfertigt eine minimale Anschlussphase; keine Rueckkehr zu Produktrechnung.
Fortgeschriebene Reihenfolge v58	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` **active**; `SM3fR` blocked until concrete `TwoSidedInv`.

0ai. Fortschreibungsentscheidung v59 nach gruenem SM3eR-r5-Build: SM3fR aktiv

Statusentscheidung	`SM3eR-r5` ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v58-Rolle erfuellt: der r4a-`ProductIdentityProof` wurde ohne neue Produktrechnung durch `provenTwoSidedInvFromSM3db7R` in `TwoSidedInv (generatedInteriorBlock W) H.candidateMatrix` ueberfuehrt. Der naechste aktive Schritt ist `SM3fR`. Eine zusaetzliche r5a-/r5b-Unterteilung wird nicht eingefuehrt.
Formaler r5-Ertrag	Geschlossen sind insbesondere `twoSidedInv_from_productIdentityProofR4a`, `twoSidedInv_from_productCancellationLedger_via_r4a`, `twoSidedInv_from_r3c2TraceCancellationLedger_via_r4a`, `regularTwoSidedInv_from_reRunProductCancellationLedger`, `variableTwoSidedInv_from_reRunProductCancellationLedger`, `regularTwoSidedInv_from_accumulatedEntryCancellationLedger` und `variableTwoSidedInv_from_accumulatedEntryCancellationLedger`. Die Implementierung liegt in `GeneratedInteriorProvenTwoSidedInvR5.lean` und wird ueber `Arithmetic/BuildAll.lean` importiert.
Praeziser Restbefund	Der gruene r5-Build erzeugt genau den Inversen-Witness, aber noch keine `InteriorEliminationData`, kein DtN-Objekt, kein `GeneratedDtN`, kein `GeneratedGeneralizedDtN`, kein `MultiSchur`, keine LevelHistory-Matrix und keine Charpoly-/Faktor-/Diskriminantendaten. Der Witness darf auch nicht nachtraeglich als freie Inversenformel oder als externes Certificate gelesen werden: er ist das Ergebnis der geschlossenen SM3db7R–SM3eR-Produktkette.
SM3fR-Freigabe	`SM3fR` ist jetzt aktiv. Die Phase konsumiert ausschliesslich den in r5 gebauten `TwoSidedInv`-Witness, die SM3aR-Partition, die SM3bR-/SM3cR-Dirichlet-/InteriorBlock-Provenienz und die SM3db7R-Handoff-Kandidatenmatrix. Sie darf daraus die minimalen Interior-Elimination-/Certificate-Daten fuer regular und variable generated Quellen bauen.

SM3fR-DoD	Minimaler DoD ist kein neues Audit-Universum, sondern ein schmaler Builderpfad: `H.candidateMatrix` wird als abgeleitete Interior-Inversenmatrix registriert; daraus werden `interiorSolve`, `boundaryOperator`, `interiorSolve_spec` und `boundaryOperator_spec` bzw. die aequivalenten generated Certificate-Felder konstruiert. Falls die vorhandene oeffentliche `InteriorEliminationData`-Klasse konsumiert wird, muss vorher die Typbruecke `generatedInteriorBlock W = D.interiorBlock` fuer das konkrete generated Dirichlet-Objekt theorematisch oder definitorisch geschlossen sein.
Beweisroute SM3fR	Keine erneute Matrixproduktrechnung, keine Kancellation, keine Summen-/Trace-/Fold-Induktion, kein `Matrix.inv`, keine freie Inversenannahme und kein `infer_instance` als versteckter Certificate-Import. Die Beweise sind Feld-/Konstruktorverbrauch aus r5-`TwoSidedInv` plus die bereits vorhandenen Dirichlet-Block-Gleichungen. Falls ein notwendiger Konstruktor im DtN-Kern privat oder falsch platziert ist, ist hoechstens eine neutrale, kleine Public-Bridge zulaessig; keine neue Status-Enum-Familie.
Verbotene Folgerung	`SM3fR` darf noch kein `DtNStrong`, kein `GeneralizedDtNStrong`, kein `MultiSchurStrong`, keine Charpoly-/Faktor-/Diskriminantendaten und keine ArithmeticSource erzeugen. Diese Consumer bleiben `SM3gR`, `SM3hR`, `SM3iR`, `SM4` und spaeteren Phasen vorbehalten.
Overhead-Regel v59	Aktive Phase ist `SM3fR`, nicht eine neue Kette `SM3fR0/SM3fR1/...`. Eine interne Mini-Bruecke ist nur erlaubt, wenn der direkte Builder wegen einer realen Typ-/Importgrenze nicht kompiliert. Dann muss sie dieselbe Phase tragen und darf nur die API-Oeffnung leisten, nicht neue mathematische Substanz simulieren.
Fortgeschriebene Reihenfolge v59	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` **active**; `SM3gR` blocked until concrete InteriorEliminationData from SM3fR.

0aj. Fortschreibungsentscheidung v60 nach bestaetigtem gruenem SM3eR-r5-Build: SM3fR bleibt aktive Minimalphase

Statusentscheidung	Der jetzt bestaetigte gruene r5-Build schliesst keine neue mathematische Phase auf, sondern bestaetigt die v59-Entscheidung. `SM3eR-r5` bleibt `closed/green`; der konkrete `TwoSidedInv (generatedInteriorBlock W) H.candidateMatrix`-Witness ist der einzige neue konsumierbare Ertrag. Die naechste aktive Phase ist weiterhin `SM3fR`. Eine Zwischenphase `SM3fR0`, `SM3fR-audit` oder `SM3fR-pre` wird nicht eingefuehrt.
Aktiver Auftrag	`SM3fR` muss aus dem r5-`TwoSidedInv`-Witness die minimalen generated Interior-Elimination-/Certificate-Daten bauen. Der direkte Zielname darf z.B. `GeneratedInteriorEliminationDataSM3fR.lean` oder `GeneratedInteriorEliminationCertificateSM3fR.lean` sein; es soll aber nur eine Implementierungsschicht entstehen. Der Pfad ist: `H.candidateMatrix` als abgeleitete Interior-Inversenmatrix registrieren, daraus `interiorSolve`, `boundaryOperator`, `interiorSolve_spec` und `boundaryOperator_spec` bzw. die generated-aequivalenten Certificate-Felder konstruieren.
Zulaessige Inputs	Erlaubt sind ausschliesslich: der r5-`TwoSidedInv`-Witness, die SM3aR-Partition, die SM3bR-generated Dirichlet-Entry-/Matrix-Provenienz, die SM3cR-`generatedInteriorBlock`-Provenienz, die SM3db7R-Handoff-Matrix `H.candidateMatrix` und bereits vorhandene regular/variable Handoff-Spezialisierungen. Kein anderer Inversen- oder Eliminationswitness darf importiert werden.
Oeffentliche Klasse vs. generated Certificate	Der erste positive Versuch ist ein generated-lokales Certificate, falls noch kein oeffentliches generated `DirichletLaplacianStrong`-Objekt mit definitorisch passendem `interiorBlock` vorhanden ist. Die oeffentliche Klasse `InteriorEliminationData D` darf nur dann direkt instanziiert werden, wenn die Bruecke `generatedInteriorBlock W = D.interiorBlock` fuer das konkrete generated Dirichlet-Objekt bereits definitorisch oder mit einem kleinen theorematischen Gleichheitslemma geschlossen ist. Diese Bruecke gehoert noch zu `SM3fR`; sie rechtfertigt keine neue Phase.
Beweisroute	Die `interiorSolve_spec`-Seite soll direkt aus `TwoSidedInv.left_inv` bzw. aus der passenden Matrix-mulVec-Kompatibilitaet folgen. Die `boundaryOperator_spec`-Seite soll nur die definierte Schur-Form `boundaryBlock - boundaryInteriorBlock * H.candidateMatrix * interiorBoundaryBlock` auswerten. Falls die generated Boundary-/Interior-Block-Restriktionen noch nicht als Namen exponiert sind, werden sie in derselben SM3fR-Datei als reine Restriktionen der SM3bR-Dirichlet-Matrix definiert und per `rfl`/kleinen Feldgleichungen angebunden.
Verbotene Abkuerzungen	Kein `Matrix.inv`, kein `inv`, kein `noncomputable`, kein `classical`, kein `field_simp`, keine freie Inversenmatrix, kein `infer_instance` als versteckter `InteriorEliminationData`-Import, keine erneute Produktrechnung, keine Kancellation, keine Trace-/Fold-/BlockFold-Induktion, keine neue CandidateEntry-Formel und keine Status-Enum-Familie. Simplifizierung nur am letzten Meter fuer definitorische Ziele.
Grenze von SM3fR	`SM3fR` erzeugt noch kein `DtNStrong`, kein `GeneratedDtN`, kein `GeneratedGeneralizedDtN`, kein `MultiSchurStrong`, keine LevelHistory-Matrix, keine ArithmeticSource und keine Charpoly-/Faktor-/Diskriminantendaten. Diese bleiben fuer `SM3gR` und nachfolgende Phasen blockiert.
SM3gR-Gate	`SM3gR` wird erst freigegeben, wenn `SM3fR` mindestens ein generated-lokales Interior-Elimination-Certificate mit `interiorSolve_spec` und `boundaryOperator_spec` liefert oder, noch besser, eine saubere `InteriorEliminationData`-Instanz fuer ein konkretes generated `DirichletLaplacianStrong`-Objekt. Ohne diese Daten darf keine DtN-Phase starten.

Overhead-Regel v60	Die naechste Implementierung bleibt `SM3fR`. Nur ein realer Typ- oder Sichtbarkeitsblocker – z.B. ein privater DtN-Hilfskonstruktor oder eine fehlende generated Block-Restriktion – darf durch eine kleine Public-/Restriction-Bridge innerhalb derselben Phase beantwortet werden. Keine neue Planunterphase, solange der direkte Builderpfad syntaktisch formulierbar ist.
Fortgeschriebene Reihenfolge v60	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` **active**; `SM3gR` blocked until SM3fR-InteriorElimination-Certificate/Data.

0ak. Fortschreibungsentscheidung v61 nach gruenem SM3fR-Build: SM3gR aktiv

Statusentscheidung	`SM3fR` ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v60-Rolle erfuellt: aus dem r5-`TwoSidedInv (generatedInteriorBlock W) H.candidateMatrix`-Witness und den generated Dirichlet-/Block-Restriktionen wurde ein generated-lokales Interior-Elimination-Certificate gebaut. Das v60-Gate fuer `SM3gR` ist damit erfuellt. Die naechste aktive Phase ist `SM3gR`. Eine Zwischenphase `SM3fR-public`, `SM3gR0` oder `SM3gR-audit` wird nicht eingefuehrt.
Formaler SM3fR-Ertrag	Geschlossen sind insbesondere `generatedBoundaryBlockSM3fR`, `generatedBoundaryInteriorBlockSM3fR`, `generatedInteriorBoundaryBlockSM3fR`, `generatedInteriorInverseMatrixSM3fR`, `generatedInteriorSolveSM3fR`, `generatedBoundaryOperatorSM3fR`, `generatedInteriorSolve_spec_from_twoSidedInvSM3fR`, `generatedBoundaryOperator_specSM3fR`, `GeneratedInteriorEliminationCertificateSM3fR.fromTwoSidedInv` sowie die regular/variable Anschlusskonstruktoren aus r3c2-, ProductCancellation-, ProductIdentity- und AccumulatedEntryCancellation-Ledgern. Die Implementierung liegt in `GeneratedInteriorEliminationCertificateSM3fR.lean` und wird ueber `Arithmetic/BuildAll.lean` importiert.
Praeziser Restbefund	SM3fR erzeugt bewusst noch kein oeffentliches `DtNStrong`, kein `GeneratedGeneralizedDtN`, kein `MultiSchurStrong`, keine LevelHistory-Matrix, keine `ArithmeticSource` und keine Charpoly-/Faktor-/Diskriminantendaten. Ebenfalls ist eine oeffentliche `InteriorEliminationData D`-Instanz fuer ein konkretes `DirichletLaplacianStrong`-Objekt nicht als zwingender Ertrag zu lesen. Das ist kein Blocker: v60 erlaubt als SM3gR-Gate ausdruecklich das generated-lokale Certificate mit `interiorSolve_spec` und `boundaryOperator_spec`.
SM3gR-Klassifikation	`SM3gR` ist eine schmale H-/Builder-Phase mit rechnerischem Output, keine neue Beweisquellen- oder Audit-Phase. Sie konsumiert das SM3fR-Certificate und macht daraus die finite generated top-boundary DtN-Oberflaeche. Der bevorzugte Zielname ist eine einzelne Datei wie `GeneratedTopBoundaryDtNSM3gR.lean` oder `GeneratedDtNSM3gR.lean`. Maximal ein knappes Certificate/Export-Struct ist zulaessig; keine neue Status-Enum-Familie.
Aktiver Auftrag SM3gR	Aus einem `C : GeneratedInteriorEliminationCertificateSM3fR ... H` wird die DtN-Matrix als `C.boundaryOperator` exportiert, zusammen mit der Auswertungsfunktion `fun phi => C.boundaryOperator.mulVec phi`. Die Kernsaetze sind die Rueckbindung an die Schur-Form ueber `C.boundaryOperator_eq_schur` und die Flux-/Boundary-Spezifikation ueber `C.boundaryOperator_spec`. Regular- und variable Spezialisierungen duerfen direkt aus den bereits in SM3fR vorhandenen regular/variable Certificate-Konstruktoren gebaut werden, sofern dies ohne neue Adapterkette moeglich ist.
SM3gR-DoD	Der DoD ist erfuellt, wenn ein generated top-boundary DtN-Certificate bzw. ein aequivalenter `GeneratedDtN`-Builder existiert mit: `sourceCertificate`, `boundaryOperator`, `boundaryOperator_eq_SM3fR`, `boundaryOperator_eq_schur`, `apply`, `apply_eq_mulVec` und `boundaryFlux_spec`. Zulaessig ist ein direkter public-`DtNStrong`-Konstruktor nur dann, wenn die konkrete generated `DirichletLaplacianStrong`-Bruecke und die Blockgleichheiten definitorisch oder mit kleinen Gleichheitslemmata schliessen. Andernfalls bleibt SM3gR generated-lokal; das ist der positive Pfad und kein Fehlschlag.
Beweisroute SM3gR	Keine erneute Interior-Inversenrechnung, keine neue Produktvalidierung, keine Kancellation, keine Trace-/Fold-/BlockFold-Induktion und kein Rueckgriff auf `Matrix.inv`. Die Beweise sollen Feldprojektionen aus dem SM3fR-Certificate, `rfl`-Rueckbindungen und kleine `calc`/`rw`-Schritte sein. Die Schur-Form darf nicht neu hergeleitet werden, sondern wird aus dem bereits gruenen `boundaryOperator_eq_schur`-Feld gelesen.
Verbotene Folgerung	`SM3gR` darf noch kein `GeneratedGeneralizedDtN`, kein `GeneralizedDtNStrong`, kein `MultiSchurStrong`, keine LevelHistory-Matrix, keine `ArithmeticSource`, keine Charpoly-/Faktor-/Diskriminantendaten und keine L2/L3-Zielvergleiche erzeugen. Diese Consumer bleiben `SM3hR`, `SM3iR`, `SM4` und spaeteren Phasen vorbehalten.
Overhead-Regel v61	Keine neue Kette `SM3gR0/SM3gR1/...`. Falls der direkte public-`DtNStrong`-Weg an einer Typbruecke zum alten `DirichletLaplacianStrong`-API scheitert, wird nicht sofort eine neue Phase eroeffnet. SM3gR schliesst dann generated-lokal mit dem top-boundary DtN-Certificate. Eine spaetere public-Bridge ist nur zulaessig, wenn `SM3hR` oder `SM4` sie real konsumieren muss.
Fortgeschriebene Reihenfolge v61	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` **active**; `SM3hR` blocked until SM3gR-generated top-boundary DtN.

0al. Fortschreibungsentscheidung v62 nach gruenem SM3gR-Build: SM3hR aktiv

Statusentscheidung	`SM3gR` ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v61-Rolle erfuellt: aus einem `C : GeneratedInteriorEliminationCertificateSM3fR ... H` wurde eine generated-lokale finite top-boundary-DtN-Oberflaeche gebaut. Das v61-Gate fuer `SM3hR` ist damit erfuellt. Die naechste aktive Phase ist `SM3hR`. Eine Zwischenphase `SM3gR-public`, `SM3hR0` oder `SM3hR-audit` wird nicht eingefuehrt.
Formaler SM3gR-Ertrag	Geschlossen sind insbesondere `generatedTopBoundaryOperatorSM3gR`, `generatedTopBoundaryDtNApplySM3gR`, `GeneratedTopBoundaryDtNSM3gR`, `GeneratedTopBoundaryDtNSM3gR.fromCertificate`, `generatedTopBoundaryDtN_from_-certificateSM3gR`, `generatedTopBoundaryOperator_eq_SM3fR`, `generatedTopBoundaryApply_-eq_mulVecSM3gR` sowie die regular/variable Anschlusskonstruktoren aus den vorhandenen SM3fR-Certificate-Buildern. Die Implementierung liegt in `GeneratedTopBoundaryDtNSM3gR.lean` und wird ueber `Arithmetic/BuildAll.lean` importiert.
Praeziser Restbefund	`SM3gR` erzeugt bewusst noch kein oeffentliches `DtNStrong`, kein `GeneralizedDtNStrong`, kein `MultiSchurStrong`, keine LevelHistory-Matrix, keine `ArithmeticSource` und keine Charpoly-/Faktor-/Diskriminantendaten. Das ist kein Blocker: v61 erlaubte ausdruecklich den generated-lokalen Abschluss, falls eine public-`DtNStrong`-Bruecke zum alten `DirichletLaplacianStrong`-API nicht ohne Zusatzkette benoetigt wird.
SM3hR-Klassifikation	`SM3hR` ist eine schmale Generalized-Builder-Phase, keine neue Beweisquellen- oder Audit-Phase. Sie konsumiert den `SM3gR`-top-boundary-DtN und die bereits vorhandenen Hauptpfad-/Sektor-Provenienzdaten. Inhaltlich ist SM3hR zunaechst ein Generalized-Wrapper mit Provenienzgleichheiten, nicht eine neue DtN-Berechnung und nicht der Start von MultiSchur. Der bevorzugte Zielname ist eine einzelne Datei wie `GeneratedGeneralizedTopBoundaryDtNSM3hR.lean` oder `GeneratedGeneralizedDtNSM3hR.lean`.
Aktiver Auftrag SM3hR	Aus einem `G : GeneratedTopBoundaryDtNSM3gR ...` wird eine generated-lokale Generalized-DtN-Oberflaeche gebaut, deren raw/top-boundary-Teil definitionell oder theorematisch auf `G.boundaryOperator` und `G.apply` zurueckgeht. Minimal werden `sourceTopBoundaryDtN`, `boundaryOperator`, `apply`, `boundaryOperator_eq_SM3gR`, `apply_eq_SM3gR`, `source_eq_-generatedMainPath` bzw. aequivalente Hauptpfad-Provenienz und regular/variable Konstruktoren exponiert. Ein public-`GeneralizedDtNStrong`-Konstruktor ist nur zulaessig, wenn die benoetigten `DtNStrong`/`SectorSplitStrong`-Bruecken bereits ohne neue freie Daten und ohne Importbruch schliessen.
SM3hR-DoD	Der DoD ist erfuellt, wenn ein generated Generalized-DtN-Certificate bzw. aequivalenter Builder existiert, der seine DtN-Komponente ausschliesslich aus `SM3gR` bezieht und seine Hauptpfad-/Sektor-Provenienz aus vorhandenen generated Daten rueckbindet. Zulaessige Kernfelder sind `sourceTopBoundaryDtN`, `boundaryOperator`, `apply`, `boundaryOperator_eq_SM3gR`, `apply_eq_SM3gR`, `mainPathProvenance`, `noExternalDtNStatus` und regular/variable Builder. Nicht erforderlich ist ein public-`GeneralizedDtNStrong`, wenn dafuer erst ein neues public-`DtNStrong`-Universum konstruiert werden muesste.
Beweisroute SM3hR	Keine erneute Schur-Formel, keine Interior-Inversenrechnung, keine Produktvalidierung, keine Trace-/Fold-/BlockFold-Induktion und kein Rueckgriff auf `Matrix.inv`. Die Beweise sollen Feldprojektionen aus `SM3gR`, `rfl`-Rueckbindungen und kleine `calc`/`rw`-Schritte sein. Die Flux- und Schur-Korrektheit wird nicht neu bewiesen, sondern ueber die SM3gR-Felder transportiert.
Verbotene Folgerung	`SM3hR` darf noch kein `MultiSchurStrong`, keine SM3iR-Aggregation, keine LevelHistory-Matrix, keine `ArithmeticSource`, keine Charpoly-/Faktor-/Diskriminantendaten und keine L2/L3-Zielvergleiche erzeugen. Es darf ausserdem keine externen Interior-Certificates, keine freie DtN-Matrix, keine freie Sektor-Tabelle und keine alten SM1–SM3d-DtN-Objekte als Ersatz konsumieren.
Overhead-Regel v62	Genau eine kleine SM3hR-Datei plus `BuildAll`-Import ist der Zielkorridor. Keine neue Status-Enum-Familie, keine Audit-Unterphase, keine public-Bridge-Phase vorab. Falls die public-`GeneralizedDtNStrong`-Route an vorhandenen Typbruecken scheitert, schliesst SM3hR generated-lokal; eine public-Bridge wird erst dann geplant, wenn `SM3iR` oder `SM4` sie real konsumieren muss.
Fortgeschriebene Reihenfolge v62	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` **active**; `SM3iR` blocked until SM3hR-generated Generalized-DtN.

0am. Fortschreibungsentscheidung v63 nach gruenem SM3hR-Build: SM3iR aktiv

Statusentscheidung	SM3hR ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v62-Rolle erfuellt: aus dem SM3gR-top-boundary-DtN wurde eine generated-lokale Generalized-DtN-Oberflaeche gebaut und an die vorhandene Hauptpfad-Provenienz rueckgebunden. Das v62-Gate fuer SM3iR ist damit erfuellt. Die naechste aktive Phase ist SM3iR. Eine Zwischenphase SM3hR-public, SM3iR0 oder SM3iR-audit wird nicht eingefuehrt.
Formaler SM3hR-Ertrag	Geschlossen sind insbesondere GeneratedGeneralizedTopBoundaryDtNSM3hR, generatedGeneralizedTopBoundaryOperatorSM3hR, generatedGeneralizedTopBoundaryApplySM3hR, GeneratedGeneralizedTopBoundaryDtNSM3hR.fromTopBoundaryDtN, generatedGeneralizedTopBoundaryDtN_from_SM3gR, generatedGeneralizedTopBoundaryOperator_eq_SM3gR, generatedGeneralizedTopBoundaryApply_eq_SM3gR, boundaryOperator_from_SM3gR, boundaryOperator_from_schur, apply_from_SM3gR, apply_from_mulVec, boundaryFlux_from_SM3gR, noExternalDtN_from_SM3hR, noMultiSchur_from_SM3hR und noArithmeticTarget_from_SM3hR. Regular/variable Anschlusskonstruktoren existieren aus ProductCancellation- und AccumulatedEntryCancellation-Ledgern. Die Implementierung liegt in GeneratedGeneralizedTopBoundaryDtNSM3hR.lean und wird ueber Arithmetic/BuildAll.lean sowie Arithmetic/Notation.lean importiert.
Praeziser Restbefund	SM3hR erzeugt bewusst noch kein MultiSchurStrong, keine ArithmeticSource, keine LevelHistory-Matrix und keine Charpoly-/Faktor-/Diskriminantendaten. Ein public-GeneralizedDtNStrong ist nicht als zwingender Ertrag zu lesen. Das ist kein Blocker: v62 erlaubte ausdruecklich den generated-lokalen Abschluss, falls eine public-Bridge erst dann sinnvoll wird, wenn SM3iR oder SM4 sie real konsumieren muss.
SM3iR-Klassifikation	SM3iR ist keine neue mathematische Beweisquelle, sondern ein Reset-Ledger und Konsumfilter. Die Phase aggregiert die neu gebauten SM3aR–SM3hR-Daten in einem einzigen, gut sichtbaren Uebergabeobjekt fuer SM4. Sie soll verhindern, dass SM4 direkt eine lange Import- und Provenienzkette aus SM3aR, SM3dbR, SM3eR, SM3fR, SM3gR und SM3hR konsumieren muss. Der bevorzugte Zielname ist eine einzelne Datei wie GeneratedResetSM3LedgerSM3iR.lean oder GeneratedSM3ResetLedgerSM3iR.lean.
Aktiver Auftrag SM3iR	Baue ein schmales Ledger, das mindestens die sichtbaren Quellen fuer partition, Dirichlet-/Interior-Block-Profil, proven TwoSidedInv, SM3fR-InteriorElimination-Certificate, SM3gR-top-boundary-DtN und SM3hR-Generalized-DtN zusammenfuehrt. Zulaessige Kernfelder sind beispielsweise sourcePartition, sourceDirichletEntrySource, sourceInteriorBlockProfile, sourceTwoSidedInv, sourceInteriorEliminationCertificate, sourceTopBoundaryDtN, sourceGeneralizedTopBoundaryDtN, generalized_boundaryOperator_eq_topBoundary, topBoundary_boundaryOperator_eq_certificate, certificate_inverseMatrix_eq_handoffCandidate und regular/variable Ledger-Konstruktoren. Namen duerfen abweichen, solange diese Provenienzfunktion erfuellt ist.
SM3iR-DoD	Der DoD ist erfuellt, wenn SM4 anschliessend regular/variable GeneratedMultiSchur-Builder ausschliesslich aus dem SM3iR-Ledger konsumieren kann. Das Ledger muss die Herkunft der DtN-/Generalized-DtN-Komponente aus SM3gR/SM3hR und die Herkunft der Interior-Elimination aus SM3fR/r5 explizit tragen. Es darf keine externe Certificate-Typklasse, keine freie DtN-Matrix, keine freie Generalized-DtN-Struktur und keine alte SM1–SM3d-Quelle als Ersatz importieren.
Beweisroute SM3iR	Keine neue Schur-Formel, keine neue Interior-Inversenrechnung, keine Produktvalidierung, keine Trace-/Fold-/BlockFold-Induktion und kein Rueckgriff auf Matrix.inv. Die Beweise sollen Feldprojektionen aus SM3fR, SM3gR und SM3hR, rfl-Rueckbindungen und kleine calc/rw-Schritte sein. Falls ein Gleichheitslemma fehlt, wird es als lokale Provenienzgleichheit des Ledgers formuliert, nicht als neue Rechenphase.
Verbotene Folgerung	SM3iR darf noch kein MultiSchurStrong, keine LevelHistory-Matrix, keine ArithmeticSource, keine Charpoly-/Faktor-/Diskriminantendaten und keine L2/L3-Zielvergleiche erzeugen. Es darf ausserdem keine neuen public-DtNStrong/GeneralizedDtNStrong-Bruecken erzwingen, solange SM4 mit dem generated-lokalen Ledger arbeiten kann.
Overhead-Regel v63	Genau eine kleine SM3iR-Datei plus BuildAll- und, falls noetig, Notation-Import ist der Zielkorridor. Keine neue Status-Enum-Familie, keine Audit-Unterphase, keine public-Bridge-Vorphase und keine zweite Ledger-Kette. Wenn SM4 spaeter eine public-Bridge wirklich braucht, wird diese erst dann als realer Consumer-Bedarf geplant.
Fortgeschriebene Reihenfolge v63	r3c1a closed/green; r3c1b0 closed/green; SM3db2aR-SM3db2fR closed/green bzw. closed/green-audit; SM3eR-r3c1b1 closed/green; SM3eR-r3c1b-full closed/green; SM3eR-r3c1c closed/green; SM3eR-r3c1d green/gate-closed; SM3db3aR closed/green-audit; SM3db4aT/U/V/W closed/green; SM3eR-r3c1d-full closed/green; SM3eR-r3c1e closed/green; SM3eR-r3c2 closed/green; SM3eR-r4a closed/green; SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 closed/green; SM3fR closed/green; SM3gR closed/green; SM3hR closed/green; SM3iR **active**; SM4 blocked until SM3iR-Reset-Ledger.

0an. Fortschreibungsentscheidung v64 nach gruenem SM3iR-Build: SM4 aktiv

Statusentscheidung	SM3iR ist nach lokal gruenem Build formal geschlossen. Die Phase hat exakt die v63-Rolle erfuellt: die SM3aR–SM3hR-Kette wird jetzt in einem Reset-Ledger als einziger SM4-Konsumfilter gesammelt. Das v63-Gate fuer SM4 ist damit erfuellt. Die naechste aktive Phase ist SM4. Eine Zwischenphase SM4a, SM4-audit oder SM4-publicBridge wird nicht eingefuehrt.

Formaler SM3iR-Ertrag	Geschlossen sind insbesondere `GeneratedResetSM3LedgerSM3iR`, `GeneratedResetSM3LedgerSM3iR.fromGeneralizedTopBoundaryDtN`, `regularGeneratedResetSM3Ledger_from_accumulatedEntryCancellationLedgerSM3iR`, `variableGeneratedResetSM3Ledger_from_accumulatedEntryCancellationLedgerSM3iR`, `SM3iRGeneratedResetLedger`, `sm3iRGeneratedResetLedger`, `generalized_boundaryOperator_from_topBoundary`, `topBoundary_boundaryOperator_from_certificate`, `certificate_inverseMatrix_from_handoffCandidate`, `sm3iRGeneratedResetLedger_regular_certificate_from_SM3fR` und `sm3iRGeneratedResetLedger_variable_certificate_from_SM3fR`. Die Implementierung liegt in `GeneratedResetSM3LedgerSM3iR.lean` und wird ueber `Arithmetic/BuildAll.lean` sowie `Arithmetic/Notation.lean` importiert.
Praeziser Restbefund	`SM3iR` erzeugt bewusst noch kein `MultiSchurStrong`, keine `ArithmeticSource`, keine LevelHistory-Matrix und keine Charpoly-/Faktor-/Diskriminantendaten. Der SM3hR-Anteil ist weiterhin eine generated-lokale Generalized-DtN-Oberflaeche ohne public-`SectorSplitStrong`; deshalb darf SM4 nicht mit einer erzwungenen public-Bridge starten. Der korrekte positive Pfad ist zuerst eine generated-lokale MultiSchur-Oberflaeche, die nur SM3iR-Daten konsumiert.
SM4-Klassifikation	`SM4` ist der erste echte Consumer des Reset-SM3-Ledgers. Die Phase baut den generated-lokalen MultiSchur-Builder fuer regular und variable Quellen. Sie ist keine ArithmeticSource-Phase, kein LevelHistory-Start, kein public-`MultiSchurStrong`-Beweis und kein arithmetisches Zielobjekt. Falls spaeter ein public-`MultiSchurStrong` noetig wird, muss der Bedarf aus SM5 oder einem spaeteren echten Consumer folgen, nicht aus einer vorgezogenen Bridge-Pflicht.
Aktiver Auftrag SM4	Baue genau eine schmale Datei wie `GeneratedMultiSchurSM4.lean`. Sie importiert vorzugsweise nur `GeneratedResetSM3LedgerSM3iR`. Zulaessige Kernfelder sind `sourceResetLedger`, `sourceGeneralizedTopBoundaryDtN`, `sourceTopBoundaryDtN`, `sourceInteriorEliminationCertificate`, `boundaryOperator`, `apply`, `boundaryOperator_eq_SM3iR`, `apply_eq_SM3iR`, `apply_eq_mulVec`, `boundaryOperator_eq_schur`, `certificate_inverseMatrix_eq_handoffCandidate`, `noExternalDtN_from_SM3iR`, `noPublicMultiSchurStrongStatus` und regular/variable Konstruktoren wie `regularGeneratedMultiSchur_from_SM3iRSM4` und `variableGeneratedMultiSchur_from_SM3iRSM4`. Namen duerfen abweichen, solange der Konsum ausschliesslich ueber SM3iR laeuft.
SM4-DoD	Der DoD ist erfuellt, wenn regular und variable `GeneratedMultiSchurSM4`-Builder aus einem `SM3iRGeneratedResetLedger` entstehen, die Boundary-Operator-/Apply-/Schur-Rueckbindungen aus SM3iR tragen und die Interior-Elimination-Provenienz auf SM3fR/r5 sowie die Generalized-DtN-Provenienz auf SM3gR/SM3hR zurueckbinden. SM4 muss fuer SM5 eine einzige konsumierbare generated-MultiSchur-Oberflaeche bereitstellen, ohne alte SM1–SM3d-Quellen, externe Certificate-Typklassen, freie DtN-/Generalized-DtN-Strukturen oder freie Matrixdaten einzufuehren.
Beweisroute SM4	Keine neue Schur-Formel, keine neue Interior-Inversenrechnung, keine Interface-Eliminationsrechnung, keine Produktvalidierung und kein Rueckgriff auf `Matrix.inv`. Die Beweise sollen Feldprojektionen aus dem SM3iR-Ledger, Rueckbindungen auf SM3gR/SM3hR und kleine `rfl`/`rw`/`calc`-Schritte sein. `apply_eq_mulVec` wird von SM3hR/SM3iR geerbt; `boundaryOperator_eq_schur` wird ueber das SM3fR-Certificate aus SM3iR geerbt.
Verbotene Folgerung	`SM4` darf noch keine `ArithmeticSource`, keine LevelHistory-Matrix, keine `generatedSchurEntry`-History-Matrix, keine Charpoly-/Faktor-/Diskriminantendaten und keine L2/L3-Zielvergleiche erzeugen. Es darf kein public-`MultiSchurStrong`, kein public-`GeneralizedDtNStrong`, kein `InterfaceEliminationData`-Typklassenpaket und keine `SectorSplitStrong`-Bruecke erzwingen, solange die generated-lokale SM4-Oberflaeche den SM5-Consumer ausreichend speisen kann.
Overhead-Regel v64	Genau eine kleine SM4-Datei plus `BuildAll`- und, falls noetig, `Notation`-Import ist der Zielkorridor. Keine SM4a/SM4b-Kette, keine Audit-Unterphase, keine zweite Ledger-Kette, kein public-Bridge-Preflight. Wenn beim eigentlichen SM4-Bau ein konkretes Gleichheitslemma fehlt, wird es lokal als SM3iR/SM4-Provenienzprojektion formuliert, nicht als neue Rechenphase.
Fortgeschriebene Reihenfolge v64	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` **active**; `SM5` blocked until generated-local SM4-MultiSchur surface.

0ao. Fortschreibungsentscheidung v65 nach gruenem SM4-Build: SM5 aktiv

Statusentscheidung	`SM4` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die in v64 geforderte generated-lokale MultiSchur-Oberflaeche geliefert: regular und variable Builder entstehen aus dem `SM3iR`-Reset-Ledger, die Boundary-Operator-/Apply-/Schur-Provenienz bleibt auf SM3fR, SM3gR und SM3hR zurueckgebunden, und es wurden keine public-`MultiSchurStrong`-Bruecke, keine `ArithmeticSource`, keine LevelHistory-Matrix und keine Charpoly-/Faktor-/Diskriminantendaten erzeugt. Damit ist das v64-Gate fuer `SM5` erfuellt.
Formaler SM4-Ertrag	Geschlossen sind insbesondere `GeneratedMultiSchurSM4`, `generatedMultiSchur_from_SM3iRSM4`, `regularGeneratedMultiSchur_from_SM3iRSM4`, `variableGeneratedMultiSchur_from_SM3iRSM4`, `SM4GeneratedMultiSchurLedger`, `sm4GeneratedMultiSchurLedger` und `sm4GeneratedMultiSchurLedger_from_accumulatedEntryCancellationLedger`. Der Ertrag ist eine konsumierbare generated-MultiSchur-Oberflaeche mit `boundaryOperator`, `apply`, `apply_eq_mulVec`, `boundaryOperator_eq_schur` und Herkunftsfeldern aus SM3iR/SM3fR/SM3gR/SM3hR.

Praeziser Restbefund	SM4 ist absichtlich noch keine arithmetische Quellschicht im alten Sinn von ArithmeticSource: Die bestehende Kernstruktur verlangt noch public ToCStrongOf, ChannelSeedStrong und MultiSchurStrong. Ein unmittelbares Befuellen dieses alten Records aus SM4 wuerde entweder eine public-Bridge erzwingen oder alte Nicht-Reset-Quellen konsumieren. Deshalb ist der naechste Schritt nicht SM6 und auch kein Charpoly-Start, sondern eine schmale generated-ArithmeticSource-Oberflaeche, die SM4 als Quelle registriert, ohne den alten public-Record voreilig zu erzwingen.
SM5-Klassifikation	SM5 ist eine Source-/Adapterphase fuer generated Quellen. Sie erweitert die arithmetische Quellenklassifikation um regularGenerated und variableGenerated aus dem Reset-Pfad und stellt einen eigenen proof-carrying Consumer fuer SM4GeneratedMultiSchurLedger bereit. Sie ist keine LevelHistory-Phase, keine Matrixeintragsphase, kein Charpoly-/Faktor-/Diskriminantenpfad und keine L2/L3-Vergleichsphase.
Aktiver Auftrag SM5	Baue genau eine schmale Datei wie GeneratedArithmeticSourceSM5.lean. Sie soll vorzugsweise nur GeneratedMultiSchurSM4 importieren. Zulaessige Kernobjekte sind eine generated-lokale Source-Art, z.B. GeneratedArithmeticSourceKindSM5.regularGenerated/variableGenerated, eine Struktur wie GeneratedArithmeticSourceSM5 oder ein Ledger wie SM5GeneratedArithmeticSourceLedger, die Felder sourceSM4Ledger, regularSource, variableSource, sourceMultiSchur, boundaryOperator, apply, apply_eq_mulVec, boundaryOperator_eq_schur, source_eq_SM4, noNewToCGeneratorStatus, noPublicMultiSchurStrongForcedStatus, noLevelHistoryStatus und noCharpolyDiscriminantStatus tragen.
SM5-DoD	Der DoD ist erfuellt, wenn regular und variable generated ArithmeticSource-Witnesses ausschliesslich aus einem SM4GeneratedMultiSchurLedger entstehen und ihre MultiSchur-/Boundary-/Apply-/Schur-Daten auf SM4 zurueckgebunden sind. SM5 muss fuer SM6 genau eine konsumierbare Source-Oberflaeche bereitstellen, aus der spaeter ein GeneratedLevelStepWitness gebaut werden kann. SM5 darf keine neue ToC-/Substratquelle, keine freie Matrix, keine neue DtN-/Generalized-DtN-Struktur, keine public-MultiSchurStrong-Pflicht und keine Charpoly-/Diskriminantendaten einfuehren.
Beweisroute SM5	Keine neue Rechnung. Die Beweise sind Feldprojektionen aus dem SM4-Ledger und kleine rfl/rw/calc-Rueckbindungen auf GeneratedMultiSchurSM4. Falls eine Erweiterung von ArithmeticSourceKind im Kernmodul noetig ist, bleibt sie minimal: nur neue Konstruktoren fuer regularGenerated/variableGenerated plus triviale Kind-Lemmata. Die alte ArithmeticSource-Recordstruktur wird erst dann befuellt oder umgebaut, wenn SM5 einen realen, nicht public-bridge-freien Konsumpfad dafuer belegt.
Verbotene Folgerung	SM5 erzeugt keine LevelHistory-Matrix, keine generatedSchurEntry-History-Matrix, keine Charpoly-, Faktor- oder Diskriminantendaten, keinen L2/L3-Zielvergleich, keine public DtNStrong/GeneralizedDtNStrong/MultiSchurStrong-Bruecke, kein ChannelSeedStrong-Paket und kein neues InterfaceEliminationData-Typklassenpaket. Eine unmittelbare Reparatur des alten ArithmeticSource-Records durch freie public-Daten ist Planbruch.
Overhead-Regel v65	Genau eine kleine SM5-Datei plus BuildAll- und, falls noetig, Notation-Import ist der Zielkorridor. Keine SM5a/SM5b-Kette, keine Audit-Unterphase, keine zweite Source-Ledger-Familie und kein public-Bridge-Preflight. Falls beim SM5-Bau ein Feld der alten ArithmeticSource-API nicht aus SM4 stammt, wird dieses Feld nicht frei ersetzt; SM5 bleibt generated-lokal und dokumentiert die public-Record-Fuellung als spaeteren echten Consumer-Bedarf.
Freigaberegel SM6	SM6 wird erst freigegeben, wenn SM5 eine einzige generated Source-Oberflaeche bereitstellt, die regular/variable SM4-MultiSchur-Daten konsumierbar macht. SM6 darf danach einen GeneratedLevelStepWitness vorbereiten, aber weiterhin keine freie LevelHistory-Matrix und keine Charpoly-/Diskriminantenrechnung starten.
Fortgeschriebene Reihenfolge v65	r3c1a closed/green; r3c1b0 closed/green; SM3db2aR-SM3db2fR closed/green bzw. closed/green-audit; SM3eR-r3c1b1 closed/green; SM3eR-r3c1b-full closed/green; SM3eR-r3c1c closed/green; SM3eR-r3c1d green/gate-closed; SM3db3aR closed/green-audit; SM3db4aT/U/V/W closed/green; SM3eR-r3c1d-full closed/green; SM3eR-r3c1e closed/green; SM3eR-r3c2 closed/green; SM3eR-r4a closed/green; SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 closed/green; SM3fR closed/green; SM3gR closed/green; SM3hR closed/green; SM3iR closed/green; SM4 closed/green; SM5 **active**; SM6 blocked until generated-local SM5 ArithmeticSource surface.

0ap. Fortschreibungsentscheidung v66 nach gruenem SM5-Build: SM6 aktiv

Statusentscheidung	SM5 ist nach lokal gruenem Build formal geschlossen. Die Phase hat die in v65 geforderte generated-lokale Source-Oberflaeche geliefert: regularGeneratedArithmeticSource_from_SM4, variableGeneratedArithmeticSource_from_SM4 und SM5GeneratedArithmeticSourceLedger entstehen ausschliesslich aus dem SM4GeneratedMultiSchurLedger; Boundary-Operator, Apply-Funktion und Schur-Rueckbindung bleiben auf SM4 zurueckgebunden. Damit ist das v65-Gate fuer SM6 erfuellt.
Formaler SM5-Ertrag	Geschlossen sind insbesondere GeneratedArithmeticSourceKindSM5.regularGenerated, GeneratedArithmeticSourceKindSM5.variableGenerated, GeneratedArithmeticSourceSM5, GeneratedArithmeticSourceSM5.fromSM4, SM5GeneratedArithmeticSourceLedger, sm5GeneratedArithmeticSourceLedger und sm5GeneratedArithmeticSourceLedger_from_accumulatedEntryCancellationLedger. Der Ertrag ist eine konsumierbare generated-Source-Oberflaeche mit sourceMultiSchur, boundaryOperator, apply, apply_eq_mulVec, boundaryOperator_eq_schur, source_eq_SM4 und Statusfeldern gegen neue ToC-Generatoren, public-MultiSchurStrong-Zwang, LevelHistory und Charpoly-/Diskriminantendaten.

Praeziser Restbefund	`SM5` ist absichtlich noch keine LevelHistory-Phase. Es gibt keinen legalen Schluss von `LevelHistoryIndex L := Fin (L+1)` auf echte generated Boundary-/Schur-Indizes, keine `generatedLevelHistoryMatrix`, keine `generatedSchurEntry`-Historie und keinen Charpoly-/Diskriminantenstart. Der alte `BoundarySource.LevelStepWitness` konsumiert weiterhin den alten `ArithmeticSource`-Record; ein Rueckfall auf diese Route waere fuer den Reset-Pfad zu frueh, solange SM5 gerade nur eine generated-lokale Source-Oberflaeche bereitstellt.
SM6-Klassifikation	`SM6` ist eine Witness-/Provenienzphase fuer einzelne Level-History-Schritte. Sie darf einen generated-lokalen `GeneratedLevelStepWitnessSM6` vorbereiten, der fuer ein Level `L` und einen Index `k : Fin (L+1)` festhaelt, welche regular/variable generated Source aus SM5 konsumiert wird und welcher generated Boundary-Carrier, Boundary-Operator, Apply-Operator und Schur-Ausdruck aus SM5/SM4 stammt. `SM6` ist keine Index-Bruecke, keine Matrixeintragsphase und kein Charpoly-/Diskriminantenpfad.
Aktiver Auftrag SM6	Baue genau eine schmale Datei wie `GeneratedLevelStepWitnessSM6.lean`. Sie soll vorzugsweise nur `GeneratedArithmeticSourceSM5` importieren. Zulaessige Kernobjekte sind eine lokale Indexabkuerzung `GeneratedLevelHistoryIndexSM6 (L) := Fin (L+1)` oder ein gleichwertiger Import ohne alten Source-Consumer, eine Struktur `GeneratedLevelStepWitnessSM6`, regular/variable Spezialwitnesses und ein `SM6GeneratedLevelStepWitnessLedger`. Minimalfelder: `sourceSM5Ledger`, `level`, `levelIndex`, `source`, `source_eq_SM5`, `boundaryCarrier`, `boundaryOperator`, `boundaryOperator_eq_SM5`, `boundaryOperator_eq_schur`, `apply`, `apply_eq_SM5`, `apply_eq_mulVec`, `noCutSpecCarrierClaimStatus`, `noSchurIndexBridgeStatus`, `noLevelHistoryMatrixStatus` und `noCharpolyDiscriminantStatus`.
SM6-DoD	Der DoD ist erfuellt, wenn fuer regularGenerated und variableGenerated je ein `GeneratedLevelStepWitnessSM6` aus einem `SM5GeneratedArithmeticSourceLedger` entsteht und alle Boundary-/Apply-/Schur-Provenienzfelder auf die entsprechende SM5-Source bzw. deren SM4-MultiSchur zurueckgehen. `SM6` muss fuer SM7 genau eine konsumierbare Witness-Oberflaeche liefern, aus der spaeter eine Bruecke von `Fin (L+1)` zu echten generated Boundary-/Schur-Indizes geprueft werden kann.
Beweisroute SM6	Keine Summen, keine Matrixeintraege, keine Rekursion ueber Level und kein grosser Index-Eliminator. Die Beweise sind Feldprojektionen aus `SM5GeneratedArithmeticSourceLedger` und kleine `rfl`/`rw`/`calc`-Rueckbindungen: zuerst Source und Boundary-Operator aus SM5 fixieren, dann `apply_eq_mulVec` und `boundaryOperator_eq_schur` direkt aus den SM5-Feldern weiterreichen. `noCutSpecCarrierClaim` ist in SM6 nur ein Status/Gate: Es behauptet gerade nicht, dass der LevelHistory-Index bereits ein CutSpec- oder Boundary-Carrier ist.
Verbotene Folgerung	`SM6` erzeugt keine `RecursiveLevelHistoryCarrier`, keine `LevelHistoryMatrixPackage`, keine `generatedLevelHistoryMatrix`, keine `generatedSchurEntry`, keine Bruecke von `Fin (L+1)` zu `GeneratedBoundaryIndex`, keine freie Ersatzindex-Funktion, keine public-`ArithmeticSource`-Recordfuellung, keine public-`MultiSchurStrong`-Bruecke, kein `Matrix.inv` und keine Charpoly-/Faktor-/Diskriminantendaten.
Overhead-Regel v66	Genau eine kleine SM6-Datei plus `BuildAll`- und, falls noetig, `Notation`-Import ist der Zielkorridor. Keine `SM6a`/`SM6b`-Kette, kein Audit-Preflight und keine neue Ledger-Familie neben dem einen SM6-Ledger. Falls ein altes LevelHistory-Feld nur ueber den alten `ArithmeticSource`-Record erreichbar ist, wird es nicht frei nachgebaut; SM6 bleibt generated-lokal und dokumentiert die Brueckenfrage als SM7-Pflicht.
Freigaberegel SM7	`SM7` wird erst freigegeben, wenn SM6 fuer regular und variable generated Quellen einen `GeneratedLevelStepWitnessSM6` liefert. SM7 darf dann die Bruecke von `Fin (L+1)` zu echten `GeneratedBoundaryIndex`/Schur-Indizes pruefen oder eine proof-carrying Obstruktion ausgeben. SM7 darf weiterhin keine freie LevelHistory-Matrix und keine Charpoly-/Diskriminantenrechnung starten.
Fortgeschriebene Reihenfolge v66	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` **active**; `SM7` blocked until generated-local SM6 LevelStepWitness surface.

0aq. Fortschreibungsentscheidung v67 nach gruenem SM6-Build: SM7 aktiv

Statusentscheidung	`SM6` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die in v66 geforderte generated-lokale LevelStepWitness-Oberflaeche geliefert: regular und variable Witnesses entstehen aus dem `SM5GeneratedArithmeticSourceLedger`; `level`, `levelIndex`, `boundaryCarrier`, `boundaryOperator`, `apply` und die Schur-Rueckbindung bleiben auf SM5/SM4 zurueckgebunden. Damit ist das v66-Gate fuer `SM7` erfuellt.
Formaler SM6-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryIndexSM6`, `GeneratedLevelStepWitnessSM6`, `GeneratedLevelStepWitnessSM6.fromSM5`, `regularGeneratedLevelStepWitness_from_SM5`, `variableGeneratedLevelStepWitness_from_SM5`, `SM6GeneratedLevelStepWitnessLedger`, `sm6GeneratedLevelStepWitnessLedger`, `sm6GeneratedLevelStepWitnessLedger_from_accumulatedEntryCancellationLedger` sowie die Rueckbindungslemmata fuer Source, Boundary-Operator, Apply und die No-Bridge-/No-Matrix-/No-Charpoly-Statusfelder.

Praeziser Restbefund	SM6 beweist bewusst keine Abbildung GeneratedLevelHistoryIndexSM6 L -> GeneratedBoundaryIndex A. Der aktuelle Witness traegt zwar boundaryCarrier = GeneratedBoundaryIndex A, aber levelIndex : Fin (level+1) bleibt ein History-Koordinatentrigger und ist noch kein Boundary-Vertex, kein Schur-Index und kein Eintragstraeger. Ein direkter Start von SM8 waere daher eine freie Index- oder Matrixeintragsannahme.
SM7-Klassifikation	SM7 ist ein Bridge-Gate, keine Matrixphase. Die Phase darf genau zwei Arten von Ergebnis erzeugen: entweder eine proof-carrying GeneratedLevelHistorySchurIndexBridgeSM7-Quelle, die eine echte, derived generated Abbildung von History-Indizes zu Boundary-/Schur-Indizes mit Provenienz traegt, oder eine proof-carrying GeneratedLevelHistorySchurIndexObstructionSM7, die exakt ausweist, welche generated Indexquelle fehlt. Beides bleibt vor SM8; weder ein freier Ersatzindex noch eine Fintype-Enumerationswahl ist erlaubt.
Aktiver Auftrag SM7	Baue genau eine schmale Datei wie GeneratedLevelHistoryToSchurIndexSM7.lean. Sie soll vorzugsweise nur GeneratedLevelStepWitnessSM6 importieren. Zulaessige Kernobjekte sind Statusarten wie SM7BridgeOutcome, SM7NoFreeFintypeEnumerationStatus, SM7NoLevelHistoryMatrixStatus, SM7NoCharpolyDiscriminantStatus; eine optionale positive Struktur GeneratedLevelHistorySchurIndexBridgeSM7 mit sourceSM6Ledger, level, witnessAt, toBoundaryIndex, toBoundaryIndex_source, boundaryIndex_level_eq_cutoff und Schur-/Boundary-Provenienzfeldern; sowie eine Obstruktionsstruktur GeneratedLevelHistorySchurIndexObstructionSM7 mit sourceSM6Ledger, missingLevelToBoundaryMapSource, missingSchurIndexProvenance, noFreeReplacementIndexStatus, noMatrixStatus und nextPhaseBlockedStatus.
SM7-DoD	Der starke positive DoD ist nur erfuellt, wenn regular und variable SM6-Ledger eine echte generated Bridge liefern, deren toBoundaryIndex nicht aus Fintype.equivFin, Classical.choice, einer freien Tabelle oder einem willkuerlichen Default-Vertex stammt. Falls diese Quelle im aktuellen API-Stand nicht existiert, ist der SM7-DoD nicht ein Matrixstart, sondern ein sauberer Obstruktionsabschluss: SM8 bleibt blockiert und die naechste Planfortschreibung muss die minimal fehlende generated Indexquellen-Schicht bestimmen.
Beweisroute SM7	Zuerst wird positiv aus dem SM6-Ledger eine Witness-Familie witnessAt : (k : GeneratedLevelHistoryIndexSM6 L) -> GeneratedLevelStepWitnessSM6 ... gebaut; das ist nur sm6GeneratedLevelStepWitnessLedger mit wechselndem levelIndex. Danach werden vorhandene Boundary-Fakten nur projiziert: boundaryCarrier_is_generated, boundaryOperator_from_SM5, apply_from_mulVec und, fuer echte Boundary-Indizes, GeneratedBoundaryIndex.level_eq_cutoff. Wenn daraus keine Funktion Fin (L+1) -> GeneratedBoundaryIndex A mit Provenienz folgt, wird keine Scheingleichheit bewiesen; die fehlende Quelle wird als Status/Obstruktion mit rfl-Feldgleichungen geschlossen. Keine Summen, keine Matrixeintraege, keine Rekursion ueber Schur-Eintraege, kein grosser Index-Eliminator.
Verbotene Folgerung	SM7 erzeugt keine generatedLevelHistoryMatrix, keine generatedSchurEntry-Matrix, keine Charpoly-, Faktor- oder Diskriminantendaten, keinen L2/L3-Zielvergleich, keine publicArithmeticSource-Recordfuellung, keine public-MultiSchurStrong-Bruecke und keine freie Funktion aus Fin (L+1) in GeneratedBoundaryIndexA. Insbesondere sind Fintype.equivFin, Finset.toList-Wahlen, Default-Elemente, Classical.choice und tabellierte Ersatzindices fuer den operativen Pfad verboten.
Overhead-Regel v67	Genau eine kleine SM7-Datei plus BuildAll- und, falls noetig, Notation-Import ist der Zielkorridor. Keine SM7a/SM7b-Kette, kein separates Audit-Preflight und keine zweite Ledger-Familie. Die Bridge- und Obstruktionsmoeglichkeit werden in derselben Phase gefuehrt; erst nach dem gruenen SM7-Build wird entschieden, ob SM8 freigegeben oder eine minimale Indexquellen-Nachruestung notwendig ist.
Freigaberegel SM8	SM8 wird nur freigegeben, wenn SM7 einen bridgeSuccess-artigen Witness mit echter generated History-to-Boundary-/Schur-Provenienz liefert. Ein reiner Obstruktionsabschluss von SM7 gibt SM8 nicht frei. In diesem Fall ist der naechste Schritt nicht Matrix/Charpoly, sondern die kleinste generated Quelle, die den fehlenden Indexpfeil erzeugen kann.
Fortgeschriebene Reihenfolge v67	r3c1a closed/green; r3c1b0 closed/green; SM3db2aR-SM3db2fR closed/green bzw. closed/green-audit; SM3eR-r3c1b1 closed/green; SM3eR-r3c1b-full closed/green; SM3eR-r3c1c closed/green; SM3eR-r3c1d green/gate-closed; SM3db3aR closed/green-audit; SM3db4aT/U/V/W closed/green; SM3eR-r3c1d-full closed/green; SM3eR-r3c1e closed/green; SM3eR-r3c2 closed/green; SM3eR-r4a closed/green; SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 closed/green; SM3fR closed/green; SM3gR closed/green; SM3hR closed/green; SM3iR closed/green; SM4 closed/green; SM5 closed/green; SM6 closed/green; SM7 **active**; SM8 blocked until SM7 bridgeSuccess.

0ar. Fortschreibungsentscheidung v69 nach gruenem SM7-Build: SM6a aktiv

Statusentscheidung	SM7 ist nach lokal gruenem Build formal geschlossen. Der Build schliesst genau den in v67 erlaubten Obstruktionspfad: regular und variable Seite tragen SM7BridgeOutcome.obstructionMissingGeneratedIndexSource, keine freie Fin (L+1) -> GeneratedBoundaryIndex A-Funktion, keine Fintype-Enumeration, keine Matrix und keine Charpoly-/Diskriminantenrechnung. Damit ist SM7 gruen, aber SM8 bleibt gesperrt.
Korrektur gegen v68	Die in v68 vorgeschlagene aktive Phase SM7IndexSource war methodisch falsch lokalisiert und wird verworfen. SM7 ist ein Gate und ein Consumer der Indexquelle, nicht die natuerliche Erzeugungsschicht dieser Quelle. Die fehlende Struktur wird dort nachgeruestet, wo sie natuerlich entsteht: unmittelbar nach SM6, also als SM6a.

Provenienzregel v69	Wir patchen nicht nachtraeglich dort, wo eine Struktur benoetigt wird, sondern dort, wo die benoetigte Struktur natuerlich entsteht. Fuer die CNNA-Arithmetik bedeutet das: Ein spaeteres Gate darf eine fehlende Struktur sichtbar machen, aber die Nachruestung muss in die kleinste fruehere Schicht zurueck, in der die Daten ohne Provenienzbruch konstruktiv entstehen koennen.
Formaler SM7-Ertrag	Geschlossen sind insbesondere `SM7BridgeOutcome`, `GeneratedLevelHistorySchurIndexBridgeSM7` als positive Zielsignatur ohne freien Konstruktor, `GeneratedLevelHistorySchurIndexObstructionSM7`, `GeneratedLevelHistorySchurIndexObstructionSM7.fromSM6Witness`, `SM7GeneratedLevelHistoryToSchurIndexGate`, `sm7GeneratedLevelHistoryToSchurIndexGate` sowie die No-Free-/No-Matrix-/No-Charpoly- und `sm8BlockedUntilBridgeSuccess`-Statusfelder.
Praeziser Restbefund	Der gruene SM7-Build beweist nicht, dass ein History-Index bereits ein Boundary- oder Schur-Index ist. Das ist kein Taktikloch, sondern ein Datenloch: `GeneratedLevelHistoryIndexSM6 L = Fin (L+1)` traegt nur eine History-Koordinate. Fuer einen echten `GeneratedBoundaryIndexA` braucht es zusaetzlich einen boundary-level Carrier-Vertex bzw. eine generated Branch-/Boundary-Port-Provenienz. Ein Default-Branch, ein beliebiger `Fintype`-Vertreter oder eine tabellierte Ersatzabbildung waere derived-only-widrig.
Natuerliche Entstehungsschicht	Die Quelle gehoert nach `SM6`, weil `SM6` erstmals `GeneratedLevelHistoryIndexSM6`, `GeneratedLevelStepWitnessSM6`, generated Boundary-Carrier, Boundary-Operator, Apply-Operator und SM5/SM4-Schur-Provenienz in einer lokalen Witness-Oberflaeche zusammenfuehrt. `SM7` darf diese Quelle spaeter konsumieren, aber nicht nachtraeglich erfinden. Die alte v66-Overhead-Regel gegen eine `SM6a`/`SM6b`-Kette war vor dem SM7-Obstruktionsbefund sinnvoll; v69 ueberschreibt sie minimal, weil der Gate-Befund nun gerade eine fehlende SM6-nahe Provenienzquelle nachweist.
Naechste aktive Phase	Die naechste aktive Phase ist `SM6a - GeneratedLevelHistoryBoundaryPortSource`, nicht `SM7IndexSource` und nicht `SM8`. Ziel ist eine schmale Datei wie `GeneratedLevelHistoryBoundaryPortSourceSM6a.lean`. Sie importiert vorzugsweise nur `GeneratedLevelStepWitnessSM6` und sucht die minimal fehlende generated Boundary-Port-/Indexquelle aus SM6/SM5/SM4-Provenienz.
SM6a-DoD	Der starke positive DoD ist eine proof-carrying Struktur wie `GeneratedLevelHistoryBoundaryPortSourceSM6a`, die fuer eine SM6-Witness-Familie eine konstruktive Quelle fuer `GeneratedLevelHistoryIndexSM6 level -> GeneratedBoundaryIndex A` traegt. Minimalfelder: `sourceSM6Witness`, `witnessAt`, `historyIndex`, `toBoundaryIndex` oder aequivalente boundary-port Quelle, `toBoundaryIndex_source`, `toBoundaryIndex_level_eq_cutoff`, BoundaryCarrier-Provenienz, BoundaryOperator-/Apply-Rueckbindung an SM6/SM5 und Statusfelder gegen freie Enumeration, freie Tabellen, `Classical.choice`, `Matrix.inv`, Matrixstart und Charpoly-Daten.
SM6a-Grenze	`SM6a` erzeugt noch keine `generatedLevelHistoryMatrix`, keine `generatedSchurEntry`-Matrix, keine Charpoly-, Faktor- oder Diskriminantendaten und keinen L2/L3-Zielvergleich. Es ist eine Indexquellen-/Portquellen-Schicht, keine Matrixphase. Sie soll auch keinen SM7-BridgeSuccess in der SM7-Signatur erzwingen, falls dafuer ein Importzyklus oder eine falsche Consumer-Richtung entstuende.
Beweistaktik SM6a	Zuerst wird aus dem SM6-Ledger wieder die Witness-Familie `witnessAt : (k : GeneratedLevelHistoryIndexSM6 level) -> GeneratedLevelStepWitnessSM6 ...` gebaut. Danach wird geprueft, ob aus `GeneratedBoundaryIndex` als Subtyp von `GeneratedApproximantIndex` und aus den vorhandenen Partition-/Boundary-Feldern ein boundary-level Port mit `GeneratedBoundaryIndex.level_eq_cutoff` konstruktiv geliefert werden kann. Beweisform: kleine Feldprojektionen, `rfl`-Gleichungen, lokale `calc`-Bloecke fuer Rueckbindungen; keine globale `simp`-Flaeche, keine Summeninduktion, kein grosser Index-Eliminator, keine `Fintype.equivFin`/`Finset.toList`/Default-Auswahl.
Minimaler Rueckfall	Falls `SM6a` zeigt, dass selbst in der SM6-natuerlichen Schicht kein boundary-level Port aus der aktuellen API konstruierbar ist, endet `SM6a` nicht mit SM8-Freigabe und nicht mit einem SM7-Patch. Dann ist die naechste positive Phase `SM6b - GeneratedBoundaryPortSource` an der noch tieferen natuerlichen BranchFamily-/GeneratedMainPath-/Boundary-Partition-Schicht.
Anschlussgate	Nach einem positiven `SM6a`-Build folgt kein direkter SM8-Start. Zuerst muss das SM7-Gate als Consumer neu geprueft werden, z.B. durch eine sehr kleine Adapter-/Recheck-Schicht, die `SM6a` konsumiert und einen echten `GeneratedLevelHistorySchurIndexBridgeSM7`-Witness liefert. Erst dieser BridgeSuccess oeffnet SM8. Diese Adapter-Schicht darf keine neue Quelle erzeugen; die Quelle bleibt SM6a-Provenienz.
Overhead-Regel v69	Genau eine kleine `SM6a`-Datei plus `BuildAll`-Import ist der Zielkorridor. Keine `SM6a0/1`-Kette, kein separates Audit-Preflight und keine neue Ledger-Familie neben der einen Source-/Missing-Source-Struktur. Wenn die Quelle fehlt, wird exakt die minimal tiefere Portquelle benannt; es wird nicht im SM7-Gate gepatcht.
Freigaberegel SM8	`SM8` wird erst aktiv, wenn nach `SM6a` das SM7-Gate einen echten BridgeSuccess-Konstruktor fuer die `GeneratedLevelHistorySchurIndexBridgeSM7`-Signatur liefert. Ein SM6a-Missing-Source-Befund gibt `SM8` nicht frei; dann ist ausschliesslich `SM6b - GeneratedBoundaryPortSource` zulaessig.
Fortgeschriebene Reihenfolge v69	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` closed/green; `SM7` closed/green-obstruction; `SM6a` **active**; `SM7BridgeRecheck` blocked until SM6a positive source; `SM8` blocked until SM7 bridgeSuccess.

0as. Fortschreibungsentscheidung v70 nach gruenem SM6a-Build: SM7BridgeRecheck aktiv

Statusentscheidung	SM6a ist nach lokal gruenem Build formal geschlossen. Die Phase hat die v69-geforderte natuerliche SM6-nahe Portquelle geliefert: GeneratedLevelHistoryBoundaryPortSourceSM6a traegt eine konstruktive History-to-Boundary-Quelle, die an GeneratedLevelStepWitnessSM6, Boundary-Carrier, Boundary-Operator, Apply und die SM5/SM4-Schur-Provenienz rueckgebunden ist. Damit entfaellt der SM7-Missing-Source-Befund als aktiver Rueckspruggrund; er bleibt nur historischer Obstruktionsbefund des alten Gates.
Formaler SM6a-Ertrag	Geschlossen sind insbesondere GeneratedLevelHistoryBoundaryPortSourceSM6a, fromBoundaryMap, toBoundaryIndex, toBoundaryIndex_level_eq_cutoff, toBoundaryIndex_mem_boundaryCarrier, boundaryOperator_from_SM6, boundaryOperator_from_schur, apply_from_mulVec, regularGeneratedBoundaryZeroPortSM6a, variableGeneratedBoundaryZeroPortSM6a, regularGeneratedLevelHistoryBoundaryPortSource_from_SM6, variableGeneratedLevelHistoryBoundaryPortSource_from_SM6 und die regular/variable SM6a-Ledger-Felder. Geschlossen sind ausserdem die Statusfelder gegen freie Enumeration, freie Tabelle, Classical.choice, Matrixstart, neu eingefuehrtes Matrix.inv und Charpoly-/Faktor-/Diskriminantendaten.
Praeziser Restbefund	SM6a erzeugt keine Matrix und keinen arithmetischen Zahlenoutput. Es erzeugt auch noch nicht selbst den SM7-bridgeSuccess in der alten Gate-Signatur. Das ist methodisch korrekt: SM6a ist die Erzeugungsschicht der Quelle; SM7 bzw. ein direkt nachgelagerter Recheck ist der Consumer, der diese Quelle in die bereits vorhandene positive GeneratedLevelHistorySchurIndexBridgeSM7-Signatur ueberfuehren muss.
Naechste aktive Phase	Die naechste aktive Phase ist SM7BridgeRecheck - GeneratedLevelHistorySchurIndexBridgeFromSM6a. Ziel ist eine einzige kleine Datei, vorzugsweise GeneratedLevelHistorySchurIndexBridgeRecheckSM7a.lean, die SM6a und SM7 konsumiert und regular/variable echte GeneratedLevelHistorySchurIndexBridgeSM7-Witnesses liefert. SM8 bleibt bis zu diesem BridgeSuccess blockiert.
SM7BridgeRecheck-DoD	Der starke DoD ist ein Konstruktor wie bridgeFromSM6a oder generatedLevelHistorySchurIndexBridge_from_SM6a, der aus B : GeneratedLevelHistoryBoundaryPortSourceSM6a ... einen GeneratedLevelHistorySchurIndexBridgeSM7 ... baut. Minimalfelder: sourceSM6Witness := B.sourceSM6Witness, witnessAt := B.witnessAt, toBoundaryIndex := B.toBoundaryIndex, toBoundaryIndex_level_eq_cutoff := B.toBoundaryIndex_level_eq_cutoff, boundaryOperator_from_SM6 := B.boundaryOperator_from_SM6, boundaryOperator_from_schur := B.boundaryOperator_from_schur, witnessAt_source_eq_SM6 und witnessAt_boundaryCarrier_eq_generated. Zusaetzlich braucht es ein regular/variable Recheck-Ledger mit regularOutcome = bridgeSuccess, variableOutcome = bridgeSuccess und expliziter SM8-Freigabe erst nach diesen Bridges.
SM7BridgeRecheck-Grenze	Die Phase darf keine neue Boundary-Port-Quelle erzeugen, keinen Ersatzindex setzen, keine Fintype-Enumeration, keine freie Tabelle, kein Classical.choice, kein Matrix.inv, keine generatedLevelHistoryMatrix, keine generatedSchurEntry-Matrix und keine Charpoly-/Faktor-/Diskriminantendaten einfuehren. Sie darf die alte SM7GeneratedLevelHistoryToSchurIndexGate-Obstruktion nicht ueberschreiben, sondern nur als historisch ueberholt durch eine neue positive Consumer-Schicht neben/nach SM7 reklassifizieren.
Beweistaktik SM7BridgeRecheck	Importreihenfolge: erst GeneratedLevelHistoryBoundaryPortSourceSM6a, dann GeneratedLevelHistoryToSchurIndexSM7; besser neue Datei nach beiden Imports statt SM7 selbst mit SM6a zu vermischen. Der generische Konstruktor ist ein reiner Record-Builder. Die meisten Felder werden per Projektion aus B geschlossen; toBoundaryIndex_sourceStatus_eq, No-Free-/No-Matrix-/No-Charpoly-Felder und Outcome-Felder werden per rfl. witnessAt_boundaryCarrier_eq_generated wird punktweise aus (B.witnessAt k).boundaryCarrier_eq_generated oder aus dem in SM6a exponierten BoundaryCarrier-Feld geschlossen. Keine globale simp-Flaeche, keine Summeninduktion, kein grosser Index-Eliminator.
Anschlussgate	Nach gruenem SM7BridgeRecheck-Build wird SM8 aktiv. Dann beginnt erstmals die eigentliche Matrixphase: generatedSchurEntry und generatedLevelHistoryMatrix duerfen nur aus SM4/SM5/SM6, der SM6a-Portquelle und dem SM7BridgeRecheck-BridgeSuccess gebaut werden. Falls der Recheck wider Erwarten an einem fehlenden SM7-Feld scheitert, wird nicht in SM8 gepatcht; dann wird die minimal fehlende Consumer-Expose-Schicht direkt im Recheck benannt, nicht eine neue Quelle erfunden.
Overhead-Regel v70	Genau eine kleine Recheck-Datei plus BuildAll-Import ist der Zielkorridor. Keine neue SM7a/SM7b-Kette, kein Audit-Preflight, keine neue Obstruktionsfamilie und kein Umbau der alten SM7-Datei, solange eine nachgelagerte Adapterdatei die Signatur sauber konsumieren kann.
Freigaberegel SM8	SM8 wird erst nach lokal gruenem SM7BridgeRecheck-Build aktiv, wenn regular und variable je einen positiven GeneratedLevelHistorySchurIndexBridgeSM7-Witness tragen und ein Ledger bridgeSuccess statt obstructionMissingGeneratedIndexSource fuer den neuen Consumer-Pfad ausweist.
Fortgeschriebene Reihenfolge v70	r3c1a closed/green; r3c1b0 closed/green; SM3db2aR-SM3db2fR closed/green bzw. closed/green-audit; SM3eR-r3c1b1 closed/green; SM3eR-r3c1b-full closed/green; SM3eR-r3c1c closed/green; SM3eR-r3c1d green/gate-closed; SM3db3aR closed/green-audit; SM3db4aT/U/V/W closed/green; SM3eR-r3c1d-full closed/green; SM3eR-r3c1e closed/green; SM3eR-r3c2 closed/green; SM3eR-r4a closed/green; SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 closed/green; SM3fR closed/green; SM3gR closed/green; SM3hR closed/green; SM3iR closed/green; SM4 closed/green; SM5 closed/green; SM6 closed/green; SM7 closed/green-obstruction historisch; SM6a closed/green; SM7BridgeRecheck **active**; SM8 blocked until SM7BridgeRecheck bridgeSuccess.

0. Startvertrag

Startstand	Repo-Anhang bis inkl. S9h: finite Zustands-, Dynamik-, Kanal- und Spektraloberflaeche liegt als operativer `ExecComplex`-Strang mit lokalem $\mathbb{C}$-Spiegel vor. `CNNA/PillarA/Arithmetic` wird neu angelegt; kein alter Arithmetik-Code wird uebernommen.
Status	Der Arithmetikpfad ist A-intern, pre-handoff, kein zweiter Root-Generator und kein Handoff-Ersatz.
Quelle	Erlaubt sind nur bereits gruene Pillar-A-Objekte bis S9h: ToC/Finite/DtN/Coupling/Spectral/State-Dynamics-Channel-Seeds sowie bereits gruene Cayley-Dickson-/Hurwitz-Strukturmodule, sofern sie A-intern und importdiszipliniert konsumiert werden.
Ziel	Von ToC-Approximanten ueber finite bright-DtN/Schur/Spektrum zu quadratischen Ordnungen, CM-Gittern, Modularformen, j, Graph-Zeta-/Ramanujan-/Gap-Seeds, IDEAL∞-/REAL∞-Grapharithmetik, Heegner-/UFD-Indikatoren, q-Charakteren und Moonshine-Kandidaten – strikt derived-only.
Methode	Jeder Schritt hat einen operativen computable Strang und, falls noetig, einen getrennten noncomputable $\mathbb{C}$-Spiegel mit bewiesener Bruecke. Die Algebra-Seite des Generators (Sektorzerlegung, Cayley-Dickson/Hurwitz) und die Arithmetik-Seite des Generators (Branching, Schur, CM, Modularitaet) werden als getrennte derived-only Consumer gefuehrt und nur ueber explizite Konvergenztests zusammengefuehrt. Zusaetzlich wird eine Wirkungs-/Symmetrie-Querachse eingefuehrt: Gruppen, Monoide, Hecke-Algebren, Groupoids, lokale Baumaktionen, profinite/Galois-artige Wirkungen oder kategoriale Wirkungen zaehlen nur als CNNA-Daten, wenn zuerst ihr Wirkungsformat klassifiziert und dann Traeger, Aktion/Operator, Erhaltungssaetze und Invarianten A-intern belegt sind.

0a. Konsolidierte Leseroute

Diese Fassung ist redaktionell neu geordnet, aber inhaltlich nicht entschaerft. Die gewachsenen Gates bleiben erhalten; sie werden nur in eine Implementierungsreihenfolge gebracht: zuerst Methodik und Entscheidungsgates, danach Phasen, Modulbaum, Outcome-/Ledger-Status, Minimalziele und schliesslich externe Vergleichsanker.

MUSS	Derived-only-Provenienz, reale BoundarySource, Import-/Consumer-Disziplin, Elliptic-Function-First, ActionFormatDecision, Root-/Rootless-Abgrenzung und explizite Ledger-Outcomes. Diese Punkte sind Sperren, nicht Kommentare.
DARF	Vergleichsanker wie TomS/Jakito, LPS/Quaternionen, Leech/Conway/Monster, Hecke, Groupoids, Buildings, Galois, GT/Operaden, VOA-Orbifold-Formate und der neue IdealThread/String-Hinweis als externe bzw. heuristische Orientierung nutzen, sofern sie nicht als CNNA-Provenienz ausgegeben werden.
DARF NICHT	Alte Arithmetik-Module importieren, freie Carrier setzen, `CutSpec.full` als Level-History-Quelle verwenden, `variationToC beta` als reiches Substrat misslesen, den `IdealThread` als vollen Baum mit kleinerem b behandeln, aus dem String-Hinweis Stringtheorie ableiten, aus j direkt Moonshine folgern, aus einem Gitter direkt ein Root-System folgern oder eine globale Gruppe behaupten, wo nur lokale/partielle Wirkung vorliegt.
Implementierungslogik	AR0–AR2b-0 sind der sofort implementierbare Block. AR2b-D entscheidet die BoundarySource-Architektur. Erst danach duerfen AR2b-1/AR2b-2, AR3a/AR3b und die arithmetische Hauptkette als Ergebnisphasen weiterlaufen.

0b. Zusaetzliche Motivation: IdealThread als Nullpfad-Verduennung

Codepraezisierung	Im aktuellen Code ist `IdealThread` kein eigener neuer Substrattyp, sondern die idealisierte Fadenform eines `InfiniteThread`: pro Level wird genau eine Zelle `U.cell L` gewaehlt, und der Carrier ist singletonartig `{U.cell L}`. `referenceThread b` ist der kanonische Nullpfad im regulaeren Substrat, `variationThread beta` der entsprechende Nullpfad im levelvariablen Substrat.
Ambient statt kleinerem Baum	`IdealThread` ist nicht der volle Baum mit kleinerem Verzweigungsparameter. Fuer `RegularCell b L := Fin L -> Fin (b+1)` lebt der Nullpfad im vollen $(b+1)$-aeren Ambient-Substrat, ignoriert aber alle seitlichen Aeste. Carrier-maessig verhaelt er sich wie ein unaerer Pfad; strukturell bleibt er eine Verduennung eines reicheren Substrats.
Arithmetische Konsequenz	Ein Singleton-Carrier pro Level kann keine b- bzw. beta-abhaengige mehrdimensionale Boundary-, Schur- oder Diskriminantenstruktur tragen. Deshalb duerfen Eisenstein, Gauss und L>3 nicht aus `referenceToC`/`variationToC` als `IdealThread`-Substraten erwartet werden, sondern muessen im AR12a-SMOKE aus generated nicht-singleton Hauptpfad-Substraten fallen oder dort scheitern.
String-Hinweis	Die Analogie "Faden im Ambientraum" wird nur als zusaetzliche Motivation und als Fernziel-Watchlist fuer eine spaetere Rekonstruktion stringtheoretischer Strukturen gefuehrt: `IdealThread` kann als eindimensionaler Faden-Vergleich dienen, `RegularCell`/`LevelVariableCell` als Target-/Ambient-Struktur und `CutSpec` als moegliche spaetere Expositions-, Defekt- oder Worldsheet-Schicht. Daraus folgt keine Stringtheorie-Aussage im Arithmetikpfad.
Planfolge	AR12a-SMOKE ist die mathematische Scharnierstelle: Der Nullpfad wird als No-go verstanden, der generated Ambient-Traeger wird als Rechenquelle erzwungen, und `CutSpec` bleibt fuer einen separaten String-/Worldsheet-Fernzielpfad erhalten, ohne als aktuelles Arithmetik-Fundament zu dienen.

0c. Reset-Beschluss fuer AR12a-SMOKE ab SM1

Reset-Baseline	Der neue Implementierungsstart ist der reine AR12a-Codezustand. Alles, was nach AR12a in den bisherigen SM0–SM3d-Zips entstanden ist, wird nicht als positiver Pfad fortgeschrieben. Es darf gelesen werden, um Fehlerklassen, Namen und Importfallen zu erkennen, aber nicht als mathematische Grundlage fuer SM3f, SM4 oder spaetere Smoke-Phasen dienen.
SM1–SM3cR neu	Die Smoke-Kette wird ab SM1 neu gebaut. SM0R zaehlt nur als Infrastruktur fuer den Neustart; SM1R ist nach lokalem gruenem Build als erste mathematische Reset-Stufe geschlossen. SM1R schliesst Provenienzverdrahtung, aber noch keine Approximant-, Partition-, Matrix-, DtN- oder Inversenrechnung. SM2R ist nach lokalem gruenem Build als zweite Reset-Stufe geschlossen: es liefert den generated finite Approximant-Core, aber weiterhin keine Partition-, Matrix-, DtN- oder Inversenrechnung. SM3aR ist nach lokalem gruenem Build als dritte Reset-Stufe geschlossen: es liefert Boundary-/Interior-Partition, Membership-/Subset-Lemmata, Disjointness, Completeness und finite Restriktions-API, aber weiterhin keine Dirichlet-Entry-Formel, Matrix-, Interior-Block-, Inversen-, DtN- oder Diskriminantenrechnung. SM3bR ist nach lokalem gruenem Build als vierte Reset-Stufe geschlossen: es liefert Dirichlet-Entry-Quelle und Matrix-Entry-Funktion aus generated Adjazenz und Weight-/Policy-Provenienz, aber weiterhin keine Interior-Block-Strukturklassifikation, keine Inversen-, DtN- oder Diskriminantendaten. SM3cR ist nach lokalem gruenem Build als fuenfte Reset-Stufe geschlossen: es liefert Interior-Restriktion, Entry-Gleichung zur Dirichlet-Matrix, Nicht-Adjazenz-Nullsatz, Offdiag-/Diag-Lemmata und ein Strukturprofil; es liefert weiterhin keine Inversenformel, kein TwoSidedInv, kein Interior-Elimination-Certificate, keine DtN- oder Diskriminantendaten.
SM0R–SM3cR-Abschluss	SM0R ist geschlossen, weil der Reset-Scaffold kompilierend erzeugt wurde und nur die erlaubten Startdaten traegt: generated full-level carrier, regular/variable Smoke-Carrier, Nicht-Singleton-Witness fuer echte Verzweigung und Ledger-Sperren gegen `CutSpec.full`, freie Matrixdaten und alte SM1–SM3d-Fortschreibung. SM1R ist geschlossen, weil regular und variable generated Hauptpfade proof-carrying verbunden sind und zugleich keine CutSpec-, BoundaryPorts-, Matrix-, DtN-, Inversen- oder Diskriminantendaten eingefuehrt wurden. SM2R ist geschlossen, weil der Approximant-Core aus SM1R konstruiert wurde, Carrier/Cutoff/Truncation/Finiteness proof-carrying bereitstellt und zugleich keine Boundary-/Partition-, Matrix-, Dirichlet-, DtN-, Inversen- oder Zielanker-Daten einfuehrt. SM3aR ist geschlossen, weil Boundary- und Interior-Indizes ausschliesslich als Subcarrier des SM2R-Approximanten konstruiert wurden und zugleich keine freie `Fin n`-Boundary, kein `CutSpec.full`, keine `BoundaryPorts`, keine Matrix-, Dirichlet-, DtN-, Inversen- oder Diskriminantendaten eingefuehrt wurden. SM3bR ist geschlossen, weil die Dirichlet-Matrix ausschliesslich als `generatedDirichletMatrix` aus `generatedDirichlet_entry` exponiert wird und die Entry-Quelle generated Adjazenz plus Weight-/Policy-Provenienz traegt. SM3cR ist geschlossen, weil der Interior-Block ausschliesslich als Restriktion dieser Entry-Matrix entsteht und sein Profil entrywise aus Nicht-Adjazenz-, Offdiag- und Diagonal-/Degree-Lemmata klassifiziert wird; zugleich fuehrt SM3cR keine Inverse, kein `TwoSidedInv`, keine Interior-Elimination, keine DtN-/GeneralizedDtN-/MultiSchur- und keine Diskriminantendaten ein.
Verbotene Kontinuitaet	Gruene Builds aus SM1–SM3d-alt zaehlen nicht mehr als DoD. Insbesondere wird eine diagonal gesetzte Interior-Inverse nicht als Kandidat des positiven Pfads behalten, sondern als dokumentiertes Negativmuster: Formel vor Entry-Lemma.
Neue Leitregel	Jede Matrixphase folgt der Reihenfolge: Quelle/Restriktion definieren, Entry-Lemma der Quelle beweisen, Strukturprofil klassifizieren, erst danach Matrixformel oder Inversenformel definieren, dann eintragsweise Produkt-/Kompatibilitaetslemmata beweisen.
Naechster Block	SM3eR hat den vorgeschriebenen Shape-Audit-Pfad fuer SM3daR gruen geschlossen: `candidateEntry_eq_interiorBlock` ist der Pflichtbefund, aber `InteriorBlock * InteriorBlock = 1` ist nicht abgeleitet. SM3db0R–SM3db7R sind als positive Kandidatenbau- und Handoff-Kette geschlossen; SM3db4R bleibt historisch green/partial, waehrend SM3db4aR den geschlossenen akkumulierten DoD traegt. Der naechste Implementierungsauftrag ist der neue SM3eR-Re-Run aus `SM3dbRToSM3eRHandoff`. Die Produktrechnung darf ausschliesslich gegen `H.candidateMatrix` laufen und darf nicht auf `candidateMatrix_from_candidateEntry E` oder die alte SM3daR-Oberflaeche zurueckfallen. `SM3dbOmegaR` bleibt verboten, solange der Handoff selbst nicht mathematisch dokumentiert scheitert; ein spaeteres Produktscheitern ist zuerst ein SM3eR-Befund. Explizit verboten bleiben freie Matrixdaten, freie Interior-Block-Tabellen, freie Inversen, `Matrix.inv`, diagonale Beta-Inversen ohne Diagonalitaetsbeweis, externe Inversen-Oracles, DtN-/GeneralizedDtN-/MultiSchur-Daten und Diskriminanten.

0at. Fortschreibungsentscheidung v71 nach gruenem SM7BridgeRecheck-Build: SM8 aktiv

Statusentscheidung	`SM7BridgeRecheck` ist nach lokal gruenem Build formal geschlossen. Der v70-Zielkorridor wurde eingehalten: eine kleine nachgelagerte Consumer-/Adapterdatei `GeneratedLevelHistorySchurIndexBridgeRecheckSM7a.lean` konsumiert `GeneratedLevelHistoryBoundaryPortSourceSM6a` und `GeneratedLevelHistorySchurIndexBridgeSM7`, liefert regular/variable echte Bridge-Witnesses und setzt den neuen Consumer-Pfad auf `bridgeSuccess`. Die alte `SM7GeneratedLevelHistoryToSchurIndexGate`-Obstruktion bleibt historisch bestehen, wird aber nicht mehr als aktiver Blocker fuer SM8 gelesen.
Formaler SM7BridgeRecheck-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistorySchurIndexBridgeSM7.fromSM6a`, `regularGeneratedLevelHistorySchurIndexBridge_from_SM6a`, `variableGeneratedLevelHistorySchurIndexBridge_from_SM6a`, `SM7BridgeRecheckLedger`, `sm7BridgeRecheckLedger`, `sm7BridgeRecheck_regular_outcome`, `sm7BridgeRecheck_variable_outcome`, `sm7BridgeRecheck_regularBridge_from_SM6a`, `sm7BridgeRecheck_variableBridge_from_SM6a`, `sm7BridgeRecheck_sm8_gate_released` sowie die Statusfelder gegen freie Enumeration, Ersatzindex, LevelHistoryMatrix, Matrixstart und Charpoly-/Diskriminantendaten.
Praeziser Restbefund	`SM7BridgeRecheck` erzeugt absichtlich noch keine Matrix. Es beweist nur, dass der History-Index ueber die in `SM6a` natuerlich entstandene Portquelle legal in einen `GeneratedBoundaryIndexA` konsumiert werden darf und dass die Boundary-Operator-/Schur-Rueckbindung aus `SM6`/`SM5`/`SM4` erhalten bleibt. Damit ist die Indexquelle nicht mehr der Engpass; der naechste Engpass ist jetzt die erste wirklich berechnete Level-History-Matrixoberflaeche.
Naechste aktive Phase	Die naechste aktive Phase ist `SM8 - GeneratedLevelHistoryMatrixFromBridge`. Sie soll genau eine kleine Matrixdatei erzeugen, vorzugsweise `GeneratedLevelHistoryMatrixFromBridgeSM8.lean`. Diese Datei importiert nur die Recheck-Schicht `GeneratedLevelHistorySchurIndexBridgeRecheckSM7a` und baut aus einem `GeneratedLevelHistorySchurIndexBridgeSM7`-Witness die Matrixeintraege auf `GeneratedLevelHistoryIndexSM6B.sourceSM6Witness.level`.
SM8-DoD	Minimaler starker DoD: `generatedSchurEntrySM8 B i j := B.sourceSM6Witness.boundaryOperator (B.toBoundaryIndex i) (B.toBoundaryIndex j)` und `generatedLevelHistoryMatrixSM8 B := fun i j => generatedSchurEntrySM8 B i j`. Dazu gehoeren entrywise Rueckbindungen an `boundaryOperator`, an `B.boundaryOperator_from_SM6` und an `B.boundaryOperator_from_schur`; regular/variable Konstruktoren aus `SM7BridgeRecheckLedger`; ein schmaler `SM8GeneratedLevelHistoryMatrixLedger`; und Statusfelder, dass die Matrix aus BridgeSuccess stammt, keine freie `LevelHistoryMatrixPackage.matrix` ist und noch keine Charpoly-/Faktor-/Diskriminantendaten erzeugt.
SM8-Grenze	`SM8` darf `generatedSchurEntry` und `generatedLevelHistoryMatrix` definieren, aber keine Determinante, kein Charpoly, keine Faktorisierung, keine Diskriminante, keine L2/L3-Zielentscheidung und keinen Higher-Level-Scan. `SM8` darf keine neue Boundary-Port-Quelle, keinen Ersatzindex, keine Fintype-Enumeration, keine freie Tabelle, kein `Classical.choice`, kein `Matrix.inv` und keine externe Matrixquelle einfuehren. Die einzige zulaessige Matrix ist der Pullback der bereits bewiesenen Boundary-Schur-Matrix entlang `B.toBoundaryIndex`.
Beweistaktik SM8	Die Kernbeweise sollten punktweise bleiben. `entry_eq_boundaryOperator` und `matrix_apply` sind `rfl`. Fuer `entry_eq_SM6` wird die Matrixgleichung `B.boundaryOperator_from_SM6` mit punktweiser `congrArg`-Auswertung auf `B.toBoundaryIndex i` und `B.toBoundaryIndex j` transportiert. Analog wird `entry_eq_schur` aus `B.boundaryOperator_from_schur` geholt. Keine globale `simp`-Flaeche, keine Summeninduktion, kein grosser Index-Eliminator; falls Lean bei Matrixfunktionsgleichheit sperrt, lokal ein Lemma der Form `matrix_entry_of_eq (h : A = C) i j : A i j = C i j := congrArg (fun M => M i j) h` einfuehren.
Overhead-Regel v71	Genau eine SM8-Datei plus `BuildAll`-Import ist der Zielkorridor. Kein `SM8a0`-Audit, kein separater Preflight, kein neues Status-Enum-Buendel ueber das Minimum hinaus und kein Umbau von `SM7BridgeRecheck`. Falls ein Typgleichheitsproblem beim Zugriff auf `sourceSM6Witness.boundaryOperator` sichtbar wird, wird die minimal fehlende Expose-Gleichung in SM8 selbst als Consumer-Hilfslemma formuliert; falls eine echte Quelle fehlt, wird nicht in SM8 gepatcht, sondern zur natuerlichen Quelle zurueckgegangen.
Anschlussgate	Nach gruenem `SM8`-Build wird `SM9` aktiv. `SM9` darf dann erstmals Charpoly-/Determinanten- oder operative Polynomdaten konsumieren, aber nur aus der in `SM8` berechneten `generatedLevelHistoryMatrix`. Wenn SM8 nur einen freien Matrix-Wrapper ohne entrywise Pullback-Beweis liefert, bleibt SM9 blockiert.
Fortgeschriebene Reihenfolge v71	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR`-`SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` **active**; `SM9` blocked until SM8 entrywise matrix.

0au. Fortschreibungsentscheidung v73 nach gruenem SM8-Build: SM9a-ExecMirrorSource aktiv

Statusentscheidung	`SM8 - GeneratedLevelHistoryMatrixFromBridge` ist nach lokal gruenem Build formal geschlossen. Der erlaubte v71-Korridor wurde eingehalten: `GeneratedLevelHistoryMatrixFromBridgeSM8.lean` erzeugt `generatedSchurEntrySM8` und `generatedLevelHistoryMatrixSM8` als entrywise Pullback von `sourceSM6Witness.boundaryOperator` entlang `B.toBoundaryIndex`. Damit ist die Matrixquelle generated und nicht frei.
Formaler SM8-Ertrag	Geschlossen sind insbesondere `generatedSchurEntrySM8`, `generatedLevelHistoryMatrixSM8`, `GeneratedLevelHistoryMatrixFromBridgeSM8`, die entrywise Rueckbindungen an `boundaryOperator`, `SM6/SM5` und den Schur-Ausdruck sowie regular/variable Konstruktoren aus `SM7BridgeRecheckLedger`. SM8 erzeugt weiterhin keine Determinante, kein Charpoly, keine Faktorisierung, keine Diskriminante und keinen Higher-Level-Scan.
Praeziser Restbefund	Die neue harte Engstelle ist Berechenbarkeit. Der SM8-Pfad traegt aktuell eine $\mathbb{R}$-Matrix. In den unmittelbar konsumierten SM8-/SM6a-/SM7BridgeRecheck-Dateien ist kein Feld `execMatrix : Matrix ... ExecComplex`, kein $\mathbb{Q}$-Entryprofil und kein theorematischer Pfeil `ExecComplexBridge.toComplex execEntry = (realEntry : ℂ)` exponiert. Der vorhandene AR2b-/`BoundarySource`-Typ `LevelHistoryMatrix L := Matrix (LevelHistoryIndex L) (LevelHistoryIndex L) ExecComplex` ist wichtig, aber er ist im jetzigen generated SM8-Pullback nicht als natuerliche Quelle angeschlossen. Daher waere ein sofortiger SM9-Charpoly-Start entweder nicht auswertbar oder wuerde auf freie Testmatrizen ausweichen.
Naechste aktive Phase	Die naechste aktive Phase ist `SM9a - GeneratedLevelHistoryExecMirrorSource`. Sie soll genau eine kleine Datei erzeugen, vorzugsweise `GeneratedLevelHistoryExecMirrorSourceSM9a.lean`. Diese Phase ist kein Charpoly-Modul und kein Heegner-Test. Sie stellt die Berechenbarkeitsquelle her oder schliesst proof-carrying, dass der direkte Pfeil noch an einer frueheren natuerlichen Schicht fehlt.
SM9a-DoD	Starker DoD ist ein Strukturtyp wie `GeneratedLevelHistoryExecMirrorSourceSM9a`. Minimalfelder: `sourceSM8Matrix : GeneratedLevelHistoryMatrixFromBridgeSM8 ...`; `entryReal := sourceSM8Matrix.matrix i j`; optional bzw. zielhaft `execMatrix : Matrix (GeneratedLevelHistoryMatrixIndexSM8 sourceSM8Matrix.sourceBridge) (GeneratedLevelHistoryMatrixIndexSM8 sourceSM8Matrix.sourceBridge) ExecComplex`; `exec_entry_bridge : forall i j, ExecComplexBridge.toComplex (execMatrix i j) = ((sourceSM8Matrix.matrix i j : ℝ) : ℂ)`; `entry_eq_SM8`; `entry_eq_schur`; `source_eq_weightPolicy_or_generatedRationalProfile`; Statusfelder `execMirrorConstructedFromSM8Provenance` oder `missingGeneratedRationalEntrySource`; Sperren gegen freie `ExecComplex`-Matrix, freie $\mathbb{R} \to \mathbb{Q}$-Konversion, `Matrix.inv`, `noncomputable`, `field_simp`, `mul_inv_cancel`, Charpoly und Heegner-Ziele.
Natuerliche Nachruestung	Falls der direkte Konstruktor in SM9a nicht moeglich ist, ist der korrekte naechste Schritt nicht SM9c und nicht SM10, sondern eine minimale Rueckschicht an der Entstehungsstelle der Boundary-/Schur-Eintraege. Kandidatenreihenfolge: zuerst SM6b fuer `GeneratedLevelStepExecMirrorSource`, falls der SM6-Witness nur eine Exec-Spiegelung des bereits packaged `boundaryOperator` braucht; andernfalls SM5q fuer `GeneratedArithmeticSourceExecBoundaryOperator`, falls regular/variable ArithmeticSource die natuerliche Stelle ist; andernfalls SM4q fuer `GeneratedMultiSchurExecBoundaryOperator`, falls die Schur-Boundary-Formel selbst rational/Exec getragen werden muss; nur falls die inverse/reciprocal Herkunft fehlt, Rueckgriff auf die SM3f/SM3eR-Quelle der `inverseMatrix`-Eintraege. Diese Rueckschichten sind keine Consumer-Patches, sondern Nachruestungen dort, wo die benoetigte Struktur natuerlich entsteht.
Beweistaktik SM9a	Zuerst rein punktweise: aus `F.matrix_eq_generatedLevelHistoryMatrix`, `F.entry_eq_boundaryOperator`, `F.entry_eq_SM6` und `F.entry_eq_schur` die Real-Eintragsprovenienz isolieren. Danach keine globale `simp`-Flaeche und keine Fall-Eliminator-Kaskade: falls eine Exec-Quelle vorhanden ist, wird jedes Entry durch ein vorhandenes rationales Weight-/Degree-/Reciprocal-Profil gebaut und die Bruecke ueber `ExecComplexBridge.toComplex_ofRat`, `toComplex_add`, `toComplex_sub`, `toComplex_mul` sowie kleine `calc`-Blöcke geschlossen. Falls sie nicht vorhanden ist, wird ein Missing-Source-Witness erzeugt, der exakt die frueheste fehlende Quelle benennt. Aus $x \neq 0$ wird keine berechenbare Inverse konstruiert.
SM9a-Grenze	SM9a darf keine `Schur.l2RhoMultiplicationMatrix`, keine `Schur.l3GaussMultiplicationMatrix` und keine handgeschriebene `Matrix (Fin 2) (Fin 2) ExecComplex` als Smoke-Provenienz einsetzen. Isolierte Infoview-`#eval`-Dateien bleiben erlaubt, aber nur als Diagnostik ausserhalb `BuildAll`. Sie koennen den Weg testen, ersetzen aber keinen derived-only Pfeil von SM8 zur berechenbaren Matrix.
Anschlussgate SM9b–SM12	Erst nach einem gruenen SM9a- oder Rueckschicht-Build, der eine unbedingte `execMatrix`-Spiegelung fuer die SM8-Provenienz liefert, wird `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` freigegeben. `SM10` und `SM11` sind danach reine Vergleichsphasen gegen Eisenstein/Gauss; `SM12` wird erst danach zum Parameter-Scan ueber Regime, code-b/arity, Level und Gewicht. Die Hypothese `noOuterEnvironment` fuer Eisenstein/Gauss und `hasOuterEnvironment` fuer weitere Heegner-Faelle bleibt falsifizierbare Scan-Hypothese, nicht Zielbedingung.
Overhead-Regel v73	Genau eine SM9a-Datei plus `BuildAll`-Import ist der Zielkorridor. Kein SM9a0/SM9a1-Auditwald. SM9a muss entweder den ExecMirror bauen oder mit einem praezisen Missing-Source-Witness die minimale natuerliche Rueckschicht festlegen. Die Charpoly-Phase bleibt blockiert, solange keine berechenbare SM8-Matrix vorliegt.

Fortgeschriebene Reihenfolge v73	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` closed/green; **`SM9a-ExecMirrorSource` active**; `SM9b/SM9c/SM10/SM11/SM12` blocked until a computed SM8 ExecMirror exists.

0av. Fortschreibungsentscheidung v74 nach gruenem SM9a-Build: SM6b-ExecMirrorSource aktiv

Statusentscheidung	`SM9a - GeneratedLevelHistoryExecMirrorSource` ist nach lokal gruenem Build formal geschlossen, aber nicht als ExecMirror-Abschluss. Der gruene Befund ist ein sauberer Gate-Befund: `GeneratedLevelHistoryExecMirrorSourceSM9a.lean` konsumiert die `SM8`-Matrix, bindet die Real-Eintraege an `generatedLevelHistoryMatrixSM8`, `sourceSM6Witness.boundaryOperator`, die SM6/SM5-Quelle und den Schur-Ausdruck zurueck, und sperrt Charpoly/Heegner ohne berechenbaren Mirror.
Formaler SM9a-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryExecMirrorWitnessSM9a` als Zielinterface, `GeneratedLevelHistoryExecMirrorSourceSM9a` als proof-carrying Source-/Missing-Source-Struktur, `SM9aGeneratedLevelHistoryExecMirrorSourceLedger`, regular/variable Konstruktoren aus `SM8`, sowie die Statussaetze `sm9a_regularConstruction_blocked_missingSource`, `sm9a_variableConstruction_blocked_missingSource`, `sm9a_earliestBackfill_is_SM6b`, `sm9a_nextPhase_is_SM6b`, `sm9a_noCharpolyDiscriminant` und `sm9a_noHeegnerTarget`.
Praeziser Restbefund	Der fehlende Pfeil ist nicht mehr unspezifisch: Es fehlt eine generated rationale/operative Entry-Quelle fuer den schon in `SM6/SM8` getragenen Boundary-/Schur-Eintrag. `SM9a` darf diesen Pfeil nicht am Consumer-Ende erfinden; es darf auch nicht von der $\mathbb{R}$-Matrix nach $\mathbb{Q}$ zurueckraten. Die natuerlich frueheste bekannte Nachrueststelle ist `SM6`, weil dort `GeneratedLevelStepWitnessSM6` die packaged `boundaryOperator`-Oberflaeche samt `boundaryOperator_eq_SM5`, `boundaryOperator_eq_schur`, `apply_eq_SM5` und `apply_eq_mulVec` zum ersten Mal fuer die Level-History-Route bereitstellt.
Naechste aktive Phase	Die naechste aktive Phase ist `SM6b - GeneratedLevelStepExecMirrorSource`. Implementierungsdatei: vorzugsweise genau `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelStepExecMirrorSourceSM6b.lean` plus ein `BuildAll`-Import. `SM6b` ist keine neue Charpoly-Phase, kein SM9b-Vorgriff und kein SM5q/SM4q-Ruecksprung. Es ist der positive Versuch, aus dem SM6-Witness selbst eine proof-carrying `ExecComplex`/$\mathbb{Q}$-Entryquelle fuer den Boundary-Operator zu exponieren.
SM6b-DoD	Starker DoD ist ein Strukturtyp wie `GeneratedLevelStepExecMirrorSourceSM6b`. Minimalfelder: `sourceSM6Witness`; `boundaryOperatorReal := sourceSM6Witness.boundaryOperator`; zielhaft `execBoundaryOperator : Matrix (GeneratedBoundaryIndex A) (GeneratedBoundaryIndex A) ExecComplex`; `exec_entry_bridge : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM6Witness.boundaryOperator i j : ℝ) : ℂ)`; `entry_eq_SM5`; `entry_eq_schur`; Herkunftsstatus aus SM6/SM5/SM4; Sperren gegen freie `ExecComplex`-Matrix, freie $\mathbb{R} \to \mathbb{Q}$-Konversion, skalare Inversion, `Matrix.inv`, `noncomputable`, `field_simp`, Charpoly, Diskriminante und Heegner-Ziele. Falls diese Exec-Quelle im aktuellen API-Stand nicht konstruierbar ist, muss `SM6b` proof-carrying und minimal belegen, dass die naechste natuerliche Quelle `SM5q - GeneratedArithmeticSourceExecBoundaryOperator` ist.
Beweistaktik SM6b	Zuerst rein entrywise und strukturell: mit `sourceSM6Witness.boundaryOperator_eq_SM5`, `sourceSM6Witness.boundaryOperator_eq_schur`, `sourceSM6Witness.apply_eq_SM5` und `sourceSM6Witness.apply_eq_mulVec` die Real-Provenienz isolieren. Danach zwei Moeglichkeiten: Wenn eine operative `ExecComplex`-Quelle in SM6/SM5/SM4 bereits exponiert ist, wird `execBoundaryOperator i j` daraus definiert und die Bruecke mit `ExecComplexBridge.toComplex_ofRat`, `toComplex_add`, `toComplex_sub`, `toComplex_mul` und kleinen `calc`-Bloecken geschlossen. Wenn keine solche Quelle vorhanden ist, wird kein Ersatzwert gebaut; dann wird ein Missing-Source-Witness erzeugt, der exakt auf `SM5q` zeigt. Aus Nichtnullheit wird keine berechenbare Reziproke konstruiert.
Import- und Consumer-Disziplin	`SM6b` darf nur `GeneratedLevelStepWitnessSM6` und die minimal noetige `ExecComplexBridge`-Schicht importieren. Es darf `SM7`, `SM8` und `SM9a` nicht konsumieren, weil `SM6b` als natuerlich fruehere Erzeugungsschicht vor diesen Consumern liegt. Rueckbindung nach oben erfolgt spaeter durch einen kleinen Recheck-/Adapter-Schritt, nicht in `SM6b`.
Overhead-Regel v74	Genau eine kleine SM6b-Datei plus `BuildAll`-Import ist der Zielkorridor. Kein `SM6b0/SM6b1`-Auditwald, keine neue Status-Enum-Familie fuer jedes Mikrolemma, keine Diagnosemodule im Build. Die Datei darf einen Source-Typ, einen Ledger/Konstruktor fuer regular und variable sowie wenige Projektionssaetze enthalten. Wenn der positive Konstruktor blockiert, muss die Datei mit einem einzelnen praezisen Missing-Source-Gate enden.

Anschlussgate nach SM6b	Falls SM6b eine unbedingte execBoundaryOperator-Quelle liefert, folgt als naechstes ein kleiner Recheck an der bereits bestehenden Bridge-Kette: SM7BridgeRecheck/SM8 muessen die SM6b-Execquelle bis zur SM8-Level-History-Matrix weiterreichen, ohne die Quelle neu zu erzeugen. Erst danach wird SM9a erneut als positiver Mirror-Consumer ausgefuehrt; erst ein unbedingter SM8-ExecMirror gibt SM9b - GeneratedLevelHistoryCharpolyExecFromMirror frei. Falls SM6b blockiert, ist SM5q die naechste positive Nachruestphase, nicht SM9b.
Fortgeschriebene Reihenfolge v74	r3c1a closed/green; r3c1b0 closed/green; SM3db2aR-SM3db2fR closed/green bzw. closed/green-audit; SM3eR-r3c1b1 closed/green; SM3eR-r3c1b-full closed/green; SM3eR-r3c1c closed/green; SM3eR-r3c1d green/gate-closed; SM3db3aR closed/green-audit; SM3db4aT/U/V/W closed/green; SM3eR-r3c1d-full closed/green; SM3eR-r3c1e closed/green; SM3eR-r3c2 closed/green; SM3eR-r4a closed/green; SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 closed/green; SM3fR closed/green; SM3gR closed/green; SM3hR closed/green; SM3iR closed/green; SM4 closed/green; SM5 closed/green; SM6 closed/green; SM7 closed/green-obstruction historisch; SM6a closed/green; SM7BridgeRecheck closed/green; SM8 closed/green; SM9a closed/green-missing-source; SM6b-GeneratedLevelStepExecMirrorSource **active**; SM5q/SM4q conditional fallback only after SM6b-Missing-Source; SM9b/SM9c/SM10/SM11/SM12 blocked until a computed SM8 ExecMirror exists.

0aw. Fortschreibungsentscheidung v75 nach gruenem SM6b-Build: SM5q aktiv

Statusentscheidung	SM6b - GeneratedLevelStepExecMirrorSource ist nach lokal gruenem Build formal geschlossen, aber nicht als ExecMirror-Abschluss. Der gruene Befund ist sauber und minimal: GeneratedLevelStepExecMirrorSourceSM6b.lean bindet sourceSM6Witness.boundaryOperator an SM6/SM5/SM4-Schur zurueck, exponiert apply- und Entry-Provenienz und sperrt freie ExecComplex-Matrizen. Zugleich zeigt der Status blockedMissingGeneratedArithmeticSourceExecBoundaryOperator, dass die naechste natuerliche Quelle nicht in SM6, SM7, SM8 oder SM9 liegt, sondern eine Schicht tiefer in SM5.
Formaler SM6b-Ertrag	Geschlossen sind insbesondere GeneratedLevelStepExecMirrorSourceSM6b, GeneratedLevelStepExecMirrorWitnessSM6b als Zielinterface ohne freien Konstruktor, SM6bGeneratedLevelStepExecMirrorSourceLedger, regular/variable Konstruktoren aus dem SM6GeneratedLevelStepWitnessLedger, sowie Projektionssaetze wie entryReal_from_SM5, entryReal_from_schur, apply_from_SM5, apply_from_boundaryOperatorReal_mulVec, execMirror_blocked_missing_SM5q, nextPhase_is_SM5q und sm6b_noForbiddenShortcuts. Kein Matrix.inv, keine Rueckuebersetzung $\mathbb{R} \to \mathbb{Q}$, keine freie Exec-Matrix, keine Charpoly-/Faktor-/Diskriminantendaten.
Praeziser Restbefund	Der fehlende Pfeil lautet jetzt exakt: GeneratedArithmeticSourceSM5.boundaryOperator besitzt im aktuellen API-Stand nur Real-Provenienz ueber boundaryOperator_eq_SM4, boundaryOperator_eq_schur, apply_eq_SM4 und apply_eq_mulVec. Es gibt noch kein SM5-nahes Feld execBoundaryOperator : Matrix ... ExecComplex, kein rationales Entryprofil und keinen theorematischen Brueckensatz ExecComplexBridge.toComplex execEntry = ((realEntry : R) : C). Ein Start von SM9b waere daher weiterhin ein Consumer-Patch oder eine freie Testmatrix.
Naechste aktive Phase	Die naechste aktive Phase ist SM5q - GeneratedArithmeticSourceExecBoundaryOperator. Implementierungsdatei: genau CNNA/PillarA/Arithmetic/Smoke/GeneratedArithmeticSourceExecBoundaryOperatorSM5q.lean plus ein BuildAll-Import unmittelbar nach GeneratedArithmeticSourceSM5 und vor GeneratedLevelStepWitnessSM6. SM5q ist keine neue SM6b-Variante, kein SM9a-Recheck, kein Charpoly-Modul und kein Heegner-Test.
SM5q-DoD	Starker DoD ist ein kleiner Source-/Gate-Typ wie GeneratedArithmeticSourceExecBoundaryOperatorSM5q. Minimalfelder: sourceSM5; boundaryOperatorReal := sourceSM5.boundaryOperator; boundaryOperatorReal_eq_SM5; entry_eq_SM4; entry_eq_schur; apply_eq_SM4; apply_eq_mulVec; zielhaft nur falls bereits aus SM5/SM4-Provenienz ableitbar: execBoundaryOperator : Matrix (GeneratedBoundaryIndex A) (GeneratedBoundaryIndex A) ExecComplex und exec_entry_bridge : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM5.boundaryOperator i j : R) : C). Falls diese Exec-Quelle nicht konstruierbar ist, muss SM5q mit einem einzelnen Missing-Source-Gate auf SM4q - GeneratedMultiSchurExecBoundaryOperator schliessen.
Beweistaktik SM5q	Zuerst keine neue Rechnung: boundaryOperatorReal := sourceSM5.boundaryOperator; alle Real-Gleichungen per Feldprojektion und congrArg (fun M => M i j) aus sourceSM5.boundaryOperator_eq_SM4, sourceSM5.boundaryOperator_eq_schur, sourceSM5.apply_eq_SM4 und sourceSM5.apply_eq_mulVec. Danach nur falls ein operatives SM5-/SM4-Exec-Profil existiert: execBoundaryOperator i j direkt daraus setzen und mit ExecComplexBridge.toComplex_ofRat, toComplex_add, toComplex_sub, toComplex_mul, toComplex_sum und kleinen calc-Bloecken schliessen. Keine Konstruktion aus Nichtnullheit, keine x^{-1}-Definition, keine globale simp-Flaeche; lokale rfl, Feldprojektionen, rw/calc reichen fuer den Gate-Fall.
Import- und Consumer-Disziplin	SM5q darf nur GeneratedArithmeticSourceSM5 und die minimal noetige ExecComplexBridge-Schicht importieren. Es darf SM6, SM6b, SM7, SM8, SM9a nicht konsumieren, weil SM5q die natuerlich fruehere Erzeugungsschicht ist. Eine spaetere Rueckbindung nach oben erfolgt erst nach einem SM5q- oder SM4q-Erfolg ueber einen kleinen Recheck/Adapter.

Overhead-Regel v75	Genau eine kleine Datei plus ein `BuildAll`-Import. Kein `SM5q0/SM5q1`-Auditwald, keine neuen Statusfamilien fuer jedes Mikrolemma, keine Parallel-Ledger. Zulaessig sind ein Source-/Gate-Typ, ein regular/variable Ledger/Konstruktor und wenige Projektionssaetze. Wenn der positive Konstruktor blockiert, endet `SM5q` mit einem einzigen praezisen `SM4q`-Missing-Source-Gate.
Anschlussgate nach SM5q	Falls `SM5q` eine unbedingte `execBoundaryOperator`-Quelle liefert, folgt ein kleiner Upstream-Recheck: SM6/SM6b und anschliessend SM8/SM9a duerfen diese Quelle nur weiterreichen, nicht neu erzeugen. Erst wenn daraus ein berechneter SM8-ExecMirror vorliegt, wird `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` freigegeben. Falls `SM5q` blockiert, ist `SM4q` die naechste positive Nachruestphase; falls `SM4q` wiederum an der Schur-Formel haengt, wird danach minimal an die SM3f/SM3eR-Herkunft der `inverseMatrix`-Eintraege zurueckgegangen.
Fortgeschriebene Reihenfolge v75	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` closed/green; `SM9a` closed/green-missing-source; `SM6b` closed/green-missing-`SM5q`; `SM5q - GeneratedArithmeticSourceExecBoundaryOperator` **active**; `SM4q` conditional fallback only after SM5q-Missing-Source; `SM9b/SM9c/SM10/SM11/SM12` blocked until a computed SM8 ExecMirror exists.

0ax. Fortschreibungsentscheidung v76 nach gruenem SM5q-Build: SM4q aktiv

Statusentscheidung	`SM5q - GeneratedArithmeticSourceExecBoundaryOperator` ist nach lokal gruenem Build formal geschlossen, aber nicht als `ExecBoundaryOperator`-Abschluss. Der Build ist ein sauberer Missing-Source-Befund: `GeneratedArithmeticSourceExecBoundaryOperatorSM5q.lean` exponiert `sourceSM5.boundaryOperator` als $\mathbb{R}$-Matrix, bindet Eintraege und Anwendung an SM5/SM4-Schur zurueck, sperrt freie Exec-Matrizen und setzt `nextPhase_is_SM4q`. Damit ist die naechste positive Phase nicht SM6/SM8/SM9 und auch kein SM5q-Unterwald, sondern eine Schicht tiefer in SM4.
Formaler SM5q-Ertrag	Geschlossen sind insbesondere `GeneratedArithmeticSourceExecBoundaryOperatorSM5q`, `SM5qGeneratedArithmeticSourceExecBoundaryOperatorLedger`, regular/variable Konstruktoren aus dem `SM5GeneratedArithmeticSourceLedger`, sowie Projektionssaetze wie `boundaryOperatorReal_from_SM5`, `entryReal_from_SM4`, `entryReal_from_schur`, `apply_from_SM4`, `apply_from_boundaryOperatorReal_mulVec`, `execBoundaryOperator_blocked_missing_SM4q`, `nextPhase_is_SM4q` und `noForbiddenShortcuts_for_SM5q`. Kein `Matrix.inv`, keine Rueckuebersetzung $\mathbb{R} \to \mathbb{Q}$, keine freie Exec-Matrix, kein Charpoly-, Faktor-, Diskriminanten- oder Heegner-Datum.
Praeziser Restbefund	Der fehlende Pfeil lautet jetzt exakt: `GeneratedMultiSchurSM4.boundaryOperator` besitzt im aktuellen API-Stand eine Real-Provenienz ueber `boundaryOperator_eq_SM3iR`, `boundaryOperator_eq_topBoundary`, `boundaryOperator_eq_certificate`, `boundaryOperator_eq_schur` und `apply_eq_mulVec`. Es gibt noch kein SM4-nahes Feld `execBoundaryOperator : Matrix ... ExecComplex`, kein rationales Entryprofil fuer die Schur-Formel und keinen theorematischen Brueckensatz von einem operativen Schur-Eintrag zu $((G.boundaryOperatorij : \mathbb{R}) : \mathbb{C})$. Ein Start von `SM9b` waere daher weiterhin ein Consumer-Patch oder eine freie Testmatrix.
Naechste aktive Phase	Die naechste aktive Phase ist `SM4q - GeneratedMultiSchurExecBoundaryOperator`. Implementierungsdatei: genau `CNNA/PillarA/Arithmetic/Smoke/GeneratedMultiSchurExecBoundaryOperatorSM4q.lean` plus ein `BuildAll`-Import unmittelbar nach `GeneratedMultiSchurSM4` und vor `GeneratedArithmeticSourceSM5`. `SM4q` ist keine neue SM5q-Variante, kein SM6b-Recheck, kein SM9a-/SM9b-Consumer, kein Charpoly-Modul und kein Heegner-Test.
SM4q-DoD	Starker DoD ist ein kleiner Source-/Gate-Typ wie `GeneratedMultiSchurExecBoundaryOperatorSM4q`. Minimalfelder: `sourceSM4`; `boundaryOperatorReal := sourceSM4.boundaryOperator`; `boundaryOperatorReal_eq_SM4`; `entry_eq_SM3iR`; `entry_eq_topBoundary`; `entry_eq_certificate`; `entry_eq_schur`; `apply_eq_mulVec`; `certificate_inverseMatrix_eq_handoffCandidate`; Statusfelder fuer SM4-Schur-Provenienz, Missing-Source und naechste Phase. Zielhaft nur falls bereits aus SM4/SM3f/SM3eR-Provenienz ableitbar: `execBoundaryOperator : Matrix (GeneratedBoundaryIndex A) (GeneratedBoundaryIndex A) ExecComplex` und `exec_entry_bridge : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM4.boundaryOperator i j : R) : C)`. Falls diese Exec-Quelle nicht konstruierbar ist, muss `SM4q` mit einem einzigen Missing-Source-Gate auf die frueheste fehlende SM3-nahe Quelle schliessen.

Beweistaktik SM4q	Zuerst keine neue Rechnung: `boundaryOperatorReal := sourceSM4.boundaryOperator`; alle Real-Gleichungen per Feldprojektion und `congrArg (fun M => M i j)` aus `sourceSM4.boundaryOperator_eq_SM3iR`, `sourceSM4.boundaryOperator_eq_topBoundary`, `sourceSM4.boundaryOperator_eq_certificate` und `sourceSM4.boundaryOperator_eq_schur`. Die Anwendungsgleichung kommt direkt aus `sourceSM4.apply_eq_mulVec`; die inverseMatrix-Herkunft aus `sourceSM4.certificate_inverseMatrix_eq_handoffCandidate`. Danach nur falls operative Entryquellen fuer die drei Schur-Bloecke und die `inverseMatrix/H.candidateMatrix` existieren: `execBoundaryOperator i j` als operative Schur-Entryformel definieren, Produkte und Summen mit `ExecComplexBridge.toComplex_add`, `toComplex_sub`, `toComplex_mul`, `toComplex_sum` und kleinen `calc`-Bloecken abarbeiten. Keine Konstruktion aus Nichtnullheit, keine x^{-1}-Definition, keine globale `simp`-Flaeche, kein `field_simp`; nur `rfl`, Feldprojektionen, `rw`, `calc` und entrywise Matrixextensionalitaet.
Import- und Consumer-Disziplin	`SM4q` darf nur `GeneratedMultiSchurSM4` und die minimal noetige `ExecComplexBridge`-Schicht importieren. Es darf `GeneratedArithmeticSourceSM5`, `GeneratedLevelStepWitnessSM6`, `GeneratedLevelStepExecMirrorSourceSM6b`, `SM7`, `SM8`, `SM9a` und `SM9b` nicht konsumieren. Grund: `SM4q` ist die natuerlich fruehere Erzeugungsschicht; jede Rueckbindung nach oben erfolgt spaeter durch kleine Recheck-/Adapterphasen.
Overhead-Regel v76	Genau eine kleine Datei plus ein `BuildAll`-Import. Kein `SM4q0/SM4q1`-Auditwald, keine Parallel-Ledger und keine neuen Statusfamilien fuer jedes Mikrolemma. Zulaessig sind ein Source-/Gate-Typ, ein regular/variable Ledger/Konstruktor und wenige Projektionssaetze. Wenn der positive Konstruktor blockiert, endet `SM4q` mit einem einzigen praezisen Missing-Source-Gate.
Anschlussgate nach SM4q	Falls `SM4q` eine unbedingte `execBoundaryOperator`-Quelle liefert, folgen kleine Upstream-Rechecks: `SM5q`, `SM6b` und danach `SM8/SM9a` duerfen diese Quelle nur weiterreichen, nicht neu erzeugen. Erst ein daraus berechneter SM8-ExecMirror gibt `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` frei. Falls `SM4q` blockiert, ist die naechste positive Rueckschicht nicht ein Consumer-Patch, sondern die minimal fruehere Quelle der fehlenden Schur-Exec-Eintraege: zuerst SM3f-nahe `GeneratedInteriorEliminationCertificate`-/Block-Entry-Provenienz, danach nur falls noetig die SM3eR-/CandidateMatrix-Herkunft der `inverseMatrix`-Eintraege.
Fortgeschriebene Reihenfolge v76	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM5` closed/green; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` closed/green; `SM9a` closed/green-missing-source; `SM6b` closed/green-missing-`SM5q`; `SM5q` closed/green-missing-`SM4q`; `SM4q - GeneratedMultiSchurExecBoundaryOperator` **active**; `SM9b/SM9c/SM10/SM11/SM12` blocked until a computed SM8 ExecMirror exists.

0ay. Fortschreibungsentscheidung v77 nach gruenem SM4q-Build: SM3fq aktiv

Statusentscheidung	`SM4q - GeneratedMultiSchurExecBoundaryOperator` ist nach lokal gruenem Build formal geschlossen, aber nicht als `ExecBoundaryOperator`-Abschluss. Die Phase hat genau den geforderten SM4-nahen Befund geliefert: `boundaryOperatorReal := sourceSM4.boundaryOperator`, entrywise Rueckbindung auf `boundaryOperator_eq_SM3iR`, `boundaryOperator_eq_topBoundary`, `boundaryOperator_eq_certificate`, `boundaryOperator_eq_schur`, `apply_eq_mulVec` und `certificate_inverseMatrix_eq_handoffCandidate`; zugleich sperrt sie freie Exec-Matrizen, Rueckuebersetzung $\mathbb{R} \to \mathbb{Q}$, `Matrix.inv`, `field_simp`, Charpoly-, Diskriminanten- und Heegnerdaten.
Formaler SM4q-Ertrag	Geschlossen sind insbesondere `GeneratedMultiSchurExecBoundaryOperatorSM4q`, `SM4qGeneratedMultiSchurExecBoundaryOperatorLedger`, regular/variable Konstruktoren aus dem `SM4GeneratedMultiSchurLedger`, sowie Projektionssaetze wie `boundaryOperatorReal_from_SM4`, `entryReal_from_SM3iR`, `entryReal_from_topBoundary`, `entryReal_from_certificate`, `entryReal_from_schur`, `apply_from_SM3iR`, `apply_from_boundaryOperatorReal_mulVec`, `certificate_inverseMatrix_from_handoffCandidate`, `execBoundaryOperator_blocked_missing_SM3f_SM3eR_source`, `missing_exec_source_is_operational_schur_bridge`, `nextPhase_is_SM3f_SM3eR_exec_schur_source` und `noForbiddenShortcuts_for_SM4q`. Kein `execBoundaryOperator`-Witness wurde frei erzeugt.
Praeziser Restbefund	Der fehlende Pfeil ist nun exakt SM3f-nah: `generatedBoundaryOperatorSM3fR H = generatedBoundaryBlockSM3fR W - generatedBoundaryInteriorBlockSM3fR W * H.candidateMatrix * generatedInteriorBoundaryBlockSM3fR W` liegt nur als $\mathbb{R}$-Schur-Formel vor. Es gibt im aktuellen API-Stand keinen operativen Eintragswitness fuer die drei Dirichlet-Bloecke, keinen operativen Eintragswitness fuer `H.candidateMatrix` bzw. `GeneratedInteriorAccumulatedEntryValue T`, und deshalb keinen theorematischen Brueckensatz von einem `ExecComplex`-Schur-Eintrag zu `((generatedBoundaryOperatorSM3fR H i j : R) : C)`. Ein Start von SM5q/SM6b/SM8/SM9 waere daher weiterhin ein Consumer-Patch.

Naechste aktive Phase	Die naechste aktive Phase ist `SM3fq - GeneratedSM3fSchurExecEntrySource`. Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedSM3fSchurExecEntrySourceSM3fq.lean` plus ein `BuildAll`-Import unmittelbar nach `GeneratedInteriorEliminationCertificateSM3fR` und vor `GeneratedTopBoundaryDtNSM3gR`. `SM3fq` ist keine SM4q-Variante, kein SM5q-Recheck, kein SM6b-/SM8-/SM9-Consumer, kein Charpoly-Modul und kein Heegner-Test.
SM3fq-DoD	Starker DoD ist ein kleiner Source-/Gate-Typ, z.B. `GeneratedSM3fSchurExecEntrySourceSM3fq`. Minimalfelder: `sourceCertificate : GeneratedInteriorEliminationCertificateSM3fR ...`; Real-Blockquellen `boundaryBlockReal`, `boundaryInteriorBlockReal`, `interiorBoundaryBlockReal`, `inverseMatrixReal`, `boundaryOperatorReal`; Gleichungen `boundaryBlockReal_eq_SM3f`, `boundaryInteriorBlockReal_eq_SM3f`, `interiorBoundaryBlockReal_eq_SM3f`, `inverseMatrixReal_eq_handoffCandidate`, `boundaryOperatorReal_eq_schur`; Statusfelder fuer SM3f-Schur-Provenienz, Missing-Subsource und naechste Phase. Zielhaft nur falls bereits ableitbar: `execBoundaryOperator : Matrix (GeneratedBoundaryIndex A) (GeneratedBoundaryIndex A) ExecComplex` und `exec_entry_bridge : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((generatedBoundaryOperatorSM3fR H i j : R) : C)`. Falls die noetigen Exec-Subquellen nicht existieren, endet SM3fq mit einem einzigen praezisen Missing-Subsource-Gate auf die frueheste fehlende Quelle: `SM3bq - GeneratedDirichletExecEntrySource` fuer Dirichlet-/WeightPolicy-Eintraege bzw. `SM3eRq - GeneratedInverseCandidateExecEntrySource` fuer `H.candidateMatrix`/AccumulatedEntry-Eintraege.
Beweistaktik SM3fq	Zuerst keine neue Rechnung: Real-Blockfelder werden definitionell aus `generatedBoundaryBlockSM3fR W`, `generatedBoundaryInteriorBlockSM3fR W`, `generatedInteriorBoundaryBlockSM3fR W`, `H.candidateMatrix` und `generatedBoundaryOperatorSM3fR H` gesetzt; Real-Gleichungen sind `rfl` oder Feldprojektionen aus `sourceCertificate.inverseMatrix_eq_handoffCandidate` und `sourceCertificate.boundaryOperator_eq_schur`. Falls Exec-Subquellen vorhanden sind, wird die Schur-Entryformel entrywise definiert: `boundaryBlockExec i j - sum k, boundaryInteriorExec i k * (sum l, inverseExec k l * interiorBoundaryExec l j)` bzw. eine explizite assoziativ passende Variante. Der Bridge-Beweis erfolgt mit `ExecComplexBridge.toComplex_sub`, `toComplex_mul`, `toComplex_sum`, `Matrix.mul_apply`, `Finset.sum_congr` und kleinen `calc`-Bloecken. Nicht verwenden: globale `simp`-Flaechen, `field_simp`, `Matrix.inv`, skalare Inversion, freie Skalentabellen, klassische Hilfen oder ein grosser Index-Eliminator.
Import- und Consumer-Disziplin	`SM3fq` darf nur `GeneratedInteriorEliminationCertificateSM3fR` und die minimal noetige `ExecComplexBridge`-Schicht importieren. Es darf `GeneratedTopBoundaryDtNSM3gR`, `GeneratedGeneralizedTopBoundaryDtNSM3hR`, `GeneratedResetSM3LedgerSM3iR`, `GeneratedMultiSchurSM4`, `SM4q`, `SM5`, `SM5q`, `SM6`, `SM6b`, `SM8` und `SM9a` nicht konsumieren. Grund: `SM3fq` ist die natuerlich fruehere Schur-Entry-Erzeugungsschicht; spaetere Phasen duerfen diese Quelle nur weiterreichen.
Overhead-Regel v77	Genau eine kleine Datei plus ein `BuildAll`-Import. Kein `SM3fq0`/`SM3fq1`-Auditwald, keine parallelen Ledgerfamilien und keine Mikrostatusfamilien fuer jede Matrixklammerung. Zulaessig sind ein Source-/Gate-Typ, optional ein Witness-Typ fuer den positiven Exec-Fall, ein regular/variable Ledger/Konstruktor und wenige Projektionssaetze. Falls der positive Konstruktor blockiert, endet `SM3fq` mit genau einem Missing-Subsource-Gate, das die naechste natuerliche Erzeugungsschicht eindeutig markiert.
Anschlussgate nach SM3fq	Falls `SM3fq` eine unbedingte `execBoundaryOperator`-Quelle liefert, folgen kleine Forward-/Recheck-Phasen: `SM4q`, `SM5q`, `SM6b`, `SM8` und `SM9a` duerfen die Quelle nur konsumieren und weiterreichen, nicht neu erzeugen. Falls `SM3fq` blockiert, wird nicht in SM4/SM5/SM6/SM9 gepatcht; dann wird minimal an der ersten fehlenden Subquelle nachgeruestet, zuerst `SM3bq` fuer operative Dirichlet-/WeightPolicy-Eintraege oder, wenn diese bereits tragen, `SM3eRq` fuer operative `H.candidateMatrix`/AccumulatedEntry-Eintraege.
Fortgeschriebene Reihenfolge v77	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3fq - GeneratedSM3fSchurExecEntrySource` **active**; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM4q` closed/green-missing-`SM3fq`; `SM5` closed/green; `SM5q` closed/green-missing-`SM4q`; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` closed/green; `SM9a` closed/green-missing-source; `SM6b` closed/green-missing-`SM5q`; `SM9b/SM9c/SM10/SM11/SM12` blocked until a computed SM8 ExecMirror exists.

0az. Fortschreibungsentscheidung v78 nach gruenem SM3fq-Build: SM3bq aktiv

Statusentscheidung	`SM3fq - GeneratedSM3fSchurExecEntrySource` ist nach lokal gruenem Build formal geschlossen, aber bewusst nicht als operativer `ExecBoundaryOperator`-Abschluss. Der geschlossene Ertrag ist die SM3f-nahe Real-Provenienz der Boundary-Schur-Formel plus ein proof-carrying Missing-Subsource-Gate. Da dieses Gate eindeutig `missingSM3bqDirichletWeightPolicyExecEntriesBeforeInverseExecEntries` bzw. `sm3bqGeneratedDirichletExecEntrySource` traegt, ist die naechste aktive Phase nicht `SM4q/SM5q/SM6b/SM8/SM9a` und noch nicht `SM3eRq`, sondern `SM3bq - GeneratedDirichletExecEntrySource`.
Formaler SM3fq-Ertrag	Geschlossen sind insbesondere `GeneratedSM3fSchurExecEntrySource SM3fq`, `SM3fqSchurEntrySourceStatus`, `SM3fqExecConstructionStatus`, `SM3fqMissingSubsourceGate`, `SM3fqNextPhaseStatus`, `SM3fqNoForbiddenShortcutStatus`, `regularGeneratedSM3fSchurExecEntrySource_from_accumulatedEntryCancellationLedgerSM3fq`, `variableGeneratedSM3fSchurExecEntrySource_from_accumulatedEntryCancellationLedgerSM3fq`, `SM3fqGeneratedSM3fSchurExecEntrySourceLedger` und `sm3fqGeneratedSM3fSchurExecEntrySourceLedger`. Die Real-Blockquellen sind auf `generatedBoundaryBlockSM3fR W`, `generatedBoundaryInteriorBlockSM3fR W`, `generatedInteriorBoundaryBlockSM3fR W`, `sourceCertificate.inverseMatrix`, `H.candidateMatrix` und `sourceCertificate.boundaryOperator` zurueckgebunden.
Praeziser Restbefund	`SM3fq` beweist noch keine `ExecComplex`-Schur-Matrix. Es fehlt zuerst die operative Dirichlet-/WeightPolicy-Eintragsschicht: `generatedEntryWeight W i j`, `generatedDirichlet_degree W i`, `generatedDirichlet_entry W i j` und `generatedDirichletMatrix W` muessen als `ExecComplex`/$\mathbb{Q}$-Daten mit Bridge nach $\mathbb{C}$ exponiert werden. Erst danach ist sinnvoll pruefbar, ob die inverse Kandidatenmatrix `H.candidateMatrix`/`GeneratedInteriorAccumulatedEntryValue T` eine eigene operative Entry-Quelle `SM3eRq` braucht.
Naechste aktive Phase	`SM3bq - GeneratedDirichletExecEntrySource`. Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedDirichletExecEntrySourceSM3bq.lean`. Import in `BuildAll` unmittelbar nach `GeneratedDirichletEntry` bzw. vor den ersten Consumern von `GeneratedInteriorBlock`/`GeneratedInteriorEliminationCertificateSM3fR`. Die Phase ist eine natuerliche Nachruestung an der Stelle, an der die Dirichlet-Entry-Formel bereits entsteht, nicht ein Patch in `SM3fq`, `SM4q`, `SM5q`, `SM6b`, `SM8` oder `SM9a`.
SM3bq-DoD	Starker DoD ist ein kleiner Source-/Bridge-Typ, z.B. `GeneratedDirichletExecEntrySourceSM3bq`. Minimalfelder: `sourceWeight : GeneratedWeightPolicyEntrySource ...`; `entryWeightExec : GeneratedApproximantIndex A -> GeneratedApproximantIndex A -> ExecComplex`; `entryWeightExec_bridge : forall i j, ExecComplexBridge.toComplex (entryWeightExec i j) = ((generatedEntryWeight W i j : R) : C)`; `degreeExec : GeneratedApproximantIndex A -> ExecComplex`; `degreeExec_eq_sum`; `degreeExec_bridge`; `dirichletEntryExec : GeneratedApproximantIndex A -> GeneratedApproximantIndex A -> ExecComplex`; `dirichletEntryExec_bridge`; optional `dirichletMatrixExec` und Projektionsfelder fuer Boundary/Interior-Block-Entries, aber nur als direkte Projektionen der Dirichlet-Matrix, nicht als SM3f-Schur-Konsum. Statusfelder: source from generated adjacency/weight policy, no free rational table, no Real-to-Rat reverse cast, no scalar inverse, no Matrix.inv, no Schur product, no DtN, no Charpoly/Discriminant/Heegner.
Konstruktionsidee SM3bq	`entryWeightExec` soll nicht aus $\mathbb{R}$ zurueckuebersetzt werden. Es wird direkt aus der bereits vorhandenen rationalen Policy-Quelle gebaut: bei Nicht-Adjazenz `0`, bei Adjazenz und Diagonalgleichheit ebenfalls `0`, bei Adjazenz und `i != j` `ExecComplex.ofRat W.policy.beta`. Danach ist `degreeExec i := Finset.univ.sum (fun j => entryWeightExec i j)` und `dirichletEntryExec i j := if i = j then degreeExec i else - entryWeightExec i j`. Diese Reihenfolge stellt sicher: erst operative Weight-/Degree-/Dirichlet-Eintraege, dann spaeter Schur-Exec.
Beweistaktik SM3bq	Keine Summeninduktion ausser dem vorhandenen `ExecComplexBridge.toComplex_sum`; keine globale `simp`-Flaeche. Fuer `entryWeightExec_bridge`: `unfold generatedEntryWeight`, `unfold WeightPolicy.constantWeight`, Fallunterscheidung nur ueber `generatedAdjacent i j` und `i = j`; terminale Schritte mit `ExecComplexBridge.toComplex_zero`, `ExecComplexBridge.toComplex_ofRat`, `WeightPolicy.betaReal`, `ThermalAxis.betaReal` und kleinen `calc`-Bloecken. Fuer `degreeExec_bridge`: `rw [degreeExec_eq_sum]`, dann `ExecComplexBridge.toComplex_sum` und `Finset.sum_congr` mit `entryWeightExec_bridge`. Fuer `dirichletEntryExec_bridge`: Fallunterscheidung ueber `i = j`; diagonal `degreeExec_bridge`, offdiag `ExecComplexBridge.toComplex_neg` plus `entryWeightExec_bridge`.
Import- und Consumer-Disziplin	`SM3bq` darf `GeneratedDirichletEntry` und `ExecComplexBridge` importieren. Es darf nicht `GeneratedInteriorBlock`, `GeneratedInteriorEliminationCertificateSM3fR`, `SM3fq`, `SM3eRq`, `SM3gR`, `SM4q`, `SM5q`, `SM6b`, `SM8`, `SM9a`, Charpoly-/Diskriminantenmodule oder DtN-/MultiSchur-Consumer importieren. Falls ein Block-Projektionswitness benoetigt wird, muss er aus der Dirichlet-Matrix-Projektion folgen und nicht aus SM3f-Schur-Definitionen.
Overhead-Regel v78	Genau eine kleine Datei plus ein `BuildAll`-Import. Kein `SM3bq0`/`SM3bq1`-Auditwald, keine eigene Mikrodatei fuer jeden Block und kein Recheck spaeterer Consumer. Zulaessig sind ein Source-/Bridge-Typ, regular/variable Konstruktoren/Ledger und wenige Projektionssaetze. Falls die Bridge an einer fehlenden rationalen WeightPolicy-Projektion haengt, wird minimal an `WeightPolicy`/`GeneratedWeightPolicyEntrySource` nachgeschaerft, nicht in `SM3fq` oder spaeter gepatcht.

Anschlussgate nach SM3bq	Wenn `SM3bq` einen unbedingten `GeneratedDirichletExecEntrySourceSM3bq`-Witness liefert, folgt als naechste natuerliche positive Phase `SM3eRq - GeneratedInverseCandidateExecEntrySource`, sofern die Inversen-/AccumulatedEntry-Seite weiterhin fehlt. Erst wenn beide Seiten vorliegen, darf eine kleine `SM3fq+`-Forward-/Completion-Phase die Schur-Exec-Formel fuer `generatedBoundaryOperatorSM3fR H` zusammensetzen. Falls `SM3bq` nicht schliesst, wird die fehlende Quelle an `WeightPolicy/GeneratedDirichletEntry` exponiert; kein Rueckfall auf Real-zu-Rat-Rueckuebersetzung.
Fortgeschriebene Reihenfolge v78	`r3c1a` closed/green; `r3c1b0` closed/green; `SM3db2aR-SM3db2fR` closed/green bzw. closed/green-audit; `SM3eR-r3c1b1` closed/green; `SM3eR-r3c1b-full` closed/green; `SM3eR-r3c1c` closed/green; `SM3eR-r3c1d` green/gate-closed; `SM3db3aR` closed/green-audit; `SM3db4aT/U/V/W` closed/green; `SM3eR-r3c1d-full` closed/green; `SM3eR-r3c1e` closed/green; `SM3eR-r3c2` closed/green; `SM3eR-r4a` closed/green; `SM3eR-r4` collapsed/closed-by-r4a; `SM3eR-r5` closed/green; `SM3fR` closed/green; `SM3fq` closed/green-missing-`SM3bq`; `SM3bq - GeneratedDirichletExecEntrySource` **active**; `SM3eRq` blocked until SM3bq; `SM3fq+`/Schur-Exec-completion blocked until SM3bq and SM3eRq; `SM3gR` closed/green; `SM3hR` closed/green; `SM3iR` closed/green; `SM4` closed/green; `SM4q` closed/green-missing-`SM3fq`; `SM5` closed/green; `SM5q` closed/green-missing-`SM4q`; `SM6` closed/green; `SM7` closed/green-obstruction historisch; `SM6a` closed/green; `SM7BridgeRecheck` closed/green; `SM8` closed/green; `SM9a` closed/green-missing-source; `SM6b` closed/green-missing-`SM5q`; `SM9b/SM9c/SM10/SM11/SM12` blocked until a computed SM8 ExecMirror exists.

0ba. Fortschreibungsentscheidung v79 nach gruenem SM3bq-Build: SM3eRq aktiv

Statusentscheidung	`SM3bq - GeneratedDirichletExecEntrySource` ist nach lokal gruenem Build formal geschlossen. Der planrelevante Befund ist positiv: die operative Weight-/Degree-/Dirichlet-Matrix-Seite ist an der fruehesten natuerlichen Erzeugungsschicht nachgeruestet, naemlich unmittelbar nach `GeneratedDirichletEntry` und vor `GeneratedInteriorBlock`. Damit ist der v78-Missing-Subsource-Befund fuer die Dirichlet-/WeightPolicy-Seite erledigt. Die naechste aktive Phase ist nicht `SM3fq+`, nicht `SM4q/SM5q/SM6b/SM8/SM9a` und kein Charpoly-Test, sondern `SM3eRq - GeneratedInverseCandidateExecEntrySource`.
Formaler SM3bq-Ertrag	Geschlossen sind insbesondere `generatedEntryWeightExecSM3bq`, `generatedDirichletDegreeExecSM3bq`, `generatedDirichletEntryExecSM3bq`, `generatedDirichletMatrixExecSM3bq`, die Bridge-Saetze nach $\mathbb{C}$, `GeneratedDirichletExecEntrySourceSM3bq`, regular/variable Konstruktoren, `SM3bqGeneratedDirichletExecEntrySourceLedger` sowie Statusfelder gegen freie Rationaltabellen, Real-zu-Rat-Rueckuebersetzung, Skalarinverse, `Matrix.inv`, Schur-Produkte, DtN-/MultiSchur-Consumer und Charpoly-/Discriminant-/Heegner-Daten.
Praeziser Restbefund	Der noch fehlende operative Pfeil liegt jetzt exakt auf der Inversen-/AccumulatedEntry-Seite: `H.candidateMatrix i j = M.matrix i j = generatedInteriorAccumulatedEntryValue T i j` ist als Real-/Trace-/Fold-Herkunft vorhanden, aber noch nicht als `ExecComplex`/$\mathbb{Q}$-Eintrag mit Bridge nach $\mathbb{C}$ exponiert. Ohne diesen Pfeil kann `generatedBoundaryOperatorSM3fR H = boundaryBlock - boundaryInteriorBlock * H.candidateMatrix * interiorBoundaryBlock` nicht als operative Schur-Exec-Matrix zusammengesetzt werden.
Naechste aktive Phase	`SM3eRq - GeneratedInverseCandidateExecEntrySource`. Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedInverseCandidateExecEntrySourceSM3eRq.lean`. Import in `BuildAll` nach `SM3dbRToSM3eRHandoff` und vor `GeneratedInteriorTwoSidedInvFromSM3db7R/GeneratedInteriorEliminationCertificateSM3fR/Generat` Die Phase ist eine operative Source-/Bridge-Schicht fuer die Kandidatenmatrix, kein Produktbeweis und keine SM3f-Schur-Completion.
SM3eRq-DoD	Starker DoD ist ein kleiner Source-/Bridge-Typ, z.B. `GeneratedInverseCandidateExecEntrySourceSM3eRq`. Minimalfelder: `sourceDirichletExec : GeneratedDirichletExecEntrySourceSM3bq ... W`; `sourceHandoff : SM3dbRToSM3eRHandoff ...`; `sourceMatrix : GeneratedInteriorInverseCandidateMatrix ...`; `sourceAccumulatedEntry : GeneratedInteriorAccumulatedInverseCandidateEntry ...`; `candidateEntryExec : GeneratedInteriorIndex A -> GeneratedInteriorIndex A -> ExecComplex`; `candidateMatrixExec : Matrix (GeneratedInteriorIndex A) (GeneratedInteriorIndex A) ExecComplex`; Bridge-Saetze `candidateEntryExec_bridge_to_accumulatedValue`, `candidateMatrixExec_bridge_to_handoffCandidate` und `candidateMatrixExec_bridge_to_matrixExport`. Statusfelder: source from SM3db4aR/SM3db5R/SM3db7R provenance plus SM3bq-ready gate; no free rational table; no Real-to-Rat reverse cast; no scalar inverse; no `Matrix.inv`; no product proof; no `TwoSidedInv`; no `InteriorEliminationCertificate`; no Schur boundary operator; no DtN/MultiSchur; no Charpoly/Discriminant/Heegner.
Konstruktionsidee SM3eRq	Positiv zuerst: pruefe, ob `generatedInteriorAccumulatedEntryValue T i j` bereits aus operationalen rationalen Komponenten der Trace-/Fold-Kette gebaut werden kann. Dann definiere `candidateEntryExec` komponentenweise aus diesen vorhandenen Exec-Bausteinen und schliesse die Bridge ueber `ExecComplexBridge.toComplex_add`, `toComplex_sub`, `toComplex_mul` und ggf. `toComplex_sum`. Nicht erlaubt ist `candidateEntryExec` aus `H.candidateMatrix i j : R` zurueckzugewinnen, eine freie Rationaltabelle zu setzen oder eine Skala/Inverse aus $\mathbb{R}$ zu extrahieren.
Beweistaktik SM3eRq	Zuerst entrywise Rueckbindung: `H.candidateMatrixEntry_eq_accumulatedTraceValue`, `M.matrixEntry_from_accumulatedTraceValue` und vorhandene `sourceAccumulatedEntry`-Felder nur per Feldprojektion, `rfl`, kleinen `rw`-Schritten und `calc`. Danach Bridge-Beweise ausschliesslich ueber die vorhandenen `ExecComplexBridge`-Homomorphie-Lemmata. Keine globale `simp`-Flaeche; Simplifizierung nur terminal fuer definitorische Nullen/Einsen oder offensichtliche Cast-Ziele. Keine Matrixprodukt-Identitaet, keine Summen-/Fold-Invarianz und keine Boundary-Schur-Rechnung in SM3eRq.
Import- und Consumer-Disziplin	Erlaubt sind `ExecComplexBridge`, `GeneratedDirichletExecEntrySourceSM3bq`, `GeneratedInteriorInverseCandidateMatrix` und `SM3dbRToSM3eRHandoff` bzw. deren unmittelbare Herkunftsimports. Verboten sind `GeneratedInteriorTwoSidedInvFromSM3db7R`, `GeneratedInteriorEliminationCertificateSM3fR`, `GeneratedSM3fSchurExecEntrySourceSM3fq`, `GeneratedTopBoundaryDtNSM3gR`, `SM4/SM4q`, `SM5/SM5q`, `SM6b`, `SM8`, `SM9a` sowie Charpoly-/Diskriminantenmodule.
Ruecksprungregel	Falls SM3eRq zeigt, dass `generatedInteriorAccumulatedEntryValue` im aktuellen API-Stand nur $\mathbb{R}$-wertig, aber nicht operational rational exponiert ist, wird nicht in SM3eRq

0bb. Fortschreibungsentscheidung v80 nach gruenem SM3eRq-Build: SM3fqPlus aktiv

Statusentscheidung	`SM3eRq - GeneratedInverseCandidateExecEntrySource` ist nach lokal gruenem Build formal geschlossen. Der planrelevante Befund ist positiv: die Inversen-/AccumulatedEntry-Seite der SM3f-Schur-Formel ist nun als `ExecComplex`/$\mathbb{Q}$-Quelle exponiert und auf `generatedInteriorAccumulatedEntryValue T i j`, `H.candidateMatrix i j` und `M.matrix i j` zurueckgebunden. Damit sind die beiden in v79 als blockierend markierten operativen Subquellen geschlossen: `SM3bq` liefert die Dirichlet-/Block-Exec-Seite, `SM3eRq` liefert die inverse Kandidatenmatrix-Exec-Seite. Die naechste aktive Phase ist deshalb nicht `SM4q`, nicht `SM5q`, nicht `SM6b`, nicht `SM8`/`SM9a` und kein Charpoly-Test, sondern die kleine Completion-Schicht **`SM3fqPlus - GeneratedSM3fSchurExecCompletion`**.
Formaler SM3eRq-Ertrag	Geschlossen sind insbesondere `GeneratedInverseCandidateExecEntrySourceSM3eRq`, `generatedInteriorBlockEntryExecSM3eRq`, die operativen Pivot-/LocalStep-/Residual-/FoldUpdateOrder-Beitraege, `generatedInverseCandidateEntryExecSM3eRq`, `generatedInverseCandidateMatrixExecSM3eRq`, die Bridge-Saetze zur akkumulierten Entry-Provenienz und zur Handoff-/MatrixExport-Kandidatenmatrix, regular/variable Konstruktoren und `SM3eRqGeneratedInverseCandidateExecEntrySourceLedger`. Die Phase erzeugt bewusst keinen Boundary-Schur-Operator, keine Produktidentitaet, keinen `TwoSidedInv`, kein `InteriorEliminationCertificate`, keine DtN-/MultiSchur- und keine Charpoly-/Diskriminantendaten.
Praeziser Restbefund	Die noch fehlende operative Stelle ist jetzt nicht mehr die Quelle einzelner Dirichlet- oder Inversen-Eintraege, sondern nur noch die Zusammensetzung der SM3f-Schur-Formel auf der Boundary: `boundaryBlock - boundaryInteriorBlock * inverseMatrix * interiorBoundaryBlock`. Der vorhandene `SM3fq`-Stand traegt diese Formel nur als Real-/SM3f-Provenienz und altes Missing-Subsource-Gate. Nach `SM3bq` und `SM3eRq` muss genau dieses Gate durch eine neue Completion-Datei geschlossen oder, falls eine weitere natuerliche Missing-Source sichtbar wird, praezise an der Entstehungsschicht ausgewiesen werden.
Naechste aktive Phase	**`SM3fqPlus - GeneratedSM3fSchurExecCompletion`**. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedSM3fSchurExecCompletionSM3fqPlus.lean`. Import in `BuildAll` nach `GeneratedSM3fSchurExecEntrySourceSM3fq` und nach `GeneratedInverseCandidateExecEntrySourceSM3eRq`, aber vor `GeneratedMultiSchurExecBoundaryOperatorSM4q`, `GeneratedArithmeticSourceExecBoundaryOperatorSM5q`, `SM6b`, `SM8` und `SM9a`. Die Phase ist eine Forward-/Completion-Schicht fuer die operative Boundary-Schur-Matrix, kein neuer Inversenbeweis und kein Charpoly-Modul.
SM3fqPlus-DoD	Starker DoD ist ein kleiner Completion-/Bridge-Typ wie `GeneratedSM3fSchurExecCompletionSM3fqPlus`. Minimalfelder: `sourceSM3fq`; `sourceDirichletExec : GeneratedDirichletExecEntrySourceSM3bq ... W`; `sourceInverseExec : GeneratedInverseCandidateExecEntrySourceSM3eRq ...`; `boundaryBlockExec`; `boundaryInteriorBlockExec`; `interiorBoundaryBlockExec`; `inverseMatrixExec`; `schurCorrectionExec`; `boundaryOperatorExec : Matrix (GeneratedBoundaryIndex A) (GeneratedBoundaryIndex A) ExecComplex`. Bridge-Felder: `boundaryBlockExec_bridge`; `boundaryInteriorBlockExec_bridge`; `interiorBoundaryBlockExec_bridge`; `inverseMatrixExec_bridge`; `schurCorrectionExec_bridge`; `boundaryOperatorExec_bridge_to_SM3fSchur`; optional `boundaryOperatorExec_eq_sourceSM3fqReal_afterBridge`. Statusfelder: source from SM3bq+SM3eRq+SM3fq; no free Exec matrix; no Real-to-Rat reverse cast; no scalar inverse; no `Matrix.inv`; no new inverse oracle; no product-identity proof; no `TwoSidedInv`; no new `InteriorEliminationCertificate`; no DtN/MultiSchur consumer; no Charpoly/Discriminant/Heegner.
Konstruktionsidee SM3fqPlus	Baue die Boundary-/Interior-Block-Exec-Matrizen nur als Projektionen der `SM3bq`-Dirichlet-Matrix, also ueber `GeneratedBoundaryIndex.toApproximantIndex` und `GeneratedInteriorIndex.toApproximantIndex`; die Inversenmatrix wird nur aus `SM3eRq.candidateMatrixExec` genommen. Definiere die Schur-Korrektur entrywise, vorzugsweise als `Finset.univ.sum` ueber die beiden Interior-Indizes, also ohne freie Matrix und ohne neue Formelquelle. Danach `boundaryOperatorExec i j := boundaryBlockExec i j - schurCorrectionExec i j`.
Beweistaktik SM3fqPlus	Zuerst alle Blockprojektionen per vorhandenen `SM3bq`-Bridges auf `generatedBoundaryBlockSM3fR W`, `generatedBoundaryInteriorBlockSM3fR W` und `generatedInteriorBoundaryBlockSM3fR W` zurueckbinden. Dann `inverseMatrixExec_bridge` direkt aus `SM3eRq` bzw. aus den Projektionssaetzen `candidateMatrixExec_bridge_to_handoffCandidate_thm`/`...matrixExport_thm`. Fuer die Schur-Korrektur: `ExecComplexBridge.toComplex_sum` zweimal, `Finset.sum_congr`, `ExecComplexBridge.toComplex_mul`, anschliessend terminale Umformung auf die entrywise Form von `Matrix.mul_apply`; keine globale `simp`-Flaeche. Fuer den Operator: `ExecComplexBridge.toComplex_sub`, Bridge der Blockseite, Bridge der Korrektur, dann `sourceSM3fq_boundaryOperatorReal_eq_schur` bzw. ein kleines `calc`

0bc. Fortschreibungsentscheidung v81 nach gruenem SM3fqPlus-Build: SM4q-Recheck aktiv

Statusentscheidung	`SM3fqPlus - GeneratedSM3fSchurExecCompletion` ist nach lokal gruenem Build formal geschlossen. Der planrelevante Befund ist positiv: die operative Boundary-Schur-Matrix liegt nun an der natuerlichen SM3f/SM3fq-Schicht vor. Das schliesst genau das v80-Gate zwischen den einzelnen Exec-Subquellen aus `SM3bq`/`SM3eRq` und dem Boundary-Operator; es erzeugt keine neue Produktidentitaet, keinen `TwoSidedInv`, kein neues `InteriorEliminationCertificate`, keinen DtN-/MultiSchur-Consumer und keine Charpoly-/Diskriminantendaten.
Formaler SM3fqPlus-Ertrag	Geschlossen sind insbesondere `GeneratedSM3fSchurExecCompletionSM3fqPlus`, `generatedBoundaryBlockExecSM3fqPlus`, `generatedBoundaryInteriorBlockExecSM3fqPlus`, `generatedInteriorBoundaryBlockExecSM3fqPlus`, `generatedInverseMatrixExecSM3fqPlus`, `generatedSM3fqPlusSchurCorrectionExec`, `generatedSM3fqPlusBoundaryOperatorExec`, die Bridge-Saetze zur handoff-Schur-Formel und zur `SM3fq`-Realquelle, regular/variable Konstruktoren und `SM3fqPlusGeneratedSM3fSchurExecCompletionLedger`. Die Phase traegt explizit `nextPhaseStatus = sm4qMayRecheckBoundarySchurExecSource`.
Praeziser Restbefund	Der verbleibende Engpass liegt jetzt nicht mehr in SM3bq, SM3eRq oder SM3fqPlus. Die aktuelle `SM4q`-Datei ist historisch gruen als Missing-Source-Gate: `GeneratedMultiSchurExecBoundaryOperatorSM4q` kennt die Real-Provenienz von `GeneratedMultiSchurSM4.boundaryOperator`, markiert aber noch `blockedMissingSM3fSM3eRExecSchurEntrySource`. Nach gruenem `SM3fqPlus` muss genau dieses Gate an der SM4q-Schicht geschlossen werden. Ein Patch in `SM5q`, `SM6b`, `SM8`, `SM9a` oder einem Charpoly-Modul waere ein Consumer-Patch und wuerde die Provenienz verschieben.
Naechste aktive Phase	`SM4qRecheck - GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlus`. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q.lean`. Import in `BuildAll` unmittelbar nach `GeneratedMultiSchurExecBoundaryOperatorSM4q` und nach `GeneratedSM3fSchurExecCompletionSM3fqPlus`, aber vor `GeneratedArithmeticSourceExecBoundaryOperatorSM5q`, `SM6b`, `SM8` und `SM9a`.
SM4qRecheck-DoD	Starker DoD ist ein kleiner Forward-/Recheck-Typ, z.B. `GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q`. Minimalfelder: `sourceSM4 : GeneratedMultiSchurSM4 ...`; `sourceSM4q : GeneratedMultiSchurExecBoundaryOperatorSM4q ...`; `sourceSM3fqPlus : GeneratedSM3fSchurExecCompletionSM3fqPlus ...`; `boundaryOperatorReal := sourceSM4.boundaryOperator`; `execBoundaryOperator := sourceSM3fqPlus.boundaryOperatorExec`; Bridge `execBoundaryOperator_bridge_to_SM4 : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM4.boundaryOperator i j : R) : C)`; Projektionsfelder zur SM3iR-/TopBoundary-/Certificate-/Schur-Provenienz aus `sourceSM4q`; Statusfelder: completed from SM3fqPlus, no free Exec matrix, no Real-to-Rat reverse cast, no scalar inverse, no `Matrix.inv`, no field_simp, no new inverse oracle, no product identity, no `TwoSidedInv`, no new certificate, no DtN/MultiSchur recomputation, no Charpoly/Discriminant/Heegner.
Beweistaktik SM4qRecheck	Nicht die Schur-Summe neu expandieren. Setze `execBoundaryOperator` definitionell auf `sourceSM3fqPlus.boundaryOperatorExec`. Fuer die Bridge nutze ein kurzes `calc`: zuerst `sourceSM3fqPlus.boundaryOperatorExec_bridge_to_SM3fSchur i j`; dann gleiche `sourceSM3fqPlus.sourceSM3fq.boundaryOperatorReal` entrywise mit `sourceSM4.boundaryOperator` ab. Dieser Abgleich erfolgt ueber `sourceSM3fq_boundaryOperatorReal_eq_handoffSchurSM3fqPlus`, `sourceSM4.boundaryOperator_eq_schur` und `sourceSM4.certificate_inverseMatrix_eq_handoffCandidate`. Erwarteter Proof-Stil: `congrArg (fun M => M i j)` fuer Matrixgleichungen, kleine `rw`/`calc`-Ketten, hoechstens terminale lokale Vereinfachung fuer Subtraktion/Matrixausdruck; keine globale `simp`-Flaeche, keine Summeninduktion, keine neue Exec-Formel.
Import- und Consumer-Disziplin	Erlaubt sind `GeneratedMultiSchurExecBoundaryOperatorSM4q` und `GeneratedSM3fSchurExecCompletionSM3fqPlus`. Zulaessig ist der direkte Zugriff auf `GeneratedMultiSchurSM4`, soweit er durch `SM4q` bereits getragen wird. Verboten sind `GeneratedArithmeticSourceExecBoundaryOperatorSM5q`, `GeneratedLevelStepExecMirrorSourceSM6b`, `GeneratedLevelHistoryMatrixFromBridgeSM8`, `GeneratedLevelHistoryExecMirrorSourceSM9a`, Charpoly-/Diskriminantenmodule, Heegner-/CM-Consumer, neue Interior-Elimination-Daten und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v81	Genau eine kleine SM4q-Recheck-Datei plus ein `BuildAll`-Import. Kein `SM4q0`/`SM4q1`-Auditwald und kein paralleler SM5-/SM6-/SM8-Recheck, solange dieser direkte Forward-Versuch nicht mathematisch scheitert. Falls der Recheck blockiert, muss der Missing-Source-Befund exakt benennen, ob die fehlende Gleichung zwischen `SM4.boundaryOperator` und der SM3f-Handoff-Schur-Formel oder die Import-/Typgleichheit zwischen `SM4q` und `SM3fqPlus` fehlt.
Anschlussgate nach SM4qRecheck	Wenn `SM4qRecheck` einen unbedingten `execBoundaryOperator`-Witness mit Bridge zu `GeneratedMultiSchurSM4.boundaryOperator` liefert, folgt als naechste Forward-/Recheck-Phase `SM5qRecheck`: die `GeneratedArithmeticSourceSM5`-Schicht darf dann den SM4q-Exec-Operator konsumieren und ihre alte Missing-SM4q-Lage schliessen. Erst danach werden `SM6b`, `SM8`, `SM9a` und anschliessend `SM9b` wieder freigegeben.

Fortgeschriebene Reihenfolge v81	SM3fq closed/green-missing-SM3bq; SM3bq closed/green als Dirichlet-/WeightPolicy-Exec-Provider; SM3eRq closed/green als Inversen-/AccumulatedEntry-Exec-Provider; SM3fqPlus closed/green als Boundary-Schur-Exec-Completion; **SM4qRecheck - GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlus active**; SM5qRecheck blocked until SM4qRecheck; SM6b blocked until SM5qRecheck; SM8/SM9a blocked until computed boundary/SM8 ExecMirror; SM9b/SM9c/SM10/SM11/SM12 blocked until computed charpoly-ready ExecMirror. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM4qRecheck veraendert keine Produktidentitaet und keinen TwoSidedInv.

0bd. Fortschreibungsentscheidung v82 nach gruenem SM4qRecheck-Build: SM5qRecheck aktiv

Statusentscheidung	SM4qRecheck - GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlus ist nach lokal gruenem Build formal geschlossen. Der planrelevante Befund ist positiv: Die operative ExecComplex-Boundary-Schur-Matrix aus SM3fqPlus ist nun an die SM4q/GeneratedMultiSchurSM4-Schicht weitergereicht und entrywise nach $\mathbb{C}$ gegen sourceSM4.boundaryOperator gebridged. Damit ist das v81-Gate blockedMissingSM3fSM3eRExecSchurEntrySource an seiner natuerlichen SM4q-Schicht geschlossen; kein SM5-/SM6-/SM8-Consumer musste gepatcht werden.
Formaler SM4qRecheck-Ertrag	Geschlossen sind insbesondere GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q, der Konstruktor fromSources, sourceSM3fqBoundaryOperatorReal_eq_SM4, execBoundaryOperator_bridge_to_SM4, execBoundaryOperator_bridge_to_SM4_thm, sourceSM3fq_boundaryOperatorReal_from_SM4, completion_closed_from_SM3fqPlus, next_phase_is_SM5q_recheck, regular/variable Konstruktoren und SM4qRecheckGeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusLedger. Die Phase traegt explizit nextPhaseStatus = sm5qMayRecheckArithmeticExecBoundaryOperator.
Praeziser Restbefund	Der verbleibende Engpass liegt jetzt nicht mehr in SM3fqPlus oder SM4qRecheck. Die historisch gruene SM5q-Datei GeneratedArithmeticSourceExecBoundaryOperatorSM5q kennt die Real-Provenienz von GeneratedArithmeticSourceSM5.boundaryOperator und ihre Rueckbindung an sourceSM5.sourceMultiSchur.boundaryOperator, markiert aber noch blockedMissingGeneratedMultiSchurExecBoundaryOperatorSM4q. Nach gruenem SM4qRecheck muss genau dieses Gate an der SM5q-Schicht geschlossen werden. Ein Start von SM6b, SM8, SM9a oder einem Charpoly-/Diskriminantenmodul waere jetzt ein Consumer-Patch und wuerde die Provenienz verschieben.
Naechste aktive Phase	SM5qRecheck - GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheck. Implementierungsdatei: vorzugsweise CNNA/PillarA/Arithmetic/Smoke/GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q.lean. Import in BuildAll unmittelbar nach GeneratedArithmeticSourceExecBoundaryOperatorSM5q und nach GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q, aber vor GeneratedLevelStepExecMirrorSourceSM6b, GeneratedLevelHistoryMatrixFromBridgeSM8, GeneratedLevelHistoryExecMirrorSourceSM9a und allen Charpoly-/Diskriminanten-Consumern.
SM5qRecheck-DoD	Starker DoD ist ein kleiner Forward-/Recheck-Typ, z.B. GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q. Minimalfelder: sourceSM5 : GeneratedArithmeticSourceSM5 ... G; sourceSM5q : GeneratedArithmeticSourceExecBoundaryOperatorSM5q ... G; sourceSM4qRecheck : GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q ...; ein proof-carrying Abgleich sourceSM4qRecheck_source_eq_SM5_sourceMultiSchur : sourceSM4qRecheck.sourceSM4 = sourceSM5.sourceMultiSchur; boundaryOperatorReal := sourceSM5q.boundaryOperatorReal; execBoundaryOperator := sourceSM4qRecheck.execBoundaryOperator; Bridge execBoundaryOperator_bridge_to_SM5 : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM5.boundaryOperator i j : R) : C); Projektionsfelder entry_eq_SM4, entry_eq_schur, apply_eq_SM4, apply_eq_mulVec aus sourceSM5q; Statusfelder: completed from SM4qRecheck, no free Exec matrix, no Real-to-Rat reverse cast, no scalar inverse, no Matrix.inv, no field_simp, no new inverse oracle, no product identity, no TwoSidedInv, no new certificate, no DtN/MultiSchur recomputation, no Charpoly/Discriminant/Heegner.
Beweistaktik SM5qRecheck	Nicht die Schur-Summe neu expandieren und GeneratedArithmeticSourceSM5 nicht umdeuten. Setze execBoundaryOperator definitionell auf sourceSM4qRecheck.execBoundaryOperator. Fuer die Bridge nutze eine kurze entrywise calc-Kette: zuerst sourceSM4qRecheck.execBoundaryOperator_bridge_to_SM4 i j; dann ersetze sourceSM4qRecheck.sourceSM4.boundaryOperator ueber sourceSM4qRecheck_source_eq_SM5_sourceMultiSchur; zuletzt ersetze sourceSM5.sourceMultiSchur.boundaryOperator mit sourceSM5.boundaryOperator ueber sourceSM5.boundaryOperator_eq_SM4 bzw. sourceSM5q.boundaryOperatorReal_eq_SM5/sourceSM5q.entry_eq_SM4. Erwarteter Proof-Stil: congrArg (fun M => M i j) fuer Matrixgleichungen, congrArg (fun x : R => (x : C)) fuer den Cast nach $\mathbb{C}$, kleine rw/calc-Ketten; keine globale simp-Flaeche, keine Summeninduktion, keine neue Exec-Formel.

Import- und Consumer-Disziplin	Erlaubt sind `GeneratedArithmeticSourceSM5`, `GeneratedArithmeticSourceExecBoundaryOperatorSM5q` und `GeneratedMultiSchurExecBoundaryOperatorFromSM3fqPlusSM4q`. Zulaessig ist der direkte Zugriff auf `GeneratedMultiSchurSM4`, soweit er ueber `sourceSM5.sourceMultiSchur` oder `SM4qRecheck.sourceSM4` getragen wird. Verboten sind `GeneratedLevelStepExecMirrorSourceSM6b`, `GeneratedLevelHistoryMatrixFromBridgeSM8`, `GeneratedLevelHistoryExecMirrorSourceSM9a`, Charpoly-/Diskriminantenmodule, Heegner-/CM-Consumer, neue Interior-Elimination-Daten und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v82	Genau eine kleine SM5q-Recheck-Datei plus ein `BuildAll`-Import. Kein `SM5q0/SM5q1`-Auditwald und kein paralleler SM6-/SM8-/SM9-Recheck, solange dieser direkte Forward-Versuch nicht mathematisch scheitert. Falls der Recheck blockiert, muss der Befund exakt benennen, ob die fehlende Gleichung zwischen `sourceSM4qRecheck.sourceSM4` und `sourceSM5.sourceMultiSchur` fehlt oder ob die regular/variable Ledger aus unterschiedlichen `SM4`-Quellen konstruiert werden. Dann wird an dieser natuerlichen Ledger-/Source-Schicht nachgeschaerft, nicht in `SM6b`.
Anschlussgate nach SM5qRecheck	Wenn `SM5qRecheck` einen unbedingten `execBoundaryOperator`-Witness mit Bridge zu `GeneratedArithmeticSourceSM5.boundaryOperator` liefert, folgt als naechste Forward-/Recheck-Phase `SM6bRecheck`: die `GeneratedLevelStepWitnessSM6`-/`GeneratedLevelStepExecMirrorSourceSM6b`-Schicht darf dann den SM5q-Exec-Operator konsumieren und ihre alte Missing-SM5q-Lage schliessen. Erst danach werden `SM8`, `SM9a` und anschliessend `SM9b/SM9c/SM10/SM11/SM12` wieder freigegeben.
Fortgeschriebene Reihenfolge v82	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; **`SM5qRecheck - GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheck` active**; `SM6bRecheck` blocked until SM5qRecheck; `SM8/SM9a` blocked until computed boundary/SM8 ExecMirror; `SM9b/SM9c/SM10/SM11/SM12` blocked until computed charpoly-ready ExecMirror. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; `SM5qRecheck` veraendert keine Produktidentitaet und keinen `TwoSidedInv`.

0be. Fortschreibungsentscheidung v83 nach gruenem SM5qRecheck-Build: SM6bRecheck aktiv

Statusentscheidung	`SM5qRecheck - GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheck` ist nach lokal gruenem Build formal geschlossen. Der positive Befund ist eng begrenzt: Die operative `ExecComplex`-Boundary-Matrix aus `SM4qRecheck` ist an die `GeneratedArithmeticSourceSM5`-Schicht weitergereicht und entrywise nach $\mathbb{C}$ gegen `sourceSM5.boundaryOperator` gebridged. Damit ist das v82-Gate `blockedMissingGeneratedMultiSchurExecBoundaryOperatorSM4q` an seiner natuerlichen SM5q-Schicht geschlossen; kein SM6-/SM8-/SM9-/Charpoly-Consumer musste gepatcht werden.
Formaler SM5qRecheck-Ertrag	Geschlossen sind insbesondere `GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q`, der Konstruktor `fromSources`, `sourceSM4qRecheck_source_from_SM5`, `execBoundaryOperator_from_SM4qRecheck`, `execBoundaryOperator_bridge_to_SM5`, `entry_from_SM4qRecheck`, `entry_from_schur`, `apply_from_SM4`, `apply_from_boundaryOperatorReal_mulVec`, die regular/variable Konstruktoren, `SM5qRecheckGeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckLedger`, `sm5qRecheck_regularCompletion_closed`, `sm5qRecheck_variableCompletion_closed`, `sm5qRecheck_nextPhase_is_SM6b_recheck` und `sm5qRecheck_noForbiddenShortcuts`. Die Phase traegt explizit `nextPhaseStatus = sm6bMayRecheckLevelStepExecMirror`.
Praeziser Restbefund	Der verbleibende Engpass liegt jetzt nicht mehr in `SM4qRecheck` oder `SM5qRecheck`. Die historisch gruene Datei `GeneratedLevelStepExecMirrorSourceSM6b` kennt bereits den Real-Boundary-Operator aus `GeneratedLevelStepWitnessSM6`, seine Rueckbindung an `GeneratedArithmeticSourceSM5.boundaryOperator`, die Schur-Herkunft und die `apply`-Gleichungen. Sie markiert aber noch `blockedMissingGeneratedArithmeticSourceExecBoundaryOperator`. Nach gruenem `SM5qRecheck` muss genau dieses Gate an der SM6b-Schicht geschlossen werden. Ein Start von `SM8`, `SM9a`, `SM9b` oder einem Charpoly-/Diskriminantenmodul waere jetzt ein Consumer-Patch und wuerde die Provenienz verschieben.
Naechste aktive Phase	**`SM6bRecheck - GeneratedLevelStepExecMirrorSourceFromSM5qRecheck`**. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b.lean`. Import in `BuildAll` unmittelbar nach `GeneratedLevelStepExecMirrorSourceSM6b` und nach `GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q`, aber vor `GeneratedLevelHistoryMatrixFromBridgeSM8`, `GeneratedLevelHistoryExecMirrorSourceSM9a` und allen Charpoly-/Diskriminanten-Consumern.

SM6bRecheck-DoD	Starker DoD ist ein kleiner Forward-/Recheck-Typ, z.B. `GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b`. Minimalfelder: `sourceSM6b : GeneratedLevelStepExecMirrorSourceSM6b ... G`; `sourceSM5qRecheck : GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q ... G`; ein proof-carrying Abgleich `sourceSM5qRecheck_source_eq_SM6_source : sourceSM5qRecheck.sourceSM5 = sourceSM6b.sourceSM6Witness.source`; `boundaryOperatorReal := sourceSM6b.boundaryOperatorReal`; `execBoundaryOperator := sourceSM5qRecheck.execBoundaryOperator`; Bridge `execBoundaryOperator_bridge_to_SM6 : forall i j, ExecComplexBridge.toComplex (execBoundaryOperator i j) = ((sourceSM6b.sourceSM6Witness.boundaryOperator i j : R) : C)`; Projektionsfelder `entry_eq_SM5`, `entry_eq_schur`, `apply_eq_SM5`, `apply_eq_mulVec` aus `sourceSM6b`; Statusfelder: completed from SM5qRecheck, no free Exec matrix, no Real-to-Rat reverse cast, no scalar inverse, no `Matrix.inv`, no `field_simp`, no new inverse oracle, no product identity, no `TwoSidedInv`, no new certificate, no DtN/MultiSchur recomputation, no LevelHistoryMatrix, no Charpoly/Discriminant/Heegner.
Beweistaktik SM6bRecheck	Nicht die Schur-Summe neu expandieren, `GeneratedLevelStepWitnessSM6` nicht umdeuten und keine neue `ExecComplex`-Matrix bauen. Setze `execBoundaryOperator` definitionell auf `sourceSM5qRecheck.execBoundaryOperator`. Fuer die Bridge nutze eine kurze entrywise `calc`-Kette: zuerst `sourceSM5qRecheck.execBoundaryOperator_bridge_to_SM5 i j`; dann ersetze `sourceSM5qRecheck.sourceSM5.boundaryOperator` ueber den Source-Abgleich mit `sourceSM6b.sourceSM6Witness.source.boundaryOperator`; zuletzt ersetze mit `sourceSM6b.sourceSM6Witness.boundaryOperator` ueber `sourceSM6b.sourceSM6Witness.boundaryOperator_eq_SM5`. Erwarteter Proof-Stil: `congrArg (fun M => M i j)` fuer Matrixgleichungen, `congrArg (fun x : R => (x : C))` fuer Casts nach $\mathbb{C}$, kleine `rw`/`calc`-Ketten; keine globale `simp`-Flaeche, keine Summeninduktion, kein Index-Eliminator-Monster, keine neue Exec-Formel.
Import- und Consumer-Disziplin	Erlaubt sind `GeneratedLevelStepExecMirrorSourceSM6b`, `GeneratedArithmeticSourceExecBoundaryOperatorFromSM4qRecheckSM5q`, `GeneratedLevelStepWitnessSM6` indirekt ueber `SM6b` und die minimale `ExecComplexBridge`-Schicht. Verboten sind `GeneratedLevelHistoryMatrixFromBridgeSM8`, `GeneratedLevelHistoryExecMirrorSourceSM9a`, Charpoly-/Diskriminantenmodule, Heegner-/CM-Consumer, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v83	Genau eine kleine SM6b-Recheck-Datei plus ein `BuildAll`-Import. Kein `SM6b0`/`SM6b1`-Auditwald und kein paralleler SM8-/SM9-Recheck, solange dieser direkte Forward-Versuch nicht mathematisch scheitert. Falls der Recheck blockiert, muss der Befund exakt benennen, ob die fehlende Gleichung zwischen `sourceSM5qRecheck.sourceSM5` und `sourceSM6b.sourceSM6Witness.source` fehlt oder ob die regular/variable Ledger aus unterschiedlichen SM5-Quellen konstruiert werden. Dann wird an dieser natuerlichen Ledger-/Source-Schicht nachgeschaerft, nicht in `SM8` oder `SM9a`.
Anschlussgate nach SM6bRecheck	Wenn `SM6bRecheck` einen unbedingten `execBoundaryOperator`-Witness mit Bridge zu `GeneratedLevelStepWitnessSM6.boundaryOperator` liefert, folgt als naechste kleine Weiterreichung `SM8Recheck`: die `GeneratedLevelHistoryMatrixFromBridgeSM8`-Schicht darf dann die SM6b-Execquelle konsumieren und eine berechenbare Level-History-Matrix exponieren. Erst nach einem unbedingten SM8-ExecMirror wird `SM9aRecheck` bzw. danach `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` freigegeben.
Fortgeschriebene Reihenfolge v83	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck - GeneratedLevelStepExecMirrorSourceFromSM5qRecheck` **active**; `SM8Recheck`/`SM9aRecheck` blocked until computed SM6b ExecMirror; `SM9b`/`SM9c`/`SM10`/`SM11`/`SM12` blocked until computed charpoly-ready ExecMirror. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; `SM6bRecheck` darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bf. Fortschreibungsentscheidung v84 nach gruenem SM6bRecheck-Build: SM8Recheck aktiv

Statusentscheidung	`SM6bRecheck - GeneratedLevelStepExecMirrorSourceFromSM5qRecheck` ist nach lokal gruenem Build formal geschlossen. Der positive Befund ist bewusst eng: Die operative `ExecComplex`-Boundary-Matrix aus `SM5qRecheck` ist an die `GeneratedLevelStepExecMirrorSourceSM6b`-Schicht weitergereicht und entrywise nach $\mathbb{C}$ gegen `sourceSM6Witness.boundaryOperator` bzw. `boundaryOperatorReal` gebridged. Damit ist das v83-Gate `blockedMissingGeneratedArithmeticSourceExecBoundaryOperator` an seiner natuerlichen SM6b-Schicht geschlossen; kein SM8-/SM9-/Charpoly-Consumer musste gepatcht werden.

Formaler SM6bRecheck-Ertrag	Geschlossen sind insbesondere `GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b`, der Konstruktor `fromSources`, `sourceSM5qRecheck_source_eq_SM6_source`, `boundaryOperatorReal_eq_SM6b`, `execBoundaryOperator_eq_SM5qRecheck`, `execBoundaryOperator_bridge_to_SM6`, `execBoundaryOperator_bridge_to_SM6bReal`, `entry_eq_SM5`, `entry_eq_SM5qRecheck`, `entry_eq_schur`, `apply_eq_SM5`, `apply_eq_mulVec`, regular/variable Konstruktoren sowie die Statusfelder `completionStatus = execMirrorCompletedFromSM5qRecheck` und `nextPhaseStatus = sm8MayConsumeLevelStepExecMirror`. Die Phase erzeugt weiterhin keine `LevelHistoryMatrix`, kein Charpoly, keine Diskriminante und keinen Heegner-Test.
Praeziser Restbefund	Der Engpass liegt jetzt nicht mehr in SM5q oder SM6b. Die historisch gruene Datei `GeneratedLevelHistoryMatrixFromBridgeSM8` erzeugt eine $\mathbb{R}$-LevelHistory-Matrix als Pullback von `sourceSM6Witness.boundaryOperator` entlang `sourceBridge.toBoundaryIndex`. Sie konsumiert aber noch nicht die neue SM6bRecheck-`ExecComplex`-Boundaryquelle. `SM9a` markiert deshalb weiterhin `blockedMissingGeneratedRationalEntrySource` bzw. die fehlende berechenbare LevelHistory-Exec-Spiegelung. Ein Start von `SM9b`, `SM9c`, `SM10`, `SM11`, `SM12` oder einem Heegner-/CM-Consumer waere weiterhin ein Consumer-Patch.
Naechste aktive Phase	`SM8Recheck - GeneratedLevelHistoryExecMatrixFromSM6bRecheck`. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryExecMatrixFromSM6bRecheckSM8.lean`. Import in `BuildAll` unmittelbar nach `GeneratedLevelHistoryMatrixFromBridgeSM8` und nach `GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b`, aber vor `GeneratedLevelHistoryExecMirrorSourceSM9a` und allen Charpoly-/Diskriminanten-Consumern.
SM8Recheck-DoD	Starker DoD ist ein kleiner Pullback-/Forward-Recheck-Typ, z.B. `GeneratedLevelHistoryExecMatrixFromSM6bRecheckSM8`. Minimalfelder: `sourceSM8 : GeneratedLevelHistoryMatrixFromBridgeSM8 ...`; `sourceSM6bRecheck : GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b ...`; ein proof-carrying Abgleich `sourceSM6bRecheck_witness_eq_SM8_witness : sourceSM6bRecheck.sourceSM6b.sourceSM6Witness = sourceSM8.sourceBridge.sourceSM6Witness`; `matrixReal := sourceSM8.matrix`; `execMatrix := fun i j => sourceSM6bRecheck.execBoundaryOperator (sourceSM8.sourceBridge.toBoundaryIndex i) (sourceSM8.sourceBridge.toBoundaryIndex j)`; Bridge `execMatrix_bridge_to_SM8 : forall i j, ExecComplexBridge.toComplex (execMatrix i j) = ((sourceSM8.matrix i j : R) : C)`; Projektionsfelder `entryReal_eq_SM8`, `entryReal_eq_SM6`, `entryReal_eq_schur`, `execEntry_from_SM6bRecheck`; Statusfelder: completed from SM6bRecheck, no free Exec matrix, no Real-to-Rat conversion, no scalar inverse, no `Matrix.inv`, no `field_simp`, no new inverse oracle, no product identity, no `TwoSidedInv`, no new certificate, no DtN/MultiSchur recomputation, no Charpoly/Discriminant/Heegner.
Beweistaktik SM8Recheck	Nicht die SM8-Schur-Summe neu expandieren und keinen neuen LevelHistory-Carrier erfinden. Setze `execMatrix` definitionell als Pullback der SM6bRecheck-`execBoundaryOperator`-Eintraege entlang `sourceSM8.sourceBridge.toBoundaryIndex`. Fuer die Bridge nutze eine kurze entrywise `calc`-Kette: zuerst `sourceSM6bRecheck.execBoundaryOperator_bridge_to_SM6bReal` an den gepullbackten Boundary-Indizes; dann ersetze `sourceSM6bRecheck.sourceSM6b.boundaryOperatorReal` ueber den Witness-Abgleich mit `sourceSM8.sourceBridge.sourceSM6Witness.boundaryOperator`; zuletzt ersetze den SM8-$\mathbb{R}$-Eintrag ueber `sourceSM8.entry_eq_boundaryOperator` bzw. `matrix_eq_generatedLevelHistoryMatrix`. Erwarteter Proof-Stil: `congrArg (fun W => W.boundaryOperator i j)` nur falls der dependent Witness-Abgleich wirklich noetig ist; sonst kleine `rw`/`calc`-Ketten auf Eintragsebene. Keine globale `simp`-Flaeche, keine Summeninduktion, kein Index-Eliminator-Monster, keine $\mathbb{R} \to \mathbb{Q}$-Rueckuebersetzung und keine neue Exec-Formel.
Import- und Consumer-Disziplin	Erlaubt sind `GeneratedLevelHistoryMatrixFromBridgeSM8`, `GeneratedLevelStepExecMirrorSourceFromSM5qRecheckSM6b`, die von SM8 bereits konsumierte SM7BridgeRecheck-Quelle und die minimale `ExecComplexBridge`-Schicht. Verboten sind `GeneratedLevelHistoryExecMirrorSourceSM9a`, Charpoly-/Diskriminantenmodule, Heegner-/CM-Consumer, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, neue `LevelHistoryIndex`-Carrier und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v84	Genau eine kleine SM8-Recheck-Datei plus ein `BuildAll`-Import. Kein `SM8a`/`SM8b`-Auditwald, kein paralleler SM9a-/SM9b-Recheck und kein Charpoly-Stub, solange dieser direkte Pullback-Versuch nicht mathematisch scheitert. Falls der Recheck blockiert, muss der Befund exakt benennen, ob die fehlende Gleichung zwischen `sourceSM6bRecheck.sourceSM6b.sourceSM6Witness` und `sourceSM8.sourceBridge.sourceSM6Witness` fehlt oder ob die regular/variable SM8-Ledger aus unterschiedlichen SM6-Witness-Quellen konstruiert werden. Dann wird an dieser natuerlichen SM7/SM8-Bridge-/Witness-Schicht nachgeschaerft, nicht in SM9a oder SM9b.
Anschlussgate nach SM8Recheck	Wenn `SM8Recheck` eine unbedingte `execMatrix` mit Bridge zur $\mathbb{R}$-`GeneratedLevelHistoryMatrixFromBridgeSM8.matrix` liefert, folgt als naechste kleine Forward-/Recheck-Phase `SM9aRecheck - GeneratedLevelHistoryExecMirrorSourceFromSM8Recheck`. Diese Phase darf nur das in SM9a dokumentierte Missing-Source-Gate schliessen. Erst danach wird `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` freigegeben.

Fortgeschriebene Reihenfolge v84	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck - GeneratedLevelHistoryExecMatrixFromSM6bRecheck` **active**; `SM9aRecheck` blocked until computed SM8 ExecMatrix; `SM9b`/`SM9c`/`SM10`/`SM11`/`SM12` blocked until computed charpoly-ready ExecMirror. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; `SM8Recheck` darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bg. Fortschreibungsentscheidung v85 nach gruenem SM8Recheck-Build: SM9aRecheck aktiv

Statusentscheidung	`SM8Recheck - GeneratedLevelHistoryExecMatrixFromSM6bRecheck` ist nach lokal gruenem Build formal geschlossen. Der positive Befund ist bewusst eng: Die SM8-$\mathbb{R}$-LevelHistory-Matrix wurde nicht neu erzeugt, sondern ihre bereits vorhandene `GeneratedLevelHistoryMatrixFromBridgeSM8`-Quelle wurde mit der aus SM6bRecheck stammenden `ExecComplex`-Boundaryquelle zusammengefuehrt. Die neue Exec-Matrix ist der Pullback der SM6bRecheck-`execBoundaryOperator`-Eintraege entlang `sourceSM8.sourceBridge.toBoundaryIndex`; die Bridge nach $\mathbb{C}$ laeuft entrywise gegen `sourceSM8.matrix`. Damit ist das v84-Gate fuer eine berechenbare LevelHistory-ExecMatrix geschlossen.
Formaler SM8Recheck-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryExecMatrixFromSM6bRechecksSM8`, `generatedLevelHistoryExecMatrixFromSM6bRechecksSM8`, `fromSources`, `matrixReal_eq_SM8`, `execMatrix_eq_pullback`, `execMatrix_bridge_to_SM8`, `entryReal_eq_SM8`, `entryReal_eq_boundaryOperator`, `entryReal_eq_SM6`, `entryReal_eq_schur`, `execEntry_from_SM6bRecheck`, regular/variable Konstruktoren und `SM8RecheckGeneratedLevelHistoryExecMatrixLedger`. Statusfelder bestaetigen: completed from SM6bRecheck, keine freie Exec-Matrix, keine $\mathbb{R} \to \mathbb{Q}$-Rueckuebersetzung, keine Skalarinverse, kein `Matrix.inv`, kein `field_simp`, kein neues Inversenorakel, keine Produktidentitaet, kein `TwoSidedInv`, kein neues Certificate, keine DtN-/MultiSchur-Rekomputation und keine Charpoly-/Diskriminanten-/Heegner-Daten.
Praeziser Restbefund	Der Engpass liegt jetzt nicht mehr in `SM8`. Die historisch gruene Datei `GeneratedLevelHistoryExecMirrorSourceSM9a` kennt bereits die SM8-$\mathbb{R}$-Entry-Herkunft und die Schur-Rueckbindung, markiert aber weiterhin `blockedMissingGeneratedRationalEntrySource` bzw. `sm6bGeneratedLevelStepExecMirrorSourceRequired`. Nach gruenem `SM8Recheck` ist diese Diagnose veraltet: Die fehlende berechenbare Entry-Quelle entsteht nicht mehr als freie $\mathbb{Q}$-Quelle und nicht durch Real-Rueckkonversion, sondern natuerlich als SM8-`ExecComplex`-Matrix aus `SM8Recheck`. Ein direkter Start von `SM9b`, `SM9c`, `SM10`, `SM11`, `SM12` oder einem Heegner-/CM-Consumer waere weiterhin ein Consumer-Patch.
Naechste aktive Phase	`SM9aRecheck - GeneratedLevelHistoryExecMirrorSourceFromSM8Recheck`. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryExecMirrorSourceFromSM8RechecksSM9a.lean`. Import in `BuildAll` unmittelbar nach `GeneratedLevelHistoryExecMirrorSourceSM9a` und `GeneratedLevelHistoryExecMatrixFromSM6bRechecksSM8`, aber vor `GeneratedLevelHistoryCharpolyExecFromMirrorSM9b` bzw. allen Charpoly-/Faktor-/Diskriminanten-/Heegner-Consumern.
SM9aRecheck-DoD	Starker DoD ist eine kleine Source-Gate-Schicht, z.B. `GeneratedLevelHistoryExecMirrorSourceFromSM8RechecksSM9a`. Minimalfelder: `sourceSM9a : GeneratedLevelHistoryExecMirrorSourceSM9a ...`; `sourceSM8Recheck : GeneratedLevelHistoryExecMatrixFromSM6bRechecksSM8 ...`; ein proof-carrying Abgleich `sourceSM8Recheck.sourceSM8 = sourceSM9a.sourceSM8Matrix`; `execMatrix := sourceSM8Recheck.execMatrix`; `entryReal := sourceSM9a.entryReal`; Bridge `execMatrix_bridge_to_SM9a : forall i j, ExecComplexBridge.toComplex (execMatrix i j) = (entryReal i j : C)`; Projektionsfelder `execMatrix_bridge_to_SM8`, `entryReal_eq_SM8`, `entryReal_eq_generatedLevelHistoryMatrix`, `entryReal_eq_boundaryOperator`, `entryReal_eq_schur`, `execEntry_from_SM8Recheck`; Statusfelder: SM9a missing-source gate closed from SM8Recheck, no free Exec matrix, no Real-to-Rat conversion, no scalar inverse, no `Matrix.inv`, no `field_simp`, no test matrix, no Charpoly/Discriminant/Heegner, nextPhase `sm9bMayConsumeComputedExecMirror`.
Beweistaktik SM9aRecheck	Nicht die Schur-Formel, nicht die SM8-Matrix und nicht die Exec-Eintraege neu definieren. Baue `execMatrix` definitionell aus `sourceSM8Recheck.execMatrix`. Die zentrale Bridge ist eine entrywise `calc`-Kette: zuerst `sourceSM8Recheck.execMatrix_bridge_to_SM8 i j`; danach ersetze `sourceSM8Recheck.sourceSM8` durch `sourceSM9a.sourceSM8Matrix`; zuletzt ersetze `sourceSM9a.entryReal i j` mit `sourceSM9a.entryReal_eq_SM8 i j`. Wenn die Typen durch den Source-Abgleich nicht definitorisch fallen, verwende ein kleines Transport-/`congrArg`-Lemma auf dem SM8-Sourcefeld; keinen grossen dependent Index-Eliminator. Erwarteter Stil: lokale `rfl`, kurze `rw`/`calc`-Ketten, Feldprojektionen; keine globale `simp`-Flaeche, keine Summeninduktion, keine $\mathbb{R} \to \mathbb{Q}$-Rueckuebersetzung und keine neue Exec-Formel.

Import- und Consumer-Disziplin	Erlaubt sind `GeneratedLevelHistoryExecMirrorSourceSM9a` als historischer Real-/Gate-Consumer, `GeneratedLevelHistoryExecMatrixFromSM6bRecheckSM8` als neue berechenbare Exec-Quelle, die indirekt von SM8 getragene SM7/SM6-Provenienz und die minimale `ExecComplexBridge`-Schicht. Verboten sind Charpoly-/Diskriminantenmodule, Heegner-/CM-Consumer, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, neue LevelHistoryIndex-Carrier, Testmatrizen, `Matrix.inv`, `field_simp` und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v85	Genau eine kleine SM9a-Recheck-Datei plus ein `BuildAll`-Import. Kein `SM9a0`/`SM9a1`-Auditwald, kein paralleler `SM9b`-Stub und kein Charpoly-Test, solange dieser direkte Source-Gate-Versuch nicht mathematisch scheitert. Falls der Recheck blockiert, muss der Befund exakt benennen, ob die fehlende Gleichung zwischen `sourceSM8Recheck.sourceSM8` und `sourceSM9a.sourceSM8Matrix` fehlt oder ob die regular/variable Ledger aus unterschiedlichen SM8-Quellen konstruiert werden. Dann wird an dieser natuerlichen SM8/SM9a-Source-Schicht nachgeschaerft, nicht in `SM9b`.
Anschlussgate nach SM9aRecheck	Wenn `SM9aRecheck` einen unbedingten `execMatrix`-Witness mit Bridge zur SM9a-`entryReal`/SM8-Matrix liefert, folgt als naechste kleine Phase `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror`. Erst dort darf die berechenbare Matrix charpoly-/faktor-/diskriminantenreif konsumiert werden. `SM9c`/`SM10`/`SM11`/`SM12` bleiben bis zum gruenen SM9b-Gate gesperrt.
Fortgeschriebene Reihenfolge v85	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck - GeneratedLevelHistoryExecMirrorSourceFromSM8Recheck` **active**; `SM9b`/`SM9c`/`SM10`/`SM11`/`SM12` blocked until computed charpoly-ready ExecMirror. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; `SM9aRecheck` darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bh. Fortschreibungsentscheidung v86 nach gruenem SM9aRecheck-Build: SM9b aktiv

Statusentscheidung	`SM9aRecheck - GeneratedLevelHistoryExecMirrorSourceFromSM8Recheck` ist nach lokal gruenem Build formal geschlossen. Der positive Befund ist bewusst eng: Die SM8Recheck-`ExecComplex`-Matrix wird nicht neu berechnet, sondern als charpoly-faehige Mirror-Quelle zusammen mit der SM9a-$\mathbb{R}$-Entry-Herkunft exponiert. Die Bridge nach $\mathbb{C}$ laeuft entrywise gegen `entryReal`, `sourceSM8.matrix`, `generatedLevelHistoryMatrixSM8`, den Boundary-Operator und die Schur-Rueckbindung. Damit ist das v85-Gate `blockedMissingGeneratedRationalEntrySource` bzw. `sm6bGeneratedLevelStepExecMirrorSourceRequired` ueber die natuerliche SM8/SM9a-Schicht geschlossen.
Formaler SM9aRecheck-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryExecMirrorSourceFromSM8RecheckSM9a`, `fromSM8Recheck`, `sourceSM9a_eq_from_SM8Recheck`, `sourceSM8Recheck_source_eq_SM9a_source`, `execMatrix_eq_SM8Recheck`, `entryReal_eq_SM8Recheck`, `entryReal_eq_SM9aFromSM8Recheck`, `execMatrix_bridge_to_SM9a`, `execMatrix_bridge_to_SM8`, `execMatrix_bridge_to_SM8Recheck`, `entryReal_eq_SM8`, `entryReal_eq_generatedLevelHistoryMatrix`, `entryReal_eq_boundaryOperator`, `entryReal_eq_SM6`, `entryReal_eq_schur`, `execEntry_from_SM8Recheck`, regular/variable Konstruktoren und `SM9aRecheckGeneratedLevelHistoryExecMirrorSourceLedger`. Statusfelder bestaetigen: Missing-Source-Gate geschlossen aus SM8Recheck, SM9b darf die berechnete Mirror-Matrix konsumieren, keine freie Exec-Matrix, keine $\mathbb{R} \to \mathbb{Q}$-Konversion, keine Skalarinverse, kein `Matrix.inv`, kein `field_simp`, keine Testmatrix, kein neues Inversenorakel, keine Produktidentitaet, kein `TwoSidedInv`, kein neues Certificate, keine DtN-/MultiSchur-Rekomputation und keine Charpoly-/Diskriminanten-/Heegner-Daten.
Praeziser Restbefund	Der Engpass liegt jetzt nicht mehr in SM8/SM9a, sondern erstmals in der eigentlichen Charpoly-Auswertung. Es existiert nun eine berechenbare Matrix `execMatrix : Matrix _ _ ExecComplex` mit Real-/Schur-Provenienz. Noch nicht existiert eine aus dieser Matrix erzeugte `spectralPencil`, `operativeDeterminant` bzw. `operativeCharpolyEvaluation`-Schicht fuer einen `Schur.SpectralParameter`. Ein Start von `SM9c`, `SM10`, `SM11`, `SM12` oder Heegner-/CM-Consumern waere weiterhin ein Consumer-Patch, weil Faktor-, Diskriminanten- und Vergleichsdaten erst nach einer geschlossenen SM9b-Charpoly-Evaluation konsumierbar sind.
Naechste aktive Phase	Die naechste aktive Phase ist `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror`. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryCharpolyExecFromMirrorSM9b.lean`. Import in `BuildAll` unmittelbar nach `GeneratedLevelHistoryExecMirrorSourceFromSM8RecheckSM9a` und vor allen Faktor-/Diskriminanten-/Heegner-Modulen. Inhaltlich darf SM9b `CNNA.PillarA.Arithmetic.Schur.CharpolyExec` konsumieren, weil dort `spectralPencil`, `operativeDeterminant` und `operativeCharpolyEvaluation` bereits als operative Determinanten-/Pencil-API liegen; SM9b darf aber keine L1/L2/L3-Vergleichsledger, keine Heegnerlisten und keine CM-/elliptische-Funktionsschicht konsumieren.

SM9b-DoD	Starker DoD ist eine kleine Charpoly-Evaluationsschicht, z.B. `GeneratedLevelHistoryCharpolyExecFromMirrorSM9b`. Minimalfelder: `sourceSM9aRecheck : GeneratedLevelHistoryExecMirrorSourceFromSM8RecheckSM9a ...`; `z : Schur.SpectralParameter`; `matrix := sourceSM9aRecheck.execMatrix`; `matrix_eq_SM9aRecheck`; `spectralPencil := Schur.spectralPencil matrix z`; `spectralPencil_eq_from_matrix`; `determinant := Schur.operativeDeterminant spectralPencil`; `determinant_eq_matrix_det`; `charpolyEvaluation := Schur.operativeCharpolyEvaluation matrix z`; `charpolyEvaluation_eq_det_pencil`; Bridge-/Provenienzfelder zur SM9aRecheck-Matrix, etwa `matrix_bridge_to_SM9a`, `matrix_entry_from_SM8Recheck`, `matrix_entry_real_eq_schur`; Statusfelder: determinant/pencil evaluation closed, consumed computed SM9aRecheck mirror, no free matrix, no free charpoly polynomial, no factorization, no discriminant, no Heegner target, no CM target, no `Matrix.inv`, no `field_simp`, no new `TwoSidedInv`, nextPhase `sm9cMayConsumeCharpolyEvaluation`.
Beweistaktik SM9b	Nicht die Determinante händisch expandieren, keine Fin-2/Fin-3-Testmatrix einsetzen und keine Faktorform behaupten. Setze `matrix` definitionell auf `sourceSM9aRecheck.execMatrix`; setze `spectralPencil`, `determinant` und `charpolyEvaluation` definitionell ueber `Schur.spectralPencil`, `Schur.operativeDeterminant` und `Schur.operativeCharpolyEvaluation`. Die Kernbeweise sollten fast alle `rfl` oder kurze Feldprojektionen sein; fuer die Entry-Provenienz nutze die vorhandenen SM9aRecheck-Felder `execMatrix_bridge_to_SM9a`, `execMatrix_bridge_to_SM8`, `entryReal_eq_schur` und `execEntry_from_SM8Recheck`. Falls ein Cast-/Matrixgleichheitsziel entsteht, verwende kleine `congrArg (fun M => M i j)`-Lemmata statt eines breiten `simp`-Ansatzes. Keine Summeninduktion, kein Index-Eliminator-Monster, keine $\mathbb{R} \to \mathbb{Q}$-Rueckuebersetzung, keine neue Exec-Formel und keine lokale Reimplementation der Determinante.
Import- und Consumer-Disziplin	Erlaubt sind `GeneratedLevelHistoryExecMirrorSourceFromSM8RecheckSM9a` und `Schur.CharpolyExec`. Verboten sind `L1Baseline`, `L2Eisenstein`, `L3Gaussian`, `CharpolyC`, Faktor-/Diskriminantenmodule, Heegner-/CM-Consumer, elliptische-Funktions-/j-Consumer, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, Testmatrizen, `Matrix.inv`, `field_simp`, skalarer `inv` als neue Provenienz und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v86	Genau eine kleine SM9b-Datei plus ein `BuildAll`-Import. Kein `SM9b0`/`SM9b1`-Auditwald, kein paralleles `SM9c`, kein Faktor-/Diskriminantenstub und kein Eisenstein-/Gauss-/Heegner-Vergleich, solange die direkte `operativeCharpolyEvaluation`-Schicht nicht mathematisch scheitert. Wenn SM9b blockiert, muss der Befund exakt benennen, ob eine Instanz fuer `Fintype/DecidableEq` des LevelHistory-Index fehlt oder ob eine Typtransportroute zwischen SM9aRecheck-Index und CharpolyExec-Matrixindex fehlt. Dann wird an dieser natuerlichen SM8/SM9a/SM9b-Indexschnittstelle nachgeschaerft, nicht in SM9c oder SM10.
Anschlussgate nach SM9b	Wenn SM9b eine unbedingte `charpolyEvaluation` aus der berechneten SM9aRecheck-`execMatrix` liefert, folgt als naechste kleine Phase `SM9c - GeneratedLevelHistoryCharpolyCoefficientProfileFromSM9b` oder ein aequivalentes Coefficient-/Polynomial-Profile-Gate. Erst nach einem solchen Koeffizienten-/Polynomprofil duerfen Faktor-/Diskriminantenphasen starten. `SM10`/`SM11` bleiben Vergleichsphasen gegen Eisenstein/Gauss, und `SM12` bleibt der spaetere Parameter-/Regime-Scan; kein Heegner-Wert wird vor berechneter Charpoly-/Diskriminantenroute als Provenienz verwendet.
Fortgeschriebene Reihenfolge v86	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` **active**; `SM9c`/`SM10`/`SM11`/`SM12` blocked until computed charpoly evaluation and subsequent coefficient/discriminant gates. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9b darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bi. Fortschreibungsentscheidung v87 nach gruenem SM9b-Build: SM9c aktiv

Statusentscheidung	`SM9b - GeneratedLevelHistoryCharpolyExecFromMirror` ist nach lokal gruenem Build formal geschlossen. Der positive Befund ist eng und korrekt eingegrenzt: Die aus `SM9aRecheck` kommende `execMatrix` wird ueber `spectralPencil`, `operativeDeterminant` und `operativeCharpolyEvaluation` konsumiert; der Charpoly-Wert bei einem `Schur.SpectralParameter` ist gegen den Determinanten-Pencil zurueckgebunden. Damit ist das v86-Gate `sm9cMayConsumeCharpolyEvaluation` erfuellt.

Formaler SM9b-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryCharpolyExecFromMirrorSM9b`, `generatedLevelHistorySpectralPencilSM9b`, `generatedLevelHistoryOperativeDeterminantSM9b`, `generatedLevelHistoryCharpolyEvaluationSM9b`, `generatedLevelHistorySpectralPencilSM9b_eq_from_matrix`, `generatedLevelHistoryOperativeDeterminantSM9b_eq_matrix_det`, `generatedLevelHistoryCharpolyEvaluationSM9b_eq_det_pencil`, `generatedLevelHistoryOperativeDeterminantSM9b_eq_charpolyEvaluation`, `fromSM9aRecheck`, `matrix_from_SM9aRecheck`, `charpolyEvaluation_from_det_pencil`, `determinant_from_charpolyEvaluation`, `matrix_bridge_to_entryReal`, `entryReal_from_schur`, regular/variable Konstruktoren, `SM9bGeneratedLevelHistoryCharpolyExecLedger`, `sm9bGeneratedLevelHistoryCharpolyExecLedger`, `sm9b_nextPhase_SM9c`, `sm9b_noFactorization`, `sm9b_noDiscriminant` und `sm9b_noHeegnerTarget`. Statusfelder bestaetigen: SM9b konsumiert den berechneten SM9aRecheck-Mirror; keine freie Matrix, kein freies Charpoly-Polynom, keine Faktorisierung, keine Diskriminante, kein Heegner-/CM-Ziel, kein `Matrix.inv`, kein `field_simp`, kein neues `TwoSidedInv`, kein neues Certificate und keine DtN-/MultiSchur-Rekomputation.
Praeziser Restbefund	SM9b liefert eine *Evaluation* bei einem Parameter `z`; es liefert noch kein operatives Polynomobjekt und keine Koeffizientenquelle. Ein direkter Sprung zu Faktorisierung, Diskriminante, Eisenstein-/Gauss-Vergleich, Heegner-Test oder Parameter-Scan waere daher ein Consumer-Patch. Zugleich darf die naechste Schicht nicht einfach ein externes oder noncomputable `Matrix.charpoly`-Oracle als operative Quelle einschieben: Die aktive Quelle muss aus dem berechenbaren SM9b-Matrixpencil selbst entstehen. Der mathlib-Satz `Matrix.eval_charpoly` kann spaeter hoechstens als Vergleichs-/Auditanker dienen; die operative CNNA-Quelle bleibt der explizite Polynom-Pencil-Determinant.
Naechste aktive Phase	Die naechste aktive Phase ist `SM9c - GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b`. Implementierungsdatei: vorzugsweise `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b.lean`. Import in `BuildAll` unmittelbar nach `GeneratedLevelHistoryCharpolyExecFromMirrorSM9b` und vor allen Faktor-/Diskriminanten-/Heegner-/CM-Consumern. Inhaltlich darf SM9c die SM9b-Ledger konsumieren und eine kleine operative Polynomial-Schicht ueber `Polynomial ExecComplex` aufbauen; SM9c darf keine Faktorisierung, keine Diskriminante, keine Wurzelberechnung, keinen Heegner-Vergleich und keine L1/L2/L3-Zielmatrix konsumieren.
SM9c-DoD	Starker DoD ist ein kleines proof-carrying Polynom-/Koeffizientenprofil, z.B. `GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b`. Minimalfelder: `sourceSM9b : GeneratedLevelHistoryCharpolyExecFromMirrorSM9b ...`; `polynomialPencil : Matrix _ _ (Polynomial ExecComplex)` mit definitorischer Herkunft `Matrix.scalar _ Polynomial.X - (Polynomial.C.mapMatrix sourceSM9b.matrix)`; `charpolyPolynomial : Polynomial ExecComplex` mit `charpolyPolynomial_eq_det_pencil : charpolyPolynomial = Matrix.det polynomialPencil`; `coefficient : Nat -> ExecComplex` mit `coefficient_eq_coeff : coefficient n = charpolyPolynomial.coeff n`; `evalParameter := Schur.spectralParameterExec sourceSM9b.z`; `polynomialEvaluation := charpolyPolynomial.eval evalParameter`; `polynomialEvaluation_eq_SM9b_charpolyEvaluation`; regular/variable Konstruktoren und ein `SM9cGeneratedLevelHistoryOperationalCharpolyPolynomialProfileLedger`. Statusfelder: `polynomialProfileClosedFromSM9b`, `coefficientsDerivedFromPolynomial`, `noFreeCoefficientTable`, `noFactorization`, `noDiscriminant`, `noRootExtraction`, `noHeegnerTarget`, `noCMTarget`, `noMatrixInv`, `noFieldSimp`, `noScalarInv`, `noNoncomputableOperationalSource`, `nextPhaseDiscriminantGateOnlyAfterSM9c`.
Beweistaktik SM9c	Nicht die Determinante expandieren und keine Fin-2/Fin-3- oder L1/L2/L3-Spezialformel einsetzen. Setze `polynomialPencil` und `charpolyPolynomial` definitionell. Die Koeffizientenquelle ist nur `fun n => charpolyPolynomial.coeff n`; damit sind Koeffizienten exponiert, aber noch nicht faktorisiert oder diskriminantisch interpretiert. Fuer `polynomialEvaluation_eq_SM9b_charpolyEvaluation` sollte der Beweis ueber die Auswertung als Ringhomomorphismus laufen: `Polynomial.evalRingHom evalParameter` auf den Polynom-Pencil anwenden, dann `RingHom.map_det` bzw. das vorhandene Determinanten-Mapping nutzen und entrywise gegen `Schur.spectralPencil sourceSM9b.matrix sourceSM9b.z` identifizieren. Danach mit `sourceSM9b.spectralPencil_eq_from_matrix` und `sourceSM9b.charpolyEvaluation_eq_det_pencil` abschliessen. Erwarteter Stil: kurze `calc`-Ketten, lokale `rw`-/`change`-Schritte, `ext i j` nur fuer die Pencil-Eintragsschicht; kein globales `simp`, keine Summeninduktion, kein Index-Eliminator-Monster, keine $\mathbb{R} \to \mathbb{Q}$-Rueckuebersetzung, keine neue Exec-Formel.
Import- und Consumer-Disziplin	Erlaubt sind `GeneratedLevelHistoryCharpolyExecFromMirrorSM9b` und, falls nicht bereits transitiv verfuegbar, eine minimale Polynomial-Basis wie `Mathlib.Algebra.Polynomial.Basic`. `Mathlib.LinearAlgebra.Matrix.Charpoly.Basic` ist fuer SM9c *nicht* die operative Hauptquelle, weil die CNNA-Schicht berechenbar bleiben soll und das Polynom aus dem expliziten SM9b-Pencil-Determinanten entstehen muss. Ein spaeterer Vergleichssatz gegen `Matrix.charpoly` darf nur in einer separaten Audit-/Comparison-Schicht erfolgen, falls dafuer kein `noncomputable` in den operativen Pfad sickert. Verboten sind `Mathlib.LinearAlgebra.Matrix.Charpoly.Coeff` als Abkuerzung fuer Trace-/Determinanten-Koeffizienten, `Schur.CharpolyC`, `L1Baseline`, `L2Eisenstein`, `L3Gaussian`, Faktor-/Diskriminantenmodule, Heegner-/CM-/j-Consumer, elliptische-Funktionsschichten, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, Testmatrizen, `Matrix.inv`, `field_simp`, skalarer `inv` als neue Provenienz und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.

Overhead-Regel v87	Genau eine kleine SM9c-Datei plus ein `BuildAll`-Import. Kein `SM9c0/SM9c1`-Auditwald, kein paralleles Diskriminantenmodul, kein Faktorstub und kein Eisenstein-/Gauss-/Heegner-Vergleich, solange der direkte operative Polynom-Pencil nicht mathematisch scheitert. Wenn SM9c blockiert, muss der Befund exakt benennen, ob ein Polynomial-Import/Notationsthema, ein `RingHom.map_det`-Bridgeproblem oder eine Computability-Sperre vorliegt. Dann wird an der natuerlichen SM9b/SM9c-Polynomial-Schicht nachgeschaerft, nicht in SM10/SM11.
Anschlussgate nach SM9c	Wenn SM9c ein unbedingtes `charpolyPolynomial`- und Koeffizientenprofil aus der berechneten SM9b-Matrix liefert und die Evaluation gegen SM9b geschlossen ist, darf danach erst eine kleine Diskriminanten-/Faktorprofil-Quelle geplant werden. `SM10` und `SM11` bleiben Vergleichsphasen gegen Eisenstein/Gauss und duerfen erst starten, wenn die dafuer benoetigte Diskriminantenroute aus SM9c-derived Daten existiert. `SM12` bleibt der spaetere Parameter-/Regime-Scan; kein Heegner-Wert wird vor berechneter Charpoly-/Diskriminantenroute als Provenienz verwendet.
Fortgeschriebene Reihenfolge v87	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c - GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b` **active**; `SM10/SM11/SM12` blocked until computed operational polynomial/coefficient profile and subsequent discriminant gates. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9c darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bj. Fortschreibungsentscheidung v88 nach gruenem SM9c-Fix-Build: SM9d aktiv

Statusentscheidung	`SM9c - GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b` ist nach lokal gruenem Fix-Build formal geschlossen, aber in einem engeren und methodisch wichtigeren Sinn als v87 zuerst formuliert hatte. Der Versuch, ein operatives `Polynomial ExecComplex`-Objekt zu erzeugen, ist an einer echten Computability-Grenze gescheitert: `Polynomial.semiring/Polynomial.commRing` liefern fuer den operativen Pfad keinen ausfuehrbaren Code. Ein `noncomputable`-Patch waere Planbruch gewesen. Die gruene SM9c-Fassung schliesst daher die aus SM9b stammende Evaluationsfamilie und das Determinanten-Pencil-Gate, exponiert aber zugleich proof-carrying, dass eine eigene operationale Koeffizientenquelle noch fehlt.
Formaler SM9c-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b`, `generatedLevelHistoryOperationalCharpolyEvaluationFamilySM9c`, `generatedLevelHistoryOperationalCharpolyEvaluationAtSM9bParameterSM9c`, `generatedLevelHistoryOperationalCharpolyEvaluationAtSM9bParameterSM9c_eq_SM9b`, `generatedLevelHistoryOperationalCharpolyEvaluationFamilySM9c_eq_operative`, `generatedLevelHistoryOperationalCharpolyEvaluationAtSM9bParameterSM9c_eq_det_pencil`, `fromSM9b`, `matrix_from_SM9b`, `evaluationFamily_from_operative`, `evaluationAtSource_from_SM9b_charpolyEvaluation`, `evaluationAtSource_from_det_pencil`, regular/variable Konstruktoren, `SM9cGeneratedLevelHistoryOperationalCharpolyPolynomialProfileLedger`, `sm9cGeneratedLevelHistoryOperationalCharpolyPolynomialProfileLedger`, `sm9c_regularPolynomialEvaluation_eq_SM9b`, `sm9c_variablePolynomialEvaluation_eq_SM9b`, `sm9c_regularEvaluationAtSource_eq_det_pencil`, `sm9c_variableEvaluationAtSource_eq_det_pencil`, `sm9c_coefficientExtraction_blocked`, `sm9c_nextPhase_coefficientSourceBeforeDiscriminantGate`, `sm9c_noFactorization`, `sm9c_noDiscriminant` und `sm9c_noHeegnerTarget`. Statusfelder bestaetigen: keine freie Koeffizententabelle, keine Faktorisierung, keine Diskriminante, keine Wurzelberechnung, kein Heegner-/CM-Ziel, kein `Matrix.inv`, kein `field_simp`, kein skalarer Inversenpfad, kein `noncomputable` und kein `Matrix.charpoly`-Oracle.
Praeziser Restbefund	SM9c liefert noch keine berechenbare Koeffizientenfamilie. Die Stelle ist exakt lokalisiert: Die Evaluation `z |-> det (zI - A)` ist operational geschlossen, aber die Koeffizientenextraktion aus einem mathlib-`Polynomial ExecComplex` ist im operativen Pfad nicht zulaessig. Damit ist der naechste Schritt nicht SM10, nicht SM11 und kein Diskriminantenstub, sondern eine minimale Nachruestung der natuerlichen SM9b/SM9c-Polynomial-/Coefficient-Schicht.
Naechste aktive Phase	Die naechste aktive Phase ist `SM9d - GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c.lean`. Import in `BuildAll` direkt nach `GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b` und vor allen Faktor-/Diskriminanten-/Heegner-/CM-Consumern. Inhaltlich konsumiert SM9d nur die gruene SM9c-Schicht und die darin enthaltene SM9b-Quelle; es darf keine neue Matrix, keine neue Charpoly-Evaluation und keinen L1/L2/L3-Zielvergleich erzeugen.

SM9d-DoD	SM9d soll eine kleine proof-carrying operationale Koeffizientenquelle schaffen oder, falls die konkrete Determinanten-Koeffizientenrekursion im aktuellen API-Stand noch fehlt, exakt diese fehlende Quelle als naechste natuerliche Nachruestestelle ausweisen. Minimalfelder fuer den positiven Abschluss: `sourceSM9c : GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b ...`; `matrix := sourceSM9c.matrix`; `evaluationFamily := sourceSM9c.evaluationFamily`; ein berechenbarer Traeger fuer Koeffizienten, vorzugsweise ohne mathlib-`Polynomial`; `degreeBound` aus der endlichen SM9b/SM9c-Indexprovenienz; `coefficient : Nat -> ExecComplex` oder ein gebundener `Fin (degreeBound+1)`-Traeger mit definierter Nat-Projektion; `coefficient_source_eq_operationalDeterminantPencil`; `evaluationFromCoefficients`; `evaluationFromCoefficients_eq_SM9c_evaluationFamily` mindestens am SM9b-Parameter, falls die volle `forall z`-Aussage noch zu stark ist; regular/variable Konstruktoren und ein `SM9dGeneratedLevelHistoryOperationalCoefficientSourceLedger`. Statusfelder: `coefficientSourceClosedFromSM9c` oder andernfalls `blockedMissingOperationalCoefficientRecurrenceSource`, `noFreeCoefficientTable`, `noPolynomialSemiringRuntime`, `noMathlibPolynomialOperationalPath`, `noMatrixCharpolyOracle`, `noFactorization`, `noDiscriminant`, `noHeegnerTarget`, `noCMTarget`, `noMatrixInv`, `noFieldSimp`, `noScalarInv` und `noNoncomputableOperationalSource`.
Beweistaktik SM9d	Positiver Erstversuch: nicht wieder `Polynomial ExecComplex` einfuehren. Zuerst eine kleine operationale Polynom-/Koeffizientenrepraesentation als Datenstruktur bauen, z.B. gebundene Koeffizienten plus Horner-/Fin-Summen-Evaluation, deren Operationen selbst ausfuehrbare Definitionen sind. Der Pencil-Eintrag ist linear: diagonal entsteht aus `z - A i i`, offdiag aus `0 - A i j`. Danach wird, falls die Determinanten-Koeffizientenrekursion bereits lokal tragbar ist, die Koeffizientenquelle aus dieser expliziten Pencil-Struktur erzeugt. Beweisstil: kleine `rfl`- und Projektionssaetze fuer Matrix-/Evaluation-Herkunft; `calc`-Ketten fuer die Bruecke zur SM9c-Evaluation; keine Determinanten-Expansion ueber spezielle `Fin 2`/`Fin 3`-Zielmatrizen; keine Summeninduktion, solange die Koeffizientenrekursion nicht explizit als eigene operationale Struktur vorhanden ist; keine globalen `simp`-Flaechen. Wenn die Rekursion fehlt, wird SM9d nicht zu einem freien Koeffizienten-Oracle erweitert, sondern schliesst mit einem präzisen Missing-Source-Gate fuer eine minimale `SM9e - OperationalDeterminantCoefficientRecurrence`-Nachruestung.
Import- und Consumer-Disziplin	Erlaubt ist als Hauptimport nur `GeneratedLevelHistoryOperationalCharpolyPolynomialProfileFromSM9b`. Ein separater Import aus `Schur/CharpolyExec` ist nur zulaessig, wenn dort eine bereits operationale Koeffizienten-/Determinantenrekursion existiert oder in SM9d minimal lokal referenziert werden muss. Verboten bleiben `Mathlib.Algebra.Polynomial.*` als operative Quelle, `Mathlib.LinearAlgebra.Matrix.Charpoly.*`, `Schur.CharpolyC`, `L1Baseline`, `L2Eisenstein`, `L3Gaussian`, Faktor-/Diskriminantenmodule, Heegner-/CM-/j-Consumer, elliptische-Funktionsschichten, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, Testmatrizen, `Matrix.inv`, `field_simp`, skalarer `inv`, freie Koeffizententabellen und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.
Overhead-Regel v88	Genau eine kleine SM9d-Datei plus ein `BuildAll`-Import. Kein `SM9d0`/`SM9d1`-Auditwald, kein paralleles Diskriminantenmodul, kein Faktorstub und kein Eisenstein-/Gauss-/Heegner-Vergleich. Die Phase beginnt konstruktiv mit der kleinsten operationalen Koeffizientenquelle; nur wenn dieser positive Versuch an einer klaren fehlenden Determinanten-Koeffizientenrekursion scheitert, wird diese fehlende Rekursion als eigene minimale SM9e-Quelle formuliert.
Anschlussgate nach SM9d	Erst wenn SM9d eine unbedingte, nicht freie, berechenbare Koeffizientenquelle aus SM9c/SM9b-Daten liefert und diese gegen die SM9c-Evaluation zurueckgebunden ist, darf eine kleine Faktor-/Diskriminantenquelle geplant werden. `SM10` und `SM11` bleiben Vergleichsphasen gegen Eisenstein/Gauss und duerfen erst starten, wenn eine derived-only Diskriminantenroute aus SM9d-derived Koeffizienten existiert. `SM12` bleibt der spaetere Parameter-/Regime-Scan; kein Heegner-Wert wird vor berechneter Charpoly-/Diskriminantenroute als Provenienz verwendet.
Fortgeschriebene Reihenfolge v88	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family und Coefficient-Missing-Source-Gate; `SM9d - GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c` **active**; `SM10`/`SM11`/`SM12` blocked until computed operational coefficient source and subsequent discriminant gates. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9d darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bk. Fortschreibungsentscheidung v89 nach gruenem SM9d-Build: SM9e aktiv

Statusentscheidung	`SM9d - GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c` ist nach lokal gruenem Build formal geschlossen, aber nicht als berechenbare Koeffizientenquelle. Geschlossen ist die korrekte Provenienz- und Sperrschicht: `sourceSM9c`, `matrix = sourceSM9c.matrix`, `evaluationFamily = sourceSM9c.evaluationFamily`, `evaluationAtSource = Matrix.det sourceSM9c.spectralPencilAtSource`, `degreeBound = level+1` und die expliziten No-Consumer-Statusfelder. Der gruene Befund bestaetigt damit genau die v88-Fallunterscheidung: Die konkrete operationale Determinanten-Koeffizientenrekursion fehlt noch und darf nicht durch freie Koeffiziententabellen oder einen nichtcomputablen Polynomial-Pfad ersetzt werden.
Formaler SM9d-Ertrag	Geschlossen sind insbesondere `GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`, `generatedLevelHistoryOperationalDegreeBoundSM9d`, `fromSM9c`, `matrix_from_SM9c`, `evaluationFamily_from_SM9c`, `evaluationFamily_from_operative`, `evaluationAtSource_from_SM9c`, `evaluationAtSource_from_det_pencil`, `degreeBound_from_levelHistory`, `blockedMissingOperationalCoefficientRecurrenceSource`, `regularGeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`, `variableGeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`, `SM9dGeneratedLevelHistoryOperationalCoefficientSourceLedger`, `sm9dGeneratedLevelHistoryOperationalCoefficientSourceLedger`, die beiden Handoff-Konstruktoren aus SM9aRecheck bzw. r3c1-Ledger sowie die Statuslemmata `sm9d_regularEvaluationAtSource_eq_SM9c`, `sm9d_variableEvaluationAtSource_eq_SM9c`, `sm9d_regularEvaluationAtSource_eq_det_pencil`, `sm9d_variableEvaluationAtSource_eq_det_pencil`, `sm9d_blockedMissingOperationalCoefficientRecurrenceSource`, `sm9d_nextPhase_operationalCoefficientRecurrenceBeforeDiscriminantGate`, `sm9d_noFactorization`, `sm9d_noDiscriminant` und `sm9d_noHeegnerTarget`.
Praeziser Restbefund	SM9d liefert keine Funktion `coefficient : Nat -> ExecComplex`, keinen gebundenen `Fin (degreeBound+1)`-Koeffiziententraeger, keine Horner-/Finite-Summen-Auswertung aus Koeffizienten und keinen Satz `evaluationFromCoefficients_eq_SM9c_evaluationFamily`. Der fehlende Pfeil ist daher exakt: `linear spectral pencil over ExecComplex` → `operational bounded coefficient representation` → `evaluation equals Matrix.det pencil`. Ohne diesen Pfeil waere jeder Start von SM10/SM11 nur ein Faktor-/Diskriminantenconsumer ohne berechnete Koeffizientenquelle.
Naechste aktive Phase	Die naechste aktive positive Phase ist `SM9e - GeneratedOperationalDeterminantCoefficientRecurrence`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalDeterminantCoefficientRecurrenceSM9e.lean`. Import in `BuildAll` direkt nach `GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`. Inhaltlich konsumiert SM9e nur SM9d, die darin enthaltene SM9c-/SM9b-Quelle, `ExecComplex`-Ringoperationen und den bereits genutzten operativen Determinanten-/Pencil-Rahmen. SM9e erzeugt noch keine Faktorisierung, keine Diskriminante, keinen Eisenstein-/Gauss-/Heegner-/CM-Vergleich und keine neuen Schur-/DtN-/Interior-Daten.
SM9e-DoD	SM9e soll die minimale operationale Koeffizientenrekursion fuer Determinanten linearer Pencils bereitstellen. Kernobjekte: ein kleiner gebundener Koeffiziententraeger, z.B. `OperationalBoundedPolynomialSM9e degreeBound`; Konstruktoren fuer `zero`, `one`, `const`, `linear`, `add`, `neg/sub`, `boundedMul/convolution`; eine ausfuehrbare `eval` per Horner oder struktureller Fin-/Nat-Rekursion; `linearPencilEntryPolynomial` mit konstantem Term `-A i j` und linearem Term `1` nur auf der Diagonale; eine rekursive Determinanten-Koeffizientenquelle `operationalDeterminantCoefficientRecurrence`; regular/variable Konstruktoren aus dem SM9d-Ledger; und ein `SM9eGeneratedOperationalDeterminantCoefficientRecurrenceLedger`. Minimaler starker Abschluss: `coefficients_source_eq_linearPencil`, `eval_operationalDeterminantCoefficientRecurrence_eq_det_pencil` mindestens am SM9d-Source-Parameter, `degreeBound_eq_SM9d`, `noFreeCoefficientTable`, `noPolynomialSemiringRuntime`, `noMathlibPolynomialOperationalPath`, `noMatrixCharpolyOracle`, `noFactorization`, `noDiscriminant`, `noHeegnerTarget`, `noCMTarget`, `noMatrixInv`, `noFieldSimp`, `noScalarInv` und `noNoncomputableOperationalSource`.
Beweistaktik SM9e	Zuerst keine Matrixdeterminante beweisen, sondern die Koeffizientenarithmetik klein und executable machen: alle Operationen auf dem gebundenen Traeger definitional oder per kurzer Strukturrekursion; Evaluationslemmata fuer `add/neg/sub/mul` nur lokal, mit `ExecComplex.ext` und rationaler Komponentenrechnung, falls noetig mit gezieltem `ring` auf $\mathbb{Q}$-Komponenten, nicht mit globalem `simp`. Danach den linearen Pencil entrywise mit `rfl`/kleinen `change`-Schritten an SM9d.matrix zurueckbinden. Fuer die Determinantenrekursion ist der robuste positive Pfad eine Laplace- oder Listen-/Finset-Rekursion ueber den konkreten endlichen Indextraeger; die Evaluation dieser Rekursion wird per struktureller Rekursion gegen dieselbe Determinantenformel geschlossen. Wenn ein mathlib-Determinanten-Expand-Theorem konsumiert wird, muss dessen Import in Lean 4.28.0 lokal verifiziert werden und es darf nur als Beweisbridge dienen, nicht als Koeffizientenquelle.
Import- und Consumer-Disziplin	Hauptimport: `GeneratedLevelHistoryOperationalCoefficientSourceFromSM9c`. Zulaessig sind nur bereits im Pfad vorhandene oder lokal verifizierte Imports fuer endliche Matrixdeterminanten, Finset/List-Rekursion und `ExecComplex`. Verboten bleiben `Mathlib.Algebra.Polynomial.*` als operative Quelle, `Mathlib.LinearAlgebra.Matrix.Charpoly.*`, `Schur.CharpolyC`, `L1Baseline`, `L2Eisenstein`, `L3Gaussian`, Faktor-/Diskriminantenmodule, Heegner-/CM-/j-Consumer, elliptische-Funktionsschichten, neue Interior-Elimination-Daten, neue DtN-/MultiSchur-Daten, Testmatrizen, `Matrix.inv`, `field_simp`, skalarer `inv`, freie Koeffiziententabellen, freie Gradannahmen und jede Rueckuebersetzung aus $\mathbb{R}$ nach $\mathbb{Q}$.

Overhead-Regel v89	Genau eine kleine SM9e-Datei plus ein `BuildAll`-Import. Kein `SM9e0/SM9e1`-Auditwald, solange der positive Rekursionsversuch nicht an einem konkret benannten API-Hindernis scheitert. Kein paralleles Diskriminantenmodul, kein Faktorstub, kein Eisenstein-/Gauss-/Heegner-Vergleich. Falls die volle generische Determinantenbridge zu gross ist, ist eine saubere zweistufige Grenze zulaessig: SM9e schliesst die operationale Koeffizientenrekursion und die Auswertung der Rekursion; ein spaeteres minimales SM9f darf nur die Bridge `eval recurrence = SM9d/SM9c det pencil` schliessen. SM9f darf nicht vor SM9e als freier Consumer entstehen.
Anschlussgate nach SM9e	Erst wenn SM9e eine unbedingte, nicht freie, berechenbare Koeffizientenquelle aus dem SM9d/SM9c/SM9b-Pencil liefert und diese gegen die Determinanten-Evaluation zurueckgebunden ist, darf eine kleine Faktor-/Diskriminantenquelle geplant werden. Die naheliegende spaetere Phase waere dann ein eigenes Diskriminanten-/Faktor-Gate, nicht sofort SM10. `SM10` und `SM11` bleiben Vergleichsphasen gegen Eisenstein/Gauss und duerfen erst starten, wenn eine derived-only Diskriminantenroute aus SM9e-derived Koeffizienten existiert. `SM12` bleibt der spaetere Parameter-/Regime-Scan.
Fortgeschriebene Reihenfolge v89	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family und Coefficient-Missing-Source-Gate; `SM9d` closed/green als Operational-Coefficient-Source-Missing-Recurrence-Gate; `SM9e - GeneratedOperationalDeterminantCoefficientRecurrence` **active**; `SM10/SM11/SM12` blocked until computed operational coefficient recurrence, coefficient-source bridge and subsequent discriminant gate. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9e darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bl. Fortschreibungsentscheidung v90 nach gruenem SM9e-Build: SM9f aktiv

Statusentscheidung	`SM9e - GeneratedOperationalDeterminantCoefficientRecurrence` ist nach lokal gruenem Build formal geschlossen. Der Warnungsfix bestaetigt zugleich die richtige operative Richtung: keine Umstellung auf `noncomputable`, sondern eine listen-/fuelgesteuerte Rekurrenz ohne `Finset.toList`-Runtime-Abhaengigkeit. Geschlossen ist damit die berechenbare Koeffizientenrekurrenz aus dem SM9d-Pencil. Nicht geschlossen ist die mathematische Bridge zur SM9d-/SM9c-Determinantenevaluation; sie ist im Code explizit als `determinantBridgeDeferredToSM9f` markiert.
Formaler SM9e-Ertrag	Geschlossen sind insbesondere `SM9eOperationalCoefficientRecurrenceStatus`, `SM9eDeterminantBridgeStatus`, `OperationalBoundedPolynomialSM9e`, `coefficientAt`, `hornerAux`, `eval`, `zero`, `one`, `const`, `linear`, `add`, `neg`, `sub`, `scale`, `convolutionCoeffAux`, `boundedMul`, `operationalLinearPencilEntryPolynomialSM9e`, `operationalDeterminantColumnExpansionAuxSM9e`, `operationalDeterminantCoefficientRecurrenceFromRowsColsFuelSM9e`, `operationalDeterminantCoefficientRecurrenceFromOrderSM9e`, `operationalDeterminantCoefficientRecurrenceSM9e`, `GeneratedOperationalDeterminantCoefficientRecurrenceSM9e`, `fromSM9d`, regular/variable Konstruktoren, `SM9eGeneratedOperationalDeterminantCoefficientRecurrenceLedger`, die Handoff-Konstruktoren aus SM9aRecheck bzw. r3c1-Ledger sowie die Statuslemmata `sm9e_regularCoefficients_source_eq_linearPencil`, `sm9e_variableCoefficients_source_eq_linearPencil`, `sm9e_regularEvaluationAtSource_eq_coefficients`, `sm9e_variableEvaluationAtSource_eq_coefficients`, `sm9e_determinantBridge_deferred_to_SM9f`, `sm9e_noFactorization`, `sm9e_noDiscriminant` und `sm9e_noHeegnerTarget`.
Praeziser Restbefund	SM9e beweist noch nicht, dass die Evaluation der operationalen Koeffizientenrekurrenz mit `Matrix.det` des SM9d-/SM9c-Pencils uebereinstimmt. Der fehlende Pfeil ist nun exakt: `OperationalBoundedPolynomialSM9e.eval(operationalDeterminantCoefficientRecurrenceSM9edegreeBoundA)z` → `Matrix.det` des zugehoerigen linearen Spectral-Pencils. Ohne diesen Satz ist zwar die Koeffizientenquelle operational vorhanden, aber noch nicht als Determinantenkoeffizientenquelle gegen SM9c validiert. Daher bleiben Faktor-, Diskriminanten-, Eisenstein-/Gauss-, Heegner-, CM- und j-Targets weiterhin blockiert.
Naechste aktive Phase	Die naechste aktive positive Minimalphase ist `SM9f--GeneratedOperationalDeterminantEvaluationBridgeFromSM9e`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalDeterminantEvaluationBridgeFromSM9e.lean`. Import in `BuildAll` direkt nach `GeneratedOperationalDeterminantCoefficientRecurrenceSM9e` und vor jedem Faktor-/Diskriminanten-/Heegner-/CM-Consumer. Inhaltlich konsumiert SM9f nur SM9e, dessen SM9d-/SM9c-Quelle, den linearen Pencil und lokal verifizierte Determinanten-Basislemmata. SM9f erzeugt keine neue Koeffizientenrekurrenz, keine neue Matrix, keine Faktorisierung, keine Diskriminante, keinen Eisenstein-/Gauss-/Heegner-/CM-Vergleich und keine neuen Schur-/DtN-/Interior-Daten.

SM9f-DoD	SM9f soll genau die fehlende Bridge schliessen. Kernobjekte: `SM9fDeterminantEvaluationBridgeStatus`; eine Struktur wie `GeneratedOperationalDeterminantEvaluationBridgeFromSM9e`; Feld `sourceSM9e`; regular/variable Bridge-Felder `evaluationFromCoefficients_eq_SM9dEvaluationAtSource` und/oder staerker `evaluationFromCoefficients_eq_SM9cEvaluationFamily`; theorematische Rueckbindung an `sourceSM9d.evaluationAtSource` und `sourceSM9d.sourceSM9c.evaluationFamily`; ein `SM9fGeneratedOperationalDeterminantEvaluationBridgeLedger`; Statusfelder `bridgeClosedBeforeDiscriminantGate`, `noNewCoefficientRecurrence`, `noFreeDeterminantOracle`, `noPolynomialSemiringRuntime`, `noMathlibPolynomialOperationalPath`, `noMatrixCharpolyOracle`, `noFactorization`, `noDiscriminant`, `noHeegnerTarget`, `noCMTarget`, `noMatrixInv`, `noFieldSimp`, `noScalarInv` und `noNoncomputableOperationalSource`. Minimaler starker Abschluss ist die regular/variable Gleichheit am SM9d-Source-Parameter; der bevorzugte Abschluss ist die `forall z`-Bridge gegen die SM9c-EvaluationFamily, falls sie ohne neue Consumer tragbar ist.
Beweistaktik SM9f	Positiver Erstversuch: kein neuer operativer Algorithmus. Zuerst eine reine Evaluationsnormalform fuer SM9e bauen: `evaluationFromCoefficients_eq_eval`, `coefficients_source_eq_linearPencil`, `degreeBound_eq_SM9d` und die SM9d-/SM9c-Pencil-Felder nur per Projektion, `rfl`, kleinen `change`-Schritten und kurzen `calc`-Ketten zusammenfuehren. Danach die determinantielle Substanz isoliert beweisen: entweder ueber ein lokal importiertes, in Lean 4.28.0 verifiziertes `Matrix.det`-Basislemma fuer die Leibniz-/Laplace-Expansion oder ueber eine eigene Hilfsnormalform, die die SM9e-Listenrekurrenz und die benutzte Determinantenformel auf denselben endlichen Indextraeger `Fin n`/`List.finRange n` bringt. Keine globale `simp`-Flaeche; `simp` hoechstens auf dem letzten Meter fuer definitorische Listen-/Bool-Ziele. Keine `Equiv.Perm.sign`- oder `Finset.toList`-Runtime-Definition im operativen Pfad; mathlib-Determinantenlemmata duerfen nur in `Prop`-Beweisen konsumiert werden, nicht als neue Berechnungsquelle.
Ruecksprungregel	Falls die Bridge nicht beweisbar ist, wird nicht bei SM10/SM11 oder im Diskriminanten-Gate gepatcht. Die minimale Rueckkopplung liegt dann in SM9e selbst: entweder fehlt eine Evaluation-Homomorphie fuer `OperationalBoundedPolynomialSM9e` unter `add/neg/boundedMul`, oder die listen-/Laplace-Rekurrenz braucht eine zusaetzliche proof-carrying Expansionsnormalform. In diesem Fall waere eine einzige positive Nachschaerfung `SM9ePlus - OperationalBoundedPolynomialEvaluationLemmas` bzw. `SM9eLaplaceExpansionNormalForm` zulaessig; kein Faktor-/Diskriminanten-Stub darf als Workaround entstehen.
Overhead-Regel v90	Genau eine kleine SM9f-Datei plus ein `BuildAll`-Import. Kein `SM9f0`/`SM9f1`-Auditwald, solange der direkte Bridge-Versuch nicht an einem konkret benannten Beweis-/API-Hindernis scheitert. Keine neuen Planmodule fuer Diskriminanten, Faktoren, Eisenstein, Gauss, Heegner, CM oder j.
Anschlussgate nach SM9f	Erst wenn SM9f eine unbedingte, nicht freie Bridge von den SM9e-Koeffizienten zur SM9d-/SM9c-Determinantenevaluation liefert, darf die naechste kleine Phase ein eigenes Diskriminanten-/Faktor-Gate planen. Dieses Gate ist weiterhin nicht identisch mit SM10/SM11: SM10/SM11 bleiben Vergleichsphasen gegen Eisenstein/Gauss und duerfen erst nach einer derived-only Diskriminantenroute aus SM9f-validierten Koeffizienten starten.
Fortgeschriebene Reihenfolge v90	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family und Coefficient-Missing-Source-Gate; `SM9d` closed/green als Operational-Coefficient-Source-Missing-Recurrence-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence mit deferred determinant bridge; `SM9f--GeneratedOperationalDeterminantEvaluationBridgeFromSM9e` **active**; `SM10`/`SM11`/`SM12` blocked until SM9f bridge and subsequent discriminant/factor gate. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9f darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bm. Fortschreibungsentscheidung v91 nach gruenem SM9f-Build: SM9ePlus0 aktiv

Statusentscheidung	`SM9f - GeneratedOperationalDeterminantEvaluationBridgeFromSM9e` ist nach lokal gruenem Build formal integrierbar. Geschlossen ist aber nicht die unbedingte Determinantenbridge, sondern eine saubere Requirement-/Conditional-Schicht: `GeneratedOperationalDeterminantEvaluationBridgeRequirementSM9f` exponiert das Ziel `sourceSM9e.evaluationAtSource = sourceSM9e.sourceSM9d.evaluationAtSource`, und `GeneratedOperationalDeterminantEvaluationBridgeFromSM9e.fromSM9eWithSourceBridge` baut die starke Bridge nur relativ zu einem Beweis dieser Gleichheit. Damit ist SM9f als Gate gruen, aber nicht als starker mathematischer Abschluss der Koeffizienten-zu-Determinanten-Identifikation.

Formaler SM9f-Ertrag	Geschlossen sind insbesondere `SM9fDeterminantEvaluationBridgeStatus`, `SM9fNextPhaseStatus`, `GeneratedOperationalDeterminantEvaluationBridgeRequirementSM9f`, `fromSM9e`, `bridgeTarget_eq_source_target`, `determinantEvaluationAtSource_eq_SM9cDetPencil_projection`, `GeneratedOperationalDeterminantEvaluationBridgeFromSM9e`, `fromSM9eWithSourceBridge`, regular/variable Requirement-Konstruktoren, regular/variable konditionale Bridge-Konstruktoren, `SM9fGeneratedOperationalDeterminantEvaluationBridgeRequirementLedger`, `SM9fGeneratedOperationalDeterminantEvaluationBridgeLedger`, `sm9f_requirement_regularTargetStatement_eq`, `sm9f_requirement_variableTargetStatement_eq`, `sm9f_regularEvaluationFromCoefficients_eq_SM9cDetPencilAtSource`, `sm9f_variableEvaluationFromCoefficients_eq_SM9cDetPencilAtSource`, `sm9f_noFactorization` und `sm9f_noDiscriminant`.
Praeziser Restbefund	Der fehlende Pfeil bleibt exakt die Gleichheit `R.evaluationAtSource = R.sourceSM9d.evaluationAtSource`. Nach statischer Codepruefung ist dies nicht durch ein weiteres Projektionslemma aus SM9f zu gewinnen. Der natuerliche Ursprung liegt in SM9e: `OperationalBoundedPolynomialSM9e.linear degreeBound c0 c1` speichert den konstanten Koeffizienten an Index 0 und den linearen Koeffizienten an Index 1, waehrend `hornerAux` terminal als `coefficientAt P (n+1) + z * hornerAux P z n` laeuft und `eval P z = hornerAux P z degreeBound` setzt. Fuer `degreeBound = 1` ergibt die aktuelle lineare Normalform daher `c1 + z * c0`, nicht die fuer den Spectral-Pencil benoetigte Form `c0 + z * c1`. Fuer groessere `degreeBound` wird ein linearer Traeger zusaetzlich in die hoechsten Potenzlagen verschoben. Das ist kein SM9f-Taktikproblem, sondern eine fehlende bzw. falsch orientierte Auswertungsnormalform an der SM9e-Schicht.
Naechste aktive Phase	Die naechste aktive positive Minimalphase ist `SM9ePlus0--OperationalBoundedPolynomialEvaluationNormalForm`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/OperationalBoundedPolynomialEvaluationNormalFormSM9ePlus.lean`. Die Datei importiert nur `GeneratedOperationalDeterminantCoefficientRecurrenceSM9e` und liegt natuerlich vor einer spaeteren starken SM9f-Schliessung. Sie soll keine neue Matrix, keine neue Determinantenquelle, keine Faktor-/Diskriminantenquelle und keinen Heegner-/CM-/j-Consumer erzeugen.
SM9ePlus0-DoD	Minimaler DoD ist eine proof-carrying Evaluationsnormalform: `eval_zero`, `eval_one`, `eval_const`, `eval_linear_degreeOne_current`, ein klarer Orientierungsstatus fuer die aktuelle `eval`-Definition, ein Target-Statement fuer die benoetigte Spectral-Pencil-Entry-Auswertung `-(A i j) + z * delta_ij`, und ein Ledger, das festhaelt, ob die aktuelle `eval`-Orientierung direkt tragfaehig ist oder ob eine minimale positive Korrekturquelle in SM9e gebraucht wird. Falls eine neue Vorwaerts-Auswertung noetig ist, muss sie in dieser SM9ePlus-Schicht oder in einer unmittelbar folgenden `SM9ePlus1 - ForwardEvaluationSource` aus `coefficientAt` abgeleitet werden; SM9f darf diese Struktur nicht nachtraeglich patchen.
Beweistaktik SM9ePlus0	Keine Summeninduktion ueber Determinanten und kein globales `simp`. Zuerst kleine Normalformen fuer `hornerAux` bei `0`, `1` und ggf. `2` beweisen; dann `linear` bei `degreeBound = 1` mit expliziten `Fin`-/Nat-Indexgleichungen oeffnen. Ziel ist ein kurzer `calc`-Beweis, der die aktuelle Orientierung sichtbar macht. Vereinfachungen nur am letzten Meter fuer triviale `if`-Ziele wie `0 < 2` oder `1 < 2`. Keine `@[simp]`-Attribute, kein `noncomputable`, kein `classical`, kein `Matrix.det`, kein `Matrix.inv`, kein `field_simp`, kein skalarer `inv`, kein `Finset.toList` und keine `Equiv.Perm.sign`-Runtime-Quelle.
Freigaberegel	Eine starke SM9f-Schliessung wird erst wieder freigegeben, wenn SM9ePlus0 entweder beweist, dass die bestehende `eval`-Orientierung trotz des lokalen Befunds korrekt gegen den Spectral-Pencil normalisiert, oder eine korrekt orientierte, operational abgeleitete Auswertung bereitstellt. Erst danach darf eine Phase `SM9fFull - DeterminantEvaluationBridgeFromSM9ePlus` den Beweis `R.evaluationAtSource = R.sourceSM9d.evaluationAtSource` versuchen. Diskriminanten-/Faktor-Gate, `SM10`, `SM11` und `SM12` bleiben bis dahin gesperrt.
Overhead-Regel v91	Genau eine kleine SM9ePlus0-Datei plus spaeter hoechstens ein minimaler Importumbau. Kein Auditwald und kein neuer Diskriminanten- oder Faktorstub. Wenn die Normalform den Orientierungsfehler bestaetigt, wird nicht in SM9f, SM10 oder SM11 gepatcht; die minimale Korrektur erfolgt an der SM9e-Auswertungsschicht, wo die Struktur natuerlich entsteht.
Fortgeschriebene Reihenfolge v91	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence mit deferred determinant bridge; `SM9f` closed/green als Requirement-/Conditional-Bridge, nicht als unbedingter Determinantenabschluss; `SM9ePlus0--OperationalBoundedPolynomialEvaluationNormalForm` **active**; `SM9fFull`, Diskriminanten-/Faktor-Gate, `SM10`, `SM11` und `SM12` blocked until a correctly oriented, derived-only SM9ePlus evaluation source and the subsequent unconditional SM9f bridge. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9ePlus0 darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bn. Fortschreibungsentscheidung v92 nach gruenem SM9ePlus0-Build: SM9ePlus1 aktiv

Statusentscheidung	`SM9ePlus0--OperationalBoundedPolynomialEvaluationNormalForm` ist nach lokal gruenem Build formal geschlossen. Die Phase hat keine Determinantenbridge repariert und keine Folgearithmetik vorgezogen, sondern die natuerliche Diagnose erbracht: die bestehende `eval`-Definition fuer `OperationalBoundedPolynomialSM9e` wertet den Degree-1-Traeger aktuell in umgekehrter Orientierung aus. Formal sichtbar ist `eval (linear 1 c0 c1) z = c1 + z * c0`; fuer den linearen Pencil also `delta_ij + z * (-(A i j))`. Damit ist der SM9f-Restbefund bestaetigt: der fehlende Pfeil ist eine Auswertungsquelle, nicht ein SM9f-Projektionslemma.
Formaler SM9ePlus0-Ertrag	Geschlossen sind insbesondere `SM9ePlus0EvaluationNormalFormStatus`, `SM9ePlus0EvaluationOrientationStatus`, `SM9ePlus0ForwardEvaluationStatus`, `SM9ePlus0NextPhaseStatus`, `OperationalBoundedPolynomialSM9e.eval_zero`, `OperationalBoundedPolynomialSM9e.eval_one`, `OperationalBoundedPolynomialSM9e.eval_const`, `OperationalBoundedPolynomialSM9e.eval_linear_degreeOne_current`, `operationalLinearPencilEntry_eval_degreeOne_current`, `operationalLinearPencilEntryForwardEvaluationTargetSM9ePlus0`, `OperationalBoundedPolynomialEvaluationNormalFormSM9ePlus0`, `GeneratedOperationalBoundedPolynomialEvaluationNormalFormSM9ePlus0`, `fromSM9e`, regular/variable Konstruktoren, `SM9ePlus0GeneratedOperationalBoundedPolynomialEvaluationNormalFormLedger`, `sm9ePlus0_regularCurrentOrientationStatus`, `sm9ePlus0_variableCurrentOrientationStatus`, `sm9ePlus0_regularNextPhase_eq_SM9ePlus1` und `sm9ePlus0_noDiscriminant`.
Praeziser Restbefund	SM9ePlus0 liefert bewusst keine neue korrekt orientierte Auswertung. Das Target-Statement fuer die benoetigte Spectral-Pencil-Entry-Auswertung bleibt ein Ziel und wird nicht als freier Beweis konsumiert. Der fehlende positive Pfeil ist daher: aus `coefficientAt P 0` und `coefficientAt P 1` eine operationale Vorwaertsauswertung zu erzeugen, die fuer den linearen Pencil theorematisch `-(A i j) + z * delta_ij` ergibt. Dieser Pfeil muss an der SM9e-/SM9ePlus-Schicht entstehen, nicht als nachtraeglicher Patch in SM9f, SM10 oder SM11.
Naechste aktive Phase	Die naechste aktive positive Minimalphase ist `SM9ePlus1--ForwardEvaluationSourceFromCoefficientAt`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/OperationalBoundedPolynomialForwardEvaluationSourceSM9ePlus.lean`. Importdisziplin: direkt nach `OperationalBoundedPolynomialEvaluationNormalFormSM9ePlus` und vor `GeneratedOperationalDeterminantEvaluationBridgeFromSM9e`. Inhaltlich konsumiert SM9ePlus1 nur SM9e und SM9ePlus0; sie erzeugt keine neue Matrix, keine neue Koeffizientenrekurrenz, keine neue Determinantenquelle, keine Faktorisierung, keine Diskriminante und keinen Heegner-/CM-/j-Consumer.
SM9ePlus1-DoD	Minimaler DoD ist eine proof-carrying Vorwaertsauswertung aus `coefficientAt`. Zulaessige Kernobjekte: ein Status wie `SM9ePlus1ForwardEvaluationStatus`; eine Definition wie `OperationalBoundedPolynomialSM9e.forwardEvalDegreeOne` oder, wenn ohne Overhead tragbar, `OperationalBoundedPolynomialSM9e.forwardEvalAux/forwardEval`; Theoreme `forwardEval_zero`, `forwardEval_one`, `forwardEval_const`, `forwardEval_linear_degreeOne`, `operationalLinearPencilEntry_forwardEval_degreeOne`; eine Struktur `GeneratedOperationalBoundedPolynomialForwardEvaluationSourceSM9ePlus1` mit `sourceSM9ePlus0`; regular/variable Konstruktoren; Ledger-Felder `forwardEvaluationSource_eq_coefficientAt`, `linearPencilForwardTarget_closed`, `sm9fFullReleasedOnlyAfterForwardSource`, `noNewCoefficientRecurrence`, `noNewMatrix`, `noNewDeterminantSource`, `noFactorization`, `noDiscriminant`, `noHeegnerTarget`, `noCMTarget`, `noMatrixInv`, `noFieldSimp`, `noScalarInv`, `noNoncomputableOperationalSource`.
Beweistaktik SM9ePlus1	Kein Determinantenbeweis, keine Summeninduktion und kein globales `simp`. Fuer den Minimalpfad reicht Degree 1: `forwardEvalDegreeOne P z := coefficientAt P 0 + Schur.spectralParameterExec z * coefficientAt P 1`. Danach werden `coefficientAt (linear 1 c0 c1) 0 = c0` und `coefficientAt (linear 1 c0 c1) 1 = c1` mit kleinen lokalen `change`/`rfl`/`by decide`-Schritten geschlossen. Der lineare Pencil folgt dann per kurzer `calc`-Kette aus der Definition `operationalLinearPencilEntryPolynomialSM9e`. Falls Lean bei `if i = j` eine Decidable-Instanz verlangt, diese explizit im Binder halten; keine Fallanalyse ueber `i = j` ausser fuer terminale Bool-/`if`-Ziele. Algebraische Umordnung vermeiden: die Zielrichtung soll definitional `c0 + z * c1` sein, nicht ueber Kommutativitaet gerettet werden.
Freigaberegel	Erst nach gruenem SM9ePlus1-Build wird eine kleine Phase `SM9fFull--DeterminantEvaluationBridgeFromSM9ePlus` freigegeben. SM9fFull darf dann die bisher konditionale Gleichung `sourceSM9e.evaluationAtSource = sourceSM9e.sourceSM9d.evaluationAtSource` nur ueber die neue korrekt orientierte SM9ePlus-Auswertungsquelle schliessen. Diskriminanten-/Faktor-Gate, `SM10`, `SM11` und `SM12` bleiben bis nach diesem unbedingten Bridge-Abschluss blockiert.
Overhead-Regel v92	Genau eine kleine SM9ePlus1-Datei plus ein `BuildAll`-Import. Kein `SM9ePlus1a`/`SM9ePlus1b`-Auditwald, solange der Degree-1-Forward-Evaluator nicht an einem konkret benannten API-Hindernis scheitert. Keine neue generische Polynomtheorie, kein mathlib-`Polynomial`, keine Det-/Charpoly-/Factor-Stubmodule. Wenn der generische `forwardEval` zu breit wird, ist der kleinste positive Abschluss `forwardEvalDegreeOne` fuer den linearen Pencil vorzuziehen; die Determinantenebene bleibt SM9fFull vorbehalten.

Fortgeschriebene Reihenfolge v92	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence mit deferred determinant bridge; `SM9f` closed/green als Requirement-/Conditional-Bridge; `SM9ePlus0` closed/green als Evaluationsnormalform-/Orientierungsdiagnose; **`SM9ePlus1--ForwardEvaluationSourceFromCoefficientAt` active**; `SM9fFull`, Diskriminanten-/Faktor-Gate, `SM10`, `SM11` und `SM12` blocked until a correctly oriented, derived-only SM9ePlus forward-evaluation source and the subsequent unconditional SM9f bridge. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9ePlus1 darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bo. Fortschreibungsentscheidung v93 nach gruenem SM9ePlus1-Build: SM9fFull aktiv

Statusentscheidung	`SM9ePlus1--ForwardEvaluationSourceFromCoefficientAt` ist nach lokal gruenem Build formal geschlossen. Die Phase hat die in v92 geforderte minimale Forward-Source geliefert: `OperationalBoundedPolynomialSM9e.forwardEvalDegreeOne` ist aus `coefficientAt P 0` und `coefficientAt P 1` aufgebaut, die Degree-1-Basisfaelle sind geschlossen, und `operationalLinearPencilEntry_forwardEval_degreeOne` traegt das Ziel `-(A i j) + z * delta_ij`. Damit ist die Orientierungsdiagnose aus SM9ePlus0 nicht durch einen SM9f-Patch verdeckt, sondern an der natuerlichen SM9ePlus-Schicht positiv beantwortet.
Formaler SM9ePlus1-Ertrag	Geschlossen sind insbesondere `SM9ePlus1ForwardEvaluationStatus`, `SM9ePlus1SM9fFullReleaseStatus`, `SM9ePlus1NextPhaseStatus`, `OperationalBoundedPolynomialSM9e.forwardEvalDegreeOne`, `forwardEvalDegreeOne_eq_coefficientAt`, `forwardEval_zero`, `forwardEval_one`, `forwardEval_const`, `forwardEval_linear_degreeOne`, `operationalLinearPencilEntryForwardEvaluationTargetSM9ePlus1`, `operationalLinearPencilEntry_forwardEval_degreeOne`, `OperationalBoundedPolynomialForwardEvaluationSourceSM9ePlus1`, `GeneratedOperationalBoundedPolynomialForwardEvaluationSourceSM9ePlus1`, `fromSM9ePlus0`, regular/variable Konstruktoren, `forwardEvaluationTarget_closed_projection`, `nextPhase_eq_SM9fFull`, `SM9ePlus1GeneratedOperationalBoundedPolynomialForwardEvaluationSourceLedger`, `sm9ePlus1GeneratedOperationalBoundedPolynomialForwardEvaluationSourceLedger_from_SM9eLedger`, regular/variable Target-Projektionen, `sm9ePlus1_regularNextPhase_eq_SM9fFull` und `sm9ePlus1_noDiscriminant`.
Praeziser Restbefund	SM9ePlus1 beweist bewusst noch keine Determinantenbridge. Insbesondere ist die alte Feldgleichung `sourceSM9e.evaluationAtSource = sourceSM9e.sourceSM9d.evaluationAtSource` nicht automatisch durch SM9ePlus1 geschlossen, weil `sourceSM9e.evaluationAtSource` weiterhin aus der historischen `OperationalBoundedPolynomialSM9e.eval`-Orientierung stammen kann. Der naechste positive Schritt darf deshalb nicht blind den alten konditionalen SM9f-Konstruktor mit einem freien `hBridge` fuettern, sondern muss die neue Forward-Source explizit als Brueckenquelle konsumieren.
Naechste aktive Phase	Die naechste aktive positive Phase ist **`SM9fFull--DeterminantEvaluationBridgeFromSM9ePlus`**. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalDeterminantForwardEvaluationBridgeFromSM9ePlus.lean`. Importdisziplin: Import von `GeneratedOperationalDeterminantEvaluationBridgeFromSM9e` und `OperationalBoundedPolynomialForwardEvaluationSourceSM9ePlus`; Einbindung in `BuildAll` direkt nach der bisherigen SM9f-/SM9ePlus1-Schicht und weiterhin vor jedem Faktor-/Diskriminanten-/Heegner-/CM-/j-Consumer.
SM9fFull-DoD	Minimaler starker DoD ist ein unbedingter regular/variable Bridge-Abschluss, der die SM9d-/SM9c-Determinantenevaluation aus der SM9e-Koeffizientenrekurrenz ueber die neue SM9ePlus-Forward-Auswertung erreicht. Zulaessige Kernobjekte sind ein Status wie `SM9fFullDeterminantForwardBridgeStatus`, eine Struktur wie `GeneratedOperationalDeterminantForwardEvaluationBridgeFromSM9ePlus`, Felder fuer `sourceSM9fRequirement`, `sourceSM9ePlus1`, `forwardEvaluationAtSource`, `forwardEvaluationAtSource_eq_SM9dEvaluationAtSource`, `forwardEvaluationAtSource_eq_SM9cEvaluationFamilyAtSource` und `forwardEvaluationAtSource_eq_SM9cDetPencilAtSource`, regular/variable Konstruktoren und ein `SM9fFullGeneratedOperationalDeterminantForwardEvaluationBridgeLedger`. Der bevorzugte Abschluss ist eine `forall z`-Bridge gegen die SM9c-EvaluationFamily; der minimale Abschluss ist die Gleichheit am SM9d-Source-Parameter.
Nichtziel SM9fFull	SM9fFull erzeugt keine neue Matrix, keine neue Koeffizientenrekurrenz, keine neue lineare Pencil-Formel, keine Faktorisierung, keine Diskriminante, keinen Eisenstein-/Gauss-/Heegner-/CM-/j-Vergleich, keine DtN-/MultiSchur-/Interior-Daten und keinen `TwoSidedInv`. Es darf auch nicht die alte `evaluationAtSource`-Orientierung durch Kommutativitaet oder fragile Rewriteketten retten. Wenn eine alte Gleichung weiter benoetigt wird, muss sie als abgeleitete Projektion aus der Forward-Bridge erscheinen, nicht als freies Argument.

Beweistaktik SM9fFull	Positiver Erstversuch mit minimalem Overhead: zuerst reine Projektions- und Definitionsebenen zusammenfuehren, also `coefficients_source_eq_linearPencil`, `degreeBound_eq_SM9d`, `matrix_eq_SM9c`, `sourceParameter_eq_SM9c` und die SM9d-Felder nur per `rfl`, Feldprojektionen und kurzen `calc`-Ketten verwenden. Danach die Forward-Auswertung explizit an die SM9ePlus1-Quelle binden. Fuer die determinantielle Substanz wird nicht global simplifiziert: entweder ein lokal in Lean 4.28.0 vorhandenes `Matrix.det`-Laplace-/Leibniz-Basislemma nur im Prop-Beweis konsumieren, oder eine eigene kleine Normalform bauen, die `operationalDeterminantColumnExpansionAuxSM9e`/Listenrekurrenz und `Matrix.det` auf denselben endlichen Traeger bringt. Keine Summeninduktion im ersten Schritt, bevor die Entry- und Listenform exakt exponiert ist.
Vorausschauendes Beweisrisiko	Der gruen geschlossene SM9ePlus1-Minimalpfad ist Degree-1-entrywise. Fuer die volle Determinantenrekurrenz koennte Lean zusaetzlich eine forward-orientierte Homomorphieschicht fuer `OperationalBoundedPolynomialSM9e.add`, `neg`, `scale`, `boundedMul` und ggf. `listProduct` verlangen. Das ist kein Grund fuer ein Diskriminanten- oder Faktorstub. Falls der direkte SM9fFull-Versuch an genau dieser Stelle scheitert, ist die minimale positive Rueckkopplung `SM9ePlus2--OperationalBoundedPolynomialForwardEvaluationAlgebraLemmas`; sie liegt an der natuerlichen Evaluationsebene und nicht im SM9f-Consumer.
Ruecksprungregel	Falls das Hindernis nicht die Forward-Homomorphie, sondern die Determinantenexpansion selbst ist, wird nicht bei SM10/SM11 oder im Diskriminanten-Gate gepatcht. Dann ist die minimale positive Nachruestung eine proof-carrying `SM9eLaplaceExpansionNormalForm`, welche die SM9e-Listen-/Column-Expansion und die verwendete `Matrix.det`-Formel auf demselben Indextraeger synchronisiert. Erst nach einer solchen explizit benannten Huerde wird die SM9fFull-Phase gespalten; ohne konkretes Hindernis bleibt SM9fFull eine einzige kleine Bridge-Phase.
Freigaberegel	Nach einem gruenen SM9fFull-Build ist das Diskriminanten-/Faktor-Gate die naechste kleine Planphase. `SM10`, `SM11` und `SM12` bleiben auch dann noch nicht automatisch aktiv: zuerst muss ein eigenes derived-only Gate festlegen, welche validierten Koeffizienten, welche Faktorform und welche Diskriminantenquelle konsumiert werden duerfen.
Overhead-Regel v93	Kein `SM9fFull0`/`SM9fFull1`-Auditwald vor einem realen Lean-Hindernis. Eine neue Datei plus ein `BuildAll`-Import ist der Zielumfang. Keine mathlib-`Polynomial`-Runtime, kein `noncomputable`, kein `classical`, kein `Matrix.inv`, kein `field_simp`, kein skalares Inversen-Oracle, kein `Finset.toList`- oder `Equiv.Perm.sign`-Runtime-Umweg. Mathlib-Determinantenlemmata sind nur als Prop-Beweisbausteine zulaessig, nicht als neue operative Datenquelle.
Fortgeschriebene Reihenfolge v93	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence mit deferred determinant bridge; `SM9f` closed/green als Requirement-/Conditional-Bridge; `SM9ePlus0` closed/green als Evaluationsnormalform-/Orientierungsdiagnose; `SM9ePlus1` closed/green als korrekt orientierte Degree-1-Forward-Source; `SM9fFull--DeterminantEvaluationBridgeFromSM9ePlus` **active**; Diskriminanten-/Faktor-Gate, `SM10`, `SM11` und `SM12` blocked until a proven unconditional SM9fFull bridge. Alle frueheren r3c1-/SM3db-/SM3eR-/SM3fR-Produktphasen behalten ihren closed/green-Status; SM9fFull darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bp. Fortschreibungsentscheidung v94 nach gruenem SM9fFull-Build: SM9g aktiv

Statusentscheidung	`SM9fFull--DeterminantEvaluationBridgeFromSM9ePlus` ist nach lokal gruenem Build formal geschlossen. Der neue Bridge-Ledger konsumiert die SM9ePlus1-Forward-Source, fuehrt regular und variable Bruecken, projiziert die SM9d-/SM9c-Determinantenevaluation am Source-Parameter und setzt den naechsten Status auf `discriminantFactorGateAfterForwardBridge`. Damit ist die v93-Sperre gegen die alte `OperationalBoundedPolynomialSM9e.eval`-Orientierung im positiven Pfad abgearbeitet.
Formaler SM9fFull-Ertrag	Geschlossen sind insbesondere `SM9fFullDeterminantForwardBridgeStatus`, `SM9fFullNextPhaseStatus`, `SM9fFullOldEvalOrientationPatchStatus`, `GeneratedOperationalDeterminantForwardEvaluationBridgeFromSM9ePlus`, `fromForwardSource`, regular/variable Konstruktoren, `SM9fFullGeneratedOperationalDeterminantForwardEvaluationBridgeLedger`, `sm9fFullGeneratedOperationalDeterminantForwardEvaluationBridgeLedger_from_SM9eLedger`, Projektionen `forwardEvaluationAtSource_eq_SM9dEvaluationAtSource`, `forwardEvaluationAtSource_eq_SM9cEvaluationFamilyAtSource`, `forwardEvaluationAtSource_eq_SM9cDetPencilAtSource`, `linearPencilForwardTarget_closed_projection`, `bridgeClosedFromSM9ePlusSource`, `sm9fFull_nextPhase_eq_discriminantFactorGate` und `sm9fFull_noDiscriminant`.

Praeziser Restbefund	SM9fFull ist bewusst keine Faktor- oder Diskriminantenphase. Die Phase bestaetigt die legale Determinantenquelle, aber sie erzeugt weder eine quadratische Faktorform noch einen Diskriminantenwert, keine Konventionsbruecke zur Fundamentaldiskriminante und keinen Vergleich gegen -3, -4 oder Heegner-Daten. Ein direkter Sprung nach SM10 oder SM11 waere daher ein Consumer-Patch: die Vergleichsphasen wuerden dann noch nicht wissen, welche Koeffizienten, welche Faktorform und welche Diskriminantenquelle formal freigegeben sind.
Naechste aktive Phase	Die naechste aktive positive Phase ist `SM9g--DiscriminantFactorGateFromSM9fFull`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalDiscriminantFactorGateFromSM9fFull.lean`. Importdisziplin: genau die SM9fFull-Bridge und die bereits vorhandenen SM9c/SM9d/SM9e-Quellen konsumieren; keine Imports aus Eisenstein-/Gauss-/Heegner-/CM-/j-Consumerdateien. Einbindung in `BuildAll` direkt nach SM9fFull und weiterhin vor SM10/SM11/SM12.
SM9g-DoD	Minimaler DoD ist ein kleines proof-carrying Gate, das die erlaubten Quellen benennt, aber noch nicht vergleicht: `sourceSM9fFullLedger`; regular/variable Determinantenquelle aus `forwardEvaluationAtSource_eq_SM9cDetPencilAtSource`; Koeffizientenquelle aus der SM9d-/SM9e-Provenienz; Status `factorSourceMustBeDerivedFromValidatedCoefficients`; Status `discriminantSourceMustBeDerivedFromValidatedFactorOrQuadraticCoefficientWindow`; Sperren gegen freie Faktorform, freie Diskriminante, freie Zielwerte und externe Vergleichsmatrizen; naechster Status `SM10L2ComparisonBlockedUntilGateSourceIsConsumable` bzw. `SM10ReadyAfterGate` nur falls die Quellen wirklich konsumierbar exponiert sind.
Nichtziel SM9g	SM9g erzeugt keine neue Matrix, keine neue Determinantenrekurrenz, keine neue Charpoly-Rechnung, keinen Faktorbeweis, keinen Diskriminantenwert, keinen Eisenstein-/Gauss-/Heegner-/CM-/j-Vergleich, keine Konventionsumrechnung zu Fundamentaldiskriminanten und keinen `TwoSidedInv`. Insbesondere duerfen `L2GeneratedEisenstein` und `L3GeneratedGaussian` weiterhin nur Ziel-/Vergleichsconsumer bleiben und nicht als Quelle fuer Faktor- oder Diskriminantendaten importiert werden.
Beweistaktik SM9g	Keine Determinanten- oder Summeninduktion. Zuerst einen reinen Struktur-Ledger bauen: `let R := L.regularBridge`, `let V := L.variableBridge`; dann die vorhandenen SM9fFull-Projektionen per Feldzugriff oder kurze `calc`-Ketten exponieren. Fuer Koeffizienten nicht in mathlib-`Polynomial` wechseln, sondern die vorhandene operative SM9d-/SM9e-Koeffizientenquelle weiterreichen. Falls Lean fuer eine konkrete quadratische Koeffizientenfenster-API keine Felder anbietet, endet SM9g mit einem eng benannten Missing-Source-Status statt mit einem freien Faktorstub.
Vorausschauendes Beweisrisiko	Wahrscheinliches Hindernis ist nicht mehr die Determinantenbridge, sondern die Frage, ob die vorhandene operative Koeffizientenquelle die fuer eine quadratische Faktor-/Diskriminantenroute benoetigten Koeffizienten als konsumierbares Fenster exponiert. Falls ja, kann nach SM9g die L=2-Eisenstein-Vergleichsphase SM10 starten. Falls nein, ist die minimale positive Nachruestung nicht SM10, sondern eine kleine Quellenphase `SM9g1--OperationalQuadraticCoefficientWindowFromSM9dSM9e`; diese laege an der natuerlichen Koeffizientenquelle, nicht im Vergleichsconsumer.
Freigaberegel	Nach gruenem SM9g-Build wird SM10 nur freigegeben, wenn das Gate eine konkrete, aus SM9fFull/SM9d/SM9e kommende Faktor-/Diskriminantenquelle als konsumierbar markiert. Bleibt nur ein Missing-Source-Status uebrig, folgt zuerst SM9g1. SM11 bleibt bis nach einem tragfaehigen SM10- oder gemeinsam nutzbaren Gate-Ausgang blockiert; SM12 bleibt bis nach SM10/SM11-Status blockiert.
Overhead-Regel v94	Genau eine kleine SM9g-Datei plus ein `BuildAll`-Import. Kein `SM9g0/SM9gAudit/SM9gFull`-Wald vor einem realen Lean-Hindernis. Keine neue generische Polynomtheorie, kein mathlib-`Polynomial`-Runtime-Pfad, kein `noncomputable`, kein `classical`, kein `Matrix.inv`, kein `field_simp`, kein skalares Inversen-Oracle, keine freien Faktor-/Diskriminantentabellen und keine Zielwertannahmen.
Fortgeschriebene Reihenfolge v94	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence; `SM9f` closed/green als Requirement-/Conditional-Bridge; `SM9ePlus0` closed/green als Evaluationsnormalform-/Orientierungsdiagnose; `SM9ePlus1` closed/green als korrekt orientierte Degree-1-Forward-Source; `SM9fFull` closed/green als Forward-Determinantenbridge; `SM9g--DiscriminantFactorGateFromSM9fFull` **active**; `SM10`, `SM11` und `SM12` blocked until the SM9g gate exposes a legal factor/discriminant source. Alle frueheren Produkt-, DtN-, MultiSchur- und ExecMirror-Phasen behalten ihren closed/green-Status; SM9g darf keine Produktidentitaet und keinen `TwoSidedInv` veraendern.

0bq. Fortschreibungsentscheidung v95 nach gruenem SM9g-Build: SM9g1 aktiv

Statusentscheidung	`SM9g--DiscriminantFactorGateFromSM9fFull` ist nach lokal gruenem Build formal geschlossen. Der gruen gemeldete Code bestaetigt die im v94-Plan vorgesehene Gate-Rolle: SM9g benennt die legalen Determinanten- und Koeffizientenquellen, erzeugt aber bewusst noch keine Faktorform, keinen Diskriminantenwert und keinen Eisenstein-/Gauss-/Heegner-/CM-/j-Vergleich. Da der Gate-Ausgang den Status `missingConsumableWindowFromValidatedCoefficients` und als naechste Phase `operationalQuadraticCoefficientWindowFromSM9dSM9e` traegt, ist der positive Pfad nicht SM10, sondern zuerst SM9g1.
Formaler SM9g-Ertrag	Geschlossen sind insbesondere `SM9gDeterminantSourceStatus`, `SM9gCoefficientSourceStatus`, `SM9gQuadraticCoefficientWindowStatus`, `SM9gFactorSourcePolicyStatus`, `SM9gDiscriminantSourcePolicyStatus`, `SM9gSM10ReleaseStatus`, `SM9gNextPhaseStatus`, `SM9gGeneratedOperationalDiscriminantFactorGateLedger`, `sm9gGeneratedOperationalDiscriminantFactorGateLedger`, `sm9gGeneratedOperationalDiscriminantFactorGateLedger_from_SM9eLedger`, `sm9g_regularDeterminantSource_eq_SM9cDetPencil`, `sm9g_variableDeterminantSource_eq_SM9cDetPencil`, `sm9g_regularCoefficientSource_eq_operationalRecurrence`, `sm9g_variableCoefficientSource_eq_operationalRecurrence`, `sm9g_factorSourceMustBeDerivedFromValidatedCoefficients`, `sm9g_discriminantSourceMustBeDerivedFromValidatedFactorOrQuadraticCoefficientWindow`, `sm9g_sm10BlockedUntilGateSourceIsConsumable`, `sm9g_nextPhase_eq_operationalQuadraticCoefficientWindow` und `sm9g_noFreeFactorOrDiscriminant`.
Praeziser Restbefund	SM9g beweist nicht, dass ein konkretes quadratisches Fenster bereits konsumierbar ist. Es beweist vielmehr, dass Faktor- und Diskriminantenquellen aus validierten Koeffizienten bzw. aus einem validierten Faktor-/Quadratikfenster entstehen muessen. Damit ist der direkte SM10-Start weiterhin ein Consumer-Patch: SM10 wuerde sonst Zielwerte mit noch nicht freigegebenen Koeffizientenpositionen vergleichen.
Naechste aktive Phase	Die naechste aktive positive Phase ist `SM9g1--OperationalQuadraticCoefficientWindowFromSM9dSM9e`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalQuadraticCoefficientWindowFromSM9dSM9e.lean`. Importdisziplin: genau SM9g als Gate-Quelle konsumieren; SM9d/SM9e nur ueber die in SM9g exponierten Koeffizientenquellen bzw. vorhandene lokale Projektionen anfassen. Einbindung in `BuildAll` direkt nach SM9g und weiterhin vor SM10/SM11/SM12.
SM9g1-DoD	Minimaler DoD ist ein kleines proof-carrying Koeffizientenfenster, noch keine Diskriminantenphase: ein Strukturtyp wie `OperationalQuadraticCoefficientWindowSM9g1` mit `sourceSM9gGate`, regular/variable `constantCoeff`, `linearCoeff`, `quadraticCoeff`; Gleichungen dieser drei Eintraege gegen `OperationalBoundedPolynomialSM9e.coefficientAt` der in SM9g validierten `regularCoefficientSource`/`variableCoefficientSource`; Rueckbindung an `operationalDeterminantCoefficientRecurrenceSM9e`; Status `quadraticWindowValidatedFromSM9dSM9e`; Sperren gegen freie Faktorform, freie Diskriminante, freie Zielwerte, externe Vergleichsmatrizen und mathlib-`Polynomial`-Runtime.
Nichtziel SM9g1	SM9g1 erzeugt keinen Faktorbeweis, keinen Diskriminantenwert, keine Fundamentaldiskriminanten-Konventionsbruecke, keinen Vergleich gegen -3, -4 oder Heegner-Daten, keine neue Determinantenrekurrenz, keine neue Matrix, keine Charpoly-Neuberechnung, keinen `TwoSidedInv` und keinen Import aus Eisenstein-/Gauss-/CM-/j-Consumerdateien. Falls ein Koeffizient wegen `degreeBound` ausserhalb des Bounds liegt, darf nur die bereits definierte `coefficientAt`-Nullkonvention exponiert werden; es darf kein Grad- oder Quadratikinhalt behauptet werden, der nicht aus dem bestehenden Bound folgt.
Beweistaktik SM9g1	Kein globales `simp`, keine Summeninduktion, keine Polynomsemiring-Instanz. Zuerst `let G := ...` und `let Pr := G.regularCoefficientSource`, `let Pv := G.variableCoefficientSource`. Dann die Fensterkoeffizienten definitorisch als `OperationalBoundedPolynomialSM9e.coefficientAt Pr 0/1/2` bzw. analog variabel setzen. Die Herkunftsgleichungen sollten per `rfl`, Feldprojektion und kurzen `calc`-Ketten gegen `sm9g_regularCoefficientSource_eq_operationalRecurrence`/`sm9g_variableCoefficientSource_eq_operationalRecurrence` geschlossen werden. Keine Fallunterscheidung ueber alle Fin-Indizes; wenn ein Bound-Witness noetig wird, nur die drei konkreten Nat-Positionen 0, 1, 2 lokal behandeln.
Vorausschauendes Beweisrisiko	Das wahrscheinlichste Hindernis ist nicht die Eintragsdefinition, sondern ein Typgleichheitsproblem zwischen dem degreeBound der SM9g-Quelle und dem degreeBound der SM9e-Rekurrenz. Die robuste Route ist daher, den Fenster-Typ ueber dem exakt in SM9g gespeicherten `regularCoefficientSource`/`variableCoefficientSource` zu parametrisieren, nicht ueber einer neu aus SM9d rekonstruierten Degree-Gleichung. Falls Lean trotzdem eine bessere Exposition verlangt, wird minimal in SM9g1 eine lokale Projection-Lemma-Schicht ergaenzt; nicht in SM10 und nicht durch freie Koeffiziententabellen.
Freigaberegel	Nach gruenem SM9g1-Build wird SM10 nur dann freigegeben, wenn das Fenster als konsumierbare Quelle fuer die quadratische Faktor-/Diskriminantenroute markiert ist. Falls SM9g1 lediglich die drei Koeffizientenpositionen exponiert, aber noch keinen daraus abgeleiteten Diskriminantenquellstatus liefert, folgt als naechste kleine Phase `SM9h--OperationalQuadraticDiscriminantSourceFromWindow`. Falls SM9g1 wider Erwarten zeigt, dass selbst die drei Positionen nicht stabil typisierbar sind, wird nicht im Vergleichsconsumer gepatcht, sondern die minimale Rueckkopplung an SM9e/SM9g-Projektionen formuliert.

Overhead-Regel v95	Genau eine kleine SM9g1-Datei plus ein `BuildAll`-Import. Kein SM9g1a/SM9g1Audit/SM9g1Full-Wald vor einem realen Lean-Hindernis. Keine mathlib-`Polynomial`-Runtime, kein `noncomputable`, kein `classical`, kein `Matrix.inv`, kein `field_simp`, kein skalares Inversen-Oracle, keine freien Koeffizienten-/Faktor-/Diskriminantentabellen und keine Zielwertannahmen.
Fortgeschriebene Reihenfolge v95	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence; `SM9f` closed/green als Requirement-/Conditional-Bridge; `SM9ePlus0` closed/green als Evaluationsnormalform-/Orientierungsdiagnose; `SM9ePlus1` closed/green als korrekt orientierte Degree-1-Forward-Source; `SM9fFull` closed/green als Forward-Determinantenbridge; `SM9g` closed/green als Discriminant-/Factor-Source-Gate mit Missing-Window-Status; `SM9g1--OperationalQuadraticCoefficientWindowFromSM9dSM9e` **active**; `SM10`, `SM11` und `SM12` blocked until a consumable quadratic coefficient/factor/discriminant source is proven.

0br. Fortschreibungsentscheidung v96 nach gruenem SM9g1-Build: SM9h aktiv

Statusentscheidung	`SM9g1--OperationalQuadraticCoefficientWindowFromSM9dSM9e` ist nach lokal gruenem Build formal geschlossen. Der gruene Build bestaetigt die in v95 geforderte kleine Fensterschicht: Aus dem SM9g-Gate werden regular und variable Koeffizientenquellen konsumiert; daraus werden jeweils die Positionen `0`, `1` und `2` als `constantCoeff`, `linearCoeff` und `quadraticCoeff` exponiert. Da der SM9g1-Ledger als naechste Phase `operationalQuadraticDiscriminantSourceFromWindow` traegt und `SM10` bis zu einer solchen Quelle blockiert, ist der positive Pfad nicht SM10, sondern zuerst SM9h.
Formaler SM9g1-Ertrag	Geschlossen sind insbesondere `OperationalQuadraticCoefficientWindowSM9g1`, `fromCoefficientSource`, `SM9g1GeneratedOperationalQuadraticCoefficientWindowLedger`, `sm9g1GeneratedOperationalQuadraticCoefficientWindowLedger`, `sm9g1GeneratedOperationalQuadraticCoefficientWindowLedger_from_SM9eLedger`, `regularWindow`/`variableWindow`, die Fenster-Gleichungen gegen `OperationalBoundedPolynomialSM9e.coefficientAt` an den Nat-Positionen `0`, `1`, `2`, die Rueckbindungen gegen `operationalDeterminantCoefficientRecurrenceSM9e`, `sm9g1_nextPhase_eq_operationalQuadraticDiscriminantSourceFromWindow`, `sm9g1_sm10BlockedUntilOperationalDiscriminantSourceFromWindow`, `sm9g1_noFreeFactorDiscriminantOrTarget` und `sm9g1_noPolynomialRuntimeOrForbiddenAlgebra`.
Praeziser Restbefund	SM9g1 beweist kein Diskriminantenobjekt. Das ist kein Fehler, sondern die geplante Grenze: Das Fenster macht die drei Koeffizienten konsumierbar, aber es erzeugt noch nicht die aus diesen Koeffizienten abgeleitete Diskriminantenquelle. Ein direkter SM10-Start waere daher weiterhin ein Vergleichs-Consumer ohne eigene Quelle fuer $b^2 - 4ac$ bzw. ohne freigegebenen Diskriminantenstatus.
Naechste aktive Phase	Die naechste aktive positive Phase ist `SM9h--OperationalQuadraticDiscriminantSourceFromWindow`. Vorgeschlagene Implementierungsdatei: `CNNA/PillarA/Arithmetic/Smoke/GeneratedOperationalQuadraticDiscriminantSourceFromSM9g1Window.lean`. Importdisziplin: genau SM9g1 konsumieren; keine Rueckimporte aus SM10/SM11/SM12, keine Eisenstein-/Gauss-/CM-/j-Consumer, keine neue Matrix, keine neue Rekurrenz. Einbindung in `BuildAll` direkt nach SM9g1 und weiterhin vor SM10/SM11/SM12.
SM9h-DoD	Minimaler DoD ist eine kleine proof-carrying Diskriminantenquellen-Schicht: ein Strukturtyp wie `OperationalQuadraticDiscriminantSourceSM9h` mit `sourceWindow`, `discriminantValue` und einer definitorischen Gleichung zur Fensterformel `linearCoeff * linearCoeff - (ExecComplex.ofRat 4) * quadraticCoeff * constantCoeff`. Der Ledger muss regular und variable Diskriminantenquellen aus `SM9g1GeneratedOperationalQuadraticCoefficientWindowLedger` bauen, beide Formeln auf die SM9g1-Fensterkoeffizienten zurueckbinden, den Status `discriminantSourceValidatedFromSM9g1Window` tragen und die Sperren gegen freie Faktorform, freie Zielwerte, externe Vergleichsmatrizen, mathlib-`Polynomial`-Runtime, `Matrix.inv`, `field_simp`, skalare Inversen, `noncomputable` und `classical` fortschreiben.
Nichtziel SM9h	SM9h erzeugt keinen Faktorbeweis, keine Fundamentaldiskriminanten-Konventionsbruecke, keinen Vergleich gegen -3, -4 oder Heegner-Daten, keine CM-/Perioden-/j-Daten, keine neue Charpoly-Rechnung, keine neue Determinantenrekurrenz, keine neue Matrix, keinen `TwoSidedInv` und keine Aussage, dass die quadratische Koeffizientenlage tatsaechlich ein irreduzibles oder normiertes quadratisches Polynom beschreibt. SM9h darf nur die operative Diskriminantenquelle aus dem vorhandenen Fenster bauen.

Beweistaktik SM9h	Kein globales `simp`, keine Summeninduktion, kein `ring`, kein `field_simp`. Zuerst eine lokale definitorische Funktion `operationalQuadraticDiscriminantFromWindowSM9h W := W.linearCoeff * W.linearCoeff - (ExecComplex.ofRat 4) * W.quadraticCoeff * W.constantCoeff`. Dann regular/variable Quellen durch Anwenden dieser Funktion auf `L.regularWindow`/`L.variableWindow`. Die primaeren Gleichungen sollten per `rfl` schliessen; die Rueckbindung auf `coefficientAt`-Formeln erfolgt mit gezielten `rw`-Schritten ueber die SM9g1-Fensterfelder und kurzen `calc`-Ketten. Keine Algebra-Normalisierung erzwingen, solange beide Seiten syntaktisch in derselben Koeffizientenreihenfolge formuliert werden.
Vorausschauendes Beweisrisiko	Das groesste Risiko ist eine versehentliche algebraische Umordnung der Diskriminantenformel. Wegen fehlender Notwendigkeit fuer Kommutativitaets-/Ringnormalisierung sollte SM9h die Formel in genau einer syntaktischen Reihenfolge fixieren und alle Exporte auf diese Reihenfolge zurueckbinden. Falls spaeter eine andere Konvention benoetigt wird, gehoert die Konventionsbruecke nicht in SM9h, sondern in eine nachgelagerte, explizit benannte Normalisierungs-/Fundamentaldiskriminantenphase.
Freigaberegel	Nach gruenem SM9h-Build wird SM10 nur dann freigegeben, wenn der Ledger eine konsumierbare regular/variable Diskriminantenquelle aus dem SM9g1-Fenster traegt und weiterhin keine freien Zielwerte oder externen Vergleichsdaten einfuehrt. Falls SM9h nur eine rohe Formel, aber noch keinen freigegebenen Source-Status liefert, folgt als naechste kleine Phase eine knappe `SM9h1`-Source-Status-Bruecke; kein SM10-Patch und kein Faktor-/Zielwert-Consumer.
Overhead-Regel v96	Genau eine kleine SM9h-Datei plus ein `BuildAll`-Import. Kein SM9hAudit/SM9hFull/SM9i-Wald vor einem realen Lean-Hindernis. Keine freie Diskriminantentabelle, keine Zielwertannahme, keine Factorization-API, keine mathlib-`Polynomial`-Runtime, kein `noncomputable`, kein `classical`, kein `Matrix.inv`, kein `field_simp`, kein skalares Inversen-Oracle und keine CM-/Heegner-/j-Interpretation.
Fortgeschriebene Reihenfolge v96	`SM3fq` closed/green-missing-`SM3bq`; `SM3bq` closed/green als Dirichlet-/WeightPolicy-Exec-Provider; `SM3eRq` closed/green als Inversen-/AccumulatedEntry-Exec-Provider; `SM3fqPlus` closed/green als Boundary-Schur-Exec-Completion; `SM4qRecheck` closed/green als MultiSchur-Exec-Forward; `SM5qRecheck` closed/green als ArithmeticSource-Exec-Forward; `SM6bRecheck` closed/green als LevelStep-Exec-Forward; `SM8Recheck` closed/green als LevelHistory-ExecMatrix-Pullback; `SM9aRecheck` closed/green als ExecMirror-Source-Gate; `SM9b` closed/green als Charpoly-Evaluation-Gate; `SM9c` closed/green als Operational-Evaluation-Family; `SM9d` closed/green als Operational-Coefficient-Source-Gate; `SM9e` closed/green als Operational-Determinant-Coefficient-Recurrence; `SM9f` closed/green als Requirement-/Conditional-Bridge; `SM9ePlus0` closed/green als Evaluationsnormalform-/Orientierungsdiagnose; `SM9ePlus1` closed/green als korrekt orientierte Degree-1-Forward-Source; `SM9fFull` closed/green als Forward-Determinantenbridge; `SM9g` closed/green als Discriminant-/Factor-Source-Gate mit Missing-Window-Status; `SM9g1` closed/green als Operational-Quadratic-Coefficient-Window; `SM9h--OperationalQuadraticDiscriminantSourceFromWindow` **active**; `SM10`, `SM11` und `SM12` blocked until a consumable operational discriminant/factor comparison source is proven.

1. Entscheidungsgates und harte Methodik

1.1 Harte Regeln

1. Code-Primat: aktive Lean-Dateien bleiben die einzige formale Quelle; Prosa ist Begleitdokumentation.
2. Kein alter Arithmetik-Code. Aus der alten Planung werden nur Diagnosen, Gates und negative Muster uebernommen.
3. Kein freier Abschluss: keine Felder oder Annahmen der Form `matrixRat`, `matrix_real_eq`, `hcharpoly`, `factorization`, `charpoly_eq_schurNumerator`, `quotient` im finalen Pfad.
4. Kein falscher Carrier: `Fin 3`/`Fin 4` darf nie als Ersatz fuer den realen finite bright-boundary-Traeger eingefuehrt werden. Erst wird der reale Traeger konstruiert, dann werden Profilaequivalenzen bewiesen.
5. Parameterdisziplin: `spectralParameter : Rat` fuer Charpoly/Determinanten; modular-q ist separat $q_{\mathrm{mod}} = \exp(2\pi i\tau)$.
6. Elliptic-Function-First-Regel: Der Weg zu Δ und j laeuft derived-only zuerst ueber Perioden- und Lattice-Daten, Weierstrass-$\wp$, Eisenstein-Gittersummen G_4, G_6 und die Invarianten g_2, g_3. Sobald ein Perioden-/Lattice-Datum aus dem Schur-/BoundarySource-Pfad vorliegt, muessen Λ, $\wp$, G_4, G_6, g_2, g_3, Δ und erst dann j geschlossen werden. Diese Schicht ist zwingende lokale Provenienz fuer Δ und j, aber kein globales Bindeglied zu Leech-, Conway-, Monster-, VOA- oder sporadischen Strukturen. q-Expansionen, analytische Zahlentheorie, L-Funktionen, VOA- und Moonshine-Seeds sind danach reine Consumer und benoetigen jeweils eigene Traeger-, Wirkungs- und Kompatibilitaetsnachweise. Keine j-Invariante ohne vorher geschlossene Weierstrass-/Eisenstein-Provenienz; kein Sprung von elliptischen Funktionen zu Monster/VOA ohne separates Action-/VOA-Gate.
7. Keine semantische Vorwegnahme: finite Seeds sind keine VOA, kein Monster-Modul, keine McKay–

Thompson-Serie.

8. Indikatorstatus: Heegner-Zahlen, $d = 163$, Ganzzahligkeitsphaenomene und Monster-Koeffizienten sind Brueckenindikatoren und Pruefziele, keine Eingabeaxiome des CNNA-Pfads.
9. Kein `noncomputable` im operativen Pfad; $\mathbb{C}$-Spiegelinseln sind lokal, explizit und nicht konsumierbar durch den Exec-Hauptstrang.
10. Keine globalen `@[simp]`-Flaechen, keine vermeidbaren `classical`-Abkuerzungen, keine grossen Index-Fallunterscheidungsmonster.
11. Entry-Lemma-vor-Formel-Regel: Jede explizite Matrix- oder Inversenformel im Smoke-Pfad darf erst eingefuehrt werden, nachdem die eintragsweise Struktur der Quellmatrix aus dem generated Traeger, der Partition, der Adjazenz und der Weight-/Policy-Provenienz bewiesen wurde. Kompilierender Code mit plausibler, aber nicht aus Entry-Lemmata abgeleiteter Formel gilt als Planbruch.
12. Jeder oeffentliche endliche Traeger liefert `Fintype`, `DecidableEq` und noetige Decidable-Instanzen explizit oder theorematisch.
13. Boundary-Quellen-Regel: `referenceFullMultiSchur` auf `CutSpec.full` darf nicht als Level-History-Quelle verwendet werden, bevor ein Negativ-/Kardinalitaetsaudit das Gegenteil beweist. Nach aktuellem Codebefund ist es nur Top-Boundary-Kandidat; der Arithmetikpfad muss eine echte multi-level Boundary-Quelle bauen.
14. AR2b-Boundary-Source-Bifurkations- und Entscheidungsregel: AR2b ist nicht als Entscheidung zwischen `CutSpec` und rekursivem Neubau zu lesen, sondern als Entscheidung zwischen legalen Boundary-Quellen. Alle aktuell vorhandenen `ToCStrong.ofIdeal`-Substrate, insbesondere `referenceToC b` und `variationToC beta`, sind `IdealThread`-basiert und haben pro Level Singleton-Carrier. Diese konkrete CutSpec-Hoffnung ist daher als universelles No-go zu auditieren. Offen bleiben zwei legale positive Zielrouten: Carrier-first ueber ein erst noch A-intern zu konstruierendes nicht-singleton `GeneratedSubstrate`, und operator/history-first ueber eine rekursive Level-History-Quelle. Die operator/history-first-Route ist jedoch erst nach einer expliziten Typentscheidung implementierbar: Carrierform, per-level-Beitragsquelle, Matrix-/Index-Typ, Anbindung an `MultiSchurStrong` und `BoundarySourceInterface` muessen vorher festgelegt werden. Keine Route darf freie Boundary-Punkte erzeugen; kompilierender Code mit falscher Architektur gilt als Planbruch.
15. IdealThread-Nullpfad-Regel: `IdealThread` ist als Nullpfad-Verduennung eines Ambient-Substrats zu lesen, nicht als vollwertiges nicht-singleton Substrat und nicht als voller Baum mit b=1. Fuer `RegularCell b L := Fin L -> Fin (b+1)` ist b=0 der unaere volle Fall; der `IdealThread` hingegen bleibt fuer jedes b ein Singleton-Carrier im Ambientraum. Arithmetische b-/beta-Abhaengigkeit muss daher aus generated nicht-singleton Substraten kommen. Der String-Hinweis ist nur ein Fernziel-Vergleich fuer Faden/Targetspace/Worldsheet-Strukturen und keine aktive Provenienz im Arithmetikpfad.
16. REALOQS-Statusregel: REALOQS ist Legacy und Inspirationsquelle fuer Pattern wie KMS/Gibbs/GNS/Stinespring, aber keine aktive Provenienzquelle und kein Code-Transfer in den neuen Arithmetikpfad.
17. Cayley-Dickson-Anschlussregel: Bereits gruene Cayley-Dickson-/Hurwitz-Module duerfen angeschlossen und verwendet werden, sofern sie als A-interne, proof-carrying Strukturquellen konsumiert werden und den operativen $\mathbb{C}$-Produktivstrang nicht global ersetzen.
18. Algebra-Arithmetik-Konvergenzregel: Der Arithmetik-Pfad und der Cayley-Dickson/Hurwitz-Pfad sind getrennte derived-only Consumer desselben Pillar-A-Generators. Der Arithmetik-Pfad darf Cayley-Dickson/Hurwitz-Strukturen nur ueber explizite A-interne, bereits gruene Provenienzmodule konsumieren; umgekehrt darf kein algebraischer Endpunkt als Erklaerung arithmetischer Daten vorausgesetzt werden. Jede behauptete Beruehrung beider Pfade muss als Konvergenztest formuliert werden: gemeinsamer Traeger, gemeinsame Invarianten, Transferabbildung, Kompatibilitaetsgleichung oder expliziter Obstruktionsbefund.
19. Wirkungs-/Symmetrie-Provenienzregel: Eine Gruppe, ein Monoid oder eine Automorphismenstruktur darf im Arithmetikpfad nur konsumiert werden, wenn Traeger, wirkendes Objekt, Wirkungsabbildung und Erhaltungseigenschaften A-intern konstruiert oder ausdruecklich als externer Vergleich markiert sind. Insbesondere duerfen `SL(2,Z)`, Galoisgruppen, Conway-/Monstergruppen, Lie-Typ-Gruppen oder Cayley-Dickson-Automorphismen nicht als freie Erklaerungsobjekte eingefuehrt werden. Jede

behauptete Symmetrie muss mindestens als `ActionPackage` erscheinen: `carrier`, `actingObject`, `actionMap`, Preservation-Lemmata, Invariantendaten und Obstruktionsledger. Keine Wirkung ohne Traeger; keine Wirkung ohne Preservation-Lemma; keine volle Baumautomorphismengruppe und keine freie Level-Permutation ohne Beweis, dass Cutoff, Orientierung, BoundarySource und Schur/DtN-Daten erhalten bleiben.

20. Erweiterte Wirkungsformat-Regel: CNNA darf nicht voraussetzen, dass jede Symmetrie als globale Gruppenwirkung vorliegt. Jede behauptete Symmetrie ist vor der Implementierung als Wirkungsformat zu klassifizieren: globale Gruppe, Monoid, lokale Baumaktion, selbstaehnliche Wreath-Recursion, Branch-Gruppe, Groupoid oder partielle Wirkung, Hecke-/Double-Coset-Operatoralgebra, profinite oder Galois-artige Wirkung, arboreale Vergleichswirkung, Association-Scheme-/coherent-configuration-artige Matrixalgebra, p-adische Building-Wirkung, kategoriale VOA-Orbifold-Wirkung oder reine externe Vergleichsstruktur. Erst danach duerfen Traeger, Aktion/Operator, Kompositionsgesetz, Erhaltungssaetze, Invarianten und Obstruktionen A-intern konstruiert werden. Keine globale Gruppe, wenn nur lokale oder partielle Symmetrie vorliegt; keine Hecke-Algebra ohne Double-Coset- oder Korrespondenzkomposition; keine Galois-, GT-, Building- oder VOA-Kategorie ohne abgeleiteten Traeger.
21. Root-System-Abgrenzungsregel: Eine action-basierte Symmetrie darf nicht automatisch als Weyl-, Cartan- oder Root-System-Struktur gelesen werden. Fuer jede Wirkung ist getrennt zu klassifizieren, ob sie reflexiv/root-system-artig, rootless/chiral, modular, Galois-artig, sporadisch, residuen-/Lie-Typ-artig oder nur externe Vergleichsstruktur ist. Aus hoher Symmetrie folgt keine Lie-/Weyl-Struktur. Insbesondere bleibt Leech/Conway nur Vergleich, solange CNNA keinen eigenen 24-dimensionalen geraden unimodularen rootless Lattice-Traeger samt Wirkung ableitet.
22. Root-System-Praezisionsregel: Ein Root-System ist nicht mit einer Basis gleichzusetzen. Die Wurzeln bilden ein endliches lineares Erzeugendensystem des relevanten Raums; einfache Wurzeln entstehen erst nach Wahl eines positiven Systems und koennen dann als Basis des Wurzelraums dienen. CNNA muss deshalb root set, simple-root choice, root lattice, ambient lattice, Cartan integers, reflection closure, Weyl action und rootless/chiral obstruction getrennt modellieren. Insbesondere gilt: Ein Gitter mit Basis oder Erzeugendensystem ist nicht automatisch ein Root-System-Gitter; Leech-artige Faelle sind ausdruecklich als rootless/chiral oder obstruction zu behandeln. Der Schluss hochsymmetrischer Traeger ⇒ Root-System ⇒ Weyl-Gruppe ⇒ Lie-Typ ist ohne diese Zwischengates verboten.
23. Diskriminanten-Konventionsregel: Schur-Diskriminanten, Ordnungskonventionen und imaginaer-quadratische Fundamentaldiskriminanten duerfen nicht identifiziert werden, bevor ein Satz der Form `schur_discrim_to_fundamental_discrim` die Vorzeichen- und Normierungsbruecke fuer den konkreten Fall beweist.
24. L>3-Ausgangsregel: Fuer jedes Paar (b, L) mit $L > 3$ wird vor der Klassifikation ein offener Vorhersage-/Testmatrix-Eintrag angelegt. AR11 darf genau drei Ausgaenge produzieren: quadratischer Heegner-Treffer, quadratischer Nicht-Heegner-Fall oder irreduzibler/hoehergradiger Fall. Keiner dieser Ausgaenge darf als erwartet oder ausgeschlossen gelten, bevor Charpoly, Faktorisierung und Diskriminanten-/Gradzertifikat aus der realen AR2b-Quelle berechnet sind.
25. Graph-Zeta-/Ramanujan-Regel: Ihara-Zeta, Ramanujan-Schranken und spektrale Luecken sind berechenbare Seeds, aber nicht automatisch nichttrivial. Ein reiner endlicher Baum hat keine prime closed geodesics und damit eine triviale Ihara-Seite; ein abgeschnittener rooted b-Baum ist nicht regulaer. Jede Verwendung der regulaeren Bass-Hashimoto-Formel, der Schranke |lambda| <= 2 sqrt(q) oder einer Cheeger-Deutung setzt einen expliziten Regularitaets-, Gradmatrix- oder Normalisierungsbeweis voraus.
26. ∞-Substrat-Regel: Jede Aussage ueber Ihara-Zeta, Graph-Zeta, Spektralmass, Ramanujan-Eigenschaft, L-Funktions-Seed oder Gap im unendlichen Grenzregime muss zuerst den Substratstatus deklarieren: `ReferenceThreadInfinity`, `RegularTreeIDEALInfinity`, `REALInfinityDirectedSystem`, `PeriodicOrQuotientREALInfinity`, `VariationREALInfinity beta` oder `WeightedOrMeasureGraphREALInfinity`. Kein Transfer zwischen diesen Regimen ohne expliziten Transferbeweis; keine Graph-Zeta ohne Zyklus-, Periodizitaets-, Mass-, Spur- oder Determinantenstruktur.

27. Implementierbarkeitsstatus-Regel: Jede Phase traegt einen Status `sofort-implementierbar`, `entscheidungspflichtig`, `konditional-offen` oder `blockiert`. `sofort-implementierbar` heisst: die benoetigten Traeger, Typen und Importquellen sind im aktuellen S9h-Code bekannt. `entscheidungspflichtig` heisst: vor Code muss ein Typ- und API-Entwurf beschlossen werden. `konditional-offen` heisst: die Route ist legal denkbar, aber im aktuellen Code nicht vorhanden. `blockiert` heisst: ein No-go oder fehlende Provenienz verhindert Implementierung. AR0, AR1, AR2a und AR2b-0 sind der erste sofort-implementierbare Block; AR2b-2 ist bis zur Typentscheidung nicht als Implementierungsauftrag freigegeben.
28. Jede Phase endet mit: `lake build`, Importgraph-Audit, grep auf `sorry`/`admit`/freie Axiome/operative `noncomputable`/`classical`/breite `@[simp]`; zusaetzlich gilt der Pflichtblock aus Abschnitt 1a.

1.2 Pflichtblock fuer jede Phase

Jede Phase AR0–AR28, inklusive AR2a/AR2b und der Sub-Gates AR3a/AR3b/AR6a/AR10b/AR10c/AR10d/AR12a/AR15a/AR18b/AR18c/AR23a, traegt zusaetzlich zu ihrem fachlichen Gate zwingend denselben Implementierungsvertrag:

- Code-b-/Arity-Konvention: In allen Planphasen gilt `RegularCell b L := Fin L -> Fin (b+1)`. Deshalb ist `arity = code-b + 1`. Der binaere 0/1-Baum ist `code-b=1`; `code-b=0` ist unaer; `code-b=2` ist ternaer. Spaetere Scan-Tabellen muessen `code-b` und `arity` getrennt ausweisen.
- **Beachte:** derived-ONLY; Importketten- und Consumer-Disziplin; erforderliche Notationen samt Einbindung in den Code; toolchainseitige Pruefung aller mathlib-Importe gegen Lean/mathlib 4.28.0; vollstaendiges ZIP mit allen aktualisierten Dateien fuer die lokale Kompilation.
- **Toolrealitaet:** ChatGPT hat kein installiertes Lean als lokale Wahrheitsquelle; der Editor kompiliert lokal. Aussagen ueber gruenen Build gelten erst nach lokaler Kompilation.
- **Unterlasse:** pauschale `@[simp]`-Flaechen; `noncomputable` im operativen Pfad; vermeidbare klassische Hilfen; Index-Eliminatoren mit zu vielen Fallparametern; Rueckfaelle in alte Muster; fragile Rewrite-Schritte.
- **Nutze:** robuste Pfade; explizite Konstruktoren; strukturelle Beweise; Simplifizierungen nur auf den allerletzten Meter oder bei trivialen Beweisen.

1.3 BoundarySource-Gates

Negativbefund	Die bestehende Full-Quelle exponiert boundary-semantisch nur den oberen Frontier: `BoundaryVertex` laeuft ueber `A.ports L`; `ports L` ist `boundaryCarrier L`; `boundaryCarrier L` filtert nach `parentExposedAt oder childExposedAt`; bei `CutSpec.full` verschwinden innere Expositionen. Damit ist die erwartete Aequivalenz zu `Fin (L+1)` fuer die Full-Quelle nicht Planannahme, sondern zu widerlegender Altpfad.
Planfolge	AR2 wird gespalten: AR2a beweist/auditiert den Full-Negativbefund; AR2b klaert die Boundary-Quelle. AR2b beginnt mit dem universellen No-go der vorhandenen `IdealThread`-Substrate und fuehrt danach eine explizite Entscheidungsphase `AR2b-D`. Erst dieses Gate entscheidet, ob eine Generated-Substrate-Route, eine Recursive-History-Route oder beide positiv implementiert werden duerfen. Nach positiv geschlossener BoundarySource folgt AR3a als `GeneratorActionInventory`: erst die BoundarySource klaert, worauf der Generator operiert. Direkt danach folgt AR3b als `ActionFormatDecision`: die Action-Achse entscheidet nicht nur, welche Wirkung legal ist, sondern welches Wirkungsformat ueberhaupt vorliegt – globale Gruppe, lokale Baumaktion, Groupoid, Hecke-Algebra, profinite/Galois-artige Wirkung, Matrixalgebra oder kategoriale Vergleichsstruktur.
Positive Quelle	Bevor AR2b-D und mindestens eine positive BoundarySource-Route geschlossen sind, duerfen AR3–AR10 nicht als Ergebnisphasen implementiert werden. Erlaubt sind nur Hilfsdaten, No-go-Saetze und Entscheidungsledger, die AR2b nicht semantisch vorwegnehmen.

BoundarySource-Bifurkation

Gemeinsame Frage	Pillar A muss eine endliche Boundary-/Interface-Traegerstruktur erzeugen, auf der Schur, DtN und MultiSchur arithmetisch sinnvoll operieren koennen. AR2b entscheidet nicht zwischen `CutSpec` und Rekursion als Prosa-Alternativen, sondern zwischen legalen Boundary-Quellen.
Existierende Substrate	`referenceToC b` und `variationToC beta` sind nach aktuellem Codebefund beide ueber `ToCStrong.ofIdeal` aus `IdealThread` gebaut. Da `IdealThread.carrier L` ein Singleton ist, kann kein `TrunkCut`, `WindowCut`, `AdaptiveFrontierCut`, `CylinderCut`, `SubtreeCut` oder `ExteriorCut` auf diesen vorhandenen Substraten eine echte $(L+1)$-Boundary erzeugen.
Route I: Carrier-first	`GeneratedNonSingletonSubstrateRoute`: Konstruiere zuerst ein A-intern generiertes nicht-singleton Substrat, etwa ueber `BranchAddressSubstrate`, `BoundaryStateSubstrate`, `HistoryExpandedSubstrate` oder `InterfaceExposureSubstrate`. Erst dann darf `CutSpec` echte `BoundaryPorts` gewinnen. Diese Route ist prinzipiell offen, aber im aktuellen Code noch nicht vorhanden.
Route II: Operator/history-first	`RecursiveMultiLevelBoundaryRoute`: Erzeuge rekursive per-level DtN-/Schur-Daten und lasse deren Level-History den Boundary-Traeger bilden. Diese Route ist der aktuelle positive Hauptkandidat und wird in Abschnitt 1.4 konkret entschieden: `LevelHistoryIndex L := Fin (L+1)`, `witnessAt`, `LevelStepWitness`, `LevelHistoryMatrix L` und Export ueber `BoundarySourceInterface`.
Entscheidungsphase	AR2b-D ist fuer Route II geschlossen: Level-History-Index statt neuem Substratcarrier; proof-carrying `witnessAt`; per-level Quelle aus `ArithmeticSource`/S9h-/GeneralizedDtN-/MultiSchur-Provenienz; operative `ExecComplex`-Matrix; deterministic/proof-carrying Aggregation; `MultiSchurStrong` als Provenienzanker; Export nur ueber `BoundarySourceInterface`.
Interface	Beide positiven Routen muessen in ein gemeinsames `BoundarySourceInterface` muenden. Fuer AR2b-2 fuehrt dieses Interface mindestens `index`, `indexEquiv`, `matrix`, `provenance` und `route`; die Route ist `recursiveHistory`. AR3–AR10 konsumieren nur dieses Interface, nicht die interne Entstehungsroute.
Konvergenztest	Wenn spaeter sowohl eine Generated-Route als auch eine Recursive-Route existieren, ist zu pruefen, ob `GeneratedBoundarySource` und `RecursiveLevelHistoryBoundary` dieselbe Boundary beschreiben oder ob eine Obstruktion vorliegt. Bis dahin wird die Substratroute nicht als gescheitert, sondern als konditional offen gefuehrt.
Sperren	Kein Carrier darf frei gesetzt werden; kein p.c.f.-Fraktal oder externes geometrisches Substrat darf importiert werden; `BoundaryPorts` duerfen keine Punkte erzeugen, die nicht bereits im Carrier liegen; jeder Carrier-Punkt braucht Provenienz aus Branching, Cutoff, Sektor-/Coupling- oder Schur-/DtN-Daten.

1.4 AR2b-D: entschiedene BoundarySource-Typentscheidung

AR2b-D ist ab dieser Fassung nicht mehr eine offene Fragenliste, sondern ein geschlossenes Architektur-Gate fuer die operator/history-first-Route. Die Entscheidung gilt nur fuer AR2b-2; die Generated-Substrate-Route AR2b-1 bleibt konditional offen, wird aber nicht als Implementierungsbasis fuer AR2b-2 verwendet. Damit ist der erste Implementierungsblock fest: AR0, AR1, AR2a, AR2b-0, AR2b-D; erst danach darf AR2b-2 gebaut werden.

D1 Carrier	Fuer AR2b-2 ist der Carrier kein neuer ToC-Substratcarrier. Der Carrier ist ein reiner Level-History-Indexcarrier `LevelHistoryIndex L := Fin (L+1)`. Dieser Index ist nicht die Boundary als freie Setzung; legal wird er nur zusammen mit einem Beweis-/Witness-Feld fuer jeden Levelindex. Verboten bleibt jede Aussage `BoundaryPorts ≃ Fin (L+1)` auf den vorhandenen `IdealThread`-Substraten.
D2 Recursive carrier	`RecursiveLevelHistoryCarrier source L` wird nicht als Sigma-Typ mit beliebigem Payload und nicht als neues Substrat definiert, sondern als proof-carrying Struktur ueber dem festen Index `LevelHistoryIndex L`. Minimalform: `witnessAt : (k : LevelHistoryIndex L) -> LevelStepWitness source L k`. Die Matrixdimension ist damit stabil, aber die mathematische Substanz liegt ausschliesslich in `witnessAt`.
D3 Per-level Quelle	Per-level DtN-/Schur-Beitraege stammen nicht aus `TrunkCut`, `WindowCut`, `AdaptiveFrontierCut` oder irgendeinem Cut auf `IdealThread`. Quelle ist die vorhandene S9h-/DtN-/GeneralizedDtN-/MultiSchur-Provenienz, vermittelt ueber `ArithmeticSource`. `LevelStepWitness source L k` traegt mindestens `hasDtnProvenance`, `hasSchurProvenance` und `noCutSpecCarrierClaim`; diese Props duerfen spaeter durch konkrete vorhandene Typen ersetzt werden.
D4 Matrix-/Index-Typ	Die operative Matrix fuer AR2b-2 ist `Matrix (LevelHistoryIndex L) (LevelHistoryIndex L) ExecComplex`, kurz `LevelHistoryMatrix L`. Der ℂ-Spiegel ist kein Bestandteil der AR2b-2-Operativroute, sondern eine spaetere Mirror-Schicht. Dadurch wird kein `noncomputable` in den Exec-Pfad gezogen.
D5 Aggregation	Die Multi-Level-Matrix wird deterministisch aus `source` und `witnessAt` gewonnen, nicht als freies Matrixfeld im Carrier gespeichert. Falls technisch ein Matrixfeld gebraucht wird, muss es proof-carrying sein: `matrix : LevelHistoryMatrix L` zusammen mit `matrix_from_source : MatrixDerivedFrom source witnessAt matrix`. Richtung der Konstruktion ist immer `source + witnessAt -> matrix`, nie `matrix -> source`.
D6 MultiSchur-Anbindung	`MultiSchurStrong` ist Provenienzanker, nicht ein neu frei zu befuellender Zieltyp. Die Route lautet: `ArithmeticSource -> existing S9h MultiSchur/GeneralizedDtN data -> RecursiveLevelHistoryCarrier -> LevelHistoryMatrix -> BoundarySourceInterface`. Ein Adapter wie `MultiSchurToLevelHistoryAdapter` ist erlaubt, bleibt aber adapter-only, bis exakte Gleichheitslemmata wie `recursiveLevelHistoryMatrix_eq_referenceFullMultiSchur` wirklich bewiesen sind.
D7 Interface-Export	Downstream-Phasen konsumieren nicht `RecursiveLevelHistoryCarrier` direkt, sondern `BoundarySourceInterface source L`. Das Interface fuehrt `index`, `indexEquiv : index ≃ Fin (L+1)`, `matrix : Matrix index index ExecComplex`, `provenance : BoundarySourceProvenance source L` und `route : BoundarySourceRoute`. Fuer AR2b-2 gilt `route = recursiveHistory`; fuer AR2b-1 spaeter `route = generatedSubstrate`.
Routenstatus	`GeneratedNonSingletonSubstrateRoute` bleibt legal denkbar und konditional offen, aber sie ist nicht implementierungsleitend fuer AR2b-2. AR2b-1 darf erst starten, wenn ein A-internes nicht-singleton Substrat tatsaechlich konstruiert ist. AR2b-2 ist dagegen nach D1–D7 konkret implementierbar als operator/history-first Konstruktion.

Lean-Signatur-Skizze fuer AR2b-2.

```
abbrev LevelHistoryIndex (L : Nat) := Fin (L + 1)

structure LevelStepWitness
    (source : ArithmeticSource)
    (L : Nat)
    (k : LevelHistoryIndex L) where
  hasDtnProvenance : Prop
  hasSchurProvenance : Prop
  noCutSpecCarrierClaim : Prop

structure RecursiveLevelHistoryCarrier
    (source : ArithmeticSource)
    (L : Nat) where
  witnessAt :
    (k : LevelHistoryIndex L) ->
      LevelStepWitness source L k

abbrev LevelHistoryMatrix (L : Nat) :=
```

```
  Matrix (LevelHistoryIndex L) (LevelHistoryIndex L) ExecComplex

structure BoundarySourceInterface
    (source : ArithmeticSource)
    (L : Nat) where
  index : Type
  indexEquiv : index_equiv_Fin_L_plus_1
  matrix : Matrix index index ExecComplex
  provenance : BoundarySourceProvenance source L
  route : BoundarySourceRoute
```

Schlussstatus. AR2b-D ist geschlossen, wenn diese Signatur-Entscheidung im `BoundarySourceDecisionLedger` als `RecursiveTypeDecisionClosed` dokumentiert ist. `DecisionObstruction` bleibt nur fuer den Fall, dass einer der Adapter zur vorhandenen S9h-/GeneralizedDtN-/MultiSchur-Provenienz nicht angeschlossen werden kann. Ein kompilierender Platzhalter ohne `witnessAt`-Provenienz oder mit freiem Matrixfeld gilt als Planbruch.

1.5 Algebra-Arithmetik-Konvergenzgate

Die Sektorzerlegung erzeugt algebraische Strukturkandidaten; die Verzweigungsrekursion erzeugt arithmetische Strukturkandidaten. Der Arithmetik-Pfad muss an jedem natuerlichen Kontaktpunkt pruefen, ob beide Kandidaten dieselbe abgeleitete Struktur beschreiben, ohne diese Einheit vorauszusetzen.

Algebra-Seite	A-interner Cayley-Dickson/Hurwitz-Pfad: Komplex-/Quaternion-/Oktonion-Schichten, Konjugation, Norm, AlternativeLaw, Kompositions-/Division-Slots und Hurwitz-Stopp-Seeds.
Arithmetik-Seite	A-interner Schur-/Branching-Pfad: bright-DtN, Charpolys, Diskriminanten, quadratische Ordnungen, CM-Punkte, Periodengitter, j, q-/VOA-Seeds, Graph-Zeta-/Ramanujan-/Gap-Daten.
Zulaessige Beruehrung	Nur ueber explizite Transfer- und Kompatibilitaetsobjekte: gemeinsamer Traeger, gemeinsame Invarianten, Transferabbildung, Norm-/Konjugations-/Formgleichung oder negativer Obstruktionssatz.
Sperre	Keine Prosa-Abkuerzung der Form “Quaternionen erklaeren Ramanujan”, “VOA erklaert Cayley-Dickson” oder “Moonshine folgt aus $\mathbb{O}$”. Externe Konstruktionen sind Vergleichsanker, keine CNNA-Provenienz.

Kontaktpunkte und Abhaengigkeitsgraph

```
CD/Hurwitz-Pfad ----+-> AR6a
                    +-> AR10d
                    +-> AR12a
                    +-> AR15a
                    +-> AR23a
                    +-> AR25

AR-Schur-Pfad  -----+-> AR6a  nach AR6
                    +-> AR10d nach AR10b
                    +-> AR12a nach AR12
                    +-> AR15a nach AR15
                    +-> AR23a nach AR23
                    +-> AR25
```

1.6 Wirkungs-/Symmetrie- und Wirkungsformatgate

Die Algebra-Arithmetik-Konvergenz darf nicht nur objekt- und invariantentheoretisch formuliert werden. Modularitaet, CM, Gitter-/Conway-Strukturen, VOA/Moonshine und Cayley-Dickson-Automorphismen sind Strukturen mit Gruppen-, Monoid- oder Automorphismenwirkungen. CNNA darf solche Wirkungen

nicht voraussetzen, muss aber frueh pruefen, welche Wirkungen der Pillar-A-Generator selbst auf Boundary-, Schur-, CD- und CM-Traegern erzeugt.

Gemeinsame Frage	Welche A-intern erzeugte Struktur traegt welche nachweisbare Wirkung, und welche Invarianten entstehen unter dieser Wirkung? Die Aussage "Gruppen wirken ueberall" ist fuer den Plan zu weich; zaehlen darf nur ein konkretes `ActionPackage`.
Action-Paket	Minimalfelder: `carrier`, `actingObject`, `actionMap`, Identitaets-/Kompositions- oder Monoidgesetze, Preservation-Lemmata fuer relevante Strukturen, Invariantendaten und Obstruktionsstatus.
Generator-Achse	Kandidaten sind Branching-/Tree-Wirkungen, Bright/Dark-Involution, Level-History-Transformationen, Moebius-/Schur-Transferwirkungen und CD-interne Automorphismenkandidaten. Jeder Kandidat bleibt blockiert, bis sein Traeger aus `BoundarySource` oder einer anderen A-internen Quelle feststeht.
Wirkungsformat	Jede Kandidatensymmetrie wird zuerst als Wirkungsformat entschieden: globale Gruppe, Monoid, lokale Baumaktion, selbstaehnliche Wreath-Recursion, Branch-Gruppe, Groupoid/partielle Wirkung, Hecke-/Double-Coset-Operatoralgebra, profinite/Galois-artige Wirkung, coherent-configuration-artige Matrixalgebra, p-adische Building-Wirkung, kategoriale VOA-Orbifold-Wirkung oder reine Watchlist. Diese Entscheidung ist ein Gate vor mathematischem Ausbau.
Sperren	Keine volle Automorphismengruppe des unendlichen b-aeren Baums auf endlichen REAL-Approximanten ohne Cutoff-/Orientierungs-/Boundary-Preservation. Keine freie `Sym (Fin (L+1))`-Level-Permutation, solange die Levelordnung rekursiv wesentlich ist. Legal sind zunaechst eher Restriktion, Inklusion, Level-Shift, Renormalisationsschritt oder Moebius-Transfer.
Root-/Lattice-Abgrenzung	Jede Action wird zusaetzlich danach typisiert, ob sie reflexiv/root-system-artig, rootless/chiral, modular, Galois-artig, sporadisch oder nur externe Vergleichsstruktur ist. Hohe Symmetrie reicht nicht fuer Weyl-, Cartan-, Lie-Typ- oder Leech-Behauptungen; ein Leech-Vergleich bleibt `ExternalLeechComparisonOnly`, bis ein eigener even unimodular rootless 24D-Traeger mit Action-Preservation vorliegt.
Konvergenzrolle	AR25 prueft nicht nur gemeinsame Objekte, Normen und Invarianten, sondern auch kompatible Wirkungen auf gemeinsamen oder funktoriell verbundenen Traegern. Moeglich sind starke Objekt-und-Aktions-Konvergenz, reine Objektkonvergenz, reine Aktionskonvergenz, lokale Aktionskompatibilitaet mit globaler Obstruktion oder keine Aktionskonvergenz.

Action-Abhaengigkeitsgraph

```
BoundarySource -----> AR3a GeneratorActionInventory
AR3a ---------------> AR3b ActionFormatDecision
AR3b ---------------> AR4/AR5/AR6 preservation checks
AR3b ---------------> AR13/AR15 CMActionInterface
CD/Hurwitz ---------> AR6a/AR12a/AR15a plus CDActionInterface
Moonshine path -----> AR19--AR23 plus VOAActionInterface
Actions ------------> RootSystemActionBoundary / RootlessChiralActionTest
Actions ------------> AR25 ActionConvergenceLedger
```

2. Phasenplan

Phase	Ziel	Kernarbeit	Gate
AR0	Namespace- und Import-Start (sofort)	`CNNA/PillarA/Arithmetic` leer-neu anlegen; `BuildAll.lean`, `Notation.lean` und minimale Importwurzel setzen; vorhandenen S9h-Anhang nur auditieren, nicht kopieren.	Sofort implementierbar: rein mechanisch. Kein Alt-Arithmetik-Import, kein Legacy-Code, keine semantische Notation; Build belegt nur den sauberen neuen Baum.
AR1	S9h-Provenienzquelle (sofort)	`ArithmeticSource` als duennen Wrapper/Namespace fuer bekannte gruene S9h-Objekte anlegen, insbesondere ToC-, Finite-, DtN-, Coupling-, Spectral-, Channel- und MultiSchur-Quellen; Exec- und $\mathbb{C}$-Spiegel getrennt markieren.	Sofort implementierbar: nur Wrapper, keine neuen Daten. Quelle enthaelt keine Zahlenringe, CM-Punkte, Modularformen, Diskriminanten oder freien Matrizen.

Phase	Ziel	Kernarbeit	Gate
AR2a	Full-Boundary-Negativaudit (sofort)	`referenceFullMultiSchur`, `CutSpec.full` und die Boundary-Carrier-Kette isolieren; pruefen/beweisen, dass die reale Full-Boundary nicht definitionell die gesamte Level-History exponiert und keine `Fin (L+1)`-Boundary liefert.	Sofort implementierbar: Alt-Route A wird als Negativsatz oder Auditobjekt geschlossen. Kein Lemma verfolgt mehr eine Aequivalenz `CutSpec.full.boundary = LevelHistoryCarrier L`.
AR2b-0	IdealThread-No-go (sofort)	Fuer alle aktuell vorhandenen `ToCStrong.ofIdeal`-Substrate beweisen/auditieren: `referenceToC b` und `variationToC beta` stammen aus `IdealThread`; `IdealThread.carrier L` ist singleton.	Sofort implementierbar: Outputs `ReferenceCutSpecNoGo`, `VariationCutSpecNoGo`, `ExistingIdealThreadSubstrateNoGo`. Kein vorhandener nicht-voller `CutSpec` erzeugt eine echte $(L+1)$-Boundary.
AR2b-D	BoundarySource-Typentscheidung (entschieden)	D1–D7 sind festgelegt: `LevelHistoryIndex L := Fin (L+1)` als reiner History-Index; `RecursiveLevelHistoryCarrier source L` mit `witnessAt`; per-level Quelle aus `ArithmeticSource` und vorhandener S9h-/GeneralizedDtN-/MultiSchur-Provenienz; operative `LevelHistoryMatrix L` ueber `ExecComplex`; deterministische Aggregation aus `source + witnessAt`; `MultiSchurStrong` nur als Provenienzanker; Export nur ueber `BoundarySourceInterface` mit `route = recursiveHistory`.	Status: `RecursiveTypeDecisionClosed` fuer Route II. `GeneratedSubstrateRoute` bleibt konditional offen, aber nicht implementierungsleitend fuer AR2b-2. Verboten sind freie Matrixfelder, freie `Fin (L+1)`-Boundary und jeder CutSpec-Carrier-Claim auf `IdealThread`.
AR2b-1	Generated-Substrate-Route (konditional)	Carrier-first-Route nur nach AR2b-D: ein neues A-internes nicht-singleton Substrat konstruieren, wenn jeder Carrier-Punkt aus Branching, Cutoff, Sektor/Coupling oder Schur/DtN-Provenienz stammt. Kandidaten: BranchAddress, BoundaryState, HistoryExpanded, InterfaceExposure.	Konditional offen: Route bleibt blockiert, solange kein solches Substrat gruen ist. Kein p.c.f.-Import, kein frei gesetztes `Fin (L+1)`, keine von `BoundaryPorts` erzeugten Zusatzpunkte.
AR2b-2	Recursive-History-Route (nach AR2b-D)	Jetzt konkret implementierbar nach `RecursiveTypeDecisionClosed`: `LevelStepWitness source L k` fuer jeden `k : LevelHistoryIndex L`; daraus `RecursiveLevelHistoryCarrier`; daraus eine proof-carrying oder deterministisch abgeleitete `LevelHistoryMatrix L`; danach Export als `BoundarySourceInterface source L` mit `route = recursiveHistory`.	Jeder History-Eintrag muss DtN- und Schur-Provenienz aus `ArithmeticSource`/S9h tragen und `noCutSpecCarrierClaim` belegen. `MultiSchurStrong` wird adaptiert, nicht frei erzeugt; der $\mathbb{C}$-Spiegel bleibt spaeterer Consumer.
AR2b-3	BoundarySource-Interface	`BoundarySource` als gemeinsame Konsumoberflaeche fuer jede positiv entschiedene Route definieren: Carrier, Index, Einbettung, Provenienz, DtN-/Schur-Beitrag, MultiSchur-Kompatibilitaet und `noFreeBoundaryData`.	AR3–AR10 duerfen nur `BoundarySource` konsumieren. `BoundarySourceConvergence` unterscheidet unavailable, obstruction, partial witness, recursive witness, generated–recursive compatibility und generated–recursive obstruction.
AR3	Matrixtransport auf realer Boundary	`brightBrightSchur`, DtN- und Char-Matrizen entlang der entschiedenen `BoundarySource` transportieren; Profilmatrizen nur als Pullback/Pushforward mit nachgewiesener Komponentenidentitaet definieren.	Jede Matrixkomponente ist gegen die reale AR2b-Quelle bewiesen; kein `matrix_real_eq`, kein Fake-Carrier, kein Rueckfall auf `CutSpec.full`.
AR3a	Generator-Wirkungsinventar	`GeneratorActionInventory` und `ActionPackage` anlegen: moegliche Branching-/Tree-Teilwirkungen, Bright/Dark-Involution, Level-History-Transformationen, Moebius/Schur-Transfer, CM- und CD-Aktionskandidaten inventarisieren.	Keine Wirkung ohne Traeger, Aktionsabbildung, Gesetz und Preservation-Lemma. Volle Baumautomorphismen und freie Level-Permutationen bleiben gesperrt, solange Cutoff, Orientierung oder Rekursionsordnung sie brechen.
AR3b	Wirkungsformat-Entscheidung	`ActionFormatDecisionLedger` aus dem AR3a-Inventar ableiten: jeden Kandidaten als globale Gruppe, Monoid, lokale Baumaktion, selbstaehnliche Wreath-Recursion, Branch-Gruppe, Groupoid/partielle Wirkung, Hecke-/Double-Coset-Operatoralgebra, profinite/Galois-artige Wirkung, coherent-configuration-artige Matrixalgebra, p-adische Building-Wirkung, kategoriale VOA-Orbifold-Wirkung oder reine Watchlist klassifizieren.	Entscheidungspflichtig: Ohne Wirkungsformat keine mathematische Ausarbeitung der Action-Achse. Keine globale Gruppe, wenn nur lokale/partielle Symmetrie vorliegt; keine Hecke-Algebra ohne Korrespondenzkomposition; keine Galois-, GT-, Building- oder VOA-Kategorie ohne abgeleiteten Traeger.

Phase	Ziel	Kernarbeit	Gate
AR4	Schur-/Moebius-Rekursion	Rekursion $\sigma_k = p_k/q_k$ aus BoundarySource, Branching/Cutoff und Schur-Elimination ableiten; root/interior/leaf-Gleichungen getrennt halten; Zaehler-/Nennerlemmata formulieren.	p_0, q_0 und alle Transferparameter stammen aus AR1–AR3. AR3a und AR3b liefern nur Preservation- und Format-Information, keine zusaetzlichen Zahlen- oder Spektraldaten.
AR5	Operative Charpoly/Determinante	Determinante und Charpoly ueber ExecComplex/$\mathbb{Q}$-Profilen berechnen; kleine Dimensionen explizit, allgemeine Dimensionen ueber zertifizierte finite Matrixoperationen.	Charpoly-Auswertung wird gegen den AR4-Schur-Zaehler bewiesen. Keine Annahmen `hcharpoly`, `factorization` oder `charpoly_eq_schurNumerator`.
AR6	$\mathbb{C}$-Mirror und Exec-Bruecke	Noncomputable $\mathbb{C}$-Spiegel fuer analytische Matrizen, Eigenwerte und `UpperHalfPlane`-Anschluss; Exec-zu-$\mathbb{C}$-Transfer explizit halten.	Operativer Pfad konsumiert den Mirror nicht rueckwaerts. REALOQS/KMS/GNS/Stinespring bleiben nur Schnittstelleninspiration, keine Provenienz und kein Importpfad.
AR6a	Complex-Coupling-Test	`CDComplexArithmeticBridge`: ExecComplex-/$\mathbb{C}$-Arithmetik mit der A-internen CD-Komplexschicht vergleichen; Transfer fuer Carrier, Einbettung, Konjugation, Produkt und Norm getrennt pruefen.	Keine Identifikation von ExecComplex, Mirror-$\mathbb{C}$ und CD-$\mathbb{C}$ ohne Transferlemma; keine Aussage $\mathbb{C}$ "aus Sektorkopplung" ohne bewiesene CD/AR-Kompatibilitaet.
AR7	L=1-Baseline	Linearen Kontrollfall vollstaendig berechnen: Rekursion, Matrix, Charpoly, Eigenwert-/Normform $b+1$ und Ledger-Eintrag.	Beweist nur den Rechenweg und die Transportdisziplin. Kein CM-, Quaternionen-, Moonshine- oder Geometriesignal wird aus L=1 abgeleitet.
AR8	L=2-Eisenstein-Fall	Aus der berechneten AR4/AR5-/Schur-/Charpoly-Route die quadratische Normform, Diskriminantenkonvention, moegliche Diskriminante -3, Eisenstein-Ordnung und CM-Seed ρ ableiten. Falls ein separater Schur-/Normformparameter noetig ist, heisst er nicht `b`, sondern z.B. κ oder `schurParam`.	Identifikation folgt aus bewiesener Zaehlerfaktorisierung, Normgleichung und Diskriminantenkonvention, nicht aus Winkel, Zeichnung, Mustererkennung oder Lean-`code-b`.
AR9	L=3-Gauss-Fall	Aus der berechneten AR4/AR5-/Schur-/Charpoly-Route die quadratische Normform in $\mathbb{Z}[i]$, Diskriminantenkonvention, moegliche Diskriminante -4 und CM-Seed i ableiten. Falls ein separater Schur-/Normformparameter noetig ist, heisst er nicht `b`, sondern z.B. κ oder `schurParam`.	Gauss-Fall wird eigenstaendig bewiesen; keine Kopie des L=2-Beweises, keine Gleichsetzung Feld/Ordnung ohne AR12-Konvention und kein Rueckschluss aus der 90-Grad-Zeichnung.
AR10	Identitaetsledger L=1–3	Alle bewiesenen Identitaeten fuer L=1–3 sammeln: Matrixtransport, Charpoly=Schur-Zaehler, Faktorisierung, Normgleichung, Diskriminante, Status.	Ledger ist theorem-only. Keine neuen Datenfelder, keine informellen Werte, keine spaetere Phase darf ungepruefte Zahlen aus Prosa konsumieren.
AR10b	Gap-/Ramanujan-Ledger	Aus denselben endlichen Graph-/Adjazenz-/Laplace-/DtN-/Schur-Daten Gap, normalisierte Luecke, Cheeger-Seed und Ramanujan-Defekt berechnen; Regularitaet vor klassischer Ramanujan-Schranke pruefen.	Nur A-seitiges Diagnose- und Exportdatum. Keine Dekohaerenz-, Thermalzeit-, Raumzeit- oder Geometrieinterpretation in Pillar A.
AR10d	Quaternion/Ramanujan-Test	`QuaternionRamanujanInterface`: AR10b-Daten gegen CD-abgeleitete quaternionische Ordnungs-, Norm- oder Primstrukturen testen; LPS-Aehnlichkeit als Struktur-, Numerik- oder Leerbefund trennen.	Kein LPS-Rekonstruktionssatz. Genau ein Status: `QuaternionRamanujanWitness`, `RamanujanCoincidenceOnly`, `RamanujanNoCDConstraint`.
AR10c	L>3-Testmatrix	Fuer jedes (b, L), $L > 3$, offene Zeile anlegen: Faktorgrad, Diskriminantenkandidat, moeglicher Heegner-d, Status `Open/Theorem/Refuted` und AR11-Zertifikatverweis.	Testledger hat keine Beweiskraft. L=2/L=3 sind Kalibrierung, keine Extrapolationsregel; keine Zeile dient als Input fuer AR11–AR16.
AR11	L>3-Diskriminantensuche	Allgemeine Level-Rekursion aus AR2b/AR4/AR5 anwenden; Faktorisierungs-, Minimalpolynom- und Diskriminantenzertifikate erzeugen; quadratische, hoehergradige und irreduzible Faelle trennen.	Jede L>3-Aussage ist Theorem, Zertifikat oder explizit offene Beobachtung. Unzertifizierte Numerik bleibt ausserhalb des formalen Pfads.
AR12	Quadratische Ordnungen	Ordnungen $\mathbb{Z}[\sqrt{d}]$ und $\mathbb{Z}[(1+\sqrt{d})/2]$, Normformen, Einheiten und Vorzeichen-/Fundamentaldiskriminantenkonvention formalisieren.	Pflichtlemma `schur_discrim_to_fundamental_discrim`; keine Gleichsetzung irreduzibel=prim ohne passende Klassenzahl-/UFD-Voraussetzung.

Phase	Ziel	Kernarbeit	Gate
AR12a	Diskriminante trifft Algebra	`CDArithmeticCompatibility`: $D = -4$ als Gauss/$\mathbb{Z}[i]$-Kompatibilitaet, $D = -3$ als Eisenstein/CM-Kompatibilitaet, allgemeines $D < 0$ als imaginaer-quadratische Ordnung und hoehergradige Faelle als CD-$\mathbb{C}$-Obstruktion klassifizieren; zugleich nur Perioden-/Lattice-Anschluss fuer AR13 vorbereiten.	Quadratische Ordnung wird nicht mit Cayley-Dickson-Algebra gleichgesetzt; vergleichbar sind nur Skalar-, Norm-, Konjugations- und Formdaten mit explizitem Transfer. Keine j-, q-, L-Funktions- oder Moonshine-Folge ohne AR13–AR15-Weierstrass-/Eisenstein-Provenienz.
AR12a-SMOKE	Ontischer Arithmetik-Rauchtest und Berechnungs-Strengthening	Vor AR13 wird ein harter Positive-First-Rechentest eingeschoben: ein generated nicht-singleton Hauptpfad-Substrat ueber `RegularCell b` und `LevelVariableCell beta` muss zuerst State/Dirichlet/Interior-Block/Inverse/TwoSidedInv/InteriorElimination/Data/DoNEGeneralizedDirichletSchur berechnen; erst danach folgen `LevelHistoryMatrix`, Charpoly-/Faktor-/Diskriminantendaten, L=2 Eisenstein, L=3 Gauss und L>3-Ausgaenge.	Sperre vor substantieller AR13–AR16-Fortsetzung. Kein `CutSpec.full` als Smoke-Fundament, keine freie Matrix, keine freie Diskriminante, keine Oracle-Einspeisung aus L2/L3 und keine externen Typklassen im positiven Ledger. Der Status ist positive-confirmed, partial-computed, computed-target-mismatch oder API-blocked.
AR13	CM-Punkte und Periodengitter	Aus AR12 zertifizierte $\tau \in \mathbb{H}$, Periodengitter, SL(2,$\mathbb{Z}$)-Orbit-/Fundamentalbereichsdaten und `CMActionInterface` bauen.	CM-Punkt ist Consumer der Ordnung und ab AR12a-SMOKE substantiell nur Consumer einer berechneten oder explizit obstruierten quadratischen Ordnung. Keine modulare Wirkung ohne Traeger, Aktion und Invarianzlemma.
AR14	Weierstrass-/Eisenstein-Gittersummen	Aus dem Periodengitter Λ die elliptische-Funktion-Schicht explizit fuehren: Weierstrass-$\wp$, Eisenstein-Gittersummen G_4, G_6, Invarianten $g_2 = 60G_4$, $g_3 = 140G_6$ und Δ; operative q-Approximationen nur als getrennte Schatten vorbereiten.	$\Delta \neq 0$, Konvergenz-/Definitionsbedingungen und Bridge-Lemmata explizit; keine freien g_2/g_3-Felder. AR15/AR17/AR18/AR23 duerfen erst konsumieren, wenn diese Weierstrass-/Eisenstein-Provenienz geschlossen ist.
AR15	j-Invariant	$j = 1728g_2^3/\Delta$ ausschliesslich als Consumer von AR14 definieren; Anchor-Tests $j(i) = 1728$, $j(\rho) = 0$ soweit toolchainseitig erreichbar; SL(2,$\mathbb{Z}$)-Invarianz nur ueber Action-Paket.	j ist erstes arithmetisches Endobjekt und modularer Invariant, nicht Moonshine selbst. Keine j-Invariante ohne Λ-, $\wp$-, G_4-, G_6-, g_2-, g_3- und Δ-Provenienz.
AR15a	CM-Geometrie-Interface	`CMArithmeticGeometryInterface`: nicht nur Schur-CM/j-Daten pruefen, sondern das volle Paket `LatticePeriodData` + `WeierstrassEllipticFunctionLayer` + `EisensteinLatticeSumLayer` + `WeierstrassInvariantLayer` + `JInvariantLayer`; danach elliptische Kurve, CM-Ordnungswirkung und kompatible CD-Kopplungs-/Norm-/Formdaten testen. AR15a prueft damit die lokale elliptische Funktionsebene bis j; es erzeugt noch keinen Leech-, Conway-, Monster- oder VOA-Anschluss.	Genau ein Status: `CMGeometryConvergenceWitness`, `CMGeometryPartialCompatibility`, `CMGeometryObstruction`. Arithmetische Geometrie beginnt erst hier; q-, L-Funktions- und Moonshine-Consumer bleiben bis dahin gesperrt. Ein positiver j-Befund ist nur `ArithmeticGeometryConvergence`, nicht `SporadicVOAConvergence`.
AR16	Heegner-/UFD-Indikator	Heegner-d-Liste, class-number-one-/UFD-Status, Euler-$n^2 + n + 41$-Indikator und Ganzzahligkeitsledger fuer CM-j-Werte erst aus AR12–AR15a ableiten.	Heegner-, UFD- oder 163-Signale sind reine Consumer der geschlossenen Ordnung-/Perioden-/Weierstrass-/j-Kette; sie duerfen nicht in AR0–AR15 zurueckfliessen.
AR17	q-Expansionen	$q_{\mathrm{mod}} = e^{2\pi i\tau}$, Laurent-Reihe von j, Eisenstein-q-Reihen und operative Trunkierungen mit Koeffizienten-/Fehlerledger erst nach geschlossener elliptischer-Funktion-Schicht aus AR14–AR15a.	q_{mod} wird nie mit einem rationalen Spektralparameter verwechselt; q-Reihen sind Consumer der Lattice-/Weierstrass-/Eisenstein-Provenienz, kein Ersatz fuer sie; j ist Laurent-, nicht Taylor-Objekt.
AR18	L-Funktions-Seeds	Dedekind-, Dirichlet- und Modularform-L-Seeds nur aus bewiesenen Koeffizienten, Multiplikativitaet und ggf. Eulerfaktorstruktur formulieren.	Kein Eulerprodukt ohne Proof. Riemann-ζ bleibt Spezial-/Vergleichsfall, nicht zentraler Binder.
AR18b	Ihara-/Graph-Zeta-Seed	Fuer endliche ToC-Approximanten Bass-Hashimoto-/Ihara-Daten aus realer Graph-/Adjazenzstruktur ableiten; Gradmatrix-Version vor regulaerer Spezialformel; Zyklen/Nicht-Backtracking-Walks beweisen.	Fuer Baumtrunkierungen ist der triviale Zeta-Fall explizit; keine RH-Analogie, kein Eulerprodukt und keine Graph-Zeta ohne Zyklus-/Graphstruktur.

Phase	Ziel	Kernarbeit	Gate
AR18c	IDEAL/REAL-Infinity-Envelope	Unendliche Regime klassifizieren: ReferenceThread∞, regulaerer IDEAL∞-Baum, gerichtete REAL∞-Systeme, periodische/quotientierte REAL∞, Variation/Weighted/Measure-Regime.	Keine ∞-Arithmetik ohne Substratklasse, Transferbegriff, Mass/Spur/Fundamentalbereich oder Determinante; keine physikalische Deutung in A.
AR19	Graduierter Traegerseed	Aus finite bright-DtN-, Spektral- und Statedaten eine graduierte Familie mit Level, Energie, Multiplizitaet und Operatorwirkung erzeugen.	Noch keine VOA und kein Monster-Modul; nur graded carrier plus Charaktervorbereitung.
AR20	Charakter-/Modularitaetstests	Graded traces und q-Charaktere mit j, Hauptmoduln und McKay–Thompson-Kandidaten vergleichen; Koeffizientenledger exakt fuehren.	Treffer sind nur Brueckentests. Keine Monster-, Conway- oder VOA-Behauptung ohne Traeger und Aktion.
AR21	Lorentz-/Leech-Gitter-Gate	Nur bei derived-only even/unimodular/rootless-Struktur aus AR19/AR20: 26D-lorentzische Gitter-/Weyl-Vektor-Kandidaten, 24D-Leech-Kandidaten und den Rootless/Chiral-Lattice-Test getrennt formulieren.	Kein Leech-Gitter, kein Weyl-Vektor und keine 24D-Struktur aus Koeffizientenanalogie allein. Root-system-artig und rootless/chiral muessen als verschiedene Action-Status gefuehrt werden.
AR22	VOA-/Kac-Moody-Interface	Vacuum, Grading, State-Field-Map, Locality/Jacobi, Virasoro/conformal vector und Generalized-Kac-Moody-Felder als Interface-Pflichten definieren.	Interface darf leer/offen bleiben. Keine semantische Abkuerzung zu VOA, CFT oder Monster.
AR23	Moonshine-Bruecke	Monster-/Conway-/McKay–Thompson-Bezuege nur bei bewiesenem Traeger, Charakteren und Gruppen-/Monoidwirkung als `ActionPackage` oder klar markiertem externem Theorem-Import.	Monster-/Conwaygruppen sind Pruefobjekte, nicht Grundstruktur. Keine Automorphismengruppe ohne konstruierten Traeger.
AR23a	VOA/Hurwitz-Boundary-Test	`VOAHurwitzBoundaryTest`: bei derived-only VOA/Moonshine-Kandidat Operationen, Graduierung, Formen, Normen und Symmetrien gegen CD/Hurwitz-Grenzdaten testen; zusaetzlich pruefen, dass der Kandidat nur Consumer der AR14–AR17-Weierstrass-/Eisenstein-/q-Provenienz ist; falls ein lattice-/VOA-artiger Seed entsteht, `RootlessChiralActionTest` und `LatticeActionObstruction` ausfuehren.	Keine Aussage: Moonshine folgt aus $\mathbb{O}$; keine Aussage: $\mathbb{O}$ erklaert Monster. Status: compatibility seed, obstruction oder keine Beruehrung; Action-Typ muss root-system-artig, rootless/chiral/sporadisch oder extern-vergleichend markiert sein; kein VOA/Moonshine-Shortcut ohne elliptic-function-first-Kette; elliptische Funktionen und j duerfen Input-Consumer fuer einen spaeteren Kandidaten sein, erzeugen diesen Kandidaten aber nicht automatisch.
AR24	Lie-Typ-/Residuenvergleich	Residuen ueber endlichen Koerpern, polynomiale Gruppengesetze, Permutations- und Lie-Typ-Schatten nur als Wirkungen auf abgeleiteten Residuen-, Modul- oder Charaktertraegern formulieren.	Kein direkter E8-/Lie-Sprung. Bezug muss aus AR19–AR23 oder Residuen folgen und die Action-Disziplin erfuellen.
AR25	Objekt-/Aktions-Konvergenztest	`ConvergenceOutcomeLedger` und `ActionConvergenceLedger`: AR3a, AR3b, AR6a, AR10d, AR12a, AR14, AR15a, AR23a sammeln und Objekt-, Wirkungs-, lokale, root-system-artige, rootless/chirale oder negative Konvergenz entscheiden; zusaetzlich root set, simple-root choice, root lattice, Cartan integers, reflection closure und Weyl action getrennt loggen.	Genau ein Hauptstatus: strong object+action, object-only, action-only, local compatibility/global obstruction oder no convergence; zusaetzlich ein Action-Substatus wie root-system witness/obstruction, rootless-chiral witness/obstruction, sporadic lattice candidate oder external Leech comparison only. Kein spaeter CD-, Symmetrie-, j- oder Moonshine-Prosa-Shortcut. Ein positiver elliptic-function-/j-Befund zaehlt als `ArithmeticGeometryConvergence`, aber nicht als sporadische, Leech-, Conway-, Monster- oder VOA-Konvergenz.
AR26	ArithmeticSeedExport	Duenner A-interner Export: Schur/Charpoly, Ordnung/CM/j, Heegner, q-Charaktere, Graph-Zeta/Gaps, Action-Pakete und AR25-Status.	Export ist pre-handoff und konsumiert keine A-to-* Outputs. Spaetere Pillars sehen Ledgerstatus, nicht vorausgesetzte Einheit.
AR27	Schlussaudit	Importgraph, Assumptions, Computability, Carrier, Parameter, Status, No-go-Ledger und Exportoberflaeche vollstaendig pruefen.	Erst danach gilt AR0–AR27 als geschlossen. AR28 bleibt Fernziel und kein Abschlusskriterium.

Phase	Ziel	Kernarbeit	Gate
AR28	Schur-Multiplier-Gate	Nach realer endlicher Gruppen-, VOA- oder Moonshine-Aktionsoberflaeche pruefen, ob die Aktion linear oder projektiv ist; 2-Kozyklus, Klasse in $H^2(G, \mathbb{C}^\times)$ und ggf. zentralen Cover ableiten.	Der Schur-Multiplier erklaert nicht Schur-Eliminierung. Er ist nur spaeter Darstellungs-/Anomalie-/Zentralerweiterungstest.

2a. L>3-Ausgangsrouten

Ausgang A: quadratischer Heegner-Treffer	Das AR11-Zertifikat liefert einen quadratischen Faktor bzw. eine quadratische Ordnung, deren konvertierte Fundamentaldiskriminante ueber `schur_discrim_to_fundamental_discrim` zu einem positiven Heegner-d in $\{1, 2, 3, 7, 11, 19, 43, 67, 163\}$ gehoert. Folgeschritte: AR12 schliesst Ordnung/Normform/UFD-Konvention; AR12a prueft die Algebra-Kompatibilitaet der Diskriminante; AR13 baut CM-Punkt und Periodengitter; AR14–AR15 berechnen Weierstrassdaten und j; AR15a prueft das CM-Geometrie-Interface; AR16 markiert `HeegnerHitCertificate`; AR17–AR20 duerfen q-/Charaktertests nur als Consumer anschliessen.
Ausgang B: quadratischer Nicht-Heegner-Fall	Das AR11-Zertifikat liefert eine quadratische Ordnung, aber der konvertierte Diskriminanten-/Heegnerstatus ist nicht in der class-number-one-/Heegner-Liste oder bleibt als Nicht-UFD-Fall markiert. Folgeschritte: AR12 schliesst `QuadraticNonHeegnerCertificate`; AR12a darf nur Skalar-/Norm-/Konjugationskompatibilitaet pruefen; AR13–AR15 duerfen CM-/Gitter- und j-Daten weiterhin als quadratische CM-Daten bauen; AR15a markiert Kompatibilitaet, Teilkompatibilitaet oder Obstruktion; AR16 muss den UFD-/Ganzzahligkeitsindikator negativ oder offen markieren; AR18 kann Dedekind-/Dirichlet-/Klassenfeld-nahe Seeds nur nach separater Multiplikativitaets- und Klassenstruktur ableiten.
Ausgang C: irreduzibler oder hoehergradiger Fall	Das AR11-Zertifikat liefert keine quadratische Ordnung als primaeren Faktor: Charpoly irreduzibel ueber $\mathbb{Q}$ oder Faktorgrad > 2. Folgeschritte: keine Weiterleitung in AR12–AR16 als quadratischer CM-Pfad; stattdessen `HigherDegreeFieldSeed/IrreducibleLevelCertificate` als offener Forschungszweig, mit Minimalpolynom-, Diskriminanten-, Galois-/Resolventen- und moeglichen Hilbert-/Siegel-/Automorphie-Fragen. Der L=2/L=3-Hauptpfad bleibt davon unberuehrt.

2b. AR12a-SMOKE: Ontischer Arithmetik-Rauchtest und Berechnungs-Strengthening

Status und Zweck. AR12a-SMOKE wird sofort nach AR12a und vor AR13 eingeschoben. AR13–AR16 werden bis zum Abschluss dieses Blocks als konsumierende Folgephasen behandelt und duerfen keine frei gesetzten CM-, Perioden-, Weierstrass-, j- oder Heegner-Daten verwenden. Der bisherige AR0–AR12a-Pfad hat die algebraische und proof-carrying Form der Arithmetik aufgebaut; AR12a-SMOKE erzwingt jetzt einen echten Rechentest. Die vorhandene Pillar-A-Maschinerie muss aus einem A-internen ontischen ToC-Substrat konkrete arithmetische Daten erzeugen; nach dem v12-Reset beginnt dies zwingend mit SM1R und fuehrt erst ueber SM2R, SM3aR, SM3bR und die SM3cR-Entry-Klassifikation zu einer erlaubten Inversen-/Eliminationsformel. Der Test ist jetzt ein Positive-First-Rechengate: zuerst wird der positive derived-only Pfad konstruiert und berechnet; Falsifikation, Zielabweichung oder Missing*-Status sind nur dokumentierte Rueckfallzweige nach einem praezise formulierten positiven Ziel.

Zusaetzliche Motivation: Der bisherige `IdealThread`-Pfad ist die kanonische Nullpfad-Verduennung eines reicheren Ambient-Substrats. Genau diese Verduennung macht den Pfad kontrollierbar, aber arithmetisch zu arm: sie kann die seitliche Verzweigung, aus der b-/beta-abhaengige Ring-, Gruppen- und Objektstrukturen entstehen muessten, nicht sehen. AR12a-SMOKE prueft deshalb nicht nur technische Berechenbarkeit, sondern die ontologische Frage, ob Zahlen-/Ordnungsstruktur aus einem nicht-singleton ToC-Substrat wirklich als berechnete Struktur hervortritt. Der String-Vergleich bleibt hierbei Motivations- und Fernzielhinweis: ein Faden benoetigt seinen Ambient-/Targetraum; er ist aber kein Beweis einer Stringtheorie-Rekonstruktion.

Der Rauchtest prueft	`RegularCell b` / `LevelVariableCell beta` → generated nicht-singleton ToC-Substrat im Hauptpfad → State / Dirichlet / DtN / GeneralizedDtN / MultiSchur → `LevelHistoryMatrix` aus echten Schur-/DtN-Daten → Charpoly / Faktorstatus / Diskriminante → L=2 Eisenstein, L=3 Gauss, L>3-Ausgaenge → AR12a-CDArithmeticCompatibility aus berechneter Diskriminante.
Nicht erlaubt	`referenceToC` / `variationToC` → `IdealThread` → `CutSpec.full` → `BoundaryPorts` → scheinbare Level-History → freie Testmatrix → manuell gesetzte Diskriminante.
CutSpec-Status	`CutSpec` bleibt im Projekt erhalten und wird nicht verworfen. Es wird nur nicht als Fundament dieses Rauchtests verwendet. Seine moegliche Rolle bleibt Region, Exposition, Defektflaeche, Worldsheet-/Stringtheorie-Vergleich oder spaetere Vergleichsschicht.

2b.1 Grundsatz: Substrat zuerst, Rechnung danach

AR12a-SMOKE beginnt nicht im Arithmetikordner, sondern im Hauptpfad von Pillar A. Wenn der Arithmetikpfad ein reiches Substrat benoetigt, wird dieses Substrat nicht als Arithmetik-Hack erzeugt, sondern als regulaere Pillar-A-Struktur eingebaut:

```
CNNA/PillarA/ToC/GeneratedBranchFamily.lean
CNNA/PillarA/Finite/GeneratedBranchApproximant.lean
CNNA/PillarA/Coupling/GeneratedBranchMultiSchur.lean
```

Der Arithmetikpfad konsumiert diese Strukturen erst danach ueber:

```
CNNA/PillarA/Arithmetic/Smoke/GeneratedSource.lean
```

Die Variation wird zugleich mitgefuehrt: `RegularCell b` und `LevelVariableCell beta`. Es gibt also keinen regulaeren Hauptpfad mit nachtraeglicher Sondervariation, sondern beide generated Substrate werden als Hauptpfad-Objekte vorbereitet.

2b.2 Harte Regeln fuer AR12a-SMOKE

1. Kein `CutSpec.full` im Smoke-Kern. `CutSpec` darf nur in Audit-, Vergleichs- oder Future-Use-Modulen importiert werden.
2. Kein freier Carrier. Der Carrier muss aus `RegularCell b` oder `LevelVariableCell beta` und deren `SubstrateClass`-/ToC-Struktur kommen.
3. Keine freie Matrix. Die operative Matrix muss durch eine Eintragsformel aus `source`, `witnessAt` und `MultiSchur`/DtN-Daten entstehen.
4. Keine freie Diskriminante. Diskriminanten -3, -4 oder L>3-Kandidaten duerfen erst nach Charpoly-/Faktor-/Gradzertifikat auftreten.
5. L2/L3 sind Orakel, nicht Input. Die bisherigen Eisenstein-/Gauss-Matrizen dienen nur als Vergleich und duerfen nicht in die Smoke-Matrix eingespeist werden.
6. L>3 muss einen echten Status erhalten. Erlaubt sind nur `sourceObstruction`, `matrixObstruction`, `charpolyObstruction`, `factorizationObstruction`, `degenerate`, `quadraticHeegner`, `quadraticNonHeegner`, `higherDegreeOrIrreducible`.
7. Positive-First vor Falsifikation: Jede SM-Phase versucht zuerst den positiven derived-only Rechenpfad zu schliessen. Obstruktions-, Falsifikations- oder Missing*-Outcomes duerfen nur als zweiter Statuszweig auftreten, nachdem der positive Pfad mit allen Abhaengigkeiten und Rechenschritten benannt ist. Fuer SM3dbR heisst das verbindlich: keine Anfangsfassung mit `matrixOpt = none` als Hauptausgang, keine Obstruktionsphase vor Kandidatenbau, sondern zuerst SM3db0R–SM3db7R als positive Konstruktionskette.
8. Ein gruen kompilierender API-Lift zaehlt nicht als Smoke-Erfolg, wenn eine zentrale Rechenstelle weiterhin als externe Voraussetzung oder Typklassenannahme offen bleibt. Insbesondere duerfen `RegularGeneratedInteriorEliminationCertificate` und `VariableGeneratedInterior EliminationCertificate` im finalen SM3-Ledger nicht als externe `[...]`-Annahmen stehen.
9. Keine monolithische SM3-Blackbox: Die Kette Partition/Dirichlet/InteriorBlock/Inverse/TwoSidedInv/InteriorEl darf nicht in einer einzigen undurchsichtigen Funktion wie `buildGeneratedGeneralizedDtNLedger` verschwinden. Wiederverwendbare Builder sind erlaubt und erwuenscht, aber jede mathematische

Stufe muss als eigener Datentyp/Konstruktor mit sichtbaren Input-, Output-, Provenienz- und Gleichheitslemmata auftreten.

10. Ledger-Aggregation ist nicht Mathematik: `SM3GeneratedStateDtNLedger` darf die bereits gebauten regular/variable Daten zusammenfassen, aber keine neue Inverse, keinen neuen `TwoSidedInv`-Beweis und kein neues Interior-Zertifikat versteckt erzeugen. Der Ledger-Assembler konsumiert nur Builder-Ausgaben.
11. Zielabweichung ist nicht dasselbe wie Berechnungsblockade: Wenn eine berechnete L2-/L3-Matrix das Eisenstein-/Gauss-Ziel nicht trifft, entsteht `computedTargetMismatch`; wenn die Matrix gar nicht berechnet wird, entsteht `computationBlocked` bzw. ein praeziser Missing*-Status, aber keine mathematische Falsifikation des Ziels.
12. Keine Diagonal-Inversenannahme: Eine diagonal definierte `GeneratedInteriorInverseData`, etwa $\beta^{-1} \cdot I$, ist im Reset kein positiver Default-Kandidat. Sie ist nur im Spezialfall erlaubt, wenn SM3cR zuvor aus Partition, Dirichlet-Eintragsformel, Adjazenz und Weight-/Policy-Provenienz eine echte Diagonalitaet des Interior-Blocks beweist.
13. Produktkalkuel-vor-Zertifikat-Regel: `InteriorEliminationData`, `GeneratedDtN`, `GeneratedGeneralizedDtN`, SM3iR und SM4 duerfen keine Inversen- oder Zertifikatsdaten konsumieren, die nicht durch SM3eR als `provenTwoSidedInv` geschlossen wurden. Nach dem obstruktiven SM3eR-Befund darf auch SM3dbR SM3fR nicht direkt speisen: SM3dbR liefert nur eine neue Inversen-CandidateEntry-Funktion; erst ein erneuter SM3eR-Produktbeweis darf daraus `provenTwoSidedInv` machen. Ein Build mit `candidateFormulaOnly` ist nur noch ein Negativ-/Zwischenbefund, kein Eingang fuer SM3fR.
14. CandidateEntry-Sanity-vor-Produkt-Regel: SM3eR muss vor Links-/Rechtsproduktbeweisen ein `SM3eR-0 CandidateEntryShapeAudit` schliessen. Dieses Gate beweist explizit die von SM3daR bzw. nach SM3dbR die von der neuen CandidateEntry-Schicht gelieferte Form. Der erste SM3eR-Lauf hat den Pflichtbefund `candidateEntry_eq_interiorBlock` geschlossen; damit ist SM3daR als positiver Inversenpfad obstruiert. Positive Produktvalidierung ist dann nur erlaubt, wenn links und rechts wirklich `InteriorBlock * InteriorBlock = 1` bewiesen wird. Scheitert dieser Beweis, entsteht eine formale `GeneratedInteriorTwoSidedInvObstruction`; SM3eR darf keine neue Kandidatenformel erfinden.
15. Obstruktions-Ehrlichkeit nach positivem Versuch: Wenn SM3eR oder ein spaeteres SM3db-Gate zeigt, dass eine konstruierte Formel nicht invers sein kann, ist das kein taktischer Fehler und kein Planbruch. Der aktuelle Befund bleibt: SM3daR liefert den InteriorBlock selbst und ist deshalb als positiver Inversenpfad gesperrt. Daraus folgt aber nicht, dass SM3dbR mit Obstruktion beginnt. SM3dbR muss zuerst den positiven Kandidatenbau SM3db0R–SM3db7R durchlaufen. Ruecksprungziele SM3db1R, SM3db2R, SM3db3R, SM3dR oder SM3cR werden erst nach einem dokumentierten Scheitern dieser positiven Konstruktion aktiviert. Verboten bleibt `Matrix.inv` als nicht hergeleitetes Oracle.

2b.2a Historischer Codebefund und Reset-Baseline

Die bisherigen SM0–SM3d-Zips haben wichtige Diagnoseinformationen geliefert, werden aber ab v12 nicht mehr als positiver Implementierungsstand konsumiert. Der verbindliche Startpunkt ist der AR12a-Codezustand. Die folgenden Befunde bleiben nur als historische Auditliste: Welche Objekte nuetzlich waren, welche Namen/Funktionen vermieden oder neu gebaut werden muessen, und an welcher Stelle der Fehler *Formel vor Entry-Lemma* auftrat.

`ToC/GeneratedBranchFamily.lean`. `generatedBranchFamily Cell` setzt `carrier L = Finset.univ`. Daraus entstehen `regularGeneratedFamily b`, `variableGeneratedFamily beta`, `regularGeneratedToC b` und `variableGeneratedToC beta`. Die Nicht-Singleton-Aussage ist theorematisch nur unter echter Verzweigung verfuegbar: `0 < b` bzw. `0 < beta L` liefert distinkte Zellen auf Level `L+1`.

`ToC/GeneratedBranchVariation.lean`. `GeneratedMainPath`, `regularGeneratedMainPath`, `variableGeneratedMainPath` und `RegularVariableGeneratedPathPair` belegen, dass die va-

riable Seite dieselbe generated ToC-Pipeline nutzt und keine Arithmetik-Sondervariation ist.

`Finite/GeneratedBranchApproximant.lean.` `GeneratedApproximantStrong` enthaelt `toc = ToCStrong.ofIdeal F`, `truncation = F.truncate p.L_max`, Route `generatedToCSubstrate` und `boundaryPortsNotUsedInCore`. Die Traegerlemmata `regularGeneratedApproximant_carrier_of_le` und `variableGeneratedApproximant_carrier_of_le` reduzieren bei `L <= p.L_max` auf `Finset.univ`.

`Arithmetic/Smoke/GeneratedSource.lean.` SM0R ist jetzt geschlossen und bleibt bewusst schmal: Ledger ueber Hauptpfad-Substrat, `noCutSpecFullInCore`, `substrateClassLevelCarrier`, regular/variable Smoke-Carrier und konkrete Nicht-Singleton-Witness-Daten aus der generated Family. Die Witness-Konstruktionen sind `def`s, weil sie Daten in `Type` liefern; eine spaetere Prop-Form waere ein zusaetzliches Existenzlemma und kein Ersatz fuer die konsumierbaren Witness-Daten.

`Arithmetic/Smoke/GeneratedMainPath.lean.` SM1R ist geschlossen. Es fuehrt regular und variable als kanonisches generated Paar mit `sameGeneratedToCPipeline`, `noArithmeticSpecialVariation`, Policy-/Weight-Provenienz-Hooks und negativen Ledger-Feldern gegen freie Matrix-, CutSpec- und Arithmetik-Sonderdaten. SM2R ist ebenfalls geschlossen und erzeugt daraus den endlichen Approximant-Core. SM3aR ist geschlossen und erzeugt daraus die Boundary-/Interior-Partition. SM3bR ist geschlossen und erzeugt daraus die Dirichlet-Entry-Quelle sowie die Matrix als Entry-Funktion. SM3cR ist geschlossen und erzeugt daraus die Interior-Restriktion samt entrywise Strukturprofil. Diese Phasen erzeugen noch keine Inverse, kein TwoSidedInv, keine Interior-Elimination, keine DtN- oder MultiSchur-Rechnung. Ihr Output ist die proof-carrying Hauptpfad-, Approximant-, Partition-, Dirichlet-Entry- und Interior-Block-Provenienz, aus der SM3dR jetzt nur eine strukturabhaengige Kandidaten-/Builder-Schicht ableiten darf.

`Arithmetic/Smoke/GeneratedApproximantCore.lean.` SM2R ist geschlossen. Der wiederverwendbare `GeneratedApproximantStrong`-Core liefert regular/variable Konstruktoren, Route `generatedToCSubstrate`, Carrier aus SM1R-Hauptpfad, Cutoff-/Truncation-Witness, finite Carrier-API, Fintype-/DecidableEq-Zugriff und Sperren gegen BoundaryPorts, CutSpec, Matrix- und Dirichletdaten. SM3aR konsumiert diesen Core erfolgreich partition-only und ohne Matrix-/Dirichlet-Daten.

`Arithmetic/Smoke/GeneratedBoundaryInteriorPartition.lean.` SM3aR ist geschlossen. Die Datei exponiert `GeneratedApproximantIndex`, `GeneratedBoundaryIndex`, `GeneratedInteriorIndex`, `GeneratedBoundaryInteriorPartition`, regular/variable Partition-Konstruktoren, Boundary-/Interior-Carrier, Membership-/Subset-Lemmata, `boundaryInterior_disjoint`, `boundaryInterior_complete`, finite API und `SM3aRGeneratedPartitionLedger`. Diese Daten sind die einzige legale Index- und Restriktionsoberflaeche fuer SM3bR und SM3cR. Sie enthalten selbst keine freie Adjazenzformel, keine freie Weight-Funktion, keine Dirichlet-Matrix, keinen Interior-Block, keine Inverse, keine DtN- oder Diskriminantendaten; diese Daten duerfen nur ueber spaetere proof-carrying Entry-Schichten entstehen.

`Arithmetic/Smoke/GeneratedDirichletEntry.lean.` SM3bR ist geschlossen. Die Datei exponiert `GeneratedAdjacencySource`, `GeneratedWeightPolicyEntrySource`, `generatedForwardAdjacent`, `generatedAdjacent`, `generatedAdjacent_comm`, `generatedEntryWeight`, `generatedDirichlet_degree`, `generatedDirichlet_entry`, `generatedDirichletMatrix`, `generatedDirichletMatrix_entry`, `generatedDirichlet_entry_zero_of_not_adjacent`, `zero_of_not_adjacent`, `generatedDirichlet_entry_offdiag_of_adjacent`, `offdiag_of_adjacent`, `diag_degree_or_regularized_degree`, `generatedBoundaryDirichletEntry`, `generatedInteriorDirichletEntry`, `boundary_entry_from_generated_source`, `interior_entry_from_generated_source`, `GeneratedDirichletEntrySource`, regular/variable Builder und `SM3bRGeneratedDirichletEntryLedger`. Diese Daten sind die einzige legale Dirichlet-Entry- und

Matrixoberflaeche fuer SM3cR. Sie enthalten weiterhin kein Interior-Block-Strukturprofil, keine Inversenformel, kein `TwoSidedInv`, keine Interior-Elimination, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminantendaten.

`Arithmetic/Smoke/GeneratedInteriorBlock.lean`. SM3cR ist geschlossen. Die Datei exponiert `generatedInteriorBlock_entry`, `generatedInteriorBlock`, `generatedInteriorBlock_entry_eq_dirichlet`, `generatedInteriorBlock_from_dirichlet`, `generatedInteriorBlock_zero_of_not_adjacent`, `generatedInteriorBlock_offdiag_of_parent_child_or_coupling`, `generatedInteriorBlock_diag_degree_or_regularized_degree`, `InteriorBlockStructureProfile`, `GeneratedInteriorBlockStructureProfile`, regular/variable Profilkonstruktionen und `SM3cRGeneratedInteriorBlockLedger`. Diese Daten sind die einzige legale Interior-Block- und Profiloberflaeche fuer SM3dR. Der positive Befund ist offdiag-sensitiv und tree-recursive; daraus folgt gerade keine diagonale Default-Inverse. SM3cR enthaelt weiterhin keine Inversenformel, kein `TwoSidedInv`, keine Interior-Elimination, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminantendaten.

Praezise Konsequenz. Die genannten gruenen Zwischenstaende werden nicht fortgeschrieben. Sie zeigen nur, welche API- und Proof-Pflichten im Reset explizit zu erfuellen sind. Insbesondere ist der alte SM3d-Stand mit diagonaler $\beta^{-1} \cdot I$-Inverse kein positiver Kandidat mehr, sondern ein Negativbeispiel: Die Inverse wurde vor der vollstaendigen Entry-Klassifikation des Interior-Blocks formuliert. Der Reset verlangt daher SM1R–SM3cR als geschlossene positive Entry- und Profil-Provenienz. Genau dieser Zustand ist nach dem gruenen SM3cR-Build erreicht; SM3dR darf jetzt beginnen, aber nur als strukturabhaengige Kandidaten-/Builder-Phase ohne Zertifikat und ohne DtN-Konsum.

2b.2b Verbindliche Reset-Reihenfolge fuer SM1R → SM4

Die verbindliche Reihenfolge nach dem AR12a-Baseline-Reset ist:

```
SM0R geschlossen: Minimal-Scaffold, generated carrier, Witness, keine Matrixdaten
SM1R geschlossen: regular+variable generated main path, Policy-/Weight-Provenienz, keine Reche
SM2R geschlossen: GeneratedApproximantStrong-Core, Carrier/Cutoff/Truncation/Finiteness-API
SM3aR geschlossen: Boundary/Interior partition-only aus SM2R, mit Indexgleichheiten fuer Matri
SM3bR geschlossen: Dirichlet-Laplacian entry-source-first, adjacency/weight/matrix no-free
SM3cR geschlossen: Interior-Block neu, vollstaendige entrywise Strukturklassifikation, treeRec
SM3dR geschlossen: structure-derived elimination candidate/builder nur aus SM3cR-Profil, noch
SM3daR geschlossen/obstruiert: CandidateEntryInterface ist derived-only, aber candidateEntry =
SM3eR geschlossen/obstruiert: ShapeAudit + TwoSidedInvObstruction, kein provenTwoSidedInv
SM3db0R geschlossen/gruen: Positive-First-Vertrag und SM3daR-Sperre als Startschutz, keine Obs
SM3db1R geschlossen/gruen: Generated Interior Elimination Carrier ueber GeneratedInteriorIndex
SM3db2R geschlossen/gruen: positiver lokaler generated Eliminationsschritt aus SM3db1R-Carrier
SM3db3R geschlossen/gruen: finite/tree-recursive EliminationTrace mit Terminierung und Herkunf
SM3db4R gruen/partiell: erste Trace-basierte GeneratedInteriorInverseCandidateEntry-Schale, ab
SM3db4aR closed/green: akkumulierte TraceEvaluation / GeneratedInteriorAccumulatedEntryFold ge
SM3db5R geschlossen/gruen: reiner Matrix-Export der SM3db4aR-accumulatedEntry ohne Produktbewe
SM3db6R geschlossen/gruen: Shape-/Source-Separation aus SM3db5R-Matrix; Status-Gate, kein Prod
SM3db7R closed/green: Handoff an erneutes SM3eR-Produktbeweisgate; keine Produktvalidierung
SM3dbOmegaR verboten, solange der positive Handoff-Pfad nicht dokumentiert scheitert
SM3eR-Re-Run next/active: entrywise validation, left/right products gegen H.candidateMatrix, p
SM3fR geschlossen/gruen: generated-lokales InteriorElimination-Certificate nur aus provenTwoSi
SM3gR geschlossen/gruen: generated top-boundary DtN aus SM3fR-Certificate
SM3hR geschlossen/gruen: GeneratedGeneralizedDtN ohne externe Interior-Certificate-Annahmen
SM3iR geschlossen/gruen: Reset-SM3-Ledger als reine Aggregation der neu gebauten Daten
SM4 geschlossen/gruen: GeneratedMultiSchur nur aus SM3iR-Daten, generated-lokal vor public-Bri
SM5 closed/green: generated-ArithmeticSource-Oberflaeche aus SM4; noch keine LevelHistory/Matr
```

```
SM6 closed/green: GeneratedLevelStepWitness aus SM5; noch keine Schur-Index-Bruecke/Matrix/Cha
SM7 closed/green-obstruction: LevelHistory-to-Schur-Index-Gate geschlossen, aber ohne BridgeSu
SM6a closed/green: GeneratedLevelHistoryBoundaryPortSource an der natuerlichen SM6-Entstehungs
SM7BridgeRecheck closed/green: Consumer/Adapter aus SM6a in echten SM7-BridgeSuccess; keine ne
SM8 active: generatedSchurEntry/generateLevelHistoryMatrix als Pullback entlang BridgeSuccess;
```

Die Suffixe R sind Planmarker fuer den Reset. Lean-Namen muessen nicht zwingend dieses Suffix tragen, solange der Code tatsaechlich aus dem AR12a-Baseline-Zustand neu gebaut wird und keine alten SM1–SM3d-Module importiert.

2b.2c Verwerfung der alten SM1–SM3d-Staende als positiver Pfad

Der bisherige Ablauf SM1–SM3d hat kompiliert, aber er hat eine zentrale methodische Luecke sichtbar gemacht: Eine plausible Inversenformel wurde gesetzt, bevor die eintragsweise Form des Interior-Blocks aus der generated Geometrie wirklich als Strukturprofil vorlag. Fuer ein nicht-singleton Substrat mit mehreren Levels ist ein Interior-Block aus der Restriktion des Dirichlet-Laplacians im Allgemeinen nicht diagonal; Parent/Child-Kanten zwischen Interior-Knoten erzeugen Off-Diagonal-Eintraege. Deshalb wird die alte diagonale SM3d-Form nicht repariert, sondern aus dem positiven Pfad entfernt.

Status der alten Artefakte.

- Alte SM1–SM2-Objekte koennen als Namens- und Importinventar dienen, muessen aber als mathematische Provenienz neu gebaut werden.
- Alte SM3a–SM3c-Objekte koennen zeigen, welche Partition-/Dirichlet-/InteriorBlock-Typen nuetzlich waren; ihre DoD wird im Reset verschaerft.
- Alte SM3d-Objekte mit diagonaler $\beta^{-1} \cdot I$-Inverse gelten als `rejected-as-positive` und duerfen nur in einem Obstruktions-/Negativaudit auftauchen.
- Keine alte SM-Phase darf SM3fR, SM4 oder spaetere Smoke-Phasen direkt speisen.

Neue SM3c-Definition-of-Done. SM3cR ist nicht geschlossen, sobald `GeneratedInteriorBlockData` existiert. SM3cR ist erst geschlossen, wenn die eintragsweise Struktur des Blocks aus SM3aR/SM3bR bewiesen ist. Minimal erforderlich sind:

```
generatedInteriorBlock_entry
generatedInteriorBlock_from_dirichlet
generatedInteriorBlock_zero_of_not_adjacent
generatedInteriorBlock_offdiag_of_parent_child_or_coupling
generatedInteriorBlock_diag_degree_or_regularized_degree
InteriorBlockStructureProfile
```

`InteriorBlockStructureProfile` muss mindestens die Faelle `diagonalOnly`, `degreeDiagonal`, `offdiagCoupled`, `treeRecursive`, `generalFiniteEliminationNeeded` und `obstructed` unterscheiden. Ein `unknown`-Status darf nicht in SM3dR konsumiert werden.

Neue SM3dR-Definition-of-Done nach geschlossenem SM3cR. SM3dR darf keine Inversenformel frei waehlen. Die Inversen-/Eliminationskandidatur wird aus dem SM3cR-Profil abgeleitet; im aktuellen positiven Pfad ist der tree-recursive bzw. offdiag-sensitive Fall leitend. SM3dR erzeugt daher noch kein `InteriorEliminationCertificate` und keinen `TwoSidedInv`-Beweis, sondern eine proof-carrying Kandidaten-/Builder-Schicht:

- `diagonalOnly`: diagonale Kandidatur nur mit bewiesenem Diagonalwert und Nullsatz fuer alle Off-Diagonalen.
- `degreeDiagonal`: diagonale Kandidatur mit indexabhaengigem, aus der Geometrie bewiesenem degree-/regularization-Wert.
- `offdiagCoupled` oder `treeRecursive`: rekursive Schur-/Baum-Eliminationskandidatur; keine diagonale Inverse und keine Beta-Inverse als Default.

- `generalFiniteEliminationNeeded`: eigene proof-carrying finite Eliminationskonstruktion ueber dem generated Interior-Index; kein generisches `Matrix.inv`.
- `obstructed`: positiver Pfad stoppt mit formaler Obstruktion, nicht mit Ersatzformel.

Minimaler Zielzustand sind `GeneratedInteriorEliminationCandidate`, ein Profilherkunftslemma, ein `candidateFromTreeRecursiveProfile`-Konstruktor, regular/variable Candidate-Ledger und Sperren gegen `Matrix.inv`, externe Zertifikats-Typklassen, DtN-Konsum und freie Inversenfelder.

Neue SM3daR-Definition-of-Done. SM3daR wird als zwingende Zwischenphase zwischen SM3dR und der ersten SM3eR-Validierung eingefuegt. SM3dR liefert eine Kandidatur; SM3daR muss daraus die auswertbare Candidate-Entry-Oberflaeche erzeugen, die SM3eR fuer Links- und Rechtsproduktbeweise konsumiert. Minimal erforderlich sind:

```
GeneratedInteriorEliminationCandidateEntry
candidateEntry_from_treeRecursiveProfile
candidateMatrix_from_candidateEntry
candidateEntry_profile_eq_SM3dR_source
noMatrixInv_for_candidateEntry
noFreeEntryTable
noExternalInverseOracle
```

Diese Schicht ist weiterhin vor-validierend: sie darf keinen `TwoSidedInv`-Beweis, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminanten erzeugen. Der lokale gruene SM3daR-Build zaehlt deshalb nur als formaler Interface-Abschluss. Nach dem SM3eR-ShapeAudit ist SM3daR als positiver Inversenpfad obstruiert, weil die gelieferte CandidateEntry/CandidateMatrix theorematisch dem SM3cR-InteriorBlock entspricht. Dieser Befund ist kein Lean-Fehler, sondern ein mathematischer Status: SM3daR liefert eine auswertbare Matrix, aber keine Inversenmatrix.

Neue SM3dbR-Definition-of-Done als positive Teilphasen SM3db0R–SM3db7R. SM3dbR wird nicht als pauschale Obstruktionsphase eingefuehrt, sondern als positive Kandidatenbaukette zwischen der gesperrten SM3daR-Oberflaeche und jedem positiven SM3eR-Re-Run. Die Obstruktion ist kein Startmodus, sondern nur das spaete Ledger `SM3dbOmegaR`. Der Hauptzweck ist eine neue, auswertbare und derived-only hergeleitete `GeneratedInteriorInverseCandidateEntry`; SM3eR validiert danach erst Links- und Rechtsprodukt.

SM3db0R – Positive-First-Vertrag und SM3daR-Sperre. SM3daR bleibt als erster CandidateEntry-Shape-Audit formal gruen, aber als positiver Inversenpfad gesperrt. Diese Sperre ist Startschutz und kein Ersatz fuer SM3dbR. SM3dbR muss zuerst eine neue positive `GeneratedInteriorInverseCandidateEntry`-Konstruktion versuchen. Nach geschlossenem SM3db5R darf eine Obstruktion erst nach positivem Versuch von SM3db6R eingetragen werden, wenn die anschliessende Shape-/Source-Separation mathematisch dokumentiert scheitert. Minimaler Zielzustand:

```
SM3daRPositivePathLocked
SM3dbRPositiveFirstContract
noSM3dbRStartsWithObstruction
SM3dbRNeedsNewInverseCandidateEntry
```

Verboten ist `matrixOpt = none` als primaerer SM3dbR-Ausgang. Ein `none`-Status darf nur in `SM3dbOmegaR` erscheinen.

Statusfortschreibung nach gruenem SM3db5R-Build. SM3db0R ist geschlossen/gruen und erfuellt seinen engen DoD: Die Phase sperrt SM3daR als positiven Inversenpfad, erzwingt den Positive-First-Vertrag fuer SM3dbR und legt fest, dass eine neue `GeneratedInteriorInverseCandidateEntry` benoetigt wird. SM3db1R ist geschlossen/gruen und liefert die typisierte, proof-carrying

`GeneratedInteriorEliminationCarrier`-Oberflaeche ueber `GeneratedInteriorIndex A` mit Node-, Depth- und Measure-Provenienz aus SM3cR/SM3dR. SM3db2R ist geschlossen/gruen und liefert den lokalen positiven Eliminationsschritt: Pivot-Datum aus einem SM3db1R-Carrier-Node, Residual-Daten aus dem SM3cR-InteriorBlock und StepUpdate-Daten aus SM3cR/SM3dR-Profil- und Candidate-Provenienz. SM3db3R ist geschlossen/gruen und liefert die finite/tree-recursive `GeneratedInteriorEliminationTrace`. SM3db4R ist formal gruen, aber nur partiell geschlossen: Die neue `GeneratedInteriorInverseCandidateEntry` existiert und ist Trace-basiert, ihre fruehe Entry-Auswertung war noch als lokale `rowResidual`-Verpackung lesbar. Damit war SM3db4R kein gescheiterter Schritt, sondern ein positiver Zwischenbefund, der SM3db4aR erzwang. SM3db4aR, SM3db5R, SM3db6R und SM3db7R sind nun ebenfalls geschlossen/gruen: akkumulierte Entry-Herkunft, Matrix-Export, Shape-/Source-Separation und Handoff liegen vor, aber nicht als Inverse validiert. Der naechste aktive Schritt ist der SM3eR-Re-Run; `SM3dbOmegaR` bleibt verboten, weil der positive Handoff-Pfad bis SM3db7R nicht gescheitert ist.

SM3db1R – Generated Interior Elimination Carrier. SM3db1R konstruiert einen proof-carrying Carrier fuer eine finite/tree-recursive Interior-Elimination ueber `GeneratedInteriorIndex A`. Die neue Datei soll als eigene Stufe beginnen:

```
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorEliminationCarrier.lean
```

Die Eliminationsstruktur stammt aus SM3cR-InteriorBlock-Profil, SM3dR-Kandidatur und generated Adjazenz-/Weight-/Policy-Provenienz. Sie ist keine Matrixinverse, keine freie Pivotliste und kein externes Oracle. Ein Carrier, der nur als frei gesetzte Liste oder Ordnung ueber `GeneratedInteriorIndex A` definiert wird, gilt als Planbruch: Index, Profilquelle, Kandidaturquelle, Mass/Depth und No-Free-/No-Oracle-Ledger muessen im Typ oder in unmittelbaren Konstruktor-/Provenienzfeldern sichtbar sein. Kernobjekte:

```
GeneratedInteriorEliminationNode
GeneratedInteriorEliminationDepth
GeneratedInteriorEliminationMeasure
GeneratedInteriorEliminationCarrier
generatedInteriorEliminationCarrier_from_SM3cR_SM3dR
noFreeEliminationCarrier
noExternalEliminationOrderOracle
```

Strikt nicht Bestandteil von SM3db1R sind:

```
keine Inversen-Entry-Funktion
kein Matrix-Export
kein Produktbeweis
kein TwoSidedInv
kein InteriorEliminationCertificate
kein InteriorEliminationData
kein Matrix.inv
keine freie Pivotliste
kein externes Order-/Trace-/Inverse-Oracle
keine DtN-/GeneralizedDtN-/MultiSchur-Daten
keine Diskriminanten
```

DoD: Es existiert eine typisierte, endliche/proof-carrying Eliminationsoberflaeche, die aus SM3cR/SM3dR herkommt und eine spaetere positive Trace-Konstruktion vorbereiten kann. Noch keine Inversen-Entry-Funktion. Nach lokal gruenem Build ist dieser DoD geschlossen; SM3db1R ist closed/green und darf ab jetzt von SM3db2R als Carrier-Quelle konsumiert werden, aber nicht als Inversen- oder Trace-Quelle.

SM3db2R – positiver lokaler Eliminationsschritt (closed/green). SM3db2R ist nach lokal gruenem Build geschlossen. Die Phase ist vollstaendig umgesetzt, aber nur in ihrem absichtlich engen Scope: Sie definiert einen lokalen generated Eliminationsschritt. Pivot-, Residual- und StepUpdate-Daten stammen nur aus einem SM3db1R-`GeneratedInteriorEliminationCarrier`-Node, dem vorhandenen InteriorBlock-Profil und der generated SM3cR/SM3dR-Struktur. Ein `GeneratedInteriorPivotDatum` ist sichtbar an einen SM3db1R-Carrier-Node gebunden; `GeneratedInteriorResidualDatum` und `GeneratedInteriorStepUpdate` sind lokal aus SM3cR/SM3dR-Profil- und Candidate-Provenienz konstruiert. Der Schritt fuehrt keine globale Pivotordnung, keine freie Pivotliste, keine Trace-Semantik, keine globale Inverse, kein `Matrix.inv` und kein Certificate ein. Kernobjekte:

```
GeneratedInteriorEliminationStep
GeneratedInteriorPivotDatum
GeneratedInteriorResidualDatum
GeneratedInteriorStepUpdate
generatedInteriorEliminationStep_from_profile
generatedInteriorStep_uses_only_SM3cR_SM3dR
noMatrixInv_for_eliminationStep
noFreePivotData
noGlobalPivotOrder_for_eliminationStep
noTraceSemantics_for_eliminationStep
localStep_from_SM3db1R_node
residualDatum_from_SM3cR_SM3dR_profile
stepUpdate_from_SM3cR_SM3dR_candidate
```

Strikt nicht Bestandteil von SM3db2R und daher kein Defekt sind:

```
keine GeneratedInteriorEliminationTrace
keine Terminierung
keine globale Reihenfolge
keine Inversen-Entry-Funktion
kein Matrix-Export
kein TwoSidedInv
keine DtN-/GeneralizedDtN-/MultiSchur-Rechnung
keine Charpoly-/Faktor-/Diskriminantendaten
```

DoD: Die lokale, proof-carrying Eintragsoberflaeche steht positiv und lokal-node-basiert. SM3db2R darf von SM3db3R als Step-Quelle konsumiert werden. Noch kein Produktbeweis, keine Trace, keine globale Pivotordnung, keine Inversen-Entry-Funktion und kein Matrix-Export.

SM3db3R – tree-recursive / finite EliminationTrace (closed/green). SM3db3R baut aus SM3db1R und SM3db2R eine endliche oder tree-recursive Eliminationsspur und ist nach lokal gruenem Build geschlossen. Die Spur ist die erste echte Konstruktionsquelle fuer die spaetere inverse CandidateEntry-Funktion, erzeugt diese aber noch nicht. SM3db3R fuehrt Trace- und Terminierungsdaten; es erzeugt weiterhin keine Inversen-CandidateEntry, keinen Matrix-Export, keinen Produktbeweis und kein `TwoSidedInv`. Die Trace entsteht aus SM3db1R-Carrier-Daten und SM3db2R-Lokalschritten, nicht aus einer externen Pivotliste, einem externen Order-Oracle oder einem Trace-Oracle. Geschlossene Kernobjekte:

```
GeneratedInteriorEliminationTrace
GeneratedInteriorEliminationTraceStep
GeneratedInteriorEliminationTrace_from_treeRecursiveProfile
GeneratedInteriorEliminationTrace_from_finiteCarrier
generatedInteriorEliminationTrace_terminates
generatedInteriorEliminationTrace_source_eq_SM3cR_SM3dR
noExternalTraceOracle
```

Zusaetzliche Schaerfungsobjekte fuer derived-only UND Berechnungen:

```
traceStep_from_localEliminationStep
traceStep_localStep_eq_SM3db2R_step
traceStep_pivot_bound_to_SM3db1R_node
traceStep_residual_eq_SM3cR_interiorBlock
trace_noInverseEntryFunction
trace_noMatrixExport
trace_noTwoSidedInv
```

DoD: Eine positive Trace-Schicht existiert und terminiert in der im Code gewaehlten endlichen/tree-recursive Ordnung. Dieser DoD ist nach lokal gruenem Build geschlossen. Die Schicht bleibt absichtlich unterhalb der Inversen-CandidateEntry: Sie liefert Trace-Provenienz, nicht die Inversenfunktion selbst. Wenn SM3db4R fuer die CandidateEntry-Auswertung staerkere akkumulierte Update-Semantik braucht, wird zuerst eine positive `TraceEvaluation`-/`TraceEntryFold`-Schicht oder eine minimal notwendige Nachschaerfung von SM3db3R/SM3db2R/SM3db1R vorgenommen; eine unmittelbare Obstruktion ist nicht erlaubt.

SM3db4R – erste Trace-basierte Inversen-CandidateEntry-Schale. SM3db4R definiert die erste neue `GeneratedInteriorInverseCandidateEntry` aus der SM3db3R-Eliminationsspur. Diese Schicht ist nach lokal gruenem Build formal vorhanden, aber nur partiell geschlossen: Sie besitzt Trace-Herkunft und No-Free-/No-Oracle-Schutz, exportiert keine Matrix und beweist kein Produkt. Der volle mathematische DoD ist nicht geschlossen, solange die Entry-Auswertung nur als lokale Zeilen-Residualprojektion gelesen werden kann. Insbesondere darf die Form

```
entry i j = (T.traceStepOf i).stepUpdate.rowResidual j
```

nicht als endgueltige Inversen-CandidateEntry gelten. Sie ist derived-only, aber sie akkumuliert die Elimination noch nicht sichtbar. Kernobjekte dieser partiellen Stufe:

```
GeneratedInteriorInverseCandidateEntry
GeneratedInteriorTraceEntryFold
GeneratedInteriorTraceEvaluation
inverseCandidateEntry_source_eq_eliminationTrace_thm
inverseCandidateEntry_source_eq_SM3cR_SM3dR_thm
noMatrixInv_for_inverseCandidateEntry
noFreeInverseEntryTable
noExternalInverseOracle_for_inverseCandidateEntry
SM3db4RGreenButPartial
```

DoD: SM3db4R ist weiterhin nur als `green/partial` zu fuehren. Es gibt eine auswertbare Funktion ueber `GeneratedInteriorIndex A`; die fehlende akkumulierte Auswertung wurde in SM3db4aR geschlossen. SM3db5R hat diese SM3db4aR-accumulatedEntry inzwischen als Matrix verpackt; diese Matrix bleibt ein Kandidat, nicht ein Inversenbeweis.

SM3db4aR – akkumulierte TraceEvaluation / echter Kandidatenkern. SM3db4aR ist nach lokal gruenem Build geschlossen. Die Phase schafft die akkumulierte Auswertungssemantik, die nicht nur fuer jede Zeile den lokalen Trace-Step-Residualwert abliest. Der geschlossene Kern ist ein proof-carrying Fold ueber die SM3db3R-Trace mit expliziter Update-Reihenfolge. Kernobjekte:

```
GeneratedInteriorAccumulatedTraceEvaluation
GeneratedInteriorAccumulatedEntryFold
accumulatedEntry : GeneratedInteriorIndex A -> GeneratedInteriorIndex A -> R
accumulatedEntry_from_eliminationTrace
accumulatedEntry_uses_pivotData
accumulatedEntry_uses_localSteps
```

```
accumulatedEntry_uses_residualData
accumulatedEntry_respects_foldUpdateOrder
inverseCandidateEntry_source_is_accumulatedTraceEvaluation
sm3db4R_not_only_localResidualWrapper
noFreeEntryTable_for_accumulatedEntryFold
noExternalInverseOracle_for_accumulatedEntryFold
```

Falls der im Code verwendete operative Ringtyp nicht `R` heisst, ist `R` durch genau diesen Typ zu ersetzen. SM3db4aR darf weiterhin keinen Matrix-Export, keinen Links-/Rechtsproduktbeweis, kein `TwoSidedInv`, kein `InteriorEliminationCertificate`, keine `InteriorEliminationData`, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminanten erzeugen. DoD: Die exportierbare CandidateEntry stammt theorematisch aus `GeneratedInteriorAccumulatedTraceEvaluation`/`GeneratedInteriorAccumulatedEntryFold`; ein Ledger belegt, dass sie nicht bloss lokaler `rowResidual`-Wrapper ist. Dieser DoD ist nach lokal gruenem Build geschlossen und wurde in SM3db5R als Matrix konsumiert.

SM3db5R – Matrix-Export ohne Produktbeweis (closed/green). SM3db5R ist nach lokal gruenem Build geschlossen. SM3db5R exportiert die akkumulierte CandidateEntry als Matrix. Die Matrix ist nur die entrywise Verpackung der SM3db4aR-Funktion; sie fuehrt keine neue mathematische CandidateEntry-Formel, keine freie Matrix und keine SM3daR-/InteriorBlock-Rueckfuehrung ein. SM3db5R beweist keine Links-/Rechtsinverse und erzeugt kein Interior-Elimination-Certificate. Kernobjekte:

```
inverseCandidateMatrix_from_accumulatedEntry
inverseCandidateMatrix_entry_eq_accumulatedEntry
inverseCandidateMatrix_from_inverseCandidateEntry
inverseCandidateMatrix_entry_eq_inverseCandidateEntry
inverseCandidate_profile_eq_SM3cR_SM3dR_source
noTwoSidedInv_for_SM3dbR
noInteriorEliminationCertificate_for_SM3dbR
noInteriorEliminationData_for_SM3dbR
```

DoD: SM3eR kann spaeter eine konkrete neue Kandidatenmatrix konsumieren; zuvor muss SM3db6R die Shape-/Source-Separation pruefen. SM3db5R ist erfolgreich, weil jeder Matrixeintrag definitionell oder theorematisch als SM3db4aR-accumulatedEntry nachgewiesen ist. Kein Produktbeweis wurde vorweggenommen.

SM3db6R – Shape-/Source-Separation nach positivem Kandidaten (closed/green). SM3db6R ist nach lokal gruenem Build geschlossen. Es bleibt ein nachgelagertes Shape-/Source-Separation-Gate, kein Einstiegsaudit und kein Obstruktionsstart. Die Phase konsumiert die positiv konstruierte SM3db4aR/SM3db5R-Kandidatur, bindet die Matrixeintraege an `generatedInteriorAccumulatedEntryValue` zurueck und fuehrt Status-Gates gegen SM3daR-, InteriorBlock-, localResidual- und freie Tabellen-Rueckfaelle. Wichtig: SM3db6R beweist keine semantische Nichtgleichheit `candidateMatrix != interiorBlock`; die entsprechenden Namen sind Gate-Status und nicht Wert-Ungleichheiten. Kernobjekte:

```
GeneratedInteriorInverseCandidateShapeSeparation
inverseCandidateShapeSeparation_from_matrix
inverseCandidateEntry_not_SM3daR_candidateEntry
inverseCandidateEntry_not_interiorBlock_or_involutiveBlockProof
inverseCandidateMatrix_not_interiorBlock_or_involutiveBlockProof
inverseCandidateMatrix_not_only_localResidualWrapper
inverseCandidateMatrix_entry_eq_accumulatedTraceValue_afterShapeSeparation
SM3dbRShapeSeparationLedger
```

DoD: SM3db6R ist geschlossen, weil die exportierte Kandidatenmatrix als SM3db5R-Matrix und entrywise als SM3db4aR-accumulatedEntry gefuehrt wird, ohne `Matrix.inv`, freie Tabelle, Produktbeweis, `TwoSidedInv`, Certificate, `InteriorEliminationData`, DtN-/MultiSchur-Daten oder Diskriminanten einzufuehren.

SM3db7R – SM3eR-Re-Run-Handoff (closed/green). SM3db7R ist nach lokal gruenem Build geschlossen. Die Phase erzeugt ausschliesslich den Handoff an den erneuten SM3eR-Lauf. Sie macht die SM3db6R-gefilterte Kandidatur fuer SM3eR konsumierbar und schreibt die Consumer-Disziplin fest: SM3eR konsumiert die Handoff-Kandidatur, SM3fR bleibt ohne `provenTwoSidedInv` gesperrt. SM3eR bleibt die erste Phase, die Linksprodukt, Rechtsprodukt, Diagonalterm, Offdiagterm und `provenTwoSidedInv` beweisen darf. Kernobjekte:

```
SM3dbRGeneratedInverseCandidateLedger
SM3dbRToSM3eRHandoff
sm3dbRToSM3eRHandoff_from_shapeSeparation
SM3eRMayConsumeOnlySM3dbRInverseCandidate
handoffCandidate_eq_SM3db6R_shapeSeparationMatrix
handoffCandidate_source_eq_SM3db5R_matrixExport
handoffCandidate_source_eq_SM3db4aR_accumulatedEntry
noLeftProduct_for_SM3db7R
noRightProduct_for_SM3db7R
noTwoSidedInv_for_SM3db7R
noInteriorEliminationCertificate_for_SM3db7R
noInteriorEliminationData_for_SM3db7R
```

DoD: SM3eR kann neu gestartet werden; SM3db7R hat keine Produktvalidierung vorweggenommen; SM3fR bleibt gesperrt. Eine explizit benannte Lean-Deklaration `SM3fRStillBlockedWithoutProvenTwoSidedInv` ist nicht erforderlich, solange die Sperre ueber die Handoff-/NoTwoSidedInv-/NoInteriorEliminationData-Felder theorematisch abgedeckt ist.

SM3dbOmegaR – Obstruktion nur nach positivem Handoff-Scheitern. `SM3dbOmegaR` darf nur ausgefuehrt werden, wenn die positive Handoff-Kette selbst mathematisch dokumentiert scheitert. Nach dem gruenen SM3db7R-Build liegt ein solches Scheitern nicht vor. Ein spaeteres Scheitern der Links-/Rechtsproduktrechnung ist zuerst ein SM3eR-Produktbefund, nicht automatisch ein SM3dbOmegaR-Start. Eine Obstruktion ohne vorherigen positiven Kandidaten-, Shape-Separation- und Handoff-Versuch bleibt Planbruch. Kernobjekte:

```
SM3dbRPositiveAttemptFailedForDocumentedReason
SM3dbRReturnToSM3cR
SM3dbRReturnToSM3dR
SM3dbRReturnToSM3db1R
SM3dbRFormalObstructionAfterPositiveAttempt
```

Neue SM3eR-Definition-of-Done nach SM3dbR. SM3eR bleibt die erste Phase, die `TwoSidedInv` beweisen darf. Der bereits gruene erste SM3eR-Lauf ist geschlossen, aber obstruktiv: Er belegt `candidateEntry_eq_interiorBlock` und erzeugt eine `GeneratedInteriorTwoSidedInvObstruction`, kein `provenTwoSidedInv`. Nach SM3db7R muss SM3eR erneut mit Handoff-ShapeAudit beginnen und dann Linksprodukt, Rechtsprodukt, Diagonalterm und Off-Diagonalterm gegen die SM3cR-Entry-Lemmata und die SM3db7R-Handoff-Matrix `H.candidateMatrix` schliessen. Output ist weiterhin exakt entweder `provenTwoSidedInv` oder eine formale Obstruktion. Ein `candidateFormulaOnly`-Build, ein nur linksseitiger oder nur rechtsseitiger Produktbeweis und die alte SM3daR-InteriorBlock-Kandidatur duerfen SM3fR nicht speisen.

Freigabe. Der naechste Implementierungsschritt ist der neue SM3eR-Re-Run. SM3daR und der erste SM3eR-Lauf sind formal gruen, aber der positive Inversenpfad ist weiterhin nicht bewiesen. SM3db0R–SM3db7R sind geschlossen; sie liefern eine konkrete Handoff-Kandidatenmatrix aus der akkumulierten SM3db4aR-Entry-Herkunft ueber SM3db5R und SM3db6R, aber noch keinen Matrixprodukt- oder Inversenbeweis. SM3fR, SM3gR, SM3hR, SM3iR und SM4 bleiben bis zu einem nach SM3db7R erneut durch SM3eR bewiesenen `provenTwoSidedInv` und der daraus gebauten Interior-Elimination gesperrt.

2b.2d Builder-Auslagerungsregel fuer SM3

Die SM3-Kette ist mathematisch wiederverwendbar, aber nicht als monolithische Funktion. Die Planentscheidung lautet daher:

> kleine, beweistragende, wiederverwendbare Builder fuer jede Rechenstufe; ein SM3-spezifischer Ledger-Assembler nur fuer die Smoke-Zusammenfassung.

Warum keine einzelne Grossfunktion. Eine Funktion der Form `buildGeneratedGeneralizedDtNLedger ...` waere audit-semantisch falsch, wenn sie die kritischen Fragen verdeckt: woher die Partition kommt, welche Interior-Block-Form bewiesen wurde, ob die Inverse explizit ist, ob `TwoSidedInv` wirklich gerechnet wurde und ob `DtN` aus Rechnung oder Voraussetzung entsteht. Genau diese Kette war die SM3-Luecke; sie darf nicht durch eine bequemere API erneut versteckt werden.

Wiederverwendbare Builder-Daten. Folgende Stufen werden als kleine Daten-/Builder-Schicht gefuehrt und duerfen spaeter von SM4, LevelHistory, MultiSchur, L2/L3/L>3 und regular/variable-Vergleichen wiederverwendet werden:

```
GeneratedPartitionData
GeneratedDirichletData
GeneratedInteriorBlockData
GeneratedInteriorEliminationCandidateEntryData
GeneratedInteriorInverseData
GeneratedInteriorEliminationData
GeneratedDtNData
GeneratedGeneralizedDtNData
```

Pflichtkonstruktoren. Die positiven Konstruktoren muessen stufenweise sichtbar bleiben:

```
buildGeneratedPartition
buildGeneratedDirichlet
buildGeneratedInteriorBlock
buildGeneratedInteriorEliminationCandidateEntry
buildGeneratedInteriorInverse
buildGeneratedInteriorTwoSidedInv
buildGeneratedInteriorEliminationData
buildGeneratedInteriorEliminationCertificate
buildGeneratedDtN
buildGeneratedGeneralizedDtN
```

Jeder Builder exponiert mindestens: Eingabedaten, Output, Provenienzfeld oder Provenienzlemma, Komponenten-/Eintragsgleichung und ein `noFree...`-Lemma. Beispiel: `buildGeneratedInterior EliminationCertificate` darf nur `GeneratedDirichletData`, `GeneratedInteriorBlockData`, `GeneratedInteriorInverseData` und den bewiesenen `TwoSidedInv` konsumieren.

SM3-spezifisch bleibt nur die Aggregation. Diese Objekte bleiben im Smoke-Ledger und werden nicht zu allgemeinen mathematischen Basistypen erhoben:

```
SM3GeneratedStateDtNLedger
SM3 status fields
regular/variable smoke assembly
grep/audit expectations
positive-first completion criterion
```

Der Ledger-Assembler darf etwa `buildSM3GeneratedStateDtNLedger` heissen, aber er darf nur vorhandene Builder-Ausgaben zusammenstecken. Er darf keine `infer_instance`-Suche nach Interior-Zertifikaten ausloesen und keine Matrix- oder Inversendaten neu als freie Felder einfuehren.

Explizit verbotene Auslagerungsmuster. Folgende Formen gelten als Planbruch im positiven Pfad:

```
def generatedInteriorCertificate :
    RegularGeneratedInteriorEliminationCertificate ... := by
  infer_instance

variable [RegularGeneratedInteriorEliminationCertificate ...]
variable [VariableGeneratedInteriorEliminationCertificate ...]

def buildGeneratedGeneralizedDtNLedger ... :=
  -- versteckt Partition, Interior-Block, Inverse und TwoSidedInv
```

Zulaessig ist stattdessen nur ein sichtbar komponierter Konstruktor:

```
def generatedInteriorCertificate (...) :
    RegularGeneratedInteriorEliminationCertificate b p regularWeight :=
{
  data := buildGeneratedInteriorEliminationData
    generatedDirichlet
    generatedInteriorBlock
    generatedInteriorBlockInverse
    generatedInteriorBlock_twoSidedInv
  ...
}
```

Modulverantwortung. Die bevorzugte Schichtung ist:

```
Finite/GeneratedPartition.lean
  PartitionData, buildGeneratedPartition
Finite/GeneratedDirichletLaplacian.lean
  DirichletData, buildGeneratedDirichlet
DtN/GeneratedInteriorEliminationBuilder.lean
  InteriorBlockData, InteriorInverseData, TwoSidedInv,
  InteriorEliminationData, Certificate
DtN/GeneratedDtNBuilder.lean
  GeneratedDtNData, buildGeneratedDtN
Coupling/GeneratedGeneralizedDtNBuilder.lean
  GeneratedGeneralizedDtNData, buildGeneratedGeneralizedDtN
Arithmetic/Smoke/GeneratedStateDirichletDtNLedger.lean
  buildSM3GeneratedStateDtNLedger only as assembler
```

Falls bestehende Dateien diese Verantwortung bereits tragen, wird nicht mechanisch umbenannt; dann werden die Builder dort eingefuegt, wo die mathematische Verantwortung liegt. Entscheidend ist die Sichtbarkeit der Stufen, nicht der exakte Dateiname.

Naechster Implementierungskern. Nach dem Reset ist der unmittelbare Implementierungskern SM1R. Die spaetere Builder-Kette bleibt fachlich wiederverwendbar, wird aber neu aus AR12a aufgebaut: SM1R exponiert regular/variable Policy-/Weight-Provenienz, SM2R Carrier/Cutoff, SM3aR Interior-/Boundary-Indizes, SM3bR die Dirichlet-Entry-Formel, SM3cR das Interior-Block-Strukturprofil und erst SM3dR die dazu passende Inversen-/Eliminationsformel. Der wiederverwendbare Kern `GeneratedInteriorEliminationBuilder` darf erst nach SM3cR/SM3dR mit `InteriorBlock -> structure-derived inverse/elimination -> TwoSidedInv -> InteriorEliminationData -> Certificate` gefuellt werden.

2b.2e Reset-Fortschreibung: Entry-Lemmata vor Inversenformeln

Verbindlicher Befund. Bei einem nicht-singleton generated Substrat mit mehreren Levels ist der Interior-Block des Dirichlet-Laplacians erwartungsgemaess nicht automatisch diagonal. Interior-Knoten koennen durch Parent/Child-Kanten verbunden sein; diese Kanten erscheinen als Off-Diagonal-Eintraege in der Interior-Restriktion. Eine Formel $\beta^{-1} \cdot I$ ist daher im allgemeinen Fall nicht ein plausibler positiver Kandidat, sondern nur ein Negativtest fuer den Spezialfall, dass SM3cR wirklich $B = \beta I$ beweist.

Formel-nach-Entry-Prinzip. Fuer jede explizite Matrixphase gilt ab v12:
1. Die Quellmatrix wird als Restriktion oder Konstruktion aus generated Daten definiert.
2. Ein Entry-Lemma der Quellmatrix wird bewiesen.
3. Aus den Entry-Lemmata wird ein Strukturprofil abgeleitet.
4. Erst danach wird eine passende Inversen-, Schur- oder Eliminationsformel definiert.
5. Danach werden Links-/Rechtsprodukt oder Schur-Kompatibilitaet wieder eintragsweise bewiesen.

Das gilt nicht nur fuer SM3dR, sondern auch spaeter fuer LevelHistoryMatrix, Charpoly-Matrixprofile und L2/L3-Vergleichsmatrizen.

SM3eR-Validierung. SM3eR ist kein Reparaturort fuer falsche SM3dR-Formeln. SM3eR validiert nur eine aus SM3cR abgeleitete Inversen-/Eliminationsformel. Die Beweisoberflaeche bleibt eintragsweise:

```
(B.matrix * I.matrix) i j = (1 : Matrix B.Interior B.Interior R) i j
(I.matrix * B.matrix) i j = (1 : Matrix B.Interior B.Interior R) i j
```

Falls Off-Diagonal-Kopplungen die gewaehlte Formel zerstoeren oder das ShapeAudit zeigt, dass die CandidateEntry nur den InteriorBlock selbst reproduziert, wird ein formaler Obstruktionsstatus erzeugt. Der aktuelle SM3eR-Lauf hat genau diesen Obstruktionsfall fuer SM3daR geschlossen. Nach geschlossenem SM3db4aR laeuft der naechste positive Versuch konkret ueber SM3db5R als reiner Matrix-Export der akkumulierten CandidateEntry, danach ueber SM3db6R–SM3db7R und erst dann erneut ueber SM3eR; SM3fR darf ausschliesslich `provenTwoSidedInv` konsumieren.

Kein Workaround-Prinzip. Wenn eine spaetere Phase erkennt, dass eine fruehere Entry-Struktur fehlt, wird nicht lokal gepatcht. Es wird zur fruehesten fehlenden Provenienzstufe zurueckgegangen. Fuer den aktuellen Neustart ist diese Stufe SM1R: regular und variable Pfade, Policy-/Weight-Provenienz und generated Pipeline muessen neu so gebaut werden, dass SM3bR/SM3cR die echte Interior-Block-Struktur beweisen koennen.

2b.2f Fortschreibung nach gruenem SM0R-Build: SM1R-Startvertrag (geschlossen)

Befund. SM0R ist nach lokalem Build geschlossen. Diese Aussage bedeutet nur: Der Reset-Scaffold kompiliert und traegt die erlaubte generated Startoberflaeche. Sie bedeutet nicht, dass bereits gerechnet wurde. Insbesondere sind Policy-/Weight-Provenienz, generated Approximants, Boundary/Interior-Partition, Dirichlet-Entry-Formel, Interior-Block-Profil, Inverse, `TwoSidedInv`, Interior-Elimination, DtN, GeneralizedDtN und MultiSchur noch nicht Bestandteil des positiven Pfads.

SM0R-Abschlussvertrag. SM0R bleibt eine Infrastrukturphase. Geschlossen sind nur:

- generated full-level carrier fuer `RegularCell b` und `LevelVariableCell beta`;
- regular/variable Smoke-Carrier als Konsumoberflaeche fuer den Smoke-Pfad;
- Nicht-Singleton-Witness-Daten fuer echte Verzweigung, als `def`s und nicht als Prop-Theoreme;
- BuildAll-/Notation-Anschluss;
- Ledger-Sperren gegen `CutSpec.full`, freie Matrixdaten und alte SM1–SM3d-Fortschreibung.

Nicht geschlossen sind alle Rechenschritte ab Approximant, Partition, Dirichlet, Interior, Inverse, DtN und MultiSchur.

SM1R-Ziel. SM1R ist jetzt startfreigegeben. Der Zweck ist nicht Rechnung, sondern Provenienzverdrahtung. SM1R muss die regular und variable Seite so als gleichwertige generated Hauptpfade fuehren, dass spaetere Entry-Lemmata nicht zwischen einer echten regular Route und einer arithmetischen Sondervariation unterscheiden muessen. Minimaler Zielzustand:

```
GeneratedMainPath
regularGeneratedMainPath
variableGeneratedMainPath
RegularVariableGeneratedPathPair
sameGeneratedToCPipeline
noArithmeticSpecialVariation
policyProvenanceHook
weightProvenanceHook
noCutSpecCarrierClaim
noMatrixDataYet
SM1RGeneratedVariationLedger
```

SM1R-Sperren. SM1R darf nicht `CutSpec`, `BoundaryPorts`, freie `Fin n`-Ersatzcarrier, `Matrix`, Dirichlet-, Interior-, DtN-, GeneralizedDtN-, MultiSchur-, Charpoly-, Diskriminanten-, CM- oder Zielanker-Daten einfuehren. Keine Phase darf SM1R nachtraeglich als Matrixquelle lesen. SM1R liefert nur die proof-carrying Hauptpfad-Provenienz, aus der SM2R den endlichen Approximant-Core und SM3aR–SM3cR die eintragsweise Rechenstruktur gewinnen muessen.

Policy-/Weight-Provenienz. Die Policy-/Weight-Hooks in SM1R sind keine freien Zahlenwerte und keine externen Orakel. Sie sind Platzhalter bzw. duenne Provenienzfelder, die spaeter auf generated Hauptpfad-Daten zurueckgefuehrt werden muessen. Wenn SM3bR keine Entry-Formel aus diesen Hooks beweisen kann, ist das ein Ruecksprung nach SM1R/SM2R und kein Anlass, in SM3bR eine freie Weight-Funktion zu setzen.

Abhaengigkeitsfolge. Der positive Rechenpfad bleibt damit:

```
SM0R closed scaffold
-> SM1R generated regular/variable provenance
-> SM2R generated finite approximant core
-> SM3aR boundary/interior partition
-> SM3bR Dirichlet entry formula
-> SM3cR InteriorBlock structure profile
-> SM3dR structure-derived candidate/builder layer
-> SM3daR CandidateEntryInterface, now obstructed-as-positive because candidateEntry = Interio
-> SM3eR first ShapeAudit, now closed as formal obstruction, no provenTwoSidedInv
-> SM3dbR genuine inverse CandidateEntry from SM3cR/SM3dR, not InteriorBlock itself
-> SM3eR rerun provenTwoSidedInv or formal obstruction
-> SM3fR certificates as data, not assumptions and only after provenTwoSidedInv
-> SM3gR/SM3hR DtN and GeneralizedDtN
-> SM3iR ledger
```

```
-> SM4 GeneratedMultiSchur
```

Jede Abkuerzung von SM1R direkt zu SM3dR, SM4 oder L2/L3-Vergleichsdaten ist ein Planbruch.

Audit nach SM2R. Nach SM2R muessen lokal geprueft werden: `lake build`, Importgraph, BuildAll/Notation-Anschluss, Grep auf `sorry`/`admit`/freie Axiome/operative `noncomputable`/`classical`/breite `@[simp]`, alte SM-Importe, `CutSpec`-/`BoundaryPorts`-Abhaengigkeit im Smoke-Core und jede versehentliche Matrix-/Dirichlet-/DtN-Einfuehrung. Nach dem gruenen lokalen Build gilt SM2R als geschlossen und SM3aR ist startfreigegeben.

2b.2g Geschlossener SM2R-Approximant-Core-Vertrag

Befund. SM2R ist nach lokalem gruenem Build geschlossen. Geschlossen ist jetzt nicht nur die regular/variable Hauptpfad-Provenienz aus SM1R, sondern auch der finite generated Approximant-Core mit Carrier-, Cutoff-, Truncation- und Finiteness-Provenienz. Weiterhin nicht geschlossen sind Boundary-/Interior-Partition, Dirichlet-Entry-Formel, Interior-Block-Profil, Inverse, `TwoSidedInv`, Interior-Elimination, DtN, GeneralizedDtN und MultiSchur. Deshalb ist SM3aR der naechste Pflichtschritt.

SM2R-Ziel und Abschlusskriterium. SM2R musste einen wiederverwendbaren `GeneratedApproximantStrong`-Core aus dem SM1R-Hauptpfad bauen. Es reicht nicht, ein Feld der Form `toc.approximant p` oder eine bereits existierende Trunkierung bloss weiterzureichen. Der SM2R-Core muss von SM3aR direkt konsumierbar sein, ohne dass SM3aR erneut Carrier-, Cutoff- oder Finiteness-Provenienz rekonstruieren oder freie Indizes einfuehren muss. Minimaler Zielzustand:

```
GeneratedApproximantStrong
regularGeneratedApproximantStrong
variableGeneratedApproximantStrong
approximantRoute = generatedToCSubstrate
carrier_from_mainPath
carrier_eq_univ_of_le
cutoff_from_policy_or_parameter
truncationWitness
finiteCarrierAPI
fintypeCarrier_or_forwardedFintype
DecidableEqCarrier_or_forwardedDecidableEq
noBoundaryPortsInCore
noCutSpecInCore
noMatrixDataYet
noDirichletDataYet
SM2RGeneratedApproximantLedger
```

Carrier-/Cutoff-Pflicht. Der Carrier von SM2R muss aus `RegularCell b` bzw. `LevelVariableCell beta` ueber den generated Hauptpfad kommen. Fuer jedes Level, das innerhalb des Cutoffs liegt, muss ein konsumierbares Carrier-Lemma bereitstehen, im starken Fall eine Gleichung gegen `Finset.univ`. Der Cutoff darf nicht als freier arithmetischer Parameter auftreten, sondern muss aus dem Smoke-Parameter bzw. der Policy-Provenienz kontrolliert sein. Diese Daten sind keine Matrixdaten; sie sind die Finiteness- und Indexgrundlage fuer SM3aR.

Finiteness-API. SM2R muss die endlichen Carrier so exponieren, dass SM3aR Boundary-/Interior-Indizes, Disjointness-/Completeness-Lemmata und spaetere Matrixrestriktionen ohne neue Typklassen-Orakel aufbauen kann. Erlaubt sind direkte Instanzen oder explizite Weiterleitungsfelder fuer `Fintype`, `DecidableEq` und noetige Decidable-Praedikate. Nicht erlaubt ist, dass SM3aR erst freie `Fin n`-Ersatzcarrier einfuehrt, weil SM2R die endliche API nicht bereitstellt.

Sperren. SM2R darf weiterhin keine `CutSpec`, `BoundaryPorts`, Boundary-/Interior-Partition, `Matrix`, Dirichlet-, Interior-, DtN-, GeneralizedDtN-, MultiSchur-, Charpoly-, Faktor-, Diskriminanten-, CM- oder L2/L3-Zieldaten einfuehren. Insbesondere ist `noBoundaryPortsInCore` staerker als eine Importabwesenheit: SM2R darf seine Boundary nicht indirekt ueber `CutSpec.full` oder `BoundaryPorts` lesen. SM2R liefert nur den generated finite approximant core.

Policy-/Weight-Disziplin. Die Policy-/Weight-Hooks aus SM1R bleiben in SM2R Provenienzanker. SM2R darf daraus Cutoff-/Truncation- und spaeter weiterreichbare Weight-Provenienz vorbereiten, aber keine numerische Weight-Funktion frei setzen. Wenn SM3bR aus den weitergereichten Hooks keine Dirichlet-Entry-Formel beweisen kann, ist das ein Rueckssprung nach SM1R/SM2R und kein Anlass, in SM3bR freie Gewichte einzufuehren.

SM2R-DoD. SM2R gilt nach gruenem lokalem Build als geschlossen, weil das Audit bestaetigt: kein `sorry`/`admit`/Axiom, kein operatives `noncomputable`, kein `classical`, keine breite `@[simp]`-Flaeche, kein alter SM-Import, kein CutSpec-/BoundaryPorts-Import im Core, keine Matrix-/DtN-/Dirichletdaten und eine klare Konsumoberflaeche fuer SM3aR. Falls Carrier-, Cutoff- oder Finiteness-Provenienz spaeter wider Erwarten nicht konsumierbar ist, entsteht ein expliziter SM2R-Obstruktionsstatus; SM3aR darf dann nicht mit freien Ersatzindizes beginnen.

2b.2h Fortschreibung nach gruenem SM2R-Build: SM3aR-Partition-Vertrag

Der lokale Build von SM2R war gruen. Damit war SM2R als generated finite Approximant-Core geschlossen. Zu diesem Zeitpunkt waren Carrier-, Cutoff-, Truncation- und Finiteness-Provenienz fuer regular und variable generated Hauptpfade geschlossen; noch nicht geschlossen waren Boundary-/Interior-Partition, Dirichlet-Laplacian, Entry-Formel, Interior-Block, Inverse, `TwoSidedInv`, Interior-Elimination, DtN, GeneralizedDtN, MultiSchur, Charpoly, Faktorisierung, Diskriminante und L2/L3-Zielvergleich. SM2R darf deshalb weiterhin nicht nachtraeglich als Matrixquelle oder BoundarySource gelesen werden.

SM3aR-Ziel. SM3aR war nach SM2R startfreigegeben und ist nach dem gruenen lokalen SM3aR-Build jetzt geschlossen. Das Ziel war die erste echte Rechenvorbereitung aus dem SM2R-Core: eine Boundary-/Interior-Partition des generated Approximant-Carriers. Die Partition ist nur legal, weil jeder Boundary- und Interior-Index ein Subcarrier- bzw. Membership-Zeugnis zum SM2R-Approximanten traegt. Ein freier Index wie `Fin n`, `Fin (L+1)` oder ein aus `CutSpec.full`/`BoundaryPorts` gelesener Boundary-Carrier bleibt verboten.

Minimaler Zielzustand.

```
GeneratedBoundaryInteriorPartition
regularGeneratedBoundaryInteriorPartition
variableGeneratedBoundaryInteriorPartition
GeneratedBoundaryIndex
GeneratedInteriorIndex
boundaryCarrier
interiorCarrier
boundary_subset_approximantCarrier
interior_subset_approximantCarrier
boundary_in_approximant
interior_in_approximant
boundaryInterior_disjoint
boundaryInterior_complete
GeneratedPartitionFiniteAPI
SM3aRGeneratedPartitionLedger
```

Die Namen sind Zielnamen; falls der lokale Code kuerzere Namen erzwingt, muss die gleiche mathematische Oberflaeche sichtbar bleiben: Boundary, Interior, Subset/Membership, Disjointness, Completeness, Fintype, DecidableEq und Decidable-Praedikate.

Partition-only-Sperren. SM3aR darf keine `Matrix`, keine Dirichlet-Laplacian-Entry-Formel, keine Adjazenzformel, keine Weight-Funktion, keinen Interior-Block, keine Inverse, kein `TwoSidedInv`, kein Interior-Elimination-Zertifikat, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminanten- oder Zielanker-Daten einfuehren. Die Phase bereitet Matrixprodukte nur typ- und carrierseitig vor; sie berechnet noch keine Matrixeintraege.

DoD fuer SM3aR. SM3aR gilt jetzt als geschlossen, weil der lokale Build gruen ist und das Audit bestaetigt: kein `sorry`/`admit`/Axiom, kein operatives `noncomputable`, kein `classical`, keine breite `@[simp]`-Flaeche, kein alter SM-Import, kein `CutSpec`- oder `BoundaryPorts`-Import im Partition-Core, keine Matrix-/Dirichlet-/DtN-/Inversendaten und keine freien Ersatzindizes. Falls diese Konsumoberflaeche spaeter wider Erwarten fuer SM3cR nicht ausreicht, entsteht ein expliziter SM3aR- oder SM2R-Ruecksprung; SM3cR darf dann nicht mit einer freien Interior-Tabelle oder einem freien InteriorIndex kompensieren.

2b.2i Fortschreibung nach gruenem SM3bR-Build: SM3cR-Interior-Block-Vertrag

Der lokale Build von SM3bR ist gruen. Damit ist SM3bR als Dirichlet-Entry-Source-Schicht des generated finite Approximant-Cores geschlossen. Geschlossen sind jetzt die legalen Boundary- und Interior-Indizes aus SM3aR sowie die Dirichlet-Entry-Quelle aus generated Adjazenz, Weight-/Policy-Provenienz und `generatedDirichlet_entry`. Nicht geschlossen sind weiterhin Interior-Block-Strukturprofil, Inverse, `TwoSidedInv`, Interior-Elimination, DtN, GeneralizedDtN, MultiSchur, Charpoly, Faktorisierung, Diskriminante und L2/L3-Zielvergleich. SM3bR darf deshalb nicht nachtraeglich als Interior-Block- oder Inversenquelle gelesen werden; SM3cR muss die Interior-Struktur eintragsweise neu aus SM3aR/SM3bR beweisen.

SM3bR-Abschluss. SM3bR ist nach lokal gruenem Build geschlossen. Die erste echte Entry-Rechnung aus dem positiven Reset-Pfad liegt vor: Die Dirichlet-Matrix wird nicht als freies Feld eingefuehrt, sondern als `generatedDirichletMatrix` aus `generatedDirichlet_entry`. Die Entry-Quellen sind aus SM2R/SM3aR angebunden: generated Carrier, Boundary-/Interior-Partition, Parent/Child-/Level-Adjazenz und Weight-/Policy-Provenienz. Damit ist die Quelle-vor-Formel-Pflicht fuer die Dirichlet-Ebene erfuellt, aber noch keine Aussage ueber die Interior-Block-Struktur getroffen.

Erlaubte Entstehungskette.

```
SM2R GeneratedApproximantStrong
-> SM3aR GeneratedBoundaryInteriorPartition
-> generated adjacency source
-> generated weight-policy entry source
-> generatedDirichlet_entry
-> Matrix i j := generatedDirichlet_entry i j
```

Diese Kette ist gerichtet. SM3bR darf weiterhin nicht rueckwirkend als freie Matrix gelesen werden. SM3cR konsumiert ausschliesslich die SM3bR-Entry-Oberflaeche und darf weder aus einer frei gesetzten Interior-Block-Tabelle noch aus einer plausiblen Strukturannahme auf die Eintraege zurueckschliessen. Wenn die Interior-Restriktion aus SM3aR/SM3bR nicht beweisbar klassifizierbar ist, entsteht ein Ruecksprung zu SM3bR oder SM3aR, nicht die Erlaubnis fuer eine freie Interior-Formel.

Geschlossener Zielzustand fuer SM3bR.

```
GeneratedAdjacencySource
GeneratedWeightPolicyEntrySource
generatedDirichlet_entry
generatedDirichletMatrix
zero_of_not_adjacent
offdiag_of_adjacent
diag_degree_or_regularized_degree
boundary_entry_from_generated_source
interior_entry_from_generated_source
noFreeAdjacency
noFreeWeight
noFreeMatrix
SM3bRGeneratedDirichletEntryLedger
```

Die Namen sind Zieloberflaechen, nicht zwingend exakte Lean-Namen. Entscheidend ist, dass die Matrixeintraege aus den sichtbaren Quellen abgeleitet sind und dass die Entry-Lemmata vor jeder geschlossenen Matrix- oder Profilformel stehen.

SM3cR-Ziel. SM3cR ist jetzt startfreigegeben und kritisch. Es muss die Interior-Restriktion der SM3bR-Matrix bauen und deren Eintraege aus der bereits bewiesenen Dirichlet-Entry-Quelle ableiten. Minimaler Zielzustand:

```
generatedInteriorBlock_entry
generatedInteriorBlock_from_dirichlet
generatedInteriorBlock_zero_of_not_adjacent
generatedInteriorBlock_offdiag_of_parent_child_or_coupling
generatedInteriorBlock_diag_degree_or_regularized_degree
InteriorBlockStructureProfile
SM3cRGeneratedInteriorBlockLedger
```

Das Profil muss mindestens `diagonalOnly`, `degreeDiagonal`, `offdiagCoupled`, `treeRecursive`, `generalFiniteEliminationNeeded` und `obstructed` unterscheiden. Ein `unknown`-Profil ist kein positiver Output und darf SM3dR nicht speisen.

SM3cR-Abschlussregel. SM3cR ist nicht dadurch geschlossen, dass ein Objekt wie `GeneratedInteriorBlockData` oder eine Interior-Restriktionsmatrix existiert. SM3cR ist erst geschlossen, wenn die eintragsweise Struktur des Blocks aus SM3aR/SM3bR bewiesen und klassifiziert ist. Parent/Child-Kopplungen zwischen Interior-Knoten muessen entweder als Off-Diagonal-Terme nachgewiesen oder theorematisch ausgeschlossen werden. Erst dieser Befund entscheidet, ob SM3dR eine diagonale, degree-diagonale, rekursive oder allgemeine finite Eliminationsformel verfolgen darf.

Sperren in SM3bR. SM3bR darf keine Interior-Block-Strukturklassifikation, keine Inversenformel, kein `TwoSidedInv`, kein Interior-Elimination-Zertifikat, keine DtN-/GeneralizedDtN-/MultiSchur-Daten, keine Charpoly-/Faktor-/Diskriminanten- oder L2/L3-Zielanker-Daten einfuehren. SM3bR berechnet Matrixeintraege; SM3cR klassifiziert danach erst die Interior-Block-Struktur. Insbesondere bleibt jede diagonale Interior-Inverse oder jede andere geschlossene Inversenformel verboten, solange SM3cR nicht die tatsaechliche Interior-Entry-Struktur beweist.

Geschlossener DoD fuer SM3bR. SM3bR gilt jetzt als geschlossen, weil der lokale Build gruen ist und das Audit bestaetigt: kein `sorry`/`admit`/Axiom, kein operatives `noncomputable`, kein `classical`, keine breite `@[simp]`-Flaeche, kein alter SM-Import, keine freie Matrix, keine freie Adjazenz, keine freie Weight-Funktion, kein CutSpec-/BoundaryPorts-Fundament, keine Inverse und keine DtN-Daten. Die Dirichlet-Matrix muss definitionell oder theorematisch aus `generatedDirichlet_entry` stammen, und `generatedDirichlet_entry` muss seinerseits auf SM2R/SM3aR plus generated Adjazenz-/Weight-Provenienz zurueckgefuehrt sein.

2b.2j Fortschreibung nach gruenem SM3dR-Build: SM3dR-/SM3daR-/SM3eR-Vertrag

Der lokale Build von SM3dR ist gruen. Damit ist SM3dR als Candidate-/Builder-Schicht aus dem SM3cR-Profil geschlossen. Geschlossen sind jetzt die Dirichlet-Entry-Quelle aus SM3bR, die Interior-Block-Entry-Oberflaeche aus SM3cR und die strukturabhaengige Kandidatur aus SM3dR. Nicht geschlossen sind weiterhin eine auswertbare Candidate-Entry-Funktion, `TwoSidedInv`, Interior-Elimination, DtN, GeneralizedDtN, MultiSchur, Charpoly, Faktorisierung, Diskriminante und L2/L3-Zielvergleich. SM3dR darf deshalb nicht nachtraeglich als auswertbare Inversenquelle gelesen werden; SM3daR muss zuerst eine CandidateEntryInterface-Schicht aus der SM3dR-Kandidatur ableiten.

SM3cR-Abschluss. SM3cR ist nach lokal gruenem Build geschlossen. Die erste echte Interior-Block-Klassifikation aus dem positiven Reset-Pfad liegt vor: Der Interior-Block wird nicht als freie Tabelle eingefuehrt, sondern als Restriktion der SM3bR-Dirichlet-Matrix. Die Entry-Quelle ist aus SM2R/SM3aR/SM3bR angebunden: generated Carrier, Boundary-/Interior-Partition, Parent/Child-/Level-Adjazenz, Weight-/Policy-Provenienz und `generatedDirichlet_entry`. Damit ist die Entry-Lemma-vor-Formel-Pflicht fuer die Interior-Block-Ebene erfuellt. Der aktive Profilbefund ist nicht diagonal, sondern tree-recursive/offdiag-sensitiv.

Erlaubte Entstehungskette fuer SM3dR.

```
SM2R GeneratedApproximantStrong
-> SM3aR GeneratedBoundaryInteriorPartition
-> SM3bR generatedDirichlet_entry / generatedDirichletMatrix
-> SM3cR generatedInteriorBlock_entry / InteriorBlockStructureProfile
-> SM3dR GeneratedInteriorEliminationCandidate
-> SM3daR GeneratedInteriorEliminationCandidateEntry / candidateMatrix
```

Diese Kette ist gerichtet. SM3dR konsumiert ausschliesslich die SM3cR-Profil- und Ledger-Oberflaeche; SM3daR konsumiert ausschliesslich SM3dR-Kandidatur und SM3cR-Herkunftsdaten. SM3dR darf nicht aus einer frei gesetzten Matrix, nicht aus einer freien Inversentabelle und nicht aus einer plausiblen diagonalen Formel auf eine Eliminationskandidatur schliessen. Wenn das SM3cR-Profil keine beweistragende Kandidatur erlaubt, entsteht eine Kandidatenobstruktion, nicht die Erlaubnis fuer `Matrix.inv`.

SM3dR-Ziel. SM3dR ist nach lokal gruenem Build geschlossen. Es baut eine strukturabhaengige Kandidaten-/Builder-Schicht. Minimaler Zielzustand:

```
GeneratedInteriorEliminationCandidate
GeneratedInteriorEliminationCandidateSource
candidate_from_diagonalOnly
candidate_from_degreeDiagonal
candidate_from_treeRecursiveProfile
candidate_from_generalFiniteEliminationProfile
candidate_obstructed
candidate_profile_eq_SM3cR_profile
noMatrixInv
noFreeInverse
noExternalInteriorEliminationCertificate
noDtNDataYet
SM3dRGeneratedInteriorEliminationCandidateLedger
```

Die Namen sind Zieloberflaechen, nicht zwingend exakte Lean-Namen. Entscheidend ist, dass SM3dR nur eine Kandidatur konstruiert. `TwoSidedInv`, `InteriorEliminationData`, `GeneratedDtN`, `GeneratedGeneralizedDtN`, `MultiSchur`, Charpoly und Diskriminanten bleiben verboten.

SM3dR-Fallvertrag.

- `diagonalOnly`: Kandidatur nur mit SM3cR-Diagonalitaetsbeweis und explizitem Diagonalwert; keine Annahme, dass dieser Fall vorliegt.
- `degreeDiagonal`: Kandidatur nur mit indexabhaengigem Degree-/Regularization-Wert aus SM3bR/SM3cR; keine globale Konstante ohne Beweis.
- `treeRecursive`: positiver Hauptfall; Kandidatur als rekursive Schur-/Baum-Eliminationsdatenstruktur oder als finite, proof-carrying Eliminationskonstruktion ueber dem generated Interior-Index.
- `offdiagCoupled`: keine diagonale Kandidatur; entweder Reduktion auf den tree-recursive Fall durch bewiesene Strukturlemmata oder expliziter Wechsel zu finite proof-carrying elimination.
- `generalFiniteEliminationNeeded`: nur eine eigens definierte, pruefbare finite Eliminationskonstruktion; kein `Matrix.inv` und kein Oracle.
- `obstructed`: Abschluss als Obstruktionsledger; Ruecksprungziel SM3cR/SM3dR.

SM3daR-Ziel. SM3daR ist die Brueckenphase zwischen SM3dR-Kandidatur und dem ersten SM3eR-ShapeAudit. Sie erzeugt keine Inverse im Sinne eines bewiesenen `TwoSidedInv`, sondern nur eine auswertbare Entry-Oberflaeche. Minimaler Zielzustand:

```
GeneratedInteriorEliminationCandidateEntry
candidateEntry_from_treeRecursiveProfile
candidateMatrix_from_candidateEntry
candidateEntry_profile_eq_SM3dR_source
noMatrixInv_for_candidateEntry
noFreeEntryTable
noExternalInverseOracle
```

Die Namen sind Zieloberflaechen, nicht zwingend exakte Lean-Namen. Entscheidend ist, dass `candidateEntry` und `candidateMatrix` aus SM3dR-Kandidatur und SM3cR/SM3dR-Profilherkunft stammen. Eine freie Entry-Tabelle, `Matrix.inv`, ein externes Inversen-Oracle oder ein verstecktes Interior-Elimination-Certificate sind Planbruch.

Fortschreibung nach gruenem SM3daR- und SM3eR-Build. SM3daR ist nach lokal gruenem Build formal geschlossen, aber seit dem SM3eR-ShapeAudit nicht mehr als positiver Inversenpfad zu lesen. Der kritische bewiesene Befund lautet: Die von SM3daR gelieferte CandidateEntry/CandidateMatrix ist theorematisch der SM3cR-InteriorBlock selbst. Damit ist SM3daR als Interface-Schicht korrekt, aber als Inversen-CandidateEntry obstruiert. Der erste SM3eR-Lauf ist ebenfalls gruen geschlossen, aber nur obstruktiv: `SM3eR-0 CandidateEntryShapeAudit` und `GeneratedInteriorTwoSidedInvObstruction` sind geschlossen; `provenTwoSidedInv` wurde nicht erzeugt.

SM3dbR-Ziel: positive Kette SM3db0R–SM3db7R. SM3dbR ist die neu eingefuehrte positive Kandidatenbaukette vor jedem SM3eR-Re-Run. Sie darf nicht in SM3eR reparieren, darf SM3fR nicht direkt speisen und darf nicht mit einer Obstruktion beginnen. Ziel ist eine echte derived-only Inversen-/Eliminations-CandidateEntry-Funktion, die aus dem SM3cR-InteriorBlock-Profil, der SM3dR-Kandidatur und einer endlichen bzw. tree-recursive Eliminationsspur abgeleitet wird. Die verbindliche Unterteilung ist:

```
SM3db0R  closed/green: Positive-First-Vertrag + SM3daR-Sperre als Startschutz
SM3db1R  closed/green: GeneratedInteriorEliminationCarrier
SM3db2R  closed/green: GeneratedInteriorEliminationStep from SM3db1R Carrier Node
SM3db3R  closed/green: GeneratedInteriorEliminationTrace
SM3db4R  green/partial: Trace-local GeneratedInteriorInverseCandidateEntry shell
SM3db4aR closed/green: GeneratedInteriorAccumulatedTraceEvaluation / AccumulatedEntryFold
SM3db5R  closed/green: inverseCandidateMatrix_from_accumulatedEntry
```

```
SM3db6R  closed/green: GeneratedInteriorInverseCandidateShapeSeparation
SM3db7R  closed/green: SM3dbRToSM3eRHandoff without product validation
SM3eR-r0--r2 prepared/green; SM3eR-r3a closed/green; SM3eR-r3b closed/green as conditional Can
SM3dbOmegaR forbidden unless documented positive handoff-chain failure
```

Minimaler Zielzustand der positiven Kette:

```
SM3dbRPositiveFirstContract
GeneratedInteriorEliminationCarrier
GeneratedInteriorEliminationStep
GeneratedInteriorEliminationTrace
GeneratedInteriorInverseCandidateEntry
inverseCandidateEntry_from_treeRecursiveElimination
inverseCandidateEntry_from_finiteEliminationTrace
inverseCandidateMatrix_from_accumulatedEntry
inverseCandidateMatrix_entry_eq_accumulatedEntry
inverseCandidateMatrix_from_inverseCandidateEntry
inverseCandidateMatrix_entry_eq_inverseCandidateEntry
inverseCandidate_profile_eq_SM3cR_SM3dR_source
GeneratedInteriorInverseCandidateShapeSeparation
inverseCandidateShapeSeparation_from_matrix
inverseCandidateMatrix_entry_eq_accumulatedTraceValue_afterShapeSeparation
SM3dbRShapeSeparationLedger
inverseCandidateEntry_not_SM3daR_candidateEntry
inverseCandidateEntry_not_interiorBlock_or_involutiveBlockProof
noMatrixInv_for_inverseCandidateEntry
noFreeInverseEntryTable
noExternalInverseOracle_for_inverseCandidateEntry
noTwoSidedInv_for_SM3dbR
noInteriorEliminationCertificate_for_SM3dbR
noInteriorEliminationData_for_SM3dbR
SM3dbRGeneratedInverseCandidateLedger
SM3dbRToSM3eRHandoff
```

Die neue Inversen-CandidateEntry darf nicht bloss `generatedInteriorBlock` wiederholen. SM3db6R hat diesen Punkt als Source-/Shape-Gate geschlossen, ohne daraus eine semantische Wert-Ungleichheit oder einen Produktbeweis zu machen. Falls spaeter im SM3eR-Re-Run eine Gleichheit mit dem Interior-Block produktrelevant wird, ist sie nur akzeptabel, wenn der involutive Spezialfall `InteriorBlock * InteriorBlock = 1` links und rechts aus SM3cR/SM3dR bewiesen wird. Verboten bleiben `Matrix.inv`, freie Inversen-Entry-Tabellen, externe Inversen-Oracles, operative `noncomputable`-Hilfen, `classical`-Abkuerzungen, pauschale `@[simp]`-Flaechen, `TwoSidedInv` ausserhalb von SM3eR, Interior-Elimination-Certificate, `InteriorEliminationData`, DtN-/GeneralizedDtN-/MultiSchur-Daten und Diskriminanten. Eine formale `SM3dbOmegaR`-Obstruktion darf nur nach dokumentiertem positivem Scheitern der Handoff-Kette selbst eingetragen werden; ein Produktscheitern im ReRun ist zunaechst ein SM3eR-Befund.

Abschluss SM3db7R und naechster Codeauftrag. SM3db7R ist nach lokal gruenem Build geschlossen. Die Handoff-Schicht konsumiert ausschliesslich die geschlossene SM3db6R-ShapeSeparation, damit mittelbar die SM3db5R-Kandidatenmatrix und die SM3db4aR-`accumulatedEntry`-Herkunft. Der naechste Codeauftrag ist nicht SM3fR und nicht ein direkter `provenTwoSidedInv`-Beweis, sondern die Fortsetzung des neuen SM3eR-Re-Runs in:

```
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorTwoSidedInvFromSM3db7R.lean
```

Der aktuell vorhandene ReRun-Code ist nur *handoffProductCalculusPrepared*. Legal vorbereitet sind:

```
GeneratedInteriorSM3db7RLeftProductEntry
GeneratedInteriorSM3db7RRightProductEntry
generatedInteriorSM3db7RLeftProductEntry_eq_matrixProduct
generatedInteriorSM3db7RRightProductEntry_eq_matrixProduct
GeneratedInteriorSM3db7RHandoffShapeAudit
GeneratedInteriorSM3db7RLeftProductAudit
GeneratedInteriorSM3db7RRightProductAudit
GeneratedInteriorSM3db7RProductIdentityProof
provenTwoSidedInvFromSM3db7R
GeneratedInteriorSM3db7RProductProofGate
SM3eRReRunGeneratedTwoSidedInvLedger
```

Diese Liste ist kein Abschluss der Produktidentitaet. `provenTwoSidedInvFromSM3db7R` ist nur ein Eliminator aus einem bereits vorhandenen `GeneratedInteriorSM3db7RProductIdentityProof`. Der Proof selbst fehlt noch.

SM3eR-r0 bis r5: verbindliche ReRun-Unterteilung.

```
SM3eR-r0  prepared/green: HandoffShapeAudit
SM3eR-r1  prepared/green: LeftProductEntry = Block W * H.candidateMatrix
SM3eR-r2  prepared/green: RightProductEntry = H.candidateMatrix * Block W
SM3eR-r3a closed/green: ProductNormalForm, sums over accumulatedEntry
SM3eR-r3b closed/green: conditional CancellationSurface / ledger
SM3eR-r3c0 closed/green: TraceCancellationSource / Derivation API / ledger export
SM3eR-r3c1a closed/green: BlockFold normal form
SM3eR-r3c1b0 closed/green: LocalStepSchurCancellation source audit
SM3db2aR closed/green-relative: LocalStepSchurCancellationInvariant
SM3db2bR closed/green-relative: TracePivotScaleWitness consumer
SM3db2cR audit-closed/conditional: ScaleSource adapter
SM3db2dR green-relative: ReciprocalProfile adapter
SM3db2eR0 closed/green: GeneratedDegreeFormulaAudit
SM3db2eR-full closed/green: ReciprocalProfileFromWeightPolicy audit
SM3db2fR closed/green: GeneratedOperationalDegreeReciprocalSource
SM3eR-r3c1b1 closed/green: BlockFoldKernel-to-LocalSchurResidualBridge
SM3eR-r3c1b-full closed/green: LocalStepCancellation
SM3eR-r3c1c closed/green: FoldInvariance
SM3eR-r3c1d green/gate: TerminalIdentity
SM3db3aR closed/green-audit: TracePivotTerminalCoverageSource
SM3db4aT closed/green-audit: TerminalFoldResidualCoverage
SM3db4aU closed/green-audit: TerminalIdentityFields-Audit
SM3db4aV closed/green: TerminalFoldResidualIdentityEquationSource
SM3db4aW closed/green: TerminalIdentityFieldCertificateFromEquationSource
SM3eR-r3c1d-full closed/green: TerminalCoverageSource -> TerminalIdentity
SM3eR-r3c1e closed/green after r3c1d-full
SM3eR-r3c2 closed/green consumer after r3c1e
SM3eR-r4a closed/green: ProductCancellationLedger -> ProductIdentityProof
SM3eR-r4 collapsed/closed-by-r4a; SM3eR-r5 active
SM3fR blocked until provenTwoSidedInv
```

Der naechste mathematische Implementierungsschritt ist nach v54 `SM3eR-r3c1d-full`. Die r3c1-Full-Route wurde durch die lokale Schur-/Scale-Subkette, die FoldInvariance-Schicht, das r3c1d-TerminalIdentity-Gate, den SM3db3aR-TracePivot-Coverage-Audit, SM3db4aT-TerminalFoldResidualCoverage, SM3db4aU-TerminalIdentityFields-Audit, SM3db4aV-TerminalFoldResidualIdentityEquationSource und den gruenen SM3db4aW-TerminalCoverageSource-Export verfeinert. r3c1d-full muss nun die aus W exportierten SM3db4aU-Certificates und

r3c1d-TerminalCoverageSources konsumierend in den TerminalIdentity-Witness ueberfuehren; danach folgt r3c1e. Ab v54 gilt zusaetzlich das Audit-Dichte-Gate: reine Handoff-/Adapterphasen duerfen nicht automatisch weitere Status-Enum- und Ledger-Familien vermehren.

SM3eR-r3a: abgeschlossene Produktnormalformphase. Nach gruenem lokalem Build sind die zulaessigen Normalform- und Ledgerobjekte geschlossen, z.B.:

```
generatedInteriorSM3db7RLeftProductEntry_eq_sum_accumulated
generatedInteriorSM3db7RRightProductEntry_eq_sum_accumulated
leftProductEntry_expand_handoffCandidate
rightProductEntry_expand_handoffCandidate
SM3eRProductNormalFormLedger
```

Inhaltlich muessen die Normalformen mindestens die Form tragen:

$$\mathrm{Left}_{ij} = \sum_k \mathrm{generatedInteriorBlock}\, W\, ik \cdot \mathrm{generatedInteriorAccumulatedEntryValue}\, T\, kj,$$

$$\mathrm{Right}_{ij} = \sum_k \mathrm{generatedInteriorAccumulatedEntryValue}\, T\, ik \cdot \mathrm{generatedInteriorBlock}\, W\, kj.$$

Die exakten Lean-Namen duerfen abweichen, aber der Inhalt darf nicht abgeschwaecht werden. Der Beweis ging ueber die vorhandenen Handoff-/Shape-/Matrix-/AccumulatedEntry-Lemmata, nicht ueber freie Tabellen. r3a erzeugt weiterhin keinen ProductIdentityProof und keinen TwoSidedInv.

SM3eR-r3b: geschlossene bedingte Kancellationsoberflaeche. r3b ist nach lokal gruenem Build als bedingte Kancellationsoberflaeche geschlossen. Die sechs Felder, die spaeter in `GeneratedInteriorSM3db7RProductIdentityProof` eingehen, sind relativ zu einem `GeneratedInteriorSM3db7RTraceCancellationInvariant` bzw. einem ProductCancellationLedger vorbereitet:

```
leftProduct_entry_eq_identity
rightProduct_entry_eq_identity
leftProduct_diagonal_entry
leftProduct_offdiag_entry
rightProduct_diagonal_entry
rightProduct_offdiag_entry
```

Diese Felder duerfen fuer r4 nicht als freie Prooffelder konsumiert werden. r3b belegt die Oberflaeche; r3c0 verpackt die Derivation-API, aber r3c1 muss die Invarianz ableiten. Ein freier Proof, ein freies Ledgerfeld, eine externe Inversenannahme, `Matrix.inv`, eine neue CandidateEntry-Formel, ein SM3daR-Rueckfall oder ein Rechenorakel sind Planbruch. Geschlossene bzw. zulaessige r3b-Zwischenoberflaeche:

```
GeneratedInteriorSM3db7RTraceCancellationLedger
leftProductNormalForm_diagonal_eq_one
leftProductNormalForm_offdiag_eq_zero
rightProductNormalForm_diagonal_eq_one
rightProductNormalForm_offdiag_eq_zero
leftProductEntry_eq_identity_from_normalFormCancellation
rightProductEntry_eq_identity_from_normalFormCancellation
```

SM3eR-r3c0: gruen geschlossene TraceCancellationDerivation-API. r3c0 ist nach gruenem lokalem Build als Schnittstellenphase geschlossen. Ziel und Ertrag sind nicht ein neuer ProductIdentityProof, sondern die proof-carrying Verpackung der Kancellationsableitung:

```
GeneratedInteriorSM3db7RTraceCancellationSource
GeneratedInteriorSM3db7RTraceCancellationDerivation
GeneratedInteriorSM3db7RTraceCancellationInvariant
```

```
traceCancellationInvariant_from_traceDerivation
productCancellationLedger_from_traceCancellationDerivation
regularTraceCancellationDerivation / regularTraceCancellationInvariant
variableTraceCancellationDerivation / variableTraceCancellationInvariant
SM3eRr3cTraceCancellationDerivationLedger
```

Diese API ist nur dann als unbedingter r3c-Abschluss lesbar, wenn der konkrete Derivation-Witness selbst ohne freie Kancellationsbeweise konstruiert wird. Der aktuelle r3c0-Status ist deshalb bewusst schwach: Source und Ledger-Export sind legal, aber die sechs Kancellationsfelder sind noch nicht aus der SM3db-Kette ausgerechnet.

SM3eR-r3c1: neue AccumulatedEntryCancellation-Schicht. r3c1 ist ab v36 die Fold-nahe mathematische Ableitungsphase, nicht mehr der direkte TraceCancellationDerivation-Konsum. Der gruen gemeldete Schnittstellenstand darf als geschlossen gelten fuer:

```
generatedInteriorAccumulatedLeftConvolution
generatedInteriorAccumulatedRightConvolution
GeneratedInteriorAccumulatedEntryCancellationInvariant
RegularGeneratedInteriorAccumulatedEntryCancellationInvariant
VariableGeneratedInteriorAccumulatedEntryCancellationInvariant
SM3eRr3c1AccumulatedEntryCancellationLedger
```

Der starke r3c1-full-DoD bleibt aktiv, ist aber ab v37 genauer: Vor dem Konstruktor `accumulatedEntryCancellationInvariant_from_accumulatedTraceEvaluation` oder `accumulatedEntryCancellationInvariant_from_traceFold` muss eine BlockFold-Invarianz bewiesen werden. Der aktuelle Code expandiert `generatedInteriorAccumulatedEntryValue T i j` auf Diagonal-/Block-/Trace-/Fold-Beitraege; erst ein Satz `blockFold = identity` macht daraus die sechs Felder der Invariant-Struktur ohne freie Prooffelder:

```
leftAccumulatedConvolution_eq_identity
rightAccumulatedConvolution_eq_identity
leftAccumulatedConvolution_diagonal
leftAccumulatedConvolution_offdiag
rightAccumulatedConvolution_diagonal
rightAccumulatedConvolution_offdiag
```

Die Ableitung muss aus SM3cR-InteriorBlock-Profil, SM3db2R-LocalStep, SM3db3R-Trace, SM3db4aR-AccumulatedTraceEvaluation/AccumulatedEntryFold und `generatedInteriorAccumulatedEntryValue T` kommen. Ab v45 ist die unmittelbare Beweisroute feiner: r3c1a BlockFold-Normalform ist geschlossen; r3c1b0 auditiert die fehlende LocalStepSchurCancellation-Quelle; SM3db2aR ruestet die lokale Schur-Kancellationssubstanz relativ zu Pivot-Regularitaetswitnessen nach; SM3db2bR konsumiert einen TracePivotScale-Witness und baut daraus PerPivot-Regularitaet; SM3db2cR ist audit-closed/conditional als ScaleSource-Adapter; SM3db2dR ist gruen-relativ als ReciprocalProfile-Adapter; SM3db2eR0 ist closed/green als DegreeFormulaAudit; SM3db2eR-full ist als WeightPolicy-ReciprocalProfile-Audit geschlossen; SM3db2fR ist als GeneratedOperationalDegreeReciprocalSource gruen und exportiert den TracePivotScaleWitness; r3c1b1 ist als Bruecke von r3c1a-BlockFold-Kernen zu lokalen SM3db2aR-Schur-Residualen geschlossen; r3c1b-full hat danach die lokale Schur-Step-Kancellation geschlossen; r3c1c hat die Fold-Invarianz geschlossen; r3c1d ist als TerminalIdentity-Gate gruen; SM3db3aR ist als Trace-/Pivot-TerminalCoverage-Audit gruen. SM3db4aT ist green/audit-closed fuer die terminale SM3db4aR-Fold-/Residual-Coverage. SM3db4aU ist green/audit-closed fuer die fehlenden linken/rechten Terminal-Identity-Felder und den konditionalen Certificate-Pfad. SM3db4aV ist closed/green als TerminalFoldResidualIdentityEquationSource mit EquationDatum-/Source-/Certificate-Adapter. SM3db4aW ist closed/green fuer den expliziten SM3db4aU-Certificate- und r3c1d-TerminalCoverageSource-Export. SM3eR-r3c1d-full ist closed/green als konsumierender

TerminalIdentity-Witness. Aktiv ist nun SM3eR-r3c1e als knapper Constructor vom TerminalIdentity-Witness zum AccumulatedEntryCancellation-Witness. r3c1 darf keine neue CandidateEntry-Formel, keine freie Entry-Tabelle, keine freie Matrix, kein `Matrix.inv`, keine externe Inversenannahme und keinen SM3daR-Rueckfall einfuehren. r3c1 darf keinen ProductIdentityProof, keinen TwoSidedInv, keine InteriorEliminationData und keine DtN-/Charpoly-Daten erzeugen.

SM3eR-r3c2: alte r3c1-Rolle als konditionaler TraceCancellation-Verbraucher. r3c2 ist nach gruenem Schnittstellen-Build die Verbrauchsschicht fuer einen r3c1-Witness. Zulaessige Kernobjekte sind:

```
TraceCancellationDerivation.fromAccumulatedEntryCancellationInvariant
traceCancellationDerivation_from_accumulatedEntryCancellationInvariant
traceCancellationInvariant_from_accumulatedEntryCancellationInvariant
productCancellationLedger_from_accumulatedEntryCancellationInvariant
SM3eRr3c2TraceCancellationLedger
```

r3c2 darf die sechs Kancellationsfelder nicht einzeln frei entgegennehmen; es darf nur `C : GeneratedInteriorAccumulatedEntryCancellationInvariant ...` und den Handoff `H` konsumieren. Dadurch hat r3c2 alles, was es benoetigt, sobald r3c1-full diesen `C`-Witness theorematisch baut. Bis dahin ist r3c2 konditional/green, aber kein unbedingter Ledger-Abschluss fuer r4.

SM3eR-r4a/r4/r5: konditionaler Proofkonstruktor und TwoSidedInv. `SM3eR-r4a` ist nach v58 geschlossen: `GeneratedInteriorSM3db7RProductIdentityProof` entsteht aus dem unbedingten `GeneratedInteriorSM3db7RProductCancellationLedger` durch `productIdentityProof_-from_productCancellationLedger`. Die historische r4-Rolle wird dadurch nicht als separate Phase verdoppelt, sondern als closed-by-r4a gefuehrt. `SM3eR-r5` ist nach v59 ebenfalls geschlossen: `provenTwoSidedInvFromSM3db7R Q` wurde ueber den r4a-ProductIdentityProof konsumiert und liefert den konkreten `TwoSidedInv`-Witness. `SM3fR` ist nun aktiv und konsumiert ausschliesslich diesen Witness, nicht den `ProductProofGate`, nicht r3c0 allein und nicht den blossen Status `waitingForLeftAndRightProductIdentityProofs`.

Ruecksprungregel bei Produktnormalform-Schwaeche. Falls r3c1-full zeigt, dass die vorhandene SM3db4aR-`accumulatedEntry`-/Fold-Schicht keine echte BlockFold-Identity-Invarianz traegt, wird der positive Pfad nicht lokal in r3c2 oder r4 repariert. Die vormals als SM3db4bR beschriebene AccumulatedEntryCancellationInvariant-Rolle ist ab v36 in SM3eR-r3c1 integriert; ab v37 wird sie durch BlockFold-Identity als interne Pflichtschicht geschaerft. Falls darunter Substanz fehlt, wird minimal tiefer zur schwaechsten fehlenden Schicht zurueckgesprungen:

```
SM3db3aR  active: TracePivotTerminalCoverageSource / TraceOrderAndPivotCoverageInvariant
SM3db2bR  closed/green-relative: PerPivotRegularity from TracePivotScaleWitness
SM3db2cR  audit-closed/conditional: TracePivotScaleWitnessFromGeneratedDegree adapter and Miss
SM3db2dR  green-relative: ReciprocalProfile adapter / ScaleSource from profile
SM3db2eR0 closed/green: GeneratedDegreeFormulaAudit
SM3db2eR-full closed/green: ReciprocalProfileFromWeightPolicy-Audit
SM3db2fR closed/green: GeneratedOperationalDegreeReciprocalSource
SM3eR-r3c1b1 closed/green: BlockFoldKernel-to-LocalSchurResidualBridge
SM3eR-r3c1b-full closed/green: LocalStepCancellation
SM3eR-r3c1c closed/green: FoldInvariance
SM3eR-r3c1d green/gate: TerminalIdentity; SM3db3aR active: TracePivotTerminalCoverageSource
SM3db2aR  closed/green-relative: LocalStepSchurCancellationInvariant
```

Diese Ruecksprungphasen sind keine Obstruktionsstarts. Sie werden nur aktiviert, wenn r3c1-full den Bedarf mathematisch dokumentiert.

Sperren in SM3dR, SM3daR, SM3dbR und SM3eR. SM3dR darf kein `TwoSidedInv`, kein Interior-Elimination-Certificate, keine DtN-/GeneralizedDtN-/MultiSchur-Daten und keine Diskriminantendaten einfuehren. SM3daR darf nur CandidateEntry/CandidateMatrix aus der SM3dR-Kandidatur erzeugen und ist aktuell als positiver Inversenpfad gesperrt. SM3db0R–SM3db7R duerfen eine neue derived-only Inversen-CandidateEntry-Funktion konstruieren, aber kein `TwoSidedInv`, kein Interior-Elimination-Certificate, keine `InteriorEliminationData` und keine DtN-/MultiSchur-Daten erzeugen. SM3eR darf Produktnormalformen, Kancellationsbeweise und zuletzt `provenTwoSidedInv` erzeugen, aber noch keine DtN- oder MultiSchur-Daten. SM3fR ist die erste Phase, die aus `provenTwoSidedInv` Interior-Elimination-/Certificate-Daten bauen darf.

Geschlossener DoD fuer einen positiven SM3eR-Re-Run. Ein positiver SM3eR-Re-Run ist nach v59 geschlossen, weil nach SM3db7R eine Handoff-Kandidatenmatrix vorlag, r3a die beiden Produktnormalformen geliefert hat, r3b die bedingte Kancellationsoberflaeche bereitgestellt hat, r3c0 die Derivation-/Ledger-API getragen hat, r3c1a die BlockFold-Normalform geliefert hat, r3c1b0 die fehlende lokale Schur-Kancellationsquelle exakt auditiert hat, SM3db2aR die LocalStepSchurCancellationInvariant nachgeruestet hat, SM3db2bR die per-Pivot-Regularitaet relativ zum TracePivotScaleWitness konstruiert hat, SM3db2cR die ScaleSource-Schnittstelle und das Missing-Source-Gate geschlossen hat, SM3db2dR den Adapter aus einem ReciprocalProfile bereitgestellt hat, SM3db2eR-full den fehlenden ReciprocalProfile-Pfeil als Gate-Audit geschlossen hat, SM3db2fR die operationale Degree-Reziprokquelle proof-carrying bereitgestellt hat, r3c1b1 die BlockFold-Kernel an lokale Schur-Residuen gebunden hat, r3c1b-full die lokale Step-Kancellation bewiesen hat, r3c1c die Fold-Invarianz bewiesen hat, r3c1d und r3c1d-full terminal identity geschlossen haben, r3c1e daraus einen unbedingten `GeneratedInteriorAccumulatedEntryCancellationInvariant` konstruiert hat, r3c2 daraus den `GeneratedInteriorSM3db7RTraceCancellationInvariant` und ProductCancellationLedger abgeleitet hat, r4a daraus einen konkreten `GeneratedInteriorSM3db7RProductIdentityProof` konstruiert hat und r5 `provenTwoSidedInvFromSM3db7R Q` erzeugt hat. Positiver Output ist genau dieser theorematisch konstruierte `TwoSidedInv`; deshalb ist SM3fR jetzt freigegeben, aber nur fuer den Aufbau der Interior-Elimination-/Certificate-Daten.

2b.3 Phasenpunkt-Tabelle AR12a-SMOKE

Phase	Ziel	Kernarbeit	Gate / positiver Zielzustand	Status
SM0R	Reset-Scaffold geschlossen	Minimaler Namespace, BuildAll/Notation-Anschluss, generated full-level carrier, regular/variable Smoke-Carrier, Nicht-Singleton-Witness-Daten und Ledger-Sperren gegen alte SM-Kontinuitaet.	Gruener lokaler Build; kein Import alter SM-Zips; kein CutSpec.full im Smoke-Kern; keine Matrixdaten; Witness-Konstruktionen als `def`, nicht als `theorem`.	closed / green
SM1R	Regular/Variable-Pipeline geschlossen	RegularCell und LevelVariableCell sind als gleichwertige generated Hauptpfade angeschlossen. Policy-/Weight-Provenienz-Hooks sind exponiert, ohne Matrix-, DtN-, Inversen- oder Diskriminantendaten einzufuehren.	Outputs: regular/variable path pair, `sameGeneratedPipeline`, `noArithmeticSpecialVariation`, policy/weight provenance hooks, `noCutSpecCarrierClaim`, `noMatrixDataYet`, kein BoundaryPorts-/Matrix-/DtN-Import; gruener lokaler Build.	closed / green
SM2R	Generated Approximant Core geschlossen	`GeneratedApproximantStrong` aus SM1R-Hauptpfad: regular/variable Konstruktoren, Route `generatedToCSubstrate`, Carrier aus MainPath, Cutoff, Truncation-Witness, finite Carrier-API, Fintype-/DecidableEq-Zugriff, `noBoundaryPortsInCore`, `noCutSpecInCore`.	Gruener lokaler Build; kein blosses `toc.approximant p`-Packaging; kein BoundaryPorts-/CutSpec-Fundament; keine Matrix-/Dirichletdaten; Carrier/Cutoff/Finiteness sind fuer SM3aR als Gleichheits-, Fintype- und Decidable-Daten nutzbar.	closed / green

Phase	Ziel	Kernarbeit	Gate / positiver Zielzustand	Status
SM3aR	Boundary/Interior-Partition geschlossen	Boundary- und Interior-Indizes ausschliesslich aus SM2R-`GeneratedApproximantStrong`. Zusaetzlich Indexgleichheiten, DecidableEq/Fintype-Daten, Membership-/Subset-Lemmata und Restriktionshilfen fuer spaetere Matrixprodukte.	PartitionWitness, disjoint/complete, carrier-subset, no free `Fin n`, no CutSpec.full, no BoundaryPorts, keine Matrix-/Dirichletdaten; InteriorIndex API stark genug fuer SM3bR/SM3cR; gruener lokaler Build.	closed / green
SM3bR	Dirichlet-Laplacian mit Entry-Source-Vertrag geschlossen	Matrix aus SM2R-Carrier, SM3aR-Partition, Parent/Child-/Level-Adjazenz und Weight-/Policy-Provenienz. Entry-Quelle und Entry-Lemma kommen vor jeder abgeleiteten Formel; `generatedDirichletMatrix i j = generatedDirichlet_entry i j`.	generated adjacency source, generated weight-policy entry source, generatedDirichlet_entry, generatedDirichletMatrix_entry, zero_of_not_adjacent, offdiag_of_adjacent, diag_degree_-or_regularized_degree, boundary/interior entry restriction, noFreeAdjacency/Weight/Matrix. Keine Interior-Block-Klassifikation, keine Inverse, keine DtN.	closed / green
SM3cR	Interior-Block-Strukturprofil geschlossen	Interior-Restriktion der SM3bR-Matrix. Vollstaendige entrywise Klassifikation: diagonalOnly, degreeDiagonal, offdiagCoupled, treeRecursive, generalFiniteEliminationNeeded oder obstructed.	Gruener lokaler Build; Strukturprofil vorhanden; aktiver Befund treeRecursive/offdiag-sensitiv. Keine Inverse, kein TwoSidedInv, keine Interior-Elimination, keine DtN.	closed / green
SM3dR	Strukturabhaengige Candidate-/Builder-Schicht	Kandidaten-/Eliminationsdaten werden aus dem SM3cR-Profil abgeleitet: diagonal nur im bewiesenen Spezialfall; der aktive positive Pfad ist tree-recursive/offdiag-sensitiv oder finite proof-carrying elimination.	Kein Matrix.inv, keine freie Inverse, keine diagonale Beta-Inverse, kein TwoSidedInv, keine externen Certificates, keine DtN. Output nur Candidate oder Obstruktion; noch keine auswertbare CandidateEntry-Funktion.	closed / green
SM3daR	CandidateEntry-Interface fuer den ersten SM3eR-Audit	Aus der SM3dR-Kandidatur eine derived-only Entry-Funktion und Kandidatenmatrix ableiten: GeneratedInte-riorEliminationCandidateEntry, candidateEntry_from_treeRecursiveProfi-le, candidateMatrix_from_candidateEntry und Herkunftslemma gegen SM3dR-Quelle.	Gruener lokaler Build; kein Matrix.inv, keine freie Entry-Tabelle, kein externes Inversen-Oracle, kein TwoSidedInv, kein Interior-Elimination-Certificate, keine DtN-/MultiSchur-Daten. Durch SM3eR als positiver Inversenpfad obstruiert: candidateEntry = InteriorBlock.	closed / green / obstructed-as-positive
SM3eR	ShapeAudit und obstruktiver TwoSidedInv-Befund	SM3eR-0 CandidateEntryShapeAudit fuer SM3daR geschlossen; candidateEntry_eq_interiorBlock ist der Pflichtbefund. Da kein Links-/Rechtsbeweis von InteriorBlock * InteriorBlock = 1 vorliegt, entsteht eine formale GeneratedIn-teriorTwoSidedInvObstruction.	Kein provenTwoSidedInv; keine Reparaturformel; kein Matrix.inv; kein Eingang fuer SM3fR. Ruecksprung auf SM3dbR/SM3dR/SM3cR.	closed / green / obstructed
SM3db0R	Positive-First-Vertrag und SM3daR-Sperre	SM3daR als positiven Inversenpfad sperren, aber SM3dbR ausdruecklich als positive Kandidatenbaukette starten.	Kein `matrixOpt = none` als Hauptausgang; keine Obstruktion vor SM3db1R–SM3db7R; kein TwoSidedInv.	closed / green
SM3db1R	Interior-Elimination-Carrier	Proof-carrying Carrier, Node, Depth und Measure ueber `GeneratedInteriorIndex A` aus SM3cR/SM3dR.	Kein freier Carrier, keine freie Pivotliste, kein externes Order-Oracle, keine Inversenfunktion, keine Trace-Semantik.	closed / green
SM3db2R	Lokaler positiver Eliminationsschritt	PivotDatum aus SM3db1R-Carrier-Node; ResidualDatum und StepUpdate lokal aus SM3cR/SM3dR-Profil und Candidate-Provenienz.	Geschlossen im engen Scope; kein Matrix.inv, keine globale Inverse, kein Certificate, keine freien Pivotdaten, keine globale Pivotordnung, keine Trace-Semantik, keine Inversen-Entry-Funktion.	closed / green
SM3db3R	Finite/tree-recursive EliminationTrace	Schritte aus SM3db1R-Carrier-Daten und SM3db2R-Lokalschritten zu einer terminierenden Eliminationsspur zusammensetzen; TraceIndexCarrier, TraceLength, Step-Bindung, Pivot-Bindung und Residual-Rueckbindung theorematisch fuehren.	Geschlossene positive Trace-Schicht; keine Inversen-CandidateEntry, kein Matrix-Export, kein Produktbeweis; bei spaeterer Auswertungsschwaeche nicht obstruieren, sondern TraceEvaluation/TraceEntryFold oder SM3db3R/SM3db2R/SM3db1R schaerfen.	closed / green

Phase	Ziel	Kernarbeit	Gate / positiver Zielzustand	Status
SM3db4R	Erste Trace-basierte CandidateEntry-Schale	`GeneratedInteriorInverseCandidateEntry` aus der SM3db3R-Trace ist formal gruen, aber nur partiell: Entry-Auswertung noch zu nah an lokalem `rowResidual`.	Kein Matrix-Export, kein Produktbeweis, kein Certificate/Oracle; Status nicht vollstaendig geschlossen, sondern Startpunkt fuer SM3db4aR.	green / partial
SM3db4aR	Akkumulierte TraceEvaluation / AccumulatedEntryFold	`GeneratedInteriorAccumulatedTraceEvaluation` und `GeneratedInteriorAccumulatedEntryFold` sind gebaut; `accumulatedEntry` ist aus Trace, Pivotdaten, lokalen Steps, Residualdaten und Fold-/Update-Reihenfolge theorematisch hergeleitet.	Nicht bloss lokaler `rowResidual`-Wrapper; kein Matrix.inv, keine freie Entry-Tabelle, kein Matrix-Export, kein Produktbeweis, kein TwoSidedInv, kein Certificate/Oracle.	closed / green
SM3db5R	Matrix-Export ohne Produktbeweis	Akkumulierte SM3db4aR-CandidateEntry entrywise als Matrix exponieren; jeder Matrixeintrag muss definitionell oder theorematisch die SM3db4aR-`accumulatedEntry` sein.	Keine neue CandidateEntry-Formel, keine freie Matrix, kein Links-/Rechtsprodukt, kein TwoSidedInv, kein InteriorEliminationData, kein Certificate.	closed / green
SM3db6R	Shape-/Source-Separation nach positivem Kandidaten	SM3db6R-ShapeSeparation aus SM3db5R-Matrix und SM3db4aR-accumulatedEntry geschlossen; Status-Gates gegen SM3daR-, InteriorBlock-, localResidual- und freie Tabellen-Rueckfaelle.	Source-/Shape-Gate, keine semantische Nichtgleichheit, keine Produktbeweise, kein TwoSidedInv, kein Certificate, keine InteriorEliminationData.	closed / green
SM3db7R	Handoff an erneutes SM3eR	Ledger und Handoff fuer die SM3db6R-gefilterte Kandidatur erzeugt; regular/variable Handoff auf SM3db6R, SM3db5R und SM3db4aR-Herkunft zurueckgebunden.	SM3eR darf nur diese neue Kandidatur konsumieren; keine Links-/Rechtsprodukte, kein provenTwoSidedInv; SM3fR bleibt ohne provenTwoSidedInv gesperrt.	closed / green
SM3eR-r0–r5	Neuer Produkt-ReRun aus SM3db7R-Handoff	r0 HandoffShapeAudit, r1 LeftProductEntry, r2 RightProductEntry, r3a ProductNormalForm, r3b CancellationSurface, r3c0 TraceCancellationDerivation-API, r3c1a BlockFold-Normalform, r3c1b0 SourceAudit, SM3db2aR–SM3db2fR, r3c1b1, r3c1b-full, r3c1c, r3c1d/r3c1d-full, SM3db3aR, SM3db4aT/U/V/W, r3c1e, r3c2, r4a und r5 sind geschlossen. r5 liefert den konkreten `TwoSidedInv`-Witness aus `H.candidateMatrix`.	Produktrechnung ausschliesslich gegen `H.candidateMatrix`; keine freie TraceCancellation, kein freier ProductIdentityProof, kein Matrix.inv, keine Reparaturformel. Positiver Output ist nur `TwoSidedInv`; DtN-/MultiSchur-/Charpoly-Daten bleiben nachgelagert.	closed / green

Phase	Ziel	Kernarbeit	Gate / positiver Zielzustand	Status
SM3db2bR SM3eR-r3c1b1 SM3db3aR	Pivot-Regularitaet und Kernel-Bruecke	SM3db2bR ist closed/green-relative als PerPivot-Regularity-Gate aus einem TracePivotScaleWitness. SM3db2cR ist audit-closed/conditional und exponiert den benoetigten ScaleSource-Adapter. SM3db2dR ist gruen-relativ und baut diesen ScaleSource aus einem proof-carrying ReciprocalProfile. SM3db2eR0 ist closed/green; SM3db2eR-full ist closed/green und hat keinen unbedingten ReciprocalProfile-Provider aus dem aktuellen WeightPolicy-Summen-API gefunden. SM3db2fR ist closed/green und stellt die operationale Degree-Reziprokquelle samt TracePivotScaleWitness bereit. r3c1b1 ist closed/green und hat r3c1a-BlockFold-Kerne an SM3db2aR-Schur-Residuen angeschlossen; r3c1b-full ist closed/green als lokale Step-Kuerzung; r3c1c ist closed/green als Fold-Invarianz; r3c1d ist nun gruen als TerminalIdentity-Gate; SM3db3aR ist closed/green-audit fuer die TracePivotTerminalCoverageSource; SM3db4aT ist closed/green-audit fuer TerminalFoldResidualCoverage; SM3db4aU ist closed/green-audit fuer TerminalIdentityFields-Audit; SM3db4aV ist closed/green als TerminalFoldResidualI-dentityEquationSource; SM3db4aW, r3c1d-full und r3c1e sind geschlossen; r3c2 ist closed/green fuer den mechanischen TraceCancellation-/ProductCancellationLedger-Verbrauch; r4a und r5 sind inzwischen geschlossen; der ProductIdentityProof-Consumer und der TwoSidedInv-Export liegen vor.	Keine Obstruktion zuerst; SM3db2aR ist relativ gruen, SM3db2bR entfernt die Real.instInv-/Skalarinversionsfalle und bindet Regularitaet an einen ScaleWitness, SM3db2cR dokumentiert die noch freie ScaleWitness-Abhaengigkeit, SM3db2dR begrenzt sie auf ein ReciprocalProfile; SM3db2fR stellt das fehlende operationale Reziprokdatum proof-carrying bereit; r3c1b1 hat die Kernel-Anschlussluecke entfernt; r3c1b-full hat die lokale Kancellationsluecke geschlossen; SM3db3aR hat die terminale Trace-/Pivot-Coverage-Source auditiert; SM3db4aT hat die terminale Fold-/Residual-Coverage aus SM3db4aR auditierend exponiert; SM3db4aU hat die fehlenden linken/rechten Terminal-Identity-Felder und den konditionalen Certificate-Pfad auditierend exponiert; SM3db4aV hat die terminale Fold-/Residual-Identity-Gleichungsquelle als EquationDatum-/Source-Schicht samt Certificate-Adapter bereitgestellt; Der Export-, TerminalIdentity- und AccumulatedEntryCancellation-Korridor ist geschlossen; r3c2 hat den vorhandenen r3c1e-Witness in TraceCancellationInvariant und ProductCancellationLedger ueberfuehrt; r4a hat diesen Ledger in ProductIdentityProof ueberfuehrt; r5 hat daraus den TwoSidedInv-Witness exportiert.	closed / green
SM3db OmegaR	Spaetes Obstruktionsledger	Nur falls die positive SM3dbR-Handoff-Kette selbst mathematisch dokumentiert scheitert.	Obstruktion ohne vorherigen positiven Kandidaten-, Shape-Separation- und Handoff-Versuch ist Planbruch; Ruecksprungziele SM3cR/SM3dR/SM3db1R/SM3db2R/SM3db3R/SM3d	nur konditional
SM3fR	InteriorElim-/Cert-Daten	Builder aus dem r5-`TwoSidedInv`-Witness: `H.candidateMatrix` als abgeleitete Interior-Inversenmatrix, daraus `interiorSolve`, `boundaryOperator` und die Spezifikationsfelder. Regular/variable Zertifikate sind Builderdaten, keine Typklassenannahmen.	Keine externen Certificates, kein `infer_instance`, kein `candidateFormulaOnly`, keine SM3daR-Obstruktionsdaten als Ersatz; kein DtN-/MultiSchur-/Charpoly-Output.	closed / green
SM3gR	GeneratedDtN	Generated-lokale top-boundary-DtN aus SM3fR-Certificate; `boundaryOperator`, `apply/mulVec`, Schur-Rueckbindung und Flux-Spezifikation exponiert.	Kein public-`DtNStrong`-Zwang, keine neue Inversenrechnung, keine Generalized-/MultiSchur-/Charpoly-Daten.	closed / green
SM3hR	Generated GeneralizedDtN	Schmaler Generalized-Wrapper aus SM3gR-top-boundary-DtN und vorhandenen Hauptpfad-/Sektor-Provenienzdaten.	Keine externen Interior-Zertifikate, keine freie DtN, kein MultiSchur; public-`GeneralizedDtNStrong` nur wenn die Bruecke ohne Zusatzkette schliesst.	closed / green
SM3iR	Reset-SM3-Ledger	Reine Aggregation der neu gebauten SM3aR–SM3hR-Daten als einziger SM4-Konsumfilter.	Grep: keine alten SM-Imports, keine externen Certificate-Typklassen, kein Matrix.inv, kein noncomputable/classical im operativen Pfad; keine neue Rechenquelle.	closed / green

Phase	Ziel	Kernarbeit	Gate / positiver Zielzustand	Status
SM4	Generated MultiSchur	regularGeneratedMultiSchur und variableGeneratedMultiSchur ausschliesslich aus SM3iR-Daten; generated-lokal vor public-Bridge.	Darf nur neue SM3iR-DtN/GeneralizedDtN und computed Interior-Zertifikate konsumieren; kein public-MultiSchurStrong-Zwang.	closed / green
SM5	Generated ArithmeticSource fuer generated Quellen	Generated-lokale Source-Oberflaeche fuer regularGenerated/variableGenerated aus dem SM4-Ledger.	Kein neues ToC, keine public-MultiSchurStrong-Pflicht, keine LevelHistory-/Charpoly-Daten; SM5 konsumiert SM4.	closed / green
SM6	Level-History Witness	GeneratedLevelStepWitness mit Level, generated Boundary-Carrier, SM5/SM4-DtN-/Schur-Provenienz und noCutSpecCarrierClaim-Status.	LevelHistoryIndex L := Fin (L+1) bleibt nur Index; legal wird er durch Witness-Felder, nicht durch eine freie Indexbruecke.	closed / green
SM7	LevelHistory-to-Schur-Index	Bridge-Gate geschlossen als proof-carrying Obstruktion der fehlenden Indexquelle; kein BridgeSuccess.	Kein freier Ersatzindex, keine Fintype-Enumeration, keine Matrix; SM8 bleibt blockiert.	closed / green-obstruction
SM6a	GeneratedLevelHistoryBoundaryPortSourceSM6	Natürliche SM6-nahe Quelle fuer History-to-Boundary-/Schur-Index; konstruktiver Boundary-Port aus SM6/SM5/SM4-Provenienz.	Kein Matrixstart; keine neue Quelle im SM7-Gate; positive Quelle wird vom SM7BridgeRecheck konsumiert.	closed / green
SM7BridgeRecheck	BridgeSuccess aus SM6a	Reiner Consumer/Adapter von `GeneratedLevelHistoryBoundaryPortSourceSM6a` zu `GeneratedLevelHistorySchurIndexBridgeSM7`; regular/variable `bridgeSuccess`.	Keine neue Quelle, keine Matrix, keine Charpoly-Daten; nur Projektionen aus SM6a plus vorhandene SM7-Zielsignatur.	closed / green
SM8	Echte Matrixeintraege	`generatedSchurEntry` und `generatedLevelHistoryMatrix`; Matrixeintraege als Pullback der SM5/SM4-Schur-Boundary-Matrix entlang `SM7BridgeRecheck.regularBridge/variableBridge`.	Keine freie `LevelHistoryMatrixPackage.matrix`; Eintraege kommen aus `sourceSM6Witness.boundaryOperator` und `toBoundaryIndex`; Charpoly erst ab SM9.	active / kritisch
SM9	Charpoly/Determinante	Charpoly-/Determinantenpfad auf der berechneten Matrix, nicht auf Oracle-Matrizen.	operativeCharpolyEvaluation oder echtes Polynom wird aus Smoke-Matrix gespeist.	kritisch
SM10	Positiver L=2 Eisenstein-Rechenvergleich	Erst berechnete L2-Matrix/Charpoly/Faktor-/Diskriminantenstatus aus SM8/SM9; dann Vergleich mit Eisenstein-Ziel.	Treffer: confirmed; Abweichung: targetMismatch; fehlende Rechnung: blocked.	hart
SM11	Positiver L=3 Gauss-Rechenvergleich	Erst berechnete L3-Matrix/Charpoly/Faktor-/Diskriminantenstatus aus SM8/SM9; dann Vergleich mit Gauss-Ziel.	Treffer: confirmed; Abweichung: targetMismatch; fehlende Rechnung: blocked.	hart
SM12	L>3 Smoke-Scan	Erste echte Zeilen fuer kleine Faelle: regulaer `code-b`=1,2,3, also `arity`=2,3,4, L=4,5; variabel beta(n)=1, beta(n)=2, beta(n)=n+1, L=4. Der binaere 0/1-Zeichnungsfall ist `code-b`=1.	Jede Zeile erhaelt einen Status aus HigherGeneratedSmokeOutcome; kein blosses Open; kein nacktes $b = 2$ als binaerer Baum.	explorativ-hart
SM13	AR12a-Kompatibilitaet aus berechneter Diskriminante	CDArithmeticCompatibilitySurface wird fuer L2/L3 und ggf. L>3 aus Smoke-Zertifikaten gefuettert.	Keine AR12a-Klassifikation ohne berechnete Diskriminante bzw. Grad-/Faktorstatus.	hart
SM14	CutSpec-Future-Use-Ledger	CutSpecFutureUseLedger: CutSpec bleibt fuer Regi-on/Defekt/Worldsheet/Stringtheorie/Vergleich erhalten, aber nicht als Smoke-Fundament.	Der Smoke-Kern importiert CutSpec nicht; Vergleichsmodule duerfen CutSpec isoliert referenzieren.	sofort
SM15	Entscheidung vor AR13	Ergebnisledger mit Faellen: positiveConfirmed, partialComputed, computedTargetMismatch, apiBlocked.	AR13 darf substanziell nur starten, wenn eine berechnete Datenroute oder ein expliziter Obstruktionsstatus fuer quadratische Ordnungen vorliegt.	Sperre

Definition of Done fuer SM3. SM3 ist erst gruen und inhaltlich geschlossen, wenn der positive Pfad die Kette

```
generated Dirichlet-Laplacian with entry formula
-> generated InteriorBlock with structure profile
-> SM3dR structure-derived candidate/builder layer
-> SM3daR first CandidateEntryInterface, audited by SM3eR as InteriorBlock-shape
-> SM3db0R--SM3db7R positive inverse CandidateEntry construction
-> SM3eR-r3a ProductNormalForm against H.candidateMatrix
-> SM3eR-r3b conditional diagonal/offdiag cancellation surface
```

```
-> SM3eR-r3c0 TraceCancellationDerivation API -> SM3eR-r3c1 cancellation derivation from SM3db
-> SM3eR-r4 ProductIdentityProof constructor
-> SM3eR-r5 provenTwoSidedInv or documented return to SM3db4bR/3aR/2aR
-> if obstruction after positive attempt: return to SM3db1R/SM3db2R/SM3db3R/SM3dR/SM3cR, not l
-> InteriorEliminationData
-> GeneratedDtN
-> GeneratedGeneralizedDtN
-> SM3GeneratedStateDirichletDtNLedger
```

traegt und im finalen Ledger keine externen `[RegularGeneratedInteriorEliminationCertificate ...]` bzw. `[VariableGeneratedInteriorEliminationCertificate ...]` als Kontextannahmen verbleiben. Zertifikate duerfen Felder des Ledgers sein, aber nur wenn sie direkt aus den sichtbaren Builderdaten SM3aR–SM3hR gebaut wurden. Zusaetzlich muss jeder Builder ein Provenienz-/Gleichheitslemma exponieren, sodass der Ledger nicht als versteckte Rechenquelle fungiert.

2b.4 Reset-Modulbaum

In Abschnitt 3 werden neben dem Arithmetikbaum folgende Hauptpfad- und Smoke-Module neu aus dem AR12a-Codezustand aufgebaut. Die Liste ist kein Importauftrag fuer alte SM-Zips, sondern eine Zielstruktur fuer den Reset. Alte SM0–SM3d-Dateien duerfen hoechstens als Vergleich/Audit gelesen werden; im positiven Build muessen die Reset-Module die Provenienz neu tragen.

```
-- SM0R/SM1R/SM2R geschlossen; SM3aR: Partition aus SM2R, nicht aus alten SM-Zips
CNNA/PillarA/ToC/GeneratedBranchFamily.lean
CNNA/PillarA/ToC/GeneratedBranchVariation.lean
CNNA/PillarA/Finite/GeneratedBranchApproximant.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedSource.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedVariation.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedApproximantCore.lean

-- SM3R: Positive-First-Rechenclosure neu; Entry-Lemmata vor Matrix-/Inversenformeln
CNNA/PillarA/Finite/GeneratedPartition.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedPartition.lean
CNNA/PillarA/Finite/GeneratedDirichletLaplacian.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedDirichlet.lean
CNNA/PillarA/DtN/GeneratedInteriorBlockBuilder.lean
CNNA/PillarA/DtN/GeneratedInteriorBlockStructureProfile.lean
CNNA/PillarA/DtN/GeneratedInteriorInverseBuilder.lean
CNNA/PillarA/DtN/GeneratedInteriorInverseDecision.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorEliminationCandidateEntry.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorEliminationCarrier.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorEliminationStep.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorEliminationTrace.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorInverseCandidateEntry.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorInverseCandidateMatrix.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorInverseCandidateShapeSeparation.lean
CNNA/PillarA/Arithmetic/Smoke/SM3dbRToSM3eRHandoff.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorTraceCancellationDerivation.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorBlockFoldNormalFormR3c1a.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorLocalStepSchurCancellationSourceAuditR3c1b0.lea
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorLocalStepSchurCancellationInvariantSM3db2aR.lea
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorAccumulatedEntryCancellationR3c1.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorTraceCancellationR3c2.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedInteriorTwoSidedInvFromSM3db7R.lean
CNNA/PillarA/Arithmetic/Smoke/SM3dbOmegaObstructionLedger.lean
```

```
CNNA/PillarA/DtN/GeneratedInteriorProductCalc.lean
CNNA/PillarA/DtN/GeneratedInteriorEliminationBuilder.lean
CNNA/PillarA/DtN/GeneratedDtNBuilder.lean
CNNA/PillarA/Coupling/GeneratedGeneralizedDtNBuilder.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedStateDirichletDtNLedger.lean

-- SM4+
CNNA/PillarA/Coupling/GeneratedBranchMultiSchur.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedSourceObstruction.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelWitness.lean
CNNA/PillarA/Arithmetic/Smoke/LevelHistoryToSchurIndex.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedSchurEntry.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedLevelHistoryMatrix.lean
CNNA/PillarA/Arithmetic/Smoke/GeneratedCharpoly.lean
CNNA/PillarA/Arithmetic/Smoke/L2GeneratedEisenstein.lean
CNNA/PillarA/Arithmetic/Smoke/L3GeneratedGaussian.lean
CNNA/PillarA/Arithmetic/Smoke/HigherGeneratedSmoke.lean
CNNA/PillarA/Arithmetic/Smoke/AR12aGeneratedSmokeLedger.lean

-- comparison/future only
CNNA/PillarA/Finite/CutSpecFutureUseLedger.lean
CNNA/PillarA/StringTheory/IdealThreadStringMotivation.lean
CNNA/PillarA/StringTheory/CutSpecWorldsheetWatchlist.lean
```

Importdisziplin: Der Hauptpfad darf `Arithmetic/Smoke` nicht importieren. `Arithmetic/Smoke` darf `GeneratedBranchFamily`, `GeneratedBranchVariation`, `GeneratedBranchApproximant`, `GeneratedDtNBuilder`, `GeneratedGeneralizedDtNBuilder` und `GeneratedBranchMultiSchur` konsumieren. Die Builder liegen fachlich im Hauptpfad bzw. in DtN/Coupling und duerfen den Smoke-Ledger nicht importieren. `CutSpec` darf im Smoke-Kern nicht importiert werden; CutSpec-Vergleiche liegen ausschliesslich in `CutSpecFutureUseLedger` oder ComparisonOnly-Modulen. Stringtheorie-bezogene Module sind Watchlist-/Vergleichsmodule und duerfen weder von `Arithmetic/Smoke` noch von operativen Arithmetik-Consumerpfaden importiert werden.

2b.5 Neue Status-Typen

```
inductive GeneratedSourceObstruction where
  | missingGeneratedApproximant
  | missingGeneratedBoundaryInteriorPartition
  | missingGeneratedAdjacencySource
  | missingGeneratedWeightSource
  | freeAdjacencyRejected
  | freeWeightRejected
  | freeMatrixRejected
  | missingGeneratedDirichlet
  | missingGeneratedInteriorBlock
  | missingGeneratedInteriorBlockExplicitForm
  | freeInteriorMatrixRejected
  | freeInteriorCardinalityRejected
  | missingGeneratedInteriorInverse
  | missingGeneratedInteriorTwoSidedInv
  | missingGeneratedInteriorEliminationData
  | missingGeneratedInteriorBuilderExposure
  | missingGeneratedDtN
  | missingGeneratedGeneralizedDtN
```

```
  | missingGeneratedMultiSchur

inductive GeneratedMatrixObstruction where
  | missingLevelHistoryWitness
  | missingLevelHistoryToSchurIndex
  | missingRealToExecBridge
  | missingEntryFormula

inductive SM3dbPositiveCandidateOutcome where
  | positiveFirstContractClosed
  | eliminationCarrierClosed
  | eliminationStepClosed
  | eliminationTraceClosed
  | inverseCandidateEntryClosed
  | inverseCandidateMatrixExported
  | shapeSeparated
  | handoffToSM3eR
  | lateObstructionAfterPositiveAttempt

inductive GeneratedPositiveComparisonOutcome where
  | positiveSmokeConfirmed
  | positivePathComputedButTargetMismatch
  | computationBlocked

inductive HigherGeneratedSmokeOutcome where
  | sourceObstruction
  | matrixObstruction
  | charpolyObstruction
  | factorizationObstruction
  | degenerate
  | quadraticHeegner
  | quadraticNonHeegner
  | higherDegreeOrIrreducible

inductive AR12aSmokeOutcome where
  | positiveConfirmed
  | partialComputed
  | computedTargetMismatch
  | blockedByMissingGeneratedMainPathAPI
```

2b.6 Konkrete Rechenziele

Eisenstein	Minimal: `computedDiscriminant(L=2) = -3`. Staerker: `computedCharpoly(L=2)` $= z^2 + z + 1$. Noch staerker: `computedReducedMatrix(L=2) = l2RhoMultiplicationMatrix`.
Gauss	Minimal: `computedDiscriminant(L=3) = -4`. Staerker: `computedCharpoly(L=3)` $= z^2 + 1$. Noch staerker: `computedReducedMatrix(L=3) = l3GaussianMultiplicationMatrix`.
L>3	Jeder getestete Fall muss mindestens `sourceStatus`, `matrixStatus`, `charpolyStatus`, `factorStatus`, `discriminantStatus` und eine AR12a-Route liefern. Zulaessige Routen sind `quadraticHeegner`, `quadraticNonHeegner`, `higherDegreeOrIrreducible`, `degenerate` und `obstructed`.

2b.7 Harte Interpretation des Ergebnisses

Fall A: L2 und L3 treffen.

Der bisherige AR0–AR12a-Pfad ist nicht nur algebraisch konsistent, sondern

ontisch an ein ToC-Substrat angebunden. AR13–AR15 duerfen als echte Consumer fortgesetzt werden.

Fall B: L2 oder L3 werden berechnet, treffen aber nicht.
Der bisherige Eisenstein-/Gauss-Zielanker bleibt formal als Vergleichsziel stehen, wird aber durch die generated Pillar-A-Maschinerie nicht getroffen. Das ist `computedTargetMismatch`: eine harte Zielabweichung nach positiver Rechnung, nicht bloss ein fehlendes Lemma und nicht identisch mit API-Blockade.

Fall C: L2/L3 treffen, L>3 degeneriert.
Die quadratische Kalibrierung ist real, aber der generated Substrat-/Boundary-Mechanismus ist moeglicherweise zu symmetrisch, zu trivial oder zu wenig variabel.

Fall D: L2/L3 treffen und L>3 liefert nichttriviale quadratische oder hoehergradige Daten.
Dann entsteht ein mathematischer Kernbefund: Das ToC-Substrat erzeugt Ring-/Ordnungs-/Faktorstrukturen, die nicht als externe Algebra hineingelegt wurden.

Fall E: Hauptpfad-API blockiert vor der Matrix.
Dann fehlt keine Arithmetik-Prosa, sondern eine operative Hauptpfad-Schicht: Boundary/Interior-Partition, generated Dirichlet-Laplacian, Interior-Block, explizite Inverse, TwoSidedInv, InteriorEliminationData, generated DtN, generated GeneralizedDtN, generated MultiSchur oder LevelHistory-to-Schur-Index.

2b.8 Konsequenz fuer die weitere Planung

AR13, AR14, AR15, AR15a und AR16 duerfen substanziell nur als Consumer von AR12a-SMOKE-Daten weiterlaufen. Ohne berechneten, positiv verglichenen oder explizit obstruierten Smoke-Ausgang bleiben sie Interface-/Provenienzschichten, aber keine abgeschlossenen arithmetischen Ergebnisphasen. Nach dem gruen geschlossenen SM3eR-r3c1e-Stand sind die Hauptpfad-Provenienz, die endliche Approximant-Basis, die Boundary-/Interior-Partition, die Dirichlet-Entry-/Matrix-Entry-Schicht, die Interior-Block-Strukturklassifikation, die Candidate-/Builder-Schicht, die CandidateEntry-/CandidateMatrix-Oberflaeche, der Positive-First-Startschutz, der GeneratedInteriorEliminationCarrier, der lokale positive Eliminationsschritt, die finite/tree-recursive `GeneratedInteriorEliminationTrace`, die akkumulierte TraceEvaluation-/AccumulatedEntryFold-Schicht, der reine Matrix-Export, die Shape-/Source-Separation, die Handoff-Schicht, die r3a-Produktnormalformen, die r3b-CancellationSurface, r3c0 als TraceCancellationDerivation-API, r3c1 als AccumulatedEntryCancellation-Invariant-Interface, r3c1a als BlockFold-Normalform, r3c1b0 als SourceAudit, SM3db2aR als relative LocalStepSchur-CancellationInvariant, SM3db2bR als ScaleWitness-Consumer, SM3db2cR als audit-/adapter-gruene ScaleSource-Schicht, SM3db2dR als gruen-relative ReciprocalProfile-/ScaleSource-Adapter-Schicht, SM3db2eR0 als geschlossener GeneratedDegreeFormulaAudit, SM3db2eR-full als gruen geschlossener WeightPolicy-ReciprocalProfile-Audit, SM3db2fR als gruen geschlossene operationale Degree-Reziprokquelle, r3c1b1 als gruen geschlossene BlockFoldKernel-to-LocalSchurResidualBridge, r3c1b-full als gruen geschlossene LocalStepCancellation, r3c1c als gruen geschlossene FoldInvariance, r3c1d als gruenes TerminalIdentity-Gate und r3c2 als bedingter TraceCancellation-/Ledger-Adapter formal gruen bzw. konditional verfuegbar. Zugleich ist der positive Inversenpfad weiterhin offen: r3c1e liefert jetzt den unbedingten Konstruktor von `GeneratedInteriorAccumulatedEntryCancellationInvariant` aus der TerminalIdentity-Schicht, aber noch keinen unbedingten r3c2-`ProductCancellationLedger`, keinen `GeneratedInteriorSM3db7RProductIdentityProof` und kein `provenTwoSidedInv`. Weiterhin offen sind r3c2 als mechanischer TraceCancellation-/ProductCancellationLedger-Consumer, r4-ProductIdentityProof, r5-TwoSidedInv, Interior-Elimination, DtN-/GeneralizedDtN-/MultiSchur-Rechenprovenienz und alle Charpoly-/Faktor-/Diskriminantendaten. Die naechste aktive Schicht ist r3c2: TraceCancellation/ProductCancellationLedger aus dem r3c1e-AccumulatedEntryCancellation-Witness. `SM3dbOmegaR` bleibt nur das spaete Obstruktionsledger nach dokumentiertem positivem Scheitern der Handoff-Kette selbst. Damit wird die Planung geschaerft:

```
AR0--AR12a: algebraisch-proof-carrying Aufbau
```

```
SM0R--SM1R: Reset- und Hauptpfad-Provenienz geschlossen
SM2R: generated finite Approximant-Core geschlossen
SM3aR: positive Boundary-/Interior-Partition aus SM2R geschlossen
SM3bR: geschlossene Dirichlet-Entry-Rechnung ohne freie Matrix/Adjazenz/Weights
SM3cR: geschlossene Interior-Block-Klassifikation ohne freie Interior-Tabelle
SM3dR: geschlossene Candidate-/Builder-Schicht ohne TwoSidedInv und ohne freie Inverse
SM3daR: formal geschlossene, aber positiv obstruierte CandidateEntryInterface-Schicht
SM3eR: gruen geschlossene Obstruktion; candidateEntry_eq_interiorBlock, kein provenTwoSidedInv
SM3db0R: Positive-First-Startschutz geschlossen
SM3db1R: GeneratedInteriorEliminationCarrier geschlossen; keine Trace, keine Inversen-Entry-Fu
SM3db2R: closed/green; lokaler positiver Eliminationsschritt aus SM3db1R-Node und SM3cR/SM3dR
SM3db3R: closed/green; finite/tree-recursive Trace aus SM3db1R plus SM3db2R
SM3db4R: green/partial; erste Trace-basierte GeneratedInteriorInverseCandidateEntry-Schale
SM3db4aR: closed/green; akkumulierte TraceEvaluation / GeneratedInteriorAccumulatedEntryFold
SM3db5R: closed/green; reiner Matrix-Export der SM3db4aR-accumulatedEntry
SM3db6R: closed/green; Shape-/Source-Separation ohne Produktbeweis
SM3db7R: closed/green; Handoff ohne Produktvalidierung
SM3dbOmegaR: nur nach dokumentiertem positivem Scheitern eines aktiven SM3db7R-Handoff-Versuch
SM3eR-r3a: closed/green; ProductNormalForms aus SM3db7R-Handoff
SM3eR-r3b: closed/green; bedingte CancellationSurface aus r3a-Normalformen
SM3eR-r3c0: closed/green; TraceCancellationDerivation-API
  and Ledger-Export relativ zu Derivation
SM3eR-r3c1: interface closed/green;
  AccumulatedEntryCancellationInvariant aus Trace/Fold
SM3eR-r3c1a: closed/green; BlockFold-Normalform / Expose
SM3eR-r3c1b0: closed/green; LocalStepSchurCancellationSourceAudit
SM3db2aR: closed/green-relative; LocalStepSchurCancellationInvariant
SM3db2bR: closed/green-relative; PerPivotRegularity from TracePivotScaleWitness
SM3db2cR: audit-closed/conditional; TracePivotScaleWitnessFromGeneratedDegree adapter
SM3db2dR: green-relative; ReciprocalProfile adapter / ScaleSource from profile
SM3db2eR0: closed/green; GeneratedDegreeFormulaAudit
SM3db2eR-full: closed/green; GeneratedDegreePivotReciprocalProfileFromWeightPolicy audit
SM3db2fR: closed/green; GeneratedOperationalDegreeReciprocalSource
SM3eR-r3c1b1: closed/green; BlockFoldKernel-to-LocalSchurResidualBridge
SM3eR-r3c1b-full: closed/green; lokale Step-Kuerzung
SM3eR-r3c1c: closed/green; Fold-Invarianz
SM3eR-r3c1d: green/gate; terminal identity conditional on TerminalCoverageSource
SM3db3aR: closed/green-audit; TracePivotTerminalCoverageSource
SM3db4aT: closed/green-audit; TerminalFoldResidualCoverage
SM3db4aU: closed/green-audit; TerminalIdentityFields-Audit
SM3db4aV: closed/green; TerminalFoldResidualIdentityEquationSource
SM3db4aW: closed/green; TerminalIdentityFieldCertificateFromEquationSource
SM3eR-r3c1d-full: closed/green; TerminalIdentity from SM3db4aW TerminalCoverageSource
SM3eR-r3c1e: closed/green; AccumulatedEntryCancellation-Konstruktor from TerminalIdentity
SM3eR-r3c2: closed/green; r3c1e-Witness -> TraceCancellationInvariant
  and ProductCancellationLedger
SM3eR-r4a: closed/green; Ledger -> ProductIdentityProof, structural only
SM3eR-r4: collapsed/closed-by-r4a
SM3eR-r5: active; ProductIdentityProof -> TwoSidedInv
AR12a-SMOKE: erster echter Rechentest aus ontischem ToC-Substrat
AR13--AR16: nur nach Smoke-Ergebnis als echte Consumer
```

Wenn Eisenstein, Gauss und L>3 hier wirklich aus dem ToC-Kern fallen, ist das keine kosmetische numerische Bestaetigung, sondern ein fundamentaler Test, ob CNNA Zahlen-/Ring-/Objektstruktur

als abgeleitete ToC-Arithmetik tragen kann.

3. Minimaler Modulbaum

```
CNNA/PillarA/Arithmetic/
  BuildAll.lean
  Notation.lean
  Core/Source.lean
  Core/FullBoundaryAudit.lean
  Core/LevelHistoryCarrier.lean
  BoundarySource/ExistingIdealThreadNoGo.lean
  BoundarySource/BoundarySourceInterface.lean
  BoundarySource/LevelHistoryIndex.lean
  BoundarySource/LevelStepWitness.lean
  BoundarySource/RecursiveLevelHistoryCarrier.lean
  BoundarySource/LevelHistoryMatrix.lean
  BoundarySource/MultiSchurToLevelHistoryAdapter.lean
  BoundarySource/BoundarySourceTypeDecision.lean
  BoundarySource/RecursiveLevelHistoryTypeDecision.lean
  BoundarySource/BoundarySourceDecisionLedger.lean
  BoundarySource/GeneratedSubstrateRoute.lean
  BoundarySource/BranchAddressSubstrate.lean
  BoundarySource/BoundaryStateSubstrate.lean
  BoundarySource/HistoryExpandedSubstrate.lean
  BoundarySource/InterfaceExposureSubstrate.lean
  BoundarySource/RecursiveLevelHistoryRoute.lean
  BoundarySource/BoundarySourceConvergence.lean
  Actions/GeneratorActionInventory.lean
  Actions/ActionPackage.lean
  Actions/TreeBranchAction.lean
  Actions/BrightDarkInvolutionAction.lean
  Actions/LevelHistoryAction.lean
  Actions/MobiusSchurAction.lean
  Actions/CMActionInterface.lean
  Actions/CDActionInterface.lean
  Actions/VOAActionInterface.lean
  Actions/RootSystemActionBoundary.lean
  Actions/RootSystemPrecisionLedger.lean
  Actions/RootlessChiralActionTest.lean
  Actions/LatticeActionObstruction.lean
  Actions/ActionFormatDecision.lean
  Actions/SelfSimilarTreeAction.lean
  Actions/WreathRecursionBoundary.lean
  Actions/BranchGroupObstruction.lean
  Actions/LocalSectionActionLedger.lean
  Actions/PrescribedLocalAction.lean
  Actions/LocalToGlobalTreeActionObstruction.lean
  Actions/GroupoidActionPackage.lean
  Actions/PartialSymmetryGroupoid.lean
  Actions/BoundaryGroupoidLedger.lean
  Actions/HeckeSchurOperatorInterface.lean
  Actions/DoubleCosetTransferLedger.lean
  Actions/SchurHeckeCompatibilityTest.lean
```

Actions/CoherentConfigurationShadow.lean
Actions/AssociationSchemeObstruction.lean
Actions/MultiSchurAdjacencyAlgebra.lean
Actions/ArborealGaloisComparison.lean
Actions/PreimageTreeActionObstruction.lean
Actions/BruhatTitsBuildingComparison.lean
Actions/RamanujanComplexObstruction.lean
Actions/PAdicGroupActionLedger.lean
Actions/VOAOrbifoldActionGate.lean
Actions/GCrossedCategoryComparison.lean
Actions/TwistedSectorLedger.lean
Actions/DenominatorFormulaActionTest.lean
Actions/RootMultiplicityLedger.lean
Actions/GeneralizedKacMoodyComparison.lean
Actions/GrothendieckTeichmuellerComparisonOnly.lean
Actions/OperadicGraphComplexWatchlist.lean
Actions/ActionConvergenceLedger.lean
Core/BrightDtNSource.lean
Core/MatrixTransport.lean
Schur/MobiusTransfer.lean
Schur/RecursiveSchurSource.lean
Schur/Recurrence.lean
Schur/CharpolyExec.lean
Schur/CharpolyC.lean
Graph/IharaZetaSeed.lean
Graph/RamanujanTest.lean
Graph/SpectralGapSeed.lean
Graph/CheegerSeed.lean
Infinite/InfiniteSubstrateRegistry.lean
Infinite/ReferenceThreadInfinity.lean
Infinite/RegularTreeIdealBenchmark.lean
Infinite/RealInfinityDirectedSystem.lean
Infinite/PeriodicOrQuotientREALInfinity.lean
Infinite/VariationREALInfinity.lean
Infinite/MeasureGraphZetaInterface.lean
Infinite/FiniteToInfiniteSpectralTransfer.lean
Infinite/IntegratedDensityOfStatesSeed.lean
Schur/L1Baseline.lean
Schur/L2Eisenstein.lean
Schur/L3Gaussian.lean
Schur/HigherLevelDiscriminants.lean
Schur/HigherLevelPredictionLedger.lean
Schur/HigherLevelOutcomeRouter.lean
Schur/HigherLevelCertificates.lean
Orders/QuadraticOrder.lean
Orders/DiscriminantConvention.lean
Orders/NormForms.lean
Orders/CMPoint.lean
Orders/HeegnerUFDIndicator.lean
Modular/PeriodLattice.lean
Modular/WeierstrassInvariants.lean
Modular/JInvariant.lean
Modular/HeegnerJIntegrality.lean

Modular/QExpansion.lean
Modular/LFunctionSeeds.lean
Moonshine/GradedCarrierSeed.lean
Moonshine/CharacterTests.lean
Moonshine/LorentzianLeechSeed.lean
Moonshine/VOACandidateInterface.lean
Moonshine/GeneralizedKacMoodyInterface.lean
Moonshine/MoonshineBridge.lean
Moonshine/SchurMultiplierGate.lean
CayleyDickson/NormCandidate.lean
CayleyDickson/ArithmeticCDBridge.lean
CayleyDickson/HurwitzBridge.lean
Convergence/GeneratorAspectSplit.lean
Convergence/AlgebraArithmeticInterface.lean
Convergence/CDArithmeticCompatibility.lean
Convergence/QuaternionRamanujanInterface.lean
Convergence/CMArithmeticGeometryInterface.lean
Geometry/EllipticFunctionFirstGate.lean
Geometry/WeierstrassEllipticFunctionLayer.lean
Geometry/EisensteinLatticeSumLayer.lean
Geometry/WeierstrassInvariantLayer.lean
Geometry/JInvariantLayer.lean
Convergence/VOAHurwitzBoundaryTest.lean
Convergence/ConvergenceOutcomeLedger.lean
Smoke/GeneratedSource.lean
Smoke/GeneratedSourceObstruction.lean
Smoke/GeneratedInteriorEliminationCandidateEntry.lean
Smoke/GeneratedInteriorEliminationCarrier.lean
Smoke/GeneratedInteriorEliminationStep.lean
Smoke/GeneratedInteriorEliminationTrace.lean
Smoke/GeneratedInteriorInverseCandidateEntry.lean
Smoke/GeneratedInteriorInverseCandidateMatrix.lean
Smoke/GeneratedInteriorInverseCandidateShapeSeparation.lean
Smoke/SM3dbRToSM3eRHandoff.lean
Smoke/GeneratedInteriorTraceCancellationDerivation.lean
Smoke/GeneratedInteriorBlockFoldNormalFormR3c1a.lean
Smoke/GeneratedInteriorLocalStepSchurCancellationSourceAuditR3c1b0.lean
Smoke/GeneratedInteriorLocalStepSchurCancellationInvariantSM3db2aR.lean
Smoke/GeneratedInteriorAccumulatedEntryCancellationR3c1.lean
Smoke/GeneratedInteriorTraceCancellationR3c2.lean
Smoke/GeneratedInteriorTwoSidedInvFromSM3db7R.lean
Smoke/SM3dbOmegaObstructionLedger.lean
Smoke/GeneratedLevelWitness.lean
Smoke/LevelHistoryToSchurIndex.lean
Smoke/GeneratedSchurEntry.lean
Smoke/GeneratedLevelHistoryMatrix.lean
Smoke/GeneratedCharpoly.lean
Smoke/L2GeneratedEisenstein.lean
Smoke/L3GeneratedGaussian.lean
Smoke/HigherGeneratedSmoke.lean
Smoke/AR12aGeneratedSmokeLedger.lean
Export/ArithmeticSeedExport.lean

Zusaetzliche Hauptpfad-Module fuer AR12a-SMOKE.

```
CNNA/PillarA/ToC/GeneratedBranchFamily.lean
CNNA/PillarA/Finite/GeneratedBranchApproximant.lean
CNNA/PillarA/Coupling/GeneratedBranchMultiSchur.lean
CNNA/PillarA/Finite/CutSpecFutureUseLedger.lean
CNNA/PillarA/StringTheory/IdealThreadStringMotivation.lean   -- future/comparison only
CNNA/PillarA/StringTheory/CutSpecWorldsheetWatchlist.lean     -- future/comparison only
```

4. Outcome-, Ledger- und Identitaetsstatus

4.1 Fuenf Gesamtausgaenge des AR25-Ledgers

Starke Objekt-und-Aktions-Konvergenz	Algebra-Pfad und Arithmetik-Pfad liefern kompatible Daten ueber dasselbe abgeleitete Objekt und kompatible Wirkungen darauf: gemeinsamer Traeger, gemeinsame Konjugation/Norm/Form, kompatible CM- oder Quaternionenordnung, kompatible Spektral- und Modulardaten sowie Action-Preservation. Folge: `ArithmeticGeometrySeed.lean`, `ActionConvergenceLedger.lean` und gemeinsame Invariantensprache als spaeterer Theorem-Seed.
Objektkonvergenz ohne Aktionskonvergenz	Gemeinsame Traeger- oder Invariantendaten existieren, aber keine kompatible Wirkung erhaelt alle relevanten Strukturen. Folge: Objektseed darf exportiert werden, Action-Status bleibt Obstruktion.
Aktionskonvergenz ohne Objektkonvergenz	Die Pfade liefern nicht dasselbe Objekt, aber funktoriell verbundene Traeger mit kompatiblen Wirkungen oder denselben Invarianten unter verschiedenen Wirkungen. Folge: nur Action-/Invariant-Seed, keine globale Objektvereinheitlichung.
Lokale Konvergenz, globale Obstruktion	Einzelne Treffer existieren, kleben aber nicht zu einem globalen Objekt; z.B. L=2/L=3 passen zu CM-Ordnungen, Ramanujan/Gaps zeigen keine quaternionische Struktur oder ein VOA-Seed besitzt keine CD-kompatible Form. Folge: `ConvergenceObstructionLedger.lean`; keine globale Vereinheitlichung.
Keine Objekt- oder Aktionskonvergenz	Algebra und Arithmetik bleiben orthogonale Projektionen desselben Generators. CD/Hurwitz bleibt A-interner algebraischer Strukturpfad; Arithmetik bleibt Schur/CM/Modularitaets-/VOA-Pfad; spaetere Pillars duerfen beide getrennt konsumieren, aber keine gemeinsame Erklaerung und keine gemeinsame Symmetrie voraussetzen.

Action-Substatus im AR25-Ledger. `ActionConvergenceLedger` fuehrt zusaetzlich zu den fuenf Hauptausgaengen eine feingranulare Wirkungsdiagnose: `RootSystemActionWitness`, `RootSystemActionObstruction`, `RootlessChiralActionWitness`, `RootlessChiralActionObstruction`, `SporadicLatticeActionCandidate` und `ExternalLeechComparisonOnly`. Hinzu kommt der Wirkungsformatstatus: `GlobalGroupAction`, `LocalTreeAction`, `SelfSimilarWreathAction`, `GroupoidPartialAction`, `HeckeOperatorAlgebraCandidate`, `CoherentConfigurationCandidate`, `ProfiniteGaloisComparisonOnly`, `BuildingComparisonOnly`, `VOAOrbifoldComparisonOnly`, `DenominatorFormulaComparisonOnly` und `WatchlistOnly`. Der Leech-Status `ExternalLeechComparisonOnly` ist zwingend, solange kein A-intern erzeugter 24-dimensionaler gerader unimodularer rootless Lattice-Traeger samt Action-Preservation vorliegt.

4.2 Priorisierte Minimalziele

1. **MVP-1:** AR0–AR10d. Full-Boundary-Negativaudit, Existing-IdealThread-No-go, BoundarySource-Bifurkation mit konditionaler Generated-Route und aktuell positiver Recursive-Route, AR3a-Generator-Wirkungsinventar, AR3b-Wirkungsformat-Entscheidung, Schur-Rekursion, Charpoly-Identitaeten, L=2 Eisenstein, L=3 Gauss, AR6a-Complex-Coupling-Test, Spektral-Gap-/Ramanujan-Diagnostik, AR10d-Quaternion/Ramanujan-Diagnose und L>3-Vorhersageledger.

2. **MVP-2:** AR11–AR16 mit AR12a-SMOKE-Sperre. L>3-Diskriminantenpfad, Diskriminanten-Konventionsbruecke, AR12a-Algebra-Diskriminanten-Test und danach ontischer Rauchtest aus

generated Hauptpfad-Substraten: L=2 Eisenstein, L=3 Gauss und L>3-Ausgaenge muessen berechnet, positiv verglichen oder praezise obstruiert werden. Erst danach laufen quadratische Ordnungen, CM-Gitter, Weierstrass-$\wp$-Schicht, Eisenstein-Gittersummen G_4/G_6, g_2/g_3, Δ, j, AR15a-CM-Geometrie-Interface und Heegner-/UFD-Indikatoren als echte Consumer.

3. **MVP-3:** AR17–AR23a, inklusive AR18b/AR18c. q-/Laurent-Reihen, L-/Modularform-Seeds, Ihara-/Graph-Zeta-Seed, IDEAL∞-/REAL∞-Arithmetic-Envelope, graduierter Traeger, Charaktertests, Lorentz/Leech-Gate mit Rootless/Chiral-Lattice-Test, VOA/Moonshine-Kandidateninterface und AR23a-VOA/Hurwitz-Boundary-Test.

4. **MVP-4:** AR24–AR25. Lie-Typ-/Residuenvergleich und AR25-Objekt-/Aktions-Konvergenztest als Ledger der Kontaktstellen AR3a, AR3b, AR6a, AR10d, AR12a, AR15a und AR23a, inklusive RootSystem-/RootlessChiral-/SporadicLattice-Substatus.
5. **Fernziele:** AR28 als Schur-Multiplier-Gate erst nach einer abgeleiteten Gruppen-/VOA-/Moonshine-Aktion; zusaetzlich ein String-/Worldsheet-Rekonstruktionspfad nur als Watchlist nach erfolgreicher nicht-singleton Ambient-Substrat- und CutSpec-Expositionsanalyse. Beide Fernziele haben keinen Einfluss auf die fruehe Schur-Eliminierungs- und Arithmetik-Closure.

4.3 Pruefbare Identitaetsziele

Carrier	Full-Negativaudit plus CutSpec-Spike: keine `CutSpec.full`-Aequivalenz zu `Fin (L+1)`. Fuer AR12a-SMOKE ist die positive Quelle nicht CutSpec und nicht IdealThread, sondern ein generated nicht-singleton Hauptpfad-Substrat ueber `RegularCell code-b` bzw. `LevelVariableCell beta`; `arity = code-b + 1`. `CutSpec` bleibt Vergleichs-/Future-Use-Schicht.
Schur	$\sigma_k = p_k/q_k$, alle Rekursionsgleichungen und root-Zaehler theorematisch. AR12a-SMOKE verlangt zusaetzlich eine Eintragsformel `matrix i j = realToExec (source.multiSchur.brightBrightSchur ...)` oder einen praezisen Obstruktionsstatus.
Charpoly	$\chi_{\mathrm{brightDtN},L}(\lambda) = p_0(\lambda)$, berechnet aus der realen Matrix. Im Smoke-Gate muessen L=2 Eisenstein, L=3 Gauss und erste L>3-Zeilen aus dieser Matrix berechnet und danach positiv verglichen oder als Zielabweichung dokumentiert werden.
L=2	Berechnete Smoke-/Schur-/Charpoly-Daten muessen zuerst eine quadratische Route liefern; erst dann Vergleich mit Eisenstein-Normform, Diskriminantenkonvention zur Fundamentaldiskriminante -3 und CM-Seed ρ. Kein freier Winkel- oder Zeichnungsinput.
L=3	Berechnete Smoke-/Schur-/Charpoly-Daten muessen zuerst eine quadratische Route liefern; erst dann Vergleich mit Gaussian-Normform, Konventionsbruecke zur Fundamentaldiskriminante -4 bzw. Feld $\mathbb{Q}(i)$ und CM-Seed i. Kein Rueckschluss aus 90-Grad-Zeichnung.
L>3	Vorhersage-/Testmatrix vor AR11; danach genau einer von drei Ausgaengen: quadratischer Heegner-Treffer, quadratischer Nicht-Heegner-Fall oder irreduzibler/hoehergradiger Fall. Jeder Ausgang benoetigt ein eigenes Zertifikat und eigene Folgeroute; keine Klassifikation ohne Beweis.
SM3eR-Re-Run	r0–r2 sind Produktoberflaeche, nicht Produktidentitaet. r3a Produktnormalformen, r3b CancellationSurface und r3c0 TraceCancellationDerivation-API sind geschlossen; r3c1 als echte Kancellationsableitung ist aktiv. r4a darf nur konditional den Ledger-to-ProductIdentityProof-Konstruktor vorbereiten; r4 closed, r5 TwoSidedInv und SM3fR bleiben blockiert. `ProductProofGate`, `waitingForLeftAndRightProductIdentityProofs`, r3c0 allein und ein bloss bedingter r3b-Witness sind keine SM3fR-Eingaben.
Geometrie-Heuristik	Spiralige 0/1-Baumzeichnungen, Winkel, Eisenstein-/Gauss-Gitter und CM-Bilder sind nur nachgelagerte Vergleichs-/Diagnosebilder. Im derived-only Pfad zaehlen nur berechnete Generated-DtN-/Schur-/Charpoly-/Diskriminantendaten.
Modular	$G_4, G_6, g_2, g_3, \Delta, j$ nur aus CM-/Gitterdaten; q-/Laurent-Reihen getrennt von Spektralparameter.
Heegner	Positive d-Konvention und Fundamentaldiskriminanten sauber trennen: $d \in \{1, 2, 3, 7, 11, 19, 43, 67, 163\}$, UFD/class-number-one-Status und j-Ganzzahligkeit nur als abgeleitete Indikator- und Testdaten.
Graph-Zeta	Ihara/Bass-Hashimoto nur aus realem endlichem Graphtraeger; allgemeine Gradmatrix-Version vor regulaerer Spezialformel; reiner Baumfall als trivialer Zeta-Fall; Zyklen/nicht-backtracking walks als explizite Voraussetzung.
Infinity-Envelope	Substratklassifikation vor jeder Grenzaussage: `ReferenceThreadInfinity`, `RegularTreeIDEALInfinity`, `REALInfinityDirectedSystem`, `PeriodicOrQuotientREALInfinity`, `VariationREALInfinity beta`, `WeightedOrMeasureGraphREALInfinity`; Transfer nur via explizitem Konvergenz-/Spektralmass-/Spurbeweis.
Ramanujan	Klassische Schranke |lambda| <= 2 sqrt(q) nur fuer bewiesen (q+1)-regulaere Graphen oder separat definierte Regularisierung; sonst nur Defekt-/Vergleichsdatum gegen Universal-Cover- oder normalisierte Schranken.
Gap/Cheeger	Spektrale Luecke und Cheeger-Seed aus Adjazenz/Laplacian/bright-DtN als A-seitige Diagnosedaten; physikalische Lesarten wie Mixing, Dekohaerenz oder Thermalisierung erst in spaeteren Pillars.
Moonshine	q-Charaktere und Koeffizientenledger vor VOA/Monster; Lorentz/Leech/VOA/Kac-Moody-Bruecken vor Gruppenwirkung.
Schur-Multiplier	Nur nach echter Gruppen-/VOA-Aktion: lineare vs. projektive Aktion, 2-Kozyklus, Klasse in $H^2(G, \mathbb{C}^\times)$, zentrale Erweiterung/Schur-Cover; keine Rueckprojektion auf Schur-Eliminierung.
CD/Hurwitz	Bestehende gruene CD/Hurwitz-Struktur darf angeschlossen und verwendet werden; Norm-/Kompositionskandidaten aus Spektral-/Sektorstruktur; Hurwitz-Stopp nur dort als intern geschlossen ausgeben, wo der importierte gruene Strukturpfad dies tatsaechlich theorematisch traegt.

5. Pillar-Ausblick, Infinity-Envelope und Seed-Pflichten

5.1 Pillar-Ausblick: vorbereitende Arithmetik-Seeds

Pillar B / OQS	`SpectralGapSeed`, Gibbs-/KMS- und Kanal-Pattern nur als Legacy-inspirierte Schnittstellenformen; A liefert Eigenwerte, Gap, Zustandsgewichte und Trunkierungsdaten, aber keine thermische Fachtheorie.
Pillar C / Geometrie	`CheegerSeed`, Expansion, isoperimetrische Profile und Graphmetriken als endliche A-Seeds; kontinuierliche Geometrie, Raumzeit- oder Kausalaxiome bleiben spaetere Theorem-Seeds.
Pillar D / Symmetrie/Matter	Ramanujan-Defekt, Residuen, Lie-Typ-Vergleiche, Action-Pakete/Obstruktionen und AR25-Konvergenz-/Obstruktionsstatus liefern vorbereitende arithmetisch-algebraische Tests; keine Gauge- oder Matter-Axiome in A.
Pillar E / Hochsymmetrie	Ihara-/Graph-Zeta, L-/Modularform-Seeds, graduierte Charaktere, Lorentz/Leech/VOA/Moonshine, AR23a-VOA/Hurwitz-Gate und Schur-Multiplier bleiben stufenweise Gates: erst Zeta/Charakterdaten, dann Traeger, dann Aktion, dann algebraische Boundary-/projektive-lineare Pruefung.

5.2 IDEAL∞-/REAL∞-Envelope

ReferenceThread ∞	Degenerierter Singleton-Faden pro Level; Kontrollobjekt fuer Boundary-No-go und Level-History-Rekursion; keine nichttriviale Graph-Zeta- oder Expanderquelle.
RegularTree IDEAL ∞	Universeller regulaerer Idealtraeger; Benchmark fuer Spektralmass, Resolvente, Kesten–McKay-/Ramanujan-Vergleich und Universal-Cover; keine naive prime-cycle-Ihara-Zeta.
REAL ∞ DirectedSystem	Gerichtetes System endlicher REAL-Approximanten; Ort fuer lokale Konvergenz, Spektralmass-Transfer, Randartefakt-Abzug und Stabilitaet von Charpoly-/Gap-/Zeta-Diagnostik.
Periodic/Quotient REAL ∞	Periodische, residuell finite oder quotientierte Cover-Strukturen; natuerlicher Ort fuer nichttriviale Ihara-/Graph-Zeta und L-Funktionsbruecken, aber nur mit Fundamentalbereich, Gruppenwirkung und Determinantenform.
Variation REAL ∞ β	Branching- und Carrier-Variationen; die aktuell vorhandene `IdealThread`-Variation ist Boundary-Source-No-go, waehrend kuenftige nicht-singleton Variationen als Kandidaten der `GeneratedNonSingletonSubstrateRoute` dienen koennen. Zusaetzlich Testfeld fuer Regularitaetsdefekte, nichtregulaere Spektraldiagnostik und robuste Boundary-Strukturen.
Weighted/MeasureGraph REAL ∞	Allgemeinstes Regime; nur zulaessig mit derived-only Gewichtung, invariantem Mass, Spur/Trace, Gruppoid- oder Graphing-Struktur und passendem Determinantenbegriff.

5.3 Neue A-seitige Seed-Pflichten

- `Graph/IharaZetaSeed.lean`: Bass-Hashimoto-/Ihara-Daten aus dem realen endlichen Graphen; Regularitaets- und Zyklizitaetsgate.
- `Graph/RamanujanTest.lean`: Ramanujan- oder Defekt-Test nur nach Grad-/Regularitaetsnormalisierung; LPS-/Quaternionenbezug nur als spaeter Vergleich zum CD-Pfad.
- `Graph/SpectralGapSeed.lean` und `Graph/CheegerSeed.lean`: Gap, normalisierte Luecke und isoperimetrischer Seed als Exportdaten fuer spaetere OQS-/Geometrie-Pillars.
- `Infinite/InfiniteSubstrateRegistry.lean`: Pflichtklassifikation jedes unendlichen Substrats vor Zeta-, Spektralmass-, Ramanujan-, L-Funktions- oder Gap-Aussagen.
- `Infinite/FiniteToInfiniteSpectralTransfer.lean` und `Infinite/IntegratedDensityOfStatesSeed.lean`: Transfer von endlichen REAL-Approximanten zu IDEAL∞-/REAL∞-Spektralmass nur ueber explizite Konvergenz- und Randartefaktkontrolle.
- `Infinite/PeriodicOrQuotientREALInfinity.lean` und `Infinite/MeasureGraphZetaInterface.lean`: nichttriviale unendliche Graph-Zeta nur bei abgeleiteter Periodizitaet, Quotientierung, Mass-/Spur- oder Gruppoidstruktur.
- `BoundarySource/ExistingIdealThreadNoGo.lean`: universelles No-go fuer alle aktuell vorhandenen `ToCStrong.ofIdeal`/`IdealThread`-Substrate.
- `BoundarySource/GeneratedSubstrateRoute.lean`, `BoundarySource/BranchAddressSubstrate.lean`, `BoundarySource/BoundaryStateSubstrate.lean`, `BoundarySource/HistoryExpandedSubstrate.lean` und `BoundarySource/InterfaceExposureSubstrate.lean`: konditional offene Carrier-first-Route fuer legale nicht-singleton Substrate.

- `BoundarySource/BoundarySourceTypeDecision.lean`, `BoundarySource/RecursiveLevelHistoryTypeDecision.lean` und `BoundarySource/BoundarySourceDecisionLedger.lean`: dokumentieren AR2b-D als geschlossene Entscheidung fuer Route II; Status `RecursiveTypeDecisionClosed`, solange Adapter an S9h-/GeneralizedDtN-/MultiSchur-Provenienz anschliessbar sind.
- `BoundarySource/LevelHistoryIndex.lean`, `BoundarySource/LevelStepWitness.lean`, `BoundarySource/RecursiveLevelHistoryCarrier.lean`, `BoundarySource/LevelHistoryMatrix.lean` und `BoundarySource/MultiSchurToLevelHistoryAdapter.lean`: konkrete AR2b-D-Signatur fuer Route II; `Fin (L+1)` ist nur History-Index, `witnessAt` traegt die per-level-Provenienz, `LevelHistoryMatrix L` ist die operative `ExecComplex`-Matrix, `MultiSchurStrong` bleibt Provenienzanker.
- `BoundarySource/RecursiveLevelHistoryRoute.lean`: operator/history-first-Route nach `RecursiveTypeDecisionClosed`; kein Platzhalter fuer eine freie `Fin (L+1)`-Boundary und kein frei gespeichertes Matrixfeld ohne `matrix_from_source`.
- `BoundarySource/BoundarySourceInterface.lean` und `BoundarySource/BoundarySourceConvergence.lean`: gemeinsame Konsumoberflaeche und spaeterer Test, ob Generated- und Recursive-Route dieselbe Boundary beschreiben oder eine Obstruktion liefern.
- `Actions/ActionPackage.lean`: minimale Signatur fuer Traeger, wirkendes Objekt, Wirkungsabbildung, Gesetze, Preservation-Lemmata, Invarianten und Obstruktionsstatus.
- `Actions/GeneratorActionInventory.lean`: AR3a-Inventar der aus `BoundarySource`, Schur/DtN, Bright/Dark, CD und CM ableitbaren Wirkungen; nur Ledger, keine freie Symmetriebehauptung.
- `Actions/ActionFormatDecision.lean`: AR3b-Entscheidungsphase; klassifiziert jeden AR3a-Kandidaten vor mathematischem Ausbau als global, lokal, selbstaehnlich/wreath-rekursiv, branch-artig, groupoid/partiell, Hecke/Double-Coset-operatorisch, coherent-configuration-artig, profinit/Galois-artig, building-artig, VOA-orbifold-kategorial oder reine Watchlist.
- `Actions/TreeBranchAction.lean`, `Actions/BrightDarkInvolutionAction.lean`, `Actions/LevelHistoryAction.lean` und `Actions/MobiusSchurAction.lean`: fruehe Action-Kandidaten mit Preservation-Sperren fuer Cutoff, BoundarySource, Levelordnung und Schur/DtN-Daten.
- `Actions/CMActionInterface.lean`, `Actions/CDActionInterface.lean` und `Actions/VOAActionInterface.lean`: spaetere Action-Interfaces fuer Modularitaet/CM, Cayley-Dickson und VOA/Moonshine.
- `Actions/RootSystemActionBoundary.lean`, `Actions/RootlessChiralActionTest.lean` und `Actions/LatticeActionObstruction.lean`: Abgrenzung zwischen Weyl-/Cartan-/Root-System-Wirkungen, rootless/chiralen Lattice-Symmetrien, sporadischen Kandidaten und externem Leech-Vergleich.
- `Actions/RootSystemPrecisionLedger.lean`: trennt root set, simple-root basis, root lattice, Cartan integers, reflection closure, Weyl action und rootless/chiral obstruction.
- `Actions/ActionConvergenceLedger.lean`: AR25-Wirkungsledger mit Hauptstatus plus Substatus `RootSystemActionWitness`, `RootSystemActionObstruction`, `RootlessChiralActionWitness`, `RootlessChiralActionObstruction`, `SporadicLatticeActionCandidate`, `ExternalLeechComparisonOnly`.
- `Convergence/GeneratorAspectSplit.lean`: formale Trennung von Algebra-Seite und Arithmetik-Seite als zwei Consumer desselben A-Generators.
- `Convergence/AlgebraArithmeticInterface.lean`, `Convergence/CDArithmeticCompatibility.lean` und `Convergence/QuaternionRamanujanInterface.lean`: fruehe Konvergenztests AR6a, AR10d und AR12a ohne freie Daten.
- `Convergence/CMArithmeticGeometryInterface.lean`, `Convergence/VOAHurwitzBoundaryTest.lean` und `Convergence/ConvergenceOutcomeLedger.lean`: spaetere Interface- und Ledgerpflichten AR15a, AR23a und AR25.
- `Geometry/EllipticFunctionFirstGate.lean`, `Geometry/WeierstrassEllipticFunctionLayer.lean`, `Geometry/EisensteinLatticeSumLayer.lean`, `Geometry/WeierstrassInvariantLayer.lean` und `Geometry/JInvariantLayer.lean`: AR14–AR15a-Pflichtkette $\Lambda, \wp, G_4, G_6, g_2, g_3, \Delta, j$ vor q-, L-Funktions-, VOA- oder Moonshine-Consumern.

- `ToC/GeneratedBranchFamily.lean`, `Finite/GeneratedBranchApproximant.lean` und `Coupling/GeneratedBranchMultiSchur.lean`: Hauptpfad-Erweiterungen fuer AR12a-SMOKE; sie erzeugen die ontische generated Quelle vor jeder Arithmetik-Konsumtion. `Arithmetic/Smoke/*`: konsumiert diese Quelle fuer L=2/L=3/L>3-Rechnung, ohne `CutSpec.full` als Fundament zu verwenden.

6. Externe Vergleichsanker und Herkunft der Gates

6.1 Uebernahme aus der alten Planung

Behalten	Dualstrang Exec/$\mathbb{C}$, Spektralwurzel als Ursprung, Schur/Moebius-Rekursion, L=2 Eisenstein, L=3 Gauss, CM/Gitter/Weierstrass/j, Heegner-/UFD-Indikatorblock, q-Disziplin, Nicht-VOA-Gate, Lorentz/Leech/VOA/Moonshine nur als spaetere Test-/Kandidatenphasen.
Verwerfen	Altmodule unter `PillarA/Arithmetic/*`; freies `ArithmeticSignature`-Starten; CM-Punkt als Ursprung; konditionale Charpoly-Abschluesse; externe `Fin n`-Carrier; direkte Moonshine-Semantik aus einer finite Charpoly.
Korrigieren	Die alte A14c-1-Luecke wird haerter gefasst: `referenceFullMultiSchur` auf `CutSpec.full` ist nicht mehr positive Quelle, sondern zuerst Negativ-Audit. Zusaetzlich wird der aktuelle Variation-Pfad nicht mehr als reicher Traeger behandelt, solange er ueber `ToCStrong.ofIdeal/IdealThread` laeuft. AR2b wird als Boundary-Source-Bifurkation mit Entscheidungsphase gefuehrt: AR0, AR1, AR2a und AR2b-0 sind sofort implementierbar; AR2b-2 darf erst nach Typ- und API-Entscheidung fuer `RecursiveLevelHistoryBoundarySource` implementiert werden.
Anschliessen	Cayley-Dickson/Hurwitz darf aus dem bereits gruenen Strukturpfad angeschlossen und verwendet werden; dies geschieht nicht als spaetes Anhaengsel, sondern als Algebra-Arithmetik-Konvergenzquerachse mit fruehen Gates AR6a, AR10d, AR12a, AR15a, AR23a und dem Ledger AR25. Ergaenzend wird nach AR2b eine Wirkungs-/Symmetrie-Querachse eingefuehrt: Generator-, Boundary-, Schur-, CM-, CD- und spaetere VOA-Strukturen duerfen Gruppen/Monoid/Automorphismen nur als `ActionPackage` mit Traeger, Wirkung, Preservation und Invarianten konsumieren. REALOQS bleibt dagegen Legacy-Inspiration fuer fachliche Patterns, nicht Provenienz- oder Importquelle.

6.2 TomS-/Jakito- und Quellenanker: phasenspezifische Zuordnung

Diese Zuordnung ist eine Orientierungs- und Provenienzkarte fuer die mathematische Landkarte des Plans. Sie ist keine formale Beweisquelle, kein externer Axiomenspeicher und kein Ersatz fuer mathlib-/Lean-Beweise. TomS' Beitraege liefern Navigation; der CNNA-Code muss jede konsumierte Aussage derived-only oder ueber explizit markierte externe Standardsaetze schliessen.

AR8–AR9; TomS #11, #15, #17–#20
Imaginaer-quadratische Zahlringe, zwei Ordnungstypen fuer ω, Gitter ueber $\mathbb{C}$ und die Korrektur: Gauss/Eisenstein sind nicht beliebige konkrete Gitter, sondern einfachste Gitter-/Ordnungstypen. Planfolge: `L2/L3` identifiziert Ordnungstypen und Normformen erst nach Schur-Identitaetsbeweis; keine beliebige Gitterklassifikation aus L=2/L=3 extrapolieren.

AR12; TomS #9, #11–#20, #48
Prim versus irreduzibel, UFD-Problem, Klassenzahl-/Nicht-UFD-Warnung, quadratische Ganzzahlen $\mathbb{Z}[\omega]$ und Diskriminantenkonventionen. Planfolge: Pflichtlemma `schur_discrim_to_fundamental_discrim`; Schur-Diskriminante, Ordnung, Fundamentaldiskriminante und positives Heegner-d strikt trennen.

AR14–AR15; TomS #22, #30, #65
Modulargruppe/Moebiuswirkung, Modulfunktionen auf der oberen Halbebene, Eisensteinreihen G_{2k}, $g_2 = 60G_4$, $g_3 = 140G_6$, Δ und $j = 1728g_2^3/\Delta$. Planfolge: Weierstrass- und j-Daten sind Consumer von CM-/Gitterdaten; kein CM-Punkt als freier Start.

AR16; TomS #1–#10, #48, #72
Euler-/Heegner-Indikator, -163, UFD/class-number-one-Kontext, $e^{\pi\sqrt{163}}$-Phaenomen

und Heegner-Liste. Planfolge: Heegner/UFD ist Indikatorblock nach AR12–AR15; keine Eingabe fuer Schur, CM oder j.

AR17; TomS #30, #65, #72
$q_{\mathrm{mod}} = \exp(2\pi i\tau)$, Fourier-/q-Reihen der Eisensteinreihen und Korrektur: j hat eine Laurent-Reihe, nicht Taylor-Reihe. Planfolge: harte Parametertrennung zwischen `spectralParameter : Rat` und q_{mod}; j-Expansion nur als Laurent-Objekt.

AR18; TomS #49
L-Funktionen, Eulerprodukt, Dirichlet-Charaktere und multiplikative Koeffizientenstruktur; Riemann-ζ ist Spezialfall mit trivialen Charakteren, nicht zentraler Binder. Planfolge: L-Seed nur bei bewiesener Multiplikativitaetsstruktur; kein Eulerprodukt ohne Proof.

AR10b, AR18b, AR18c; externe Graph-/Spektralanker
Nicht primaer aus TomS, sondern aus etablierter Spektralgraphentheorie: spektrale Luecke, Cheeger-/Expansion, Ramanujan-Defekt sowie Ihara-/Bass-Hashimoto-Zeta werden als A-seitige Seeds ergaenzt. Planfolge: sie bleiben berechenbare Diagnostik- und Exportdaten, keine fruehen physikalischen Schlussfolgerungen und keine automatische Nichttrivialitaet fuer reine Baumtrunkierungen. AR18c ergaenzt darauf aufbauend die unendliche Regime-Disziplin: periodische, quotientierte, gemessene oder gerichtete Grenzobjekte duerfen nur mit abgeleiteter Zusatzstruktur als Zeta- oder Spektralmass-Traeger dienen.

AR19–AR20; TomS #65, #72
Monster-Darstellungsdimensionen und Koeffizientenledger: $196884 = 1 + 196883$, weitere Dimensionen $21296876, 842609326, \ldots$. Planfolge: nur Vergleichsdimensionen und Charaktertests; kein Monster-/VOA-Schluss aus Koeffizententreffern.

AR21; TomS #64, #72
Leech-Lattice, Conway-Gruppe, gerade/unimodular/chiral/rootless-Kontext; 26D-lorentzischer Gitter-/Weyl-Vektor-Ausblick und 24D-Leech-Projektion. Planfolge: eigenes Lorentz-/Leech-Gate bleibt Pflicht; Leech folgt weder aus j noch aus Koeffizienten allein.

AR22; TomS #72, #73
VOA als Zustands-/Weltflaechenstruktur, 26D-konforme Theorie mit 24D-Leech-Kompaktifizierung, Griess-Algebra, Monster-Lie-Algebra als verallgemeinerte Kac–Moody-Struktur. Planfolge: Interface formuliert nur Pflichtfelder/Gates: grading, vacuum, state-field map, locality/Jacobi, Virasoro/conformal vector, Kac–Moody/Griess-Schnittstelle.

AR23; TomS #65, #72, #73
Monstrous Moonshine, Conway–Norton, Borcherds, Monster-Modul, Monster als Automorphismengruppe geeigneter Struktur. Planfolge: Moonshine bleibt Brueckentest nach abgeleitetem Traeger und bewiesenen Charakteren; Monstergruppe nie als Input.

AR3a–AR3b, AR13–AR15, AR19–AR25; Wirkungs-/Symmetrie-Querachse
Gruppen, Monoide und Automorphismen dienen nur als mathematische Vergleichssprache, solange sie nicht als A-interne Aktionen konstruiert sind. Externe Orientierung: modulare Gruppenwirkung auf der oberen Halbebene, Galoisgruppen als Automorphismen von Koerpererweiterungen, Conway-/Leech- und Monster-/VOA-Automorphismen sowie Lie-Typ-Gruppen als Wirkungen auf abgeleiteten Traegern. Planfolge: `ActionPackage` vor jeder Symmetriebehauptung; `GeneratorActionInventory` nach AR2b; `ActionFormatDecisionLedger` nach AR3a; `ActionConvergenceLedger` in AR25. Root-System-, Weyl-/Cartan-, rootless/chiral-

und sporadische Lattice-Deutungen werden getrennt geloggert; Leech/Conway bleibt `ExternalLeechComparisonOnly`, bis ein eigener CNNA-Traeger bewiesen ist.

AR6a, AR10d, AR12a, AR15a, AR23a, AR25; Algebra-Arithmetik-Konvergenzquerachse
Externe Motivation, nicht CNNA-Provenienz: LPS-/Pizer-/JL-nahe Ramanujan-Konstruktionen zeigen echte Bruecken zwischen Spektrum/Expansion und Quaternionenalgebren; Borcherds' Monster-Lie-Algebra zeigt den spaeteren Moonshine-Zieltyp als algebraisch strukturiertes Objekt; Gross' CM-Monographie markiert elliptische Kurven mit komplexer Multiplikation als arithmetisch-geometrischen Schnittstellenanker. Planfolge: alle drei werden nur in Konvergenztests gespiegelt; kein externer Satz wird als A-interner Schluss importiert.

AR24; TomS #43–#53, #63
Endliche einfache Gruppen: zyklisch, alternierend/symmetrisch, Lie-Typ ueber endlichen Koerpern; Lie-Typ nicht als diskrete SO(n)-Untergruppe misslesen. Planfolge: Residuen-/Lie-Typ-Block bleibt Vergleichs- und Testblock; kein direkter E8-/Lie-Sprung.

String-Fernziel; IdealThread-/CutSpec-Hinweis
Nicht als CNNA-Provenienz, sondern als aus dem Codebefund motivierter Forschungsanker: `IdealThread` ist ein eindimensionaler Nullpfad im Ambient-Substrat; `RegularCell`/`LevelVariableCell` liefern den Target-/Ambientraum; `CutSpec` bleibt eine moegliche Expositions-, Defekt- oder Worldsheet-Schicht. Planfolge: Nur nach erfolgreicher generated-Substrat- und Smoke-Berechnung darf ein separater Rekonstruktionspfad fuer stringtheoretische Strukturen entworfen werden. Keine Aussage ueber Strings, Weltflaechen, Targetspaces oder Quantisierung fliesst in AR0–AR18 zurueck.

AR28; Schur-Multiplier-Ergaenzung
Nicht aus TomS, sondern aus der Anschlussfrage: projektive versus lineare Gruppen-/VOA-Aktion, $H^2(G, \mathbb{C}^\times)$, 2-Kozyklus, zentrale Erweiterung/Schur-Cover. Planfolge: reines Fernziel nach realer Gruppen-/VOA-Aktion; keine Rueckprojektion auf Schur-Eliminierung.

Weitere Quellen- und Statusanker

- TomS' spaetere Selbstrelativierung des Threads als oberflaechliche Orientierung wird als Statussperre gelesen: die obige Zuordnung dokumentiert Navigation, nicht Beweisautoritaet.
- Bestehende Cayley-Dickson-/Hurwitz-Infrastruktur im S9h-Anhang: `CNNA/Structural/CayleyDickson/BuildAll.lean`, `S6HurwitzStopSeed`, `S6FullAlternativeLawBoundary`, Norm-/Produkt-/Division-Slot-Module. Diese duerfen als aktive A-interne Strukturquellen angeschlossen werden, sofern Importketten- und Consumer-Disziplin eingehalten sind.
- REALOQS-Legacy: nur Inspirationsquelle fuer KMS/Gibbs/GNS/Stinespring- und Kanal-Patterns; kein Code-Transfer und kein aktiver Provenienzpfad des neuen Arithmetikplans.
- Externe ∞-Grapharithmetikanker: Ihara-Zeta fuer measure graphs, periodische Graph-Zeta, Benjamini–Schramm-/Weak-Convergence, Integrated Density of States, Kesten–McKay-/Regular-Tree-Spektralmass und Fuglede–Kadison-Determinanten dienen als Vergleichslandkarte. Sie werden nicht importiert, sondern in AR18c nur als Strukturpflichten gespiegelt: Mass/Spur/Gruppoid/Fundamentalbereich/Determinante muessen CNNA-intern bereitstehen oder die Aussage bleibt blockiert.
- Externe Algebra-Arithmetik-Konvergenzanker: Charles–Goren–Lauter, *Families of Ramanujan Graphs and Quaternion Algebras*, CRM Proceedings and Lecture Notes 47 (2008), motiviert AR10d als Vergleich zu quaternionischen Ramanujan-Konstruktionen; Borcherds, *The monster Lie algebra*, *Advances in Mathematics* 83(1), 30–47 (1990), motiviert AR23a als

spaete algebraische Boundary-Pruefung; Gross, *Arithmetic on Elliptic Curves with Complex Multiplication*, LNM 776 (1980), motiviert AR15a als CM-/elliptische-Kurven-Schnittstelle. URLs: https://www.math.mcgill.ca/goren/PAPERSpublic/FinalRamanujan.pdf, https://www.sciencedirect.com/science/article/pii/000187089090067W, https://link.springer.com/book/10.1007/BFb0096754.

- Externe Wirkungs-/Symmetrie-Anker: Zagiers Darstellung von Modulformen ueber die Wirkung auf $\mathbb{H}$, Milnes Galois-Theorie als Automorphismenrahmen, Duncan–Mack-Crane zum Conway-/Leech-/Super-VOA-Pfad und Darstellungen der Monster-VOA-Automorphismen dienen nur als Vergleichslandkarte fuer AR3a/AR25. URLs: https://people.mpim-bonn.mpg.de/zagier/files/doi/10.1007/978-3-540-74119-0_1/fulltext.pdf, https://www.jmilne.org/math/CourseNotes/FT.pdf, https://arxiv.org/abs/1409.3829, https://arxiv.org/pdf/2206.15391. Rootless-/chiral-Lattice-Orientierung aus TomS #84 und der Leech-/Conway-Literatur dient nur als Warnung gegen naive Weyl-/Lie-Lesarten: root-system-artige und rootless/sporadische Wirkungen sind getrennte Ledgerfaelle, kein Beleg fuer eine CNNA-Leech-Konstruktion.
- String-/Worldsheet-Hinweis: Die IdealThread-Analyse motiviert einen kuenftigen, nun konkreteren Rekonstruktionspfad fuer Stringtheorie: Faden/Targetspace/Worldsheet werden als getrennte Strukturpflichten gelesen. `IdealThread` liefert hoechstens das Fadenvergleichsbild, generated nicht-singleton Substrate liefern den Ambient-/Target-Kandidaten, und `CutSpec` kann spaeter als Expositions-/Defekt-/Worldsheet-Schnittstelle geprueft werden. Dieser Hinweis ist explizit kein aktiver Arithmetikbeweis und kein Input fuer AR12a-SMOKE.
- Jakito #87/#88 sowie TomS #89/#90 werden als Korrekturanker fuer die Planreihenfolge genutzt: #87 erzwingt die elliptische-Funktion-Schicht vor q-, L-Funktions- und Moonshine-Consumern; #88 und TomS #90 praezisieren, dass Root-Systeme nicht als Basen zu behandeln sind und dass ein Gitter mit Basis oder Erzeugendensystem nicht automatisch ein Root-System-Gitter ist. TomS #89 bestaetigt zugleich: elliptische Funktionen gehoeren zur lokalen Δ-/j-Provenienz, sind aber kein globales Bindeglied zu sporadischen/VOA-Strukturen. Externe mathematische Orientierung: Weierstrass-$\wp$-Gleichung und Eisenstein-Gittersummen liefern g_2, g_3, Δ, j; Standard-Root-System-Definitionen trennen root set, simple roots, root lattice, Cartan integers und Weyl action. Diese Quellen sind Orientierungsanker, keine CNNA-Provenienz.
- mathlib-Orientierung: `Mathlib.NumberTheory.Modular`, `Mathlib.NumberTheory.ModularForms.*`, `Mathlib.NumberTheory.LSeries.*`, `Mathlib.NumberTheory.NumberField.*`; konkrete Importexistenz immer gegen Lean 4.28.0 im Repo pruefen.
- Alter A-Plan und A14c-Closure-Plan: nur als Negativ-/Gate-Quelle; kein Code-Transfer.

7. Fernziel-Watchlist: IdealThread, CutSpec und Stringtheorie-Rekonstruktion

Der IdealThread/String-Hinweis wird als eigenes Fernziel gefuehrt, weil er aus dem Codebefund heraus konkreter geworden ist: ein eindimensionaler Nullpfad allein traegt nicht die b-/beta-abhaengige Arithmetik, sondern benoetigt ein nicht-singleton Ambient-Substrat. Fuer eine spaetere Stringtheorie-Rekonstruktion waeren deshalb mindestens drei getrennte Proof-Obligations erforderlich: ein abgeleiteter Faden-/Worldline-Traeger, ein abgeleiteter Target-/Ambientraum und eine abgeleitete Expositions- oder Worldsheet-Schicht.

Methodische Sperren: Dieses Fernziel ist kein Input fuer AR0–AR18, keine Erklaerung der Eisenstein-/Gauss-Rechnung und keine Rechtfertigung fuer `CutSpec.full` im Arithmetikpfad. `CutSpec` bleibt als moegliche Defekt-/Region-/Worldsheet-Schnittstelle erhalten, muss aber spaeter eigene Traeger-, Expositions-, Invarianz- und Dynamiklemmata liefern. Ein positiver AR12a-SMOKE-Befund waere nur eine notwendige Plausibilisierung des Ambient-Substrats, kein Stringtheorie-Beweis.

Moegliche spaetere Module bleiben Comparison-/Watchlist-only:

```
CNNA/PillarA/StringTheory/IdealThreadStringMotivation.lean
CNNA/PillarA/StringTheory/CutSpecWorldsheetWatchlist.lean
CNNA/PillarA/StringTheory/TargetAmbientReconstructionGate.lean
```

8. Fernziel: Schur-Multiplier-Gate

Dieses Fernziel wird erst aktiv, wenn AR19–AR24 tatsaechlich eine derived-only endliche Gruppen-, VOA- oder Moonshine-Aktionsoberflaeche erzeugen. Dann ist zu pruefen, ob die Aktion linear oder nur projektiv ist. Im projektiven Fall muss der entstehende 2-Kozyklus identifiziert und seine Klasse in $H^2(G, \mathbb{C}^\times)$ gegen Trivialitaet geprueft werden. Nichttrivialitaet erzwingt eine derived-only zentrale Erweiterung bzw. einen Schur-Cover als eigene Proof-Obligation.

Methodische Sperre: Dieses Gate erklaert nicht die Schur-Eliminierung und darf nicht in AR0–AR18 zurueckprojiziert werden. Es ist ein spaeter Test auf Darstellungsanomalien und Zentralerweiterungen, falls aus dem arithmetischen Pfad echte Gruppen- oder VOA-Aktionen entstehen.